

BSc (Hons) in Information Technology Software Engineering – Year 4
SE4040 - Enterprise Application Development

**Smart Solar Microgrid Trading System - Client-Server Application (Web,
Mobile and Web Service)**



Group ID - 82

Names & Registration Numbers:

IT Number	Name	Phone Number
IT22362544	Shalon Anton Fernando	+94771052042
IT23268494	LiyanaArachchi L.A.D.D	+94717488137
IT23263512	Rukshan E A Y	+94767624847
IT23264670	A.G.L.M Wijesinghe	+94703613334

Contents

Table Of Figures	Error! Bookmark not defined.
1. Executive Summary and System Overview	4
2. System Architecture and Modelling Diagrams	5
2.1 Application High Level Architecture Diagram	5
2.2 System Use Case Diagram	6
2.3 Data Flow Diagrams	7
2.3.1 DFD Level Zero, Context Diagram	7
2.3.2 DFD Level One, Decomposition Diagram.....	8
3. Database Design	9
3.1 MongoDB Collections and Schema	9
3.1.1 Collection - Users.....	10
3.1.2 Collection - SolarStations	10
3.1.3 Collection - EnergyBookingSlots	10
3.1.4 Collection - EnergyReservation	11
3.2 Android Local SQLite Persistence Design	11
4. Source Code	12
4.1 UsersController.cs and AuthController.cs- User Management and Staff Self Service	12
4.2 StationsController.cs - Geospatial Hub and Battery Slot Dispatch	18
4.3 ReservationsController.cs, the Seven Day Rule, QR Generation and QR Verification	22
4.4 AdminProsumersController.cs, ProsumerAuthController.cs, and ProsumerController.cs - Prosume - Management and Account Control	24
5. Individual Contributions	29
5.1 Shalon Fernando (IT22362544).....	29
5.1.1 UI Screenshots	31
5.1.2 Source Code.....	41
5.2 A.G.L.M Wijesinghe (IT23264670).....	141
5.2.1 Description	141
5.2.2 Code Files.....	142
5.3 Rukshan E A Y (IT23263512)	182
5.3.1 Description	182
5.3.2 Code Files.....	183
5.3.3 UI (Screenshots)	232
5.4 Dinil LiyanaArachchi (IT23268494)	242
6. Technical Challenges and Reflections.....	346

7. Use of AI Tools & Reflection	347
8. Hosting and Deployment.....	347
8.1 MongoDB Cloud Database Setup.....	347
7.2 IIS and Kestrel Web API Deployment	348
7.3 Web and Mobile Client Build.....	351
8. Git Repository and References	351
8.1 Git Repository.....	351
8.2 References	351

Table of Figures

Figure 1 : . Application high level architecture diagram	6
Figure 2 : System use case diagram	7
Figure 3 : DFD level zero, context diagram	8
Figure 4 : DFD level one, decomposition diagram.....	9
Figure 5: Mobile map screen filter.....	31
Figure 6: Mobile Dashboard.....	31
Figure 7: Mobile dashboard page extended	32
Figure 8: Mobile prosumer location map screen.....	33
Figure 9: Onboarding	34
Figure 10: Mobile grid operator map screen	35
Figure 11: Onboarding 2	36
Figure 12: Onboarding final	37
Figure 13: Station detail view	38
Figure 14: Station detail view extended	38
Figure 15: Station deactivate business logic.....	39
Figure 16: Microgrid stations page	39
Figure 17: landing page.....	40
Figure 18: webapp dashboard page	40
Figure 19: landing page download apk section	41
Figure 20 : User Management Dashboard.....	175
Figure 21 : Filtered Table View	175
Figure 22 : Edit User Modal.....	176
Figure 23 : Create User Modal	176
Figure 24 : Prosumer Bookings List (Empty State).....	177
Figure 25 : Create New Booking Screen.....	178
Figure 26 : Booking Card (Confirmed).....	179
Figure 27 : Generated QR Code	180
Figure 28 : Cancel Booking Action.....	181

Tables

Table 1 : User collection.....	10
Table 2 : SolarStations collection	10
Table 3 : EnergyBookingSlots collection	11
Table 4 : EnergyReservation Collection	11

1. Executive Summary and System Overview

Joule is an enterprise grade renewable energy trading and microgrid management system. It connects solar energy prosumers, who generate, store and export excess photovoltaic energy, with local battery hub stations and the grid operators who service them.

The system follows the FAT service architecture pattern throughout. This means every piece of business logic lives on the central C# ASP.NET Core Web API, not on the clients. Role based authorization, reservation quota enforcement, cryptographic QR token verification, geospatial proximity calculations and concurrency control are all handled on the server. The React web client and the native Android mobile client exist purely as presentation and local caching layers. Neither client makes a decision that affects data integrity on its own.

2. System Architecture and Modelling Diagrams

The four diagrams referenced in this section, the high level architecture diagram, the use case diagram, the DFD level zero diagram and the DFD level one diagram, were built and are maintained in draw.io. They are stored as editable draw.io files in the Report and diagrams folder of the repository and exported as PNG images for direct embedding in this document. The paragraphs below describe what each diagram shows and how to read it.

2.1 Application High Level Architecture Diagram

The system uses a three tier architecture. Thin web and mobile clients talk to the ASP.NET Core Web API over secure HTTPS REST endpoints. The API is hosted on IIS with Kestrel as the reverse proxied process, and it is the only component allowed to talk to the MongoDB cluster. On the Android side, a local SQLite database holds a small cache of session and profile data so the app can restore a signed in state quickly.

The client tier contains the React web application, built with Vite and styled with Tailwind CSS, and the native Android application, built with Kotlin and Jetpack Compose. The Android app reads and writes its local SQLite cache for session tokens and profile information.

The service tier is the ASP.NET Core Web API itself. Inside it sit four logical modules, the authentication module which handles JWT issuing and BCrypt password hashing, the geospatial engine which runs the 2dsphere proximity queries, the reservation engine which enforces the seven day booking window, and the slot dispatcher which manages atomic battery slot state changes. All four modules are called by the API layer and none of them are reachable directly by a client.

The data tier is a single MongoDB database named SmartMicrogridDB. It holds four collections, Users, SolarStations, EnergyBookingSlots and EnergyReservation, described in full in section 3.

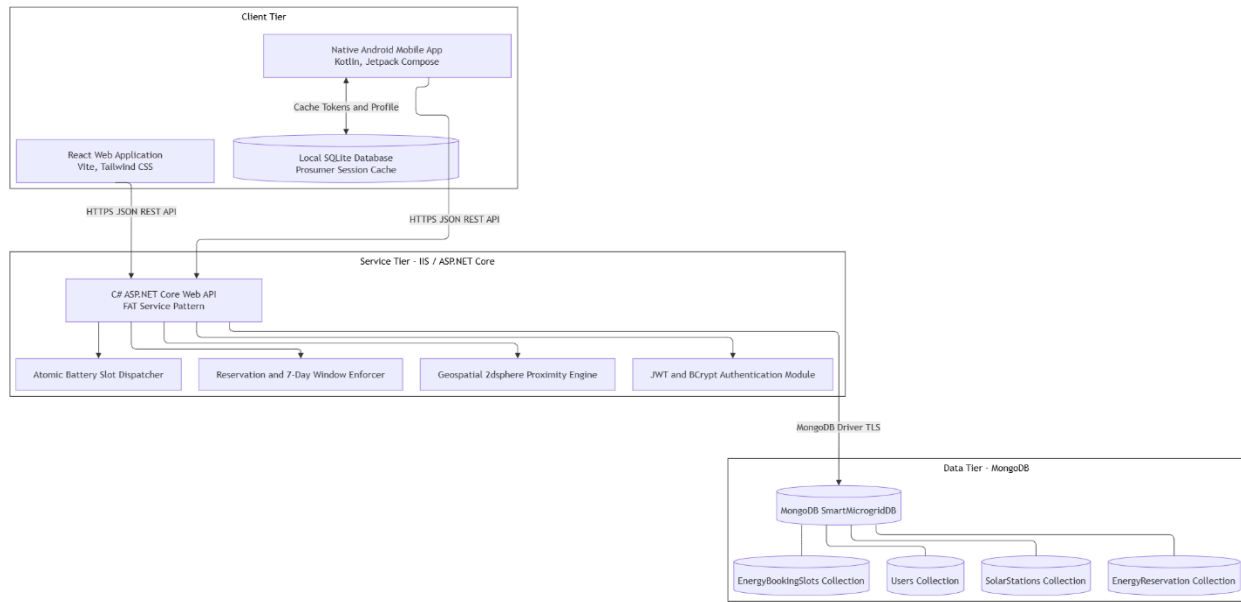


Figure 1 : . Application high level architecture diagram

<https://drive.google.com/file/d/1Z6r98nJRagH-iulA2esQBsadIOGEAKq4/view?usp=sharing>

2.2 System Use Case Diagram

The platform serves three actors, the Backoffice Administrator, the Grid Operator and the Solar Prosumer.

The Backoffice Administrator logs in to the web portal and is responsible for creating and managing staff accounts, registering and updating microgrid hub stations, provisioning battery slots at each station, approving and activating new prosumer registrations, and reviewing the global reservation list and the analytics dashboard.

The Grid Operator can log in on both the web portal and the mobile app. Their core responsibility is finding nearby hubs on the interactive map, scanning a prosumer's QR code with the device camera, and finalising the energy transfer once the token is verified by the server.

The Solar Prosumer only uses the mobile app. A prosumer registers using their National Identity Card number, logs in, searches the map for a nearby hub, creates, modifies or cancels an energy reservation within the allowed booking window, views the QR code issued for an approved reservation, manages their own profile including a profile photo, and can request that their account be deactivated.

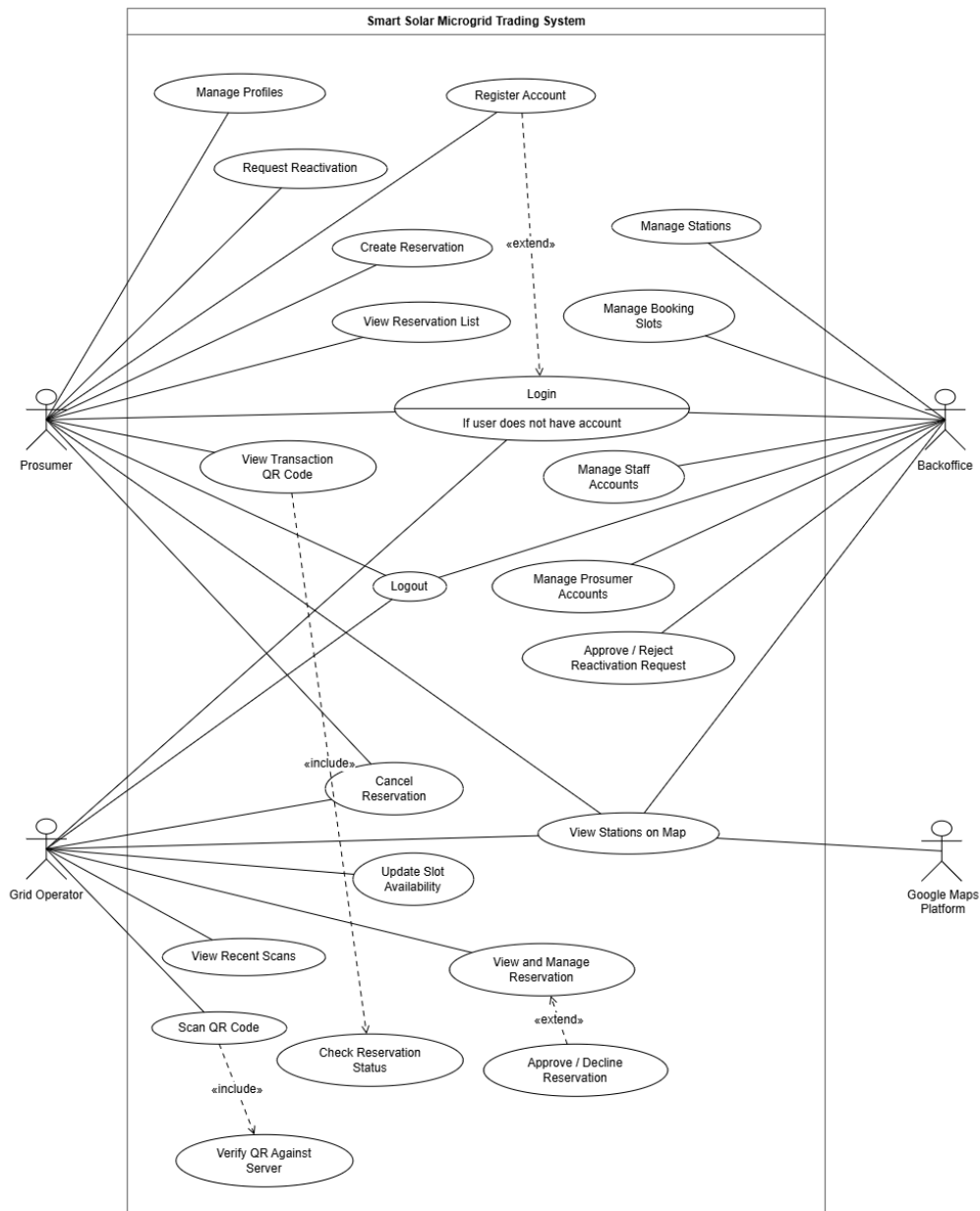


Figure 2 : System use case diagram

https://drive.google.com/file/d/1I5XyO_etypckQC3NkPUkcDxKJt65nfgL/view?usp=sharing

2.3 Data Flow Diagrams

2.3.1 DFD Level Zero, Context Diagram

At the highest level the system is a single process that exchanges data with three external entities. The Backoffice Administrator sends staff credentials, hub information and user edits, and receives audit logs, analytics and user lists back. The Solar Prosumer sends their NIC, registration details, booking requests and a profile picture, and receives JWT tokens, QR tokens, hub maps and reservation status updates. The Grid Operator sends a scanned QR token and their own credentials, and receives a validation status, a transfer confirmation and station details.

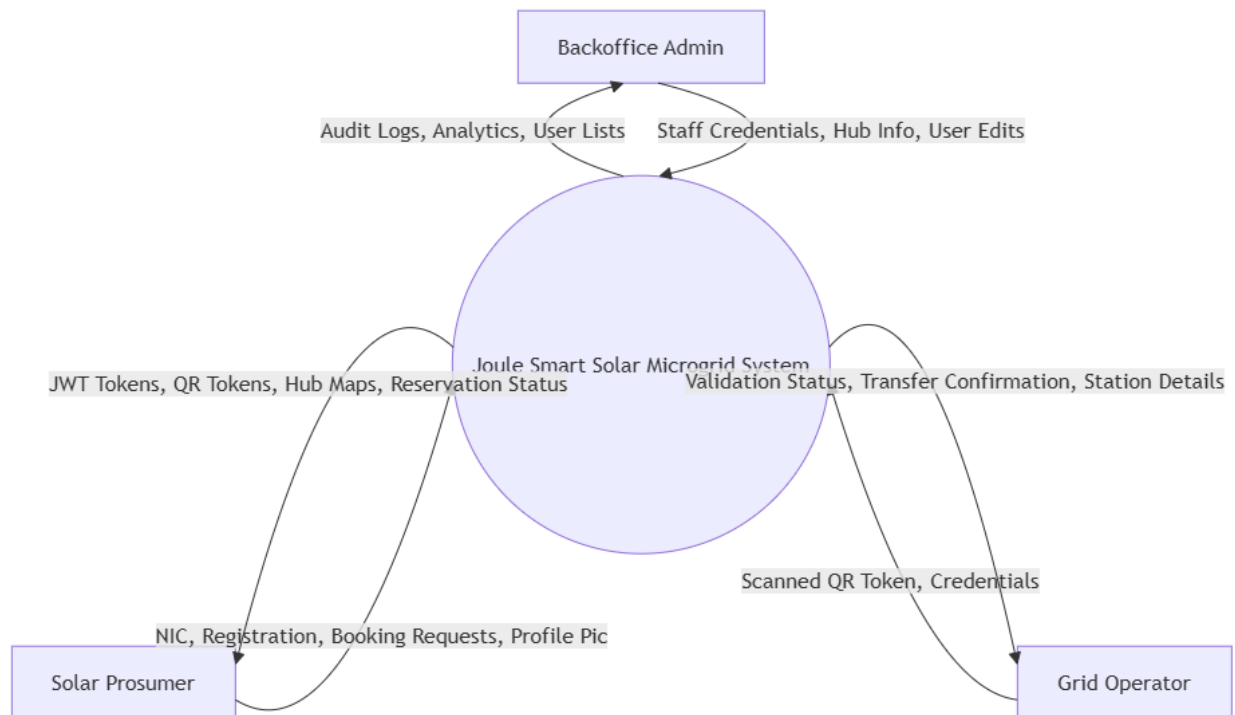


Figure 3 : DFD level zero, context diagram

<https://drive.google.com/file/d/10O8rvaHenOL9qFSaLZLBk8SMrAHc7fB/view?usp=sharing>

2.3.2 DFD Level One, Decomposition Diagram

Breaking the single process apart shows five processes running behind the API.

Process 1.0, the authentication and session controller, takes a NIC and password from the prosumer client, checks the hash against the Users collection, returns a JWT and session information, and asks the mobile app to persist that session in its local SQLite database.

Process 2.0, the geospatial hub controller, takes a station filter and GPS coordinates from the prosumer, runs a 2dsphere spatial query against the SolarStations collection, and returns the filtered list of nearby hubs.

Process 3.0, the reservation lifecycle engine, takes a booking request with a timestamp and slot ID, validates it against the seven day window and locks the slot in the EnergyBookingSlots collection, writes the new record with its QR hash into the EnergyReservation collection, and confirms the booking back to the prosumer along with the QR token.

Process 4.0, the QR dispatch and verification service, takes a scanned token from the Grid Operator client, looks up the matching reservation, transitions its status from Confirmed to Completed, releases the battery slot back to the EnergyBookingSlots collection, and returns a verified response to the operator.

Process 5.0, the backoffice administration engine, takes station and user management actions from the Backoffice portal, updates user profiles and roles in the Users collection, updates station capacity and geocodes in the SolarStations collection, provisions new battery slots, pulls global analytics from the EnergyReservation collection, and returns dashboard statistics to the portal.

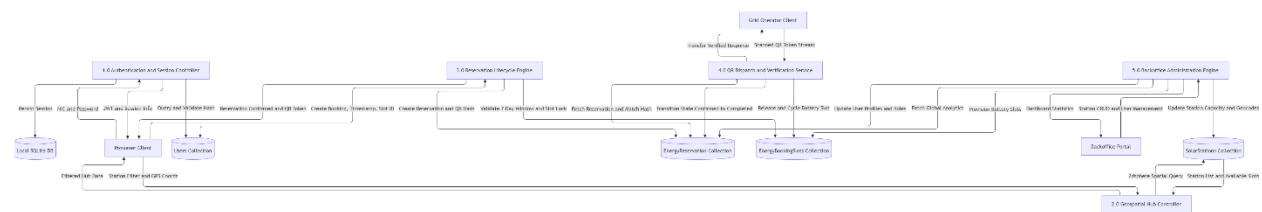


Figure 4 : DFD level one, decomposition diagram.

https://drive.google.com/file/d/1Mz7zyQdaQ6V_x0ChacVTrowqylgXY3XT/view?usp=sharing

3. Database Design

3.1 MongoDB Collections and Schema

The central database is named SmartMicrogridDB and holds four collections.

3.1.1 Collection - Users

This collection stores identity, role and profile information for Backoffice staff, Grid Operators and Prosumers in one place, distinguished by a role field. The username field is unique for staff accounts, the nic field is unique for prosumer accounts, and email is unique across everyone.

Field	Type	Description
_id	ObjectId	Primary key, generated by MongoDB
role	String	Backoffice, GridOperator or Prosumer
username	String, nullable	Login handle for staff accounts
nic	String, nullable	National Identity Card number, the primary business key for prosumers
passwordHash	String	BCrypt salted hash, work factor eleven
fullName	String	Full name of the user
email	String	Unique verified email address
phone	String, nullable	Contact telephone number
address	String, nullable	Residential or physical address
status	String	Active, PendingActivation or Deactivated
deactivationRequestedAt	DateTime, nullable	When a prosumer asked to be deactivated
createdAt	DateTime	UTC time of account creation
updatedAt	DateTime	UTC time of the last metadata change

Table 1 : User collection

3.1.2 Collection - SolarStations

This collection holds the physical microgrid hub nodes, their GPS coordinates, their operating schedule and their hardware capacity. It carries a 2dsphere geospatial index on the location field so that radius based proximity queries run quickly.

Field	Type	Description
_id	ObjectId	Primary key
name	String	Human readable hub name, for example Colombo Central Solar Hub
location	GeoJSON object	A Point with coordinates given as longitude then latitude
capacityKWh	Double	Maximum declared storage capacity in kilowatt hours
totalBatterySlots	Int32	Number of physical battery bays at the hub
operatingSchedule	Object	Opening and closing time, for example six in the morning to ten at night
status	String	Active or Inactive, the latter meaning under maintenance
createdByUserId	ObjectId	The Backoffice officer who registered the hub
createdAt	DateTime	UTC time of registration
updatedAt	DateTime	UTC time of the last update

Table 2 : SolarStations collection

3.1.3 Collection - EnergyBookingSlots

This collection represents the individual battery slot units provisioned inside each hub. A compound index on stationId and slotNumber keeps slot numbers unique within a station.

Field	Type	Description
id	ObjectId	Primary key
stationId	String	Reference to SolarStations, id
slotNumber	Int32	Sequential slot number within the station
type	String	Slot power profile, FastCharge, Standard or Bidirectional
capacityKWh	Double	Maximum rate the slot supports
status	String	Available, Occupied or Maintenance
createdAt	DateTime	UTC time of creation
updatedAt	DateTime	UTC time of the last update

Table 3 : EnergyBookingSlots collection

3.1.4 Collection - EnergyReservation

This collection captures the full lifecycle of an energy transfer booking, from creation through to the QR verified transfer.

Field	Type	Description
id	ObjectId	Primary key
reservationCode	String	Human friendly identifier, for example RES-20260928-8472
nic	String	Prosumer NIC, links to Users, nic
stationId	String	Chosen hub, links to SolarStations, id
slotId	String	Chosen battery slot, links to EnergyBookingSlots, id
scheduledTime	DateTime	Scheduled UTC arrival time, must fall within today and today plus seven days
qrToken	String	Cryptographically signed token embedded in the QR code
status	String	Pending, Confirmed, Completed or Cancelled
cancellationReason	String, nullable	Reason given for a cancellation
verifiedAt	DateTime, nullable	UTC time the QR code was scanned
verifiedByOperatorId	String, nullable	The Grid Operator who verified the transfer
createdAt	DateTime	UTC time the booking was created
updatedAt	DateTime	UTC time of the last status change

Table 4 : EnergyReservation Collection

3.2 Android Local SQLite Persistence Design

To keep the mobile app responsive and to avoid asking a prosumer to sign in on every launch, the Android client keeps a small local SQLite database named SmartSolarGrid.db.

```
CREATE TABLE prosumer_session (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nic TEXT NOT NULL UNIQUE,
  full_name TEXT NOT NULL,
  token TEXT NOT NULL,
  status TEXT NOT NULL,
  profile_picture TEXT,
  last_login_timestamp INTEGER NOT NULL
);
```

The session data access object, ProsumerSessionDao.kt, follows a simple three step flow. When login succeeds against the prosumer login endpoint, the DAO inserts or replaces a row holding the JWT, the NIC, the name, the status and a timestamp. On a cold app launch, the home activity

reads that row and, if a valid token is present, skips straight to the dashboard instead of showing the login screen. When the user logs out, or their account is deactivated, the DAO clears that row and the device falls back to asking for credentials again.

4. Source Code

The following listings show the C# ASP.NET Core controllers that implement the core business rules of the FAT service.

4.1 UsersController.cs and AuthController.cs- User Management and Staff Self Service

File, UsersController.cs. Author - Migara.

```
// =====  
// File: UsersController.cs
```



```

// Purpose: Handles CRUD operations for system users (Backoffice/GridOperator).
//      Enforces role-based access and interacts with MongoDB.
// Author: Migara
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using MongoDB.Driver;
using SmartMicrogrid.Api.Data;
using SmartMicrogrid.Api.Models;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Authorize] // Requires valid JWT token for ALL endpoints by default
    public class UsersController : ControllerBase
    {
        private readonly IMongoCollection<User> _users;

        // Inject MongoDB context
        public UsersController(MongoDbContext context)
        {
            _users = context.Users;
        }

        // GET: api/users
        // Retrieves all web application users without exposing password hashes.
        [HttpGet]
        public async Task<ActionResult<IEnumerable<UserResponseDto>>> GetAllUsers()
        {
            // Fetch all users from MongoDB
            var users = await _users.Find(u => true).ToListAsync();

            // Map to response DTO (excludes password hash for security)
            var usersWithoutPasswords = users.Select(u => new UserResponseDto
            {
                Id = u.Id,
                Username = u.Username,
                Role = u.Role,
                FullName = u.FullName,
                Email = u.Email,
                Status = u.Status,
                CreatedAt = u.CreatedAt,
                UpdatedAt = u.UpdatedAt
            });

            return Ok(usersWithoutPasswords);
        }
    }
}

```

```

}

// POST: api/users
// Creates a new user, hashes password, and saves to MongoDB.
// AllowAnonymous is used here so the FIRST admin user can be created
// without a token. All subsequent user creation should be done by an authenticated admin.
[AllowAnonymous]
[HttpPost]
public async Task<ActionResult<UserResponseDto>> CreateUser([FromBody] CreateUserDto dto)
{
    // Validate if username already exists
    var existingUser = await _users.Find(u => u.Username == dto.Username).FirstOrDefaultAsync();
    if (existingUser != null)
    {
        return Conflict(new { message = "Username already exists." });
    }

    // Validate email uniqueness
    var existingEmail = await _users.Find(u => u.Email == dto.Email).FirstOrDefaultAsync();
    if (existingEmail != null)
    {
        return Conflict(new { message = "Email already exists." });
    }

    // Hash the password using BCrypt
    string passwordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password);

    // Create the user object
    var newUser = new User
    {
        Username = dto.Username,
        PasswordHash = passwordHash,
        Role = dto.Role,
        FullName = dto.FullName,
        Email = dto.Email,
        Status = "Active",
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    // Save to MongoDB
    await _users.InsertOneAsync(newUser);

    // Return success as DTO (without password hash)
    var response = new UserResponseDto
    {
        Id = newUser.Id,
        Username = newUser.Username,

```

```

        Role = newUser.Role,
        FullName = newUser.FullName,
        Email = newUser.Email,
        Status = newUser.Status,
        CreatedAt = newUser.CreatedAt,
        UpdatedAt = newUser.UpdatedAt
    };

    return CreatedAtAction(nameof(GetAllUsers), new { id = newUser.Id }, response);
}

// PUT: api/users/{id}
// Updates user details.
[HttpPut("{id}")]
public async Task<IActionResult> UpdateUser(string id, [FromBody] UpdateUserDto dto)
{
    // Find the user
    var filter = Builders<User>.Filter.Eq(u => u.Id, id);
    var user = await _users.Find(filter).FirstOrDefaultAsync();

    if (user == null)
    {
        return NotFound(new { message = "User not found." });
    }

    // Update user fields
    var update = Builders<User>.Update
        .Set(u => u.FullName, dto.FullName)
        .Set(u => u.Email, dto.Email)
        .Set(u => u.Role, dto.Role)
        .Set(u => u.UpdatedAt, DateTime.UtcNow);

    var result = await _users.UpdateOneAsync(filter, update);

    if (result.MatchedCount == 0)
    {
        return NotFound(new { message = "User not found." });
    }

    return NoContent(); // 204 Success
}

// PUT: api/users/{id}/deactivate
// Soft deletes the user by changing status to Deactivated.
[HttpPut("{id}/deactivate")]
public async Task<IActionResult> DeactivateUser(string id)
{
    // Find the user

```

```

        var filter = Builders<User>.Filter.Eq(u => u.Id, id);
        var user = await _users.Find(filter).FirstOrDefaultAsync();

        if (user == null)
        {
            return NotFound(new { message = "User not found." });
        }

        // Update status to Deactivated
        var update = Builders<User>.Update
            .Set(u => u.Status, "Deactivated")
            .Set(u => u.UpdatedAt, DateTime.UtcNow);

        var result = await _users.UpdateOneAsync(filter, update);

        if (result.MatchedCount == 0)
        {
            return NotFound(new { message = "User not found." });
        }

        return NoContent(); // 204 Success
    }
}

// =====
// File: AuthController.cs
// Purpose: Handles user authentication, password verification, and JWT token generation.
// Author: Migara
// =====

using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using MongoDB.Driver;
using SmartMicrogrid.Api.Data;
using SmartMicrogrid.Api.Models;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly IMongoCollection<User> _users;
    }
}

```

```

private readonly IConfiguration _config;

// Inject MongoDB context and configuration
public AuthController(MongoDbContext context, IConfiguration config)
{
    _users = context.Users;
    _config = config;
}

// POST: api/auth/login
// Authenticates user and returns JWT token
[HttpPost("login")]
public async Task<ActionResult<LoginResponseDto>> Login([FromBody] LoginDto loginDto)
{
    // Find user by username
    var user = await _users.Find(u => u.Username == loginDto.Username).FirstOrDefaultAsync();

    // Validate user exists, is active, and password matches
    if (user == null || user.Status != "Active" || !BCrypt.Net.BCrypt.Verify(loginDto.Password,
user.PasswordHash))
    {
        return Unauthorized(new { message = "Invalid username, password, or account is
deactivated." });
    }

    // Generate JWT token
    var token = GenerateJwtToken(user);

    // Return token and user info
    return Ok(new LoginResponseDto
    {
        Token = token,
        Role = user.Role,
        FullName = user.FullName,
        UserId = user.Id
    });
}

// Generates a JWT token for the authenticated user
private string GenerateJwtToken(User user)
{
    var jwtKey = _config["Jwt:Key"] ?? "DefaultSecretKeyAtLeast32CharactersLong!";
    var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey));
    var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id),

```

```

        new Claim("role", user.Role),
        new Claim("fullName", user.FullName),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddHours(2),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
}
}

```

4.2 StationsController.cs - Geospatial Hub and Battery Slot Dispatch

File, StationsController.cs. Author, Shalon.

```

// =====
// File: StationsController.cs
// Purpose: HTTP endpoints for microgrid hub (station) management —
//   list/search, detail-with-slots, create, partial update,
//   and the nearby-stations query (moved from Migara
//   2026-09-21, see architecture.md §3/§4). Thin controller:
//   all business rules and validation live in StationService
//   per the FAT service pattern; this file only maps
//   requests/exceptions to HTTP.
// Author: Shalon
// =====
using System.IdentityModel.Tokens.Jwt;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers;

[ApiController]
[Route("api/stations")]
public class StationsController : ControllerBase
{
    private readonly StationService _stationService;

```

```

public StationsController(StationService stationService)
{
    _stationService = stationService;
}

// GET /api/stations?search=&status= - lists stations for any authenticated role.
[HttpGet]
public async Task<IActionResult> GetAll([FromQuery] string? search, [FromQuery] string? status)
{
    try
    {
        var stations = await _stationService.GetAllAsync(search, status);
        return Ok(stations);
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// GET /api/stations/nearby?lat=&lng=&radiusKm= - active stations within radiusKm, closest
// first. Moved from Migara to Shalon 2026-09-21 — see architecture.md §3/§4. Registered before
// "{id}" in this file only for readability; ASP.NET Core's routing always matches the literal
// "nearby" segment ahead of the "{id}" parameter regardless of declaration order.
[HttpGet("nearby")]
public async Task<IActionResult> GetNearby([FromQuery] double lat, [FromQuery] double lng,
[FromQuery] double radiusKm)
{
    try
    {
        var stations = await _stationService.GetNearbyAsync(lat, lng, radiusKm);
        return Ok(stations);
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// GET /api/stations/{id} - full station detail plus its slots.
[HttpGet("{id}")]
public async Task<IActionResult> GetById(string id)
{
    var station = await _stationService.GetByIdAsync(id);
    if (station is null)
    {
        return NotFound(new { message = $"Station '{id}' not found." });
    }
}

```

```

    }

    return Ok(station);
}

// POST /api/stations - creates a new station. Backoffice only — Migara's JWT auth (merged
// 2026-09-26) now exists, so the TODO to restore [Authorize] and read the real caller id is done.
[Authorize(Roles = "Backoffice")]
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateStationRequest request)
{
    try
    {
        var id = await _stationService.CreateAsync(request, createdByUserId: CallerId());
        return CreatedAtAction(nameof(GetById), new { id }, new CreatedIdResponse { Id = id });
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// PUT /api/stations/{id} - partial update of station fields. Backoffice only.
[Authorize(Roles = "Backoffice")]
[HttpPut("{id}")]
public async Task<IActionResult> Update(string id, [FromBody] UpdateStationRequest request)
{
    try
    {
        await _stationService.UpdateAsync(id, request);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// PUT /api/stations/{id}/deactivate - blocks with 409 if the station has active slots/reservations.
// Backoffice only.
[Authorize(Roles = "Backoffice")]
[HttpPut("{id}/deactivate")]
public async Task<IActionResult> Deactivate(string id)
{

```



```

    try
    {
        await _stationService.DeactivateAsync(id);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (ConflictException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

// PUT /api/stations/{id}/activate - flips an Inactive station back to Active. No conflict check
// needed (unlike deactivate): reactivating never blocks on anything. Backoffice only.
[Authorize(Roles = "Backoffice")]
[HttpPut("{id}/activate")]
public async Task<IActionResult> Activate(string id)
{
    try
    {
        await _stationService.ActivateAsync(id);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

// GET /api/stations/{id}/reservations-overview - read-only "who's booked what" panel.
// Currently always returns [] — see StationService.GetReservationsOverviewAsync's TODO(Dinil).
[HttpGet("{id}/reservations-overview")]
public async Task<IActionResult> GetReservationsOverview(string id)
{
    try
    {
        var overview = await _stationService.GetReservationsOverviewAsync(id);
        return Ok(overview);
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

```

```
// Reads the caller's user id from the JWT "sub" claim Migara's AuthController sets — only
// ever called from an [Authorize]-protected action, so the claim is guaranteed present.
private string? CallerId() => User.FindFirst(JwtRegisteredClaimNames.Sub)?.Value;
}
```

4.3 ReservationsController.cs, the Seven Day Rule, QR Generation and QR Verification

File, ReservationsController.cs. Authors, Dinil

```
// =====
// File: ReservationsController.cs
// Purpose: Reservation lifecycle endpoints – create, update,
//         cancel, fetch, list, and QR verification. All
//         business rules delegated to ReservationService
//         (FAT service pattern). No validation in clients.
// Author: Dinil
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers;

[ApiController]
[Route("api/reservations")]
// [Authorize]
public class ReservationsController : ControllerBase
{
    private readonly ReservationService _service;

    public ReservationsController(ReservationService service)
    {
        _service = service;
    }

    // Creates a reservation under the caller's prosumer identity.
    [HttpPost]
    //[Authorize(Roles = "Prosumer")]
    public async Task<IActionResult> Create([FromBody] CreateReservationRequest req)
    {
        var nic = User.FindFirst("nic")?.Value
            ?? throw new ServiceException(401, "MISSING_NIC", "JWT missing nic claim.");
        var result = await _service.CreateAsync(nic, req);
        return Ok(result);
    }

    // Updates a reservation (12-hour rule, 7-day window re-check).
    [HttpPut("{id}")]
    //[Authorize(Roles = "Prosumer")]
    public async Task<IActionResult> Update(string id, [FromBody] UpdateReservationRequest req)
    {
        var nic = User.FindFirst("nic")?.Value!;
```

```

        await _service.UpdateAsync(id, nic, req);
        return NoContent();
    }

    // Cancels a reservation (12-hour rule, slot returned to Available).
    [HttpPut("{id}/cancel")]
    //[Authorize(Roles = "Prosumer")]
    public async Task<IActionResult> Cancel(string id, [FromBody] CancelReservationRequest
req)
    {
        var nic = User.FindFirst("nic")?.Value!;
        await _service.CancelAsync(id, nic, req.Reason);
        return NoContent();
    }

    // Fetches one reservation – owning Prosumer or GridOperator/Backoffice.
    [HttpGet("{id}")]
    //[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]
    public async Task<IActionResult> Get(string id)
    {
        var role = User.FindFirst("role")?.Value;
        var nic = User.FindFirst("nic")?.Value;

        var reservation = (role == "Prosumer")
            ? await _service.GetOwnedAsync(id, nic!)
            : await _service.GetByIdAsync(id);

        return Ok(MapToResponse(reservation));
    }

    // Lists reservations with filters. Prosumers are scoped to their own NIC.
    [HttpGet]
    //[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]
    public async Task<IActionResult> List(
        [FromQuery] string? nic,
        [FromQuery] string? stationId,
        [FromQuery] string? status,
        [FromQuery] DateTime? from,
        [FromQuery] DateTime? to)
    {
        var role = User.FindFirst("role")?.Value;
        var jwtNic = User.FindFirst("nic")?.Value;

        // Prosumers can only ever see their own reservations.
        var effectiveNic = (role == "Prosumer") ? jwtNic : nic;

        var list = await _service.ListAsync(effectiveNic, stationId, status, from, to);
        return Ok(list.Select(MapToResponse));
    }

    // Grid Operator verifies a scanned QR token, flipping the reservation to Completed.
    [HttpPost("verify-qr")]
    //[Authorize(Roles = "GridOperator")]
    public async Task<IActionResult> VerifyQr([FromBody] VerifyQrRequest req)
    {
        var operatorUserId = User.FindFirst("sub")?.Value
            ?? throw new ServiceException(401, "MISSING_SUB", "JWT missing sub claim.");
        var result = await _service.VerifyQrAsync(req.QrToken, operatorUserId);
        return Ok(result);
    }
}

```

```

// Maps a Reservation model to its API response shape.
private static ReservationResponse MapToResponse(Reservation r) => new()
{
    Id = r.Id,
    ProsumerNic = r.ProsumerNic,
    StationId = r.StationId,
    SlotId = r.SlotId,
    ScheduledAt = r.ScheduledAt,
    Status = r.Status,
    QrToken = r.QrToken,
    CreatedAt = r.CreatedAt,
    CompletedAt = r.CompletedAt,
    CancelReason = r.CancelReason
};
}

```

4.4 AdminProsumersController.cs, ProsumerAuthController.cs, and ProsumerController.cs - Prosume - Management and Account Control

File, AdminProsumersController.cs. Author - Rukshan E A Y

```

// =====
// File: AdminProsumersController.cs
// Purpose: Handles Backoffice administration of Prosumer profiles.
//          Enforces that ONLY Backoffice can create, deactivate,
//          activate, and reactivate accounts.
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/admin/prosumers")]
    [Authorize(Roles = "Backoffice")] // STRICT: Only Backoffice can access these endpoints
    public class AdminProsumersController : ControllerBase
    {
        private readonly ProsumerService _prosumerService;

        public AdminProsumersController(ProsumerService prosumerService)
        {
            _prosumerService = prosumerService;
        }

        // GET: api/admin/prosumers/pending
        // Returns only prosumers awaiting Backoffice activation (for the web app pending
view)
        [HttpGet("pending")]
        public async Task<ActionResult<List<ProsumerResponseDto>>> GetPendingProsumers()
        {
            var allProsumers = await _prosumerService.GetAllProsumersAsync();

```

```

        var pending = allProsumers.Where(p => p.Status == "PendingActivation").ToList();
        return Ok(pending);
    }

    // GET: api/admin/prosumers
    [HttpGet]
    public async Task<ActionResult<List<ProsumerResponseDto>>> GetAllProsumers()
    {
        var prosumers = await _prosumerService.GetAllProsumersAsync();
        return Ok(prosumers);
    }

    // GET: api/admin/prosumers/{nic}
    [HttpGet("{nic}")]
    public async Task<ActionResult<ProsumerResponseDto>> GetProsumerByNic(string nic)
    {
        var prosumer = await _prosumerService.GetByNicAsync(nic);
        if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

        return Ok(prosumer);
    }

    // POST: api/admin/prosumers
    // Allows Backoffice to manually create a prosumer profile
    [HttpPost]
    public async Task<IActionResult> CreateProsumer([FromBody] ProsumerRegistrationDto
dto)
    {
        var success = await _prosumerService.CreateManuallyAsync(dto);
        if (!success) return Conflict(new { message = "A prosumer with this NIC already
exists." });

        return CreatedAtAction(nameof(GetProsumerByNic), new { nic = dto.Nic }, new {
message = "Prosumer created successfully." });
    }

    // PUT: api/admin/prosumers/{nic}
    [HttpPut("{nic}")]
    public async Task<IActionResult> UpdateProsumer(string nic, [FromBody]
UpdateProsumerDto dto)
    {
        var success = await _prosumerService.UpdateAsync(nic, dto);
        if (!success) return NotFound(new { message = "Prosumer not found or no changes
made." });

        return NoContent(); // 204 Success
    }

    // PUT: api/admin/prosumers/{nic}/deactivate
    [HttpPut("{nic}/deactivate")]
    public async Task<IActionResult> DeactivateProsumer(string nic)
    {
        var success = await _prosumerService.DeactivateAsync(nic);
        if (!success) return NotFound(new { message = "Prosumer not found." });

        return NoContent(); // 204 Success
    }

    // PUT: api/admin/prosumers/{nic}/activate
    // Activates a prosumer who is currently in "PendingActivation" status
    [HttpPut("{nic}/activate")]

```

```

        public async Task<IActionResult> ActivateProsumer(string nic)
        {
            var prosumer = await _prosumerService.GetByNicAsync(nic);
            if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

            if (prosumer.Status != "PendingActivation")
            {
                return BadRequest(new { message = "Only pending accounts can be activated."
            });
            }

            var success = await _prosumerService.ActivateAsync(nic);
            if (!success) return StatusCode(500, new { message = "Failed to activate
prosumer." });

            return NoContent(); // 204 Success
        }

        // PUT: api/admin/prosumers/{nic}/reactivate
        // CRITICAL RULE: Deactivated accounts can ONLY be reactivated by a Backoffice
officer.
        [HttpPut("{nic}/reactivate")]
        public async Task<IActionResult> ReactivateProsumer(string nic)
        {
            var prosumer = await _prosumerService.GetByNicAsync(nic);
            if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

            if (prosumer.Status != "Deactivated")
            {
                return BadRequest(new { message = "Only deactivated accounts can be
reactivated." });
            }

            var success = await _prosumerService.ReactivateAsync(nic);
            if (!success) return StatusCode(500, new { message = "Failed to reactivate
prosumer." });

            return NoContent(); // 204 Success
        }
    }
}

```

File, ProsumerAuthController.cs. Author - Rukshan E A Y

```

// =====
// File: ProsumerAuthController.cs
// Purpose: Handles Prosumer authentication (Login) for the Mobile App.
//          Calls ProsumerService to enforce business rules (Status checks).
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/auth/prosumer")]

```

```

public class ProsumerAuthController : ControllerBase
{
    private readonly ProsumerService _prosumerService;
    private readonly JwtService _jwtService;

    public ProsumerAuthController(ProsumerService prosumerService, JwtService jwtService)
    {
        _prosumerService = prosumerService;
        _jwtService = jwtService;
    }

    // POST: api/auth/prosumer/login
    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginProsumerDto dto)
    {
        // 1. Call Service to handle business logic (Status checks, Password verify)
        var result = await _prosumerService.AuthenticateAsync(dto.Nic, dto.Password);

        // 2. Handle Failure Cases
        if (!result.Success)
        {
            // Return 403 Forbidden for Pending/Deactivated, 401 Unauthorized for wrong
            // credentials
            if (result.Message.Contains("pending") ||
                result.Message.Contains("deactivated"))
            {
                return StatusCode(403, new { message = result.Message });
            }
            return Unauthorized(new { message = result.Message });
        }

        // 3. Generate JWT Token on Success
        var prosumer = result.Prosumer!;
        var token = _jwtService.GenerateToken(prosumer.Id, prosumer.Role,
        prosumer.FullName, prosumer.Nic);

        // 4. Return Success Response
        return Ok(new
        {
            token = token,
            role = prosumer.Role,
            fullName = prosumer.FullName,
            nic = prosumer.Nic,
            status = prosumer.Status
        });
    }
}

// DTO for Prosumer Login Request
public class LoginProsumerDto
{
    public string Nic { get; set; } = null!;
    public string Password { get; set; } = null!;
}

```

File, ProsumerController.cs. Author - Rukshan E A Y

```

// =====
// File: ProsumerController.cs

```

```

// Purpose: Handles Prosumer self-service actions (Register, Edit, Request Deactivation).
//          Used by the Native Android Mobile Application.
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/prosumers")]
    public class ProsumerController : ControllerBase
    {
        private readonly ProsumerService _prosumerService;

        public ProsumerController(ProsumerService prosumerService)
        {
            _prosumerService = prosumerService;
        }

        // POST: api/prosumers/register
        // AllowAnonymous so the mobile app can register without a token
        [AllowAnonymous]
        [HttpPost("register")]
        public async Task<IActionResult> Register([FromBody] ProsumerRegistrationDto dto)
        {
            var success = await _prosumerService.RegisterAsync(dto);
            if (!success) return Conflict(new { message = "A prosumer with this NIC already exists." });

            return CreatedAtAction(nameof(GetProfile), new { nic = dto.Nic }, new { message = "Registration successful. Pending Backoffice activation." });
        }

        // GET: api/prosumers/{nic}
        [Authorize]
        [HttpGet("{nic}")]
        public async Task<ActionResult<ProsumerResponseDto>> GetProfile(string nic)
        {
            // Security: Ensure the user can only fetch their own profile (or let Backoffice fetch any)
            var userNic = User.FindFirstValue("nic");
            var role = User.FindFirstValue(ClaimTypes.Role);

            if (role != "Backoffice" && userNic != nic)
            {
                return Forbid(); // 403 Forbidden
            }

            var prosumer = await _prosumerService.GetByNicAsync(nic);
            if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

            return Ok(prosumer);
        }

        // PUT: api/prosumers/{nic}
        [Authorize]

```



```

[HttpPut("{nic}")]
public async Task<IActionResult> UpdateProfile(string nic, [FromBody]
UpdateProsumerDto dto)
{
    var userNic = User.FindFirstValue("nic");
    if (userNic != nic) return Forbid();

    var success = await _prosumerService.UpdateAsync(nic, dto);
    if (!success) return NotFound(new { message = "Prosumer not found." });

    return NoContent();
}

// GET: api/prosumers?status=Active
// Returns the list of Prosumers (optionally filtered by status).
// Used by the Web App's reservation form so operators can pick a
// Prosumer from a dropdown instead of typing an NIC by hand.
// Backoffice and GridOperator may call this.
//[Authorize(Roles = "Backoffice,GridOperator")]
[HttpGet]
public async Task<ActionResult<IEnumerable<ProsumerResponseDto>>> List(
    [FromQuery] string? status)
{
    var prosumers = await _prosumerService.ListAsync(status);
    return Ok(prosumers);
}

// PUT: api/prosumers/{nic}/request-deactivation
[Authorize]
[HttpPut("{nic}/request-deactivation")]
public async Task<IActionResult> RequestDeactivation(string nic)
{
    var userNic = User.FindFirstValue("nic");
    if (userNic != nic) return Forbid();

    var success = await _prosumerService.RequestDeactivationAsync(nic);
    if (!success) return NotFound(new { message = "Prosumer not found." });

    return NoContent();
}
}
}

```

5. Individual Contributions

Work on Joule was split into vertical slices across both the web and mobile platforms, so that each member owns a complete feature from the database up to the screen a user sees.

5.1 Shalon Fernando (IT22362544)

Built Microgrid Node Management on the web side, meaning the stations page, the station detail page and the map picker used when registering a new hub, along with the shared application shell, sidebar and top bar and the overall design tokens used across the web app. On mobile, I built the Prosumer Dashboard with its live counters, the interactive Google Maps hub screen with

an expandable distance and capacity filter, and the custom icon set used throughout the app. On the backend, I wrote the Stations controller, the dashboard controller, the Haversine distance calculation and the two dsphere index configuration. Relevant files include StationsPage.jsx, StationDetailPage.jsx, MapPicker.jsx, AppShell.jsx, Sidebar.jsx, TopBar.jsx, MapScreen.kt, ProsumerDashboardScreen.kt, StationsController.cs and DashboardController.cs.

For the backend, I developed the station management services and dashboard services using C Sharp and ASP.NET Core. I kept the controllers simple and placed the main business logic and database operations inside the service layer.

I worked with MongoDB for retrieving station, battery slot, and reservation data. I also implemented location based searches using MongoDB geographic indexes. This allowed the system to find nearby solar stations based on the users location.

I also added checks when deactivating a station. A station cannot be deactivated when it has reserved battery slots or confirmed reservations. This helps prevent active bookings from being affected.

For nearby station searches, I used a MongoDB aggregation pipeline to find stations based on distance and also get the available battery slots in the same database request. This helped reduce unnecessary database requests.

For the web application, I developed the station management pages, station details, station creation and editing forms, map picker, dashboard, and main application layout. I added features such as search, status filters, pagination, station capacity information, battery slot information, and station activation and deactivation.

For the mobile application, I worked on the map screen and prosumer dashboard. The map uses the users location to show nearby solar stations. The dashboard shows booking information and important energy related information.

I also added an offline cache for the mobile dashboard using SQLite. This allows previously loaded dashboard information to be shown even when the application is temporarily offline, while new information can be loaded from the backend when the connection is available.

The main technologies I used were C Sharp, ASP.NET Core, MongoDB, React, Vite, Tailwind CSS, Kotlin, Jetpack Compose, Google Maps, Leaflet, SQLite, Retrofit, and Kotlin Coroutines.

Overall, my contribution covered a large part of the full stack system, from backend services and database operations to the web interface and Android application. I mainly focused on microgrid station management, location based station discovery, prosumer dashboards, and the shared application design.

5.1.1 UI Screenshots

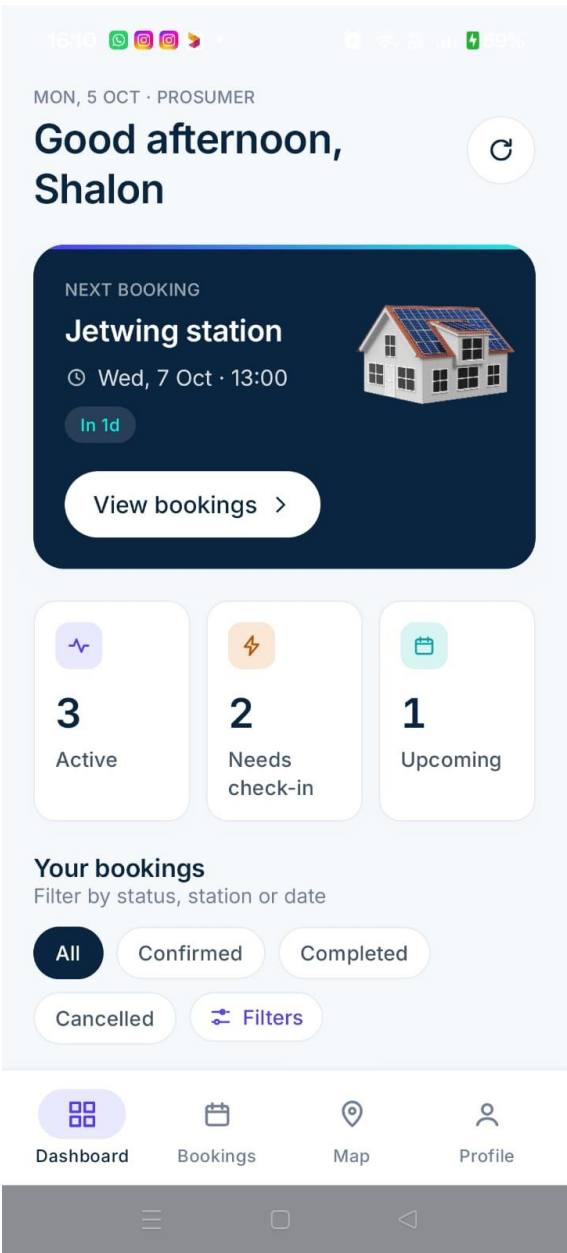


Figure 6: Mobile Dashboard

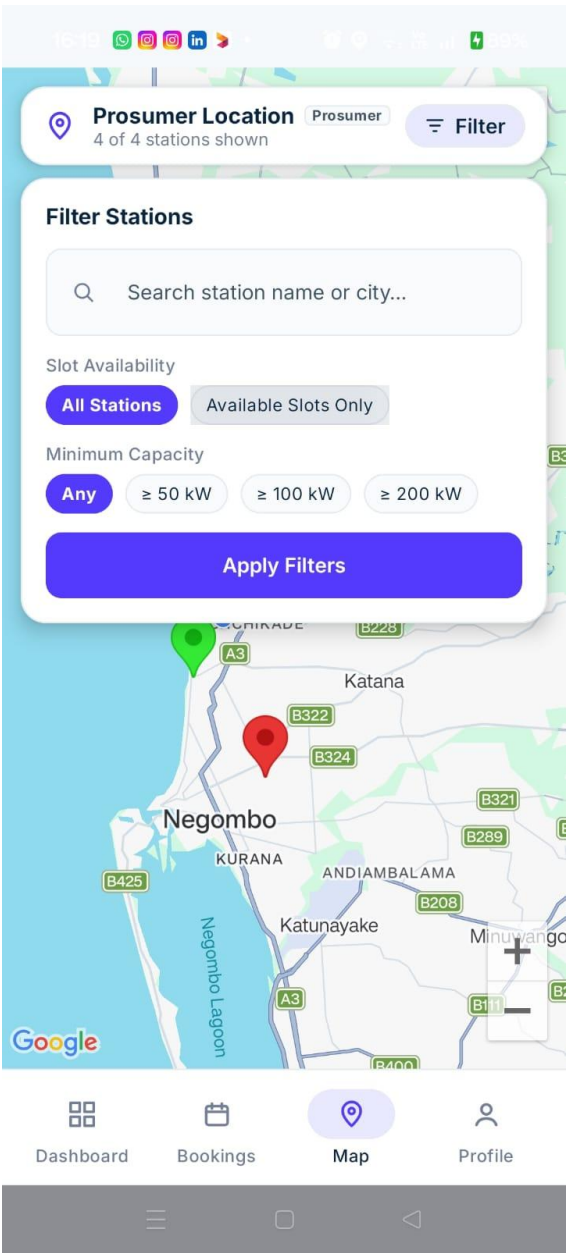


Figure 5: Mobile map screen filter

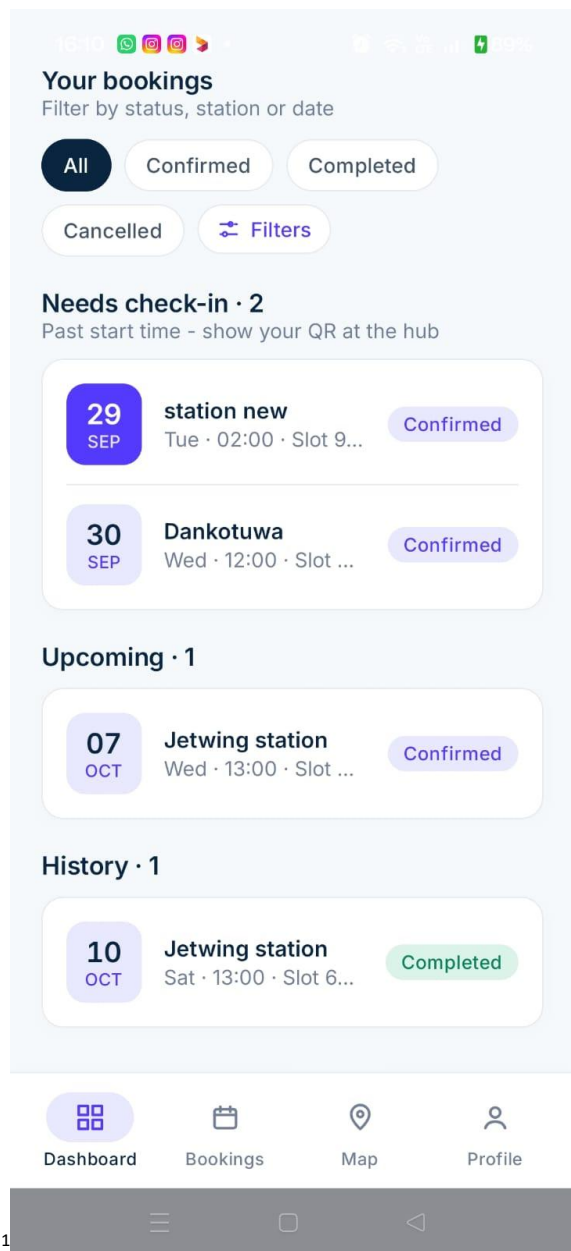


Figure 7: Mobile dashboard page extended

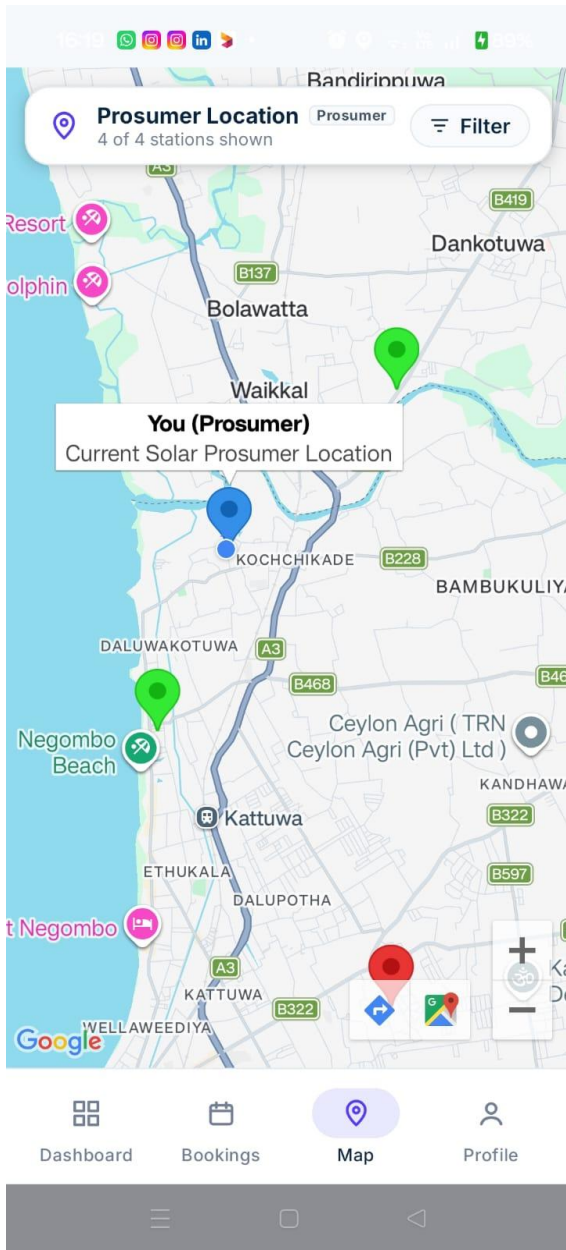


Figure 8: Mobile prosumer location map screen



Figure 9: Onboarding

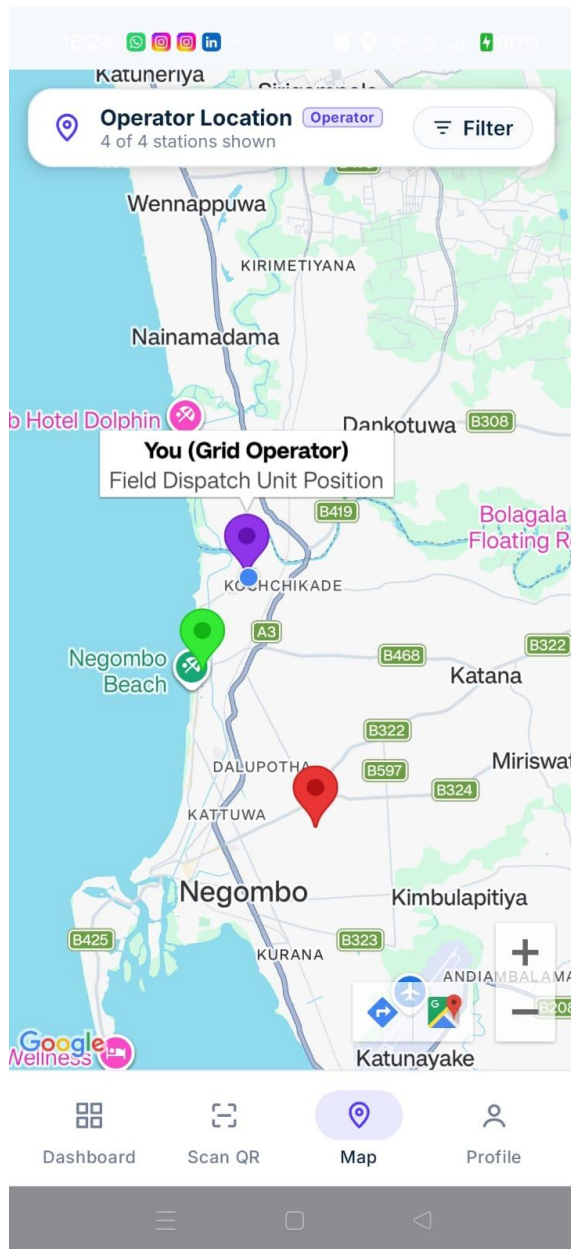


Figure 10: Mobile grid operator map screen



Figure 11: Onboarding 2

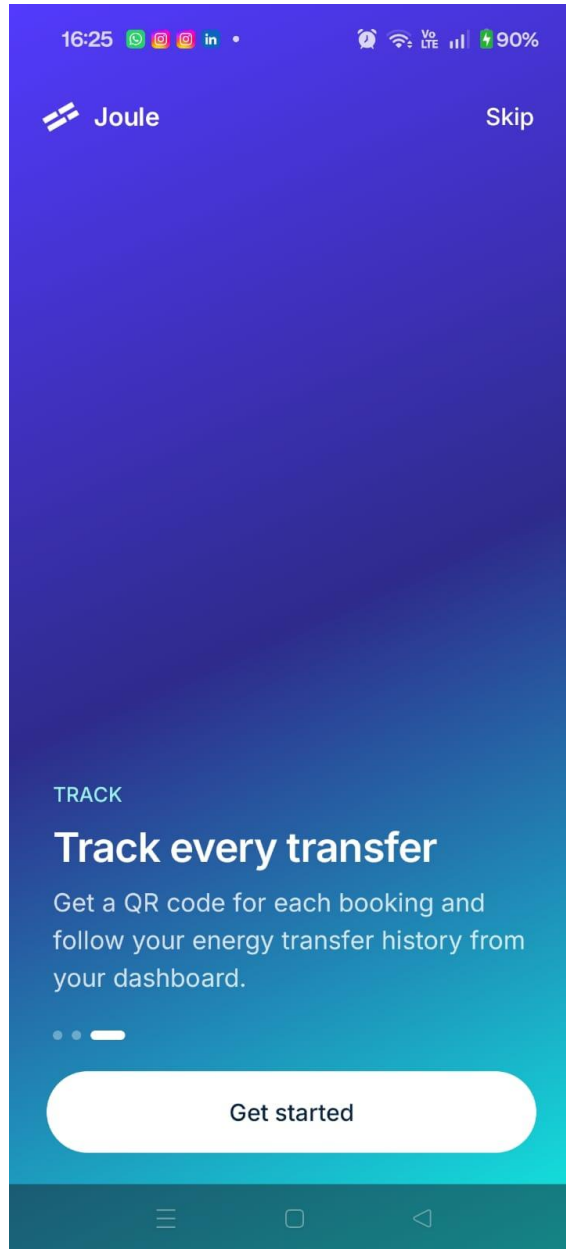


Figure 12: Onboarding final

Joule

Dashboard

Microgrid hubs

Reservations

Prosumers

Pending Prosumers

Users

SF

Shalon Fernando

Backoffice

Sign out

← Back to Microgrid Hubs

Jetwing station

Active

Deactivate Station

GP: 7.2487, 79.8431 · ID: 6ac34cb83c5ec06967f56aa1

TOTAL CAPACITY

DECLARED SLOTS

ACTIVE SLOT CONFIGS

10000 kWh

50

3

Operating Schedule

Opening and closing window for prosumer battery reservation check-ins.

Opens at

Closes at

Update Schedule

06:43 AM

06:43 PM

Battery Slots Provisioning

Individual battery slots and live availability state.

+ New Slot

SLOT #	BATTERY TYPE	CAPACITY	STATUS	ACTIONS
Slot 1	Charging	150 kWh	Reserved	Edit Slot

Opens at

Closes at

Update Schedule

06:43 AM

06:43 PM

Figure 13: Station detail view

Joule

Dashboard

Microgrid hubs

Reservations

Prosumers

Pending Prosumers

Users

SF

Shalon Fernando

Backoffice

Sign out

← Back to Microgrid Hubs

Jetwing station

Active

Deactivate Station

GP: 7.2487, 79.8431 · ID: 6ac34cb83c5ec06967f56aa1

TOTAL CAPACITY

DECLARED SLOTS

ACTIVE SLOT CONFIGS

10000 kWh

50

3

Operating Schedule

Opening and closing window for prosumer battery reservation check-ins.

Opens at

Closes at

Update Schedule

06:43 AM

06:43 PM

Battery Slots Provisioning

Individual battery slots and live availability state.

+ New Slot

SLOT #	BATTERY TYPE	CAPACITY	STATUS	ACTIONS
Slot 1	Charging	150 kWh	Reserved	Edit Slot
Slot 2	Discharging	500 kWh	Available	Edit Slot
Slot 3	Charging	1000 kWh	Available	Edit Slot

Active Reservations Overview

Live schedule of prosumers booked for this microgrid hub.

SLOT #	PROSUMER NIC	SCHEDULED TIME	STATUS
Slot 3	200212803369	10/10/2026, 1:00:00 PM	Completed
Slot 1	200212803369	10/7/2026, 1:00:00 PM	Confirmed

Figure 14: Station detail view extended

Joule

Dashboard
Microgrid hubs
Reservations
Prosumers
Pending Prosumers
Users

SF Shalon Fernando Backoffice

Sign out

Back to Microgrid Hubs

Jetwing station
Active
Deactivate Station

GPS: 7.2487, 79.8431 · ID: 6ac34cb83c5ec06967f56aa1

TOTAL CAPACITY

DECLARED SLOTS

ACTIVE SLOT CONFIGS

10000 kWh
50
3

Operating Schedule

Opening and closing window for prosumer battery reservation check-ins.

Opens at

Closes at

Update Schedule

06:43 AM
06:43 PM

Battery Slots Provisioning

Individual battery slots and live availability state.

SLOT #

BATTERY TYPE

CAPACITY

STATUS

Slot 1

Charging

150 kWh

Reserved

Cannot deactivate station '6ac34cb83c5ec06967f56aa1': one or more of its slots are currently Reserved, and one or more of its reservations are currently Confirmed.

Figure 15: Station deactivate business logic

Joule

Dashboard
Microgrid hubs
Reservations
Prosumers
Pending Prosumers
Users

SF Shalon Fernando Backoffice

Sign out

Microgrid Hubs 13 nodes

Solar microgrid stations, capacity specifications, and battery slot provisioning.

+ New Station

Total Hubs

Active Nodes

Total Capacity

Battery Slots

13
9
42,916 kWh
125

Registered grid nodes
4 currently inactive
Across all solar stations
Total declared slots

Search by station name...

All 13 Active 9 Inactive 4

HUB STATION

GPS COORDINATES

CAPACITY SPEC

SLOTS

STATUS

ACTIONS

Central station

6.9458, 79.9132

2000 kWh

17%

10

Active

Details Edit Deactivate

Dankotuwa

7.2827, 79.8713

166 kWh

1%

15

Active

Details Edit Deactivate

Jetwing station

7.2487, 79.8431

10000 kWh

83%

50

Active

Details Edit Deactivate

Negombo

7.2177, 79.8655

1600 kWh

13%

5

Active

Details Edit Deactivate

Figure 16: Microgrid stations page

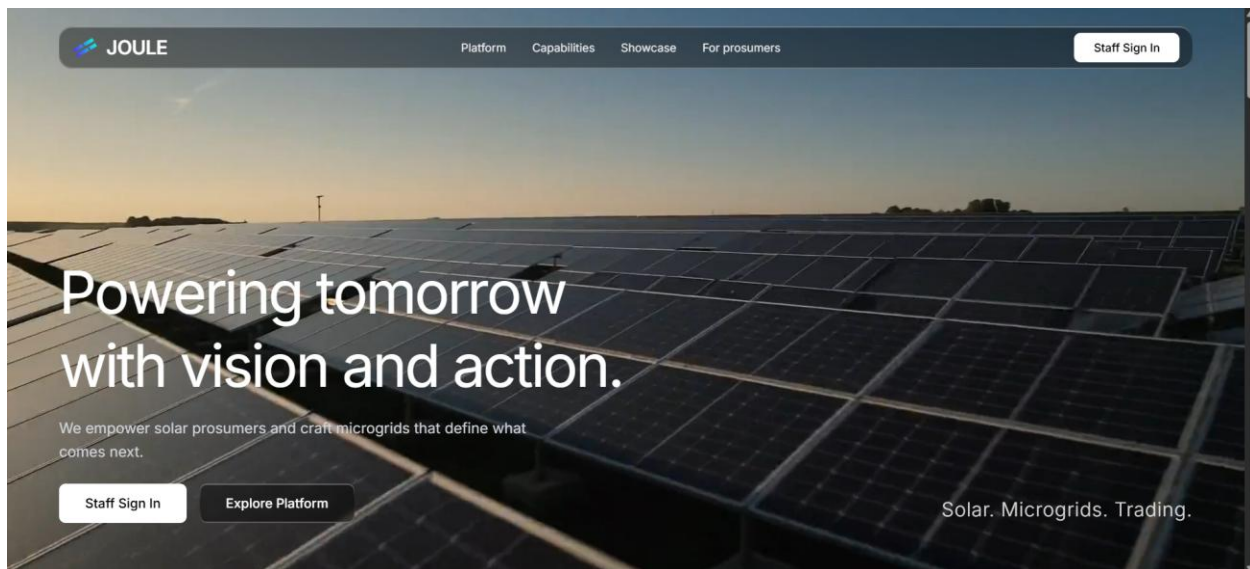


Figure 17: landing page

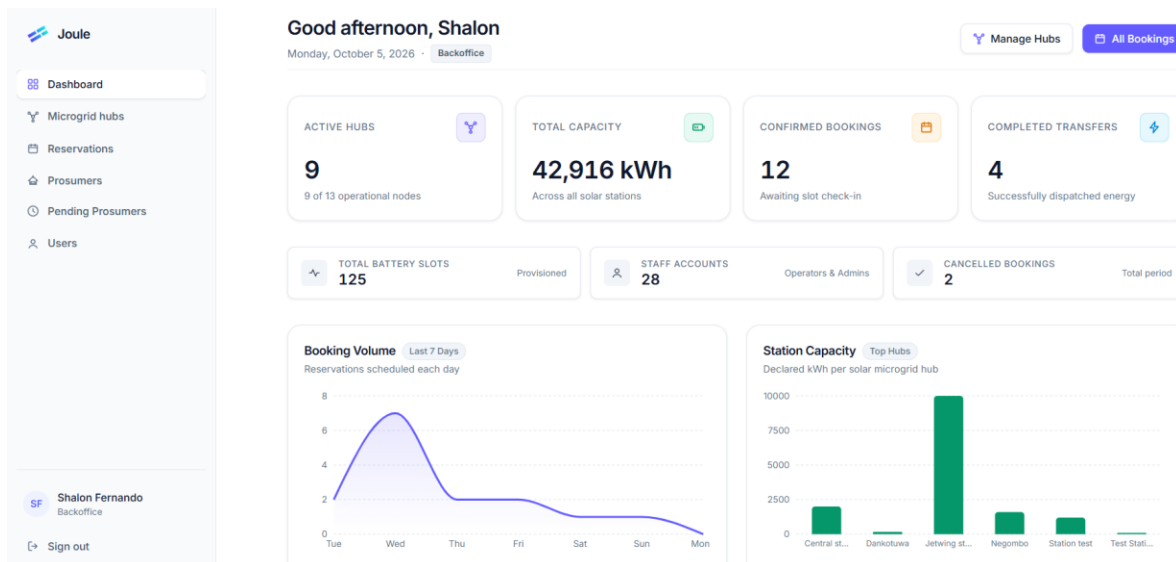


Figure 18: webapp dashboard page

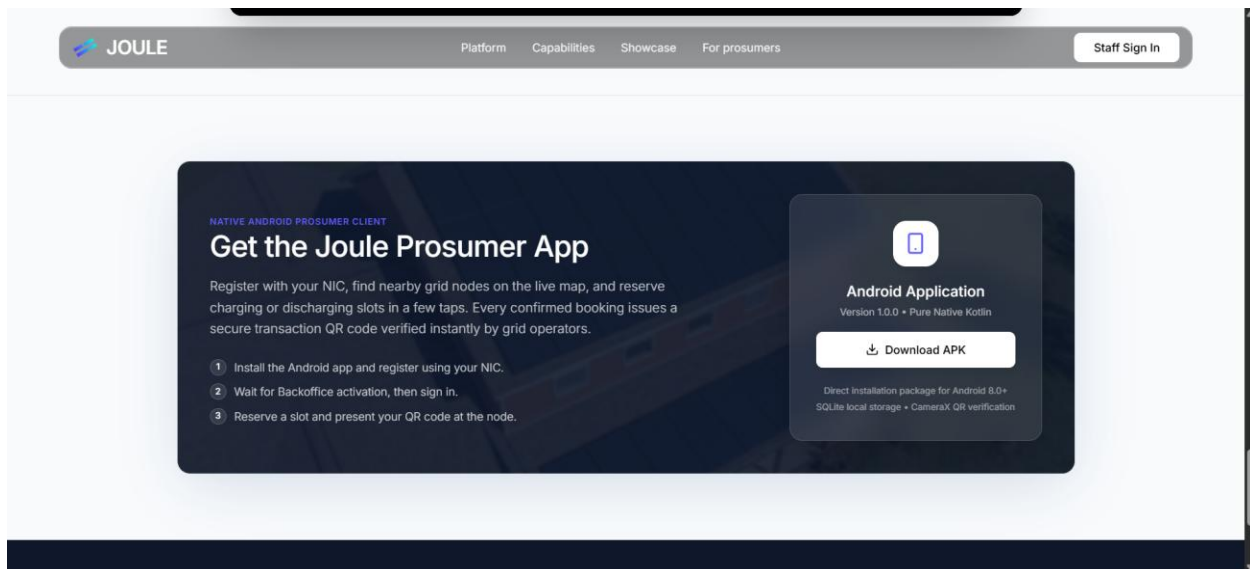


Figure 19: landing page download apk section

5.1.2 Source Code

Detailed Inventory of Authored Files & Modules:

1. Backend Web API:

- StationsController.cs: Thin REST controller exposing endpoints for station CRUD, status toggling (activate/deactivate), reservations overview, and geospatial nearby queries.
- DashboardController.cs: Thin REST controller providing prosumer dashboard metric summaries and grid operator live KPIs.
- StationService.cs: FAT business service managing 2dsphere index creation, station validation, \$geoNear aggregation pipelines, slot joins, and deactivation conflict guards.
- DashboardService.cs: FAT business service computing prosumer active/pending/approved-future reservation counts and operator station aggregations.

2. Web Portal Application:

- StationsPage.jsx: Enterprise microgrid hub management interface featuring real-time search, status filter tabs, grid/table view modes, capacity utilization progress meters, and dynamic pagination.
- StationDetailPage.jsx: Comprehensive station inspection console with hero metrics, editable operating schedule forms, battery slot provisioning tables, and live reservation logs.
- StationForm.jsx: Modal dialog for creating and updating microgrid hubs with live GPS validation and interactive map launcher.
- MapPicker.jsx: Interactive Leaflet/OpenStreetMap coordinate picker modal with live geocoding address search and draggable pin.
- DashboardPage.jsx: Main administrative analytics command center featuring 4 primary KPI cards, secondary metric strips, 7-day Recharts booking volume area charts, capacity distribution bar charts, and live activity feeds.
- AppShell.jsx: Core application layout container wrapping authenticated routes with responsive layout management.

- Sidebar.jsx: Modern left navigation rail with brand identity, icon navigation sections, active state indicators, user avatar chip, and session logout.

- TopBar.jsx: Top navigation bar containing dynamic breadcrumbs and mobile drawer toggles.

3. Native Android Mobile Application:

- MapScreen.kt: Interactive Google Map screen with custom marker hue differentiation (Azure Blue for Prosumers, Electric Violet for Grid Operators), FusedLocationProvider integration, permission handling, and an expandable animated filter panel (search, availability, capacity chips).

- ProsumerDashboardScreen.kt: Premium prosumer dashboard with personalized greeting, dark Next Booking hero card, live metric tiles, date-filtered booking history, bottom sheet filter modal, and offline SQLite cache fallback.

- OnboardingActivity.kt: Immersive 3-slide introductory onboarding walkthrough with full-bleed solar photography, gradient scrims, horizontal pager, animated pill indicators, and first-run persistence.

- SplashActivity.kt: Branded launch screen with animated Joule vector mark, automatic session resolution, and intelligent routing (Onboarding vs. HomeActivity vs. SignIn).

StationsController.cs

using System.IdentityModel.Tokens.Jwt; using Microsoft.AspNetCore.Authorization; using Microsoft.AspNetCore.Mvc; using SmartMicrogrid.Api.Models; using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers;

[ApiController] [Route("api/stations")] public class StationsController : ControllerBase { private readonly StationService _stationService;

public StationsController(StationService stationService)
{
 _stationService = stationService;
}

// GET /api/stations?search=&status= - lists stations for any authenticated role.

[HttpGet]

public async Task<IActionResult> GetAll([FromQuery] string? search,
[FromQuery] string? status)

{
 try
 {
 var stations = await _stationService.GetAllAsync(search, status);
 return Ok(stations);
 }
 catch (ValidationException ex)
 {

```

        return BadRequest(new { message = ex.Message });
    }
}

// GET /api/stations/nearby?lat=&lng=&radiusKm= - active stations within
radiusKm, closest
// first. Moved from Migara to Shalon 2026-09-21 - see architecture.md §3/§4.
Registered before
// "{id}" in this file only for readability; ASP.NET Core's routing always
matches the literal
// "nearby" segment ahead of the "{id}" parameter regardless of declaration
order.
[HttpGet("nearby")]
public async Task<IActionResult> GetNearby([FromQuery] double lat,
[FromQuery] double lng, [FromQuery] double radiusKm)
{
    try
    {
        var stations = await _stationService.GetNearbyAsync(lat, lng,
radiusKm);
        return Ok(stations);
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// GET /api/stations/{id} - full station detail plus its slots.
[HttpGet("{id}")]
public async Task<IActionResult> GetById(string id)
{
    var station = await _stationService.GetByIdAsync(id);
    if (station is null)
    {
        return NotFound(new { message = $"Station '{id}' not found." });
    }

    return Ok(station);
}

// POST /api/stations - creates a new station. Backoffice only - Migara's JWT
auth (merged
// 2026-09-26) now exists, so the TODO to restore [Authorize] and read the
real caller id is done.
[Authorize(Roles = "Backoffice")]

```

```

[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateStationRequest
request)
{
    try
    {
        var id = await _stationService.CreateAsync(request, createdByUserId:
CallerId());
        return CreatedAtAction(nameof(GetById), new { id }, new
CreatedIdResponse { Id = id });
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

```

// PUT /api/stations/{id} - partial update of station fields. Backoffice only.

```

[Authorize(Roles = "Backoffice")]
[HttpPut("{id}")]
public async Task<IActionResult> Update(string id, [FromBody]
UpdateStationRequest request)
{
    try
    {
        await _stationService.UpdateAsync(id, request);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (ValidationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

```

// PUT /api/stations/{id}/deactivate - blocks with 409 if the station has active slots/reservations.

// Backoffice only.

```

[Authorize(Roles = "Backoffice")]
[HttpPut("{id}/deactivate")]
public async Task<IActionResult> Deactivate(string id)
{

```



```

    try
    {
        await _stationService.DeactivateAsync(id);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (ConflictException ex)
    {
        return Conflict(new { message = ex.Message });
    }
}

```

// PUT /api/stations/{id}/activate - flips an Inactive station back to Active. No conflict check
 // needed (unlike deactivate): reactivating never blocks on anything.
 Backoffice only.

```

[Authorize(Roles = "Backoffice")]
[HttpPut("{id}/activate")]
public async Task<IActionResult> Activate(string id)
{
    try
    {
        await _stationService.ActivateAsync(id);
        return NoContent();
    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

```

// GET /api/stations/{id}/reservations-overview - read-only "who's booked what" panel.

// Currently always returns [] - see

StationService.GetReservationsOverviewAsync's TODO(Dinil).

```

[HttpGet("{id}/reservations-overview")]
public async Task<IActionResult> GetReservationsOverview(string id)
{
    try
    {
        var overview = await
        _stationService.GetReservationsOverviewAsync(id);
        return Ok(overview);
    }
}

```

```

    }
    catch (NotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}

// Reads the caller's user id from the JWT "sub" claim Migara's
AuthController sets - only
// ever called from an [Authorize]-protected action, so the claim is
guaranteed present.
private string? CallerId() =>
User.FindFirst(JwtRegisteredClaimNames.Sub)?.Value;

}

```

DashboardController.cs

```

using Microsoft.AspNetCore.Mvc; using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers;

[ApiController] [Route("api/dashboard")] public class DashboardController : ControllerBase { private
readonly DashboardService _dashboardService;

// Injects the service that owns all dashboard aggregation logic.
public DashboardController(DashboardService dashboardService)
{
    _dashboardService = dashboardService;
}

// GET /api/dashboard/prosumer/{nic}/summary - active/pending/approved-future
counts + recent history.
// Deliberately left open (2026-09-26): Migara's JWT auth now exists for web
Backoffice/
// GridOperator accounts, but there is still no Prosumer login/JWT anywhere
(mobile or web) -
// the mobile Dashboard screen that calls this reads a fixture NIC with no
session at all.
// Restoring [Authorize] here would 401 every mobile call with nothing to fix
it yet. Revisit
// once a Prosumer auth flow exists; this isn't a forgotten TODO, it's
blocked on that.
[HttpGet("prosumer/{nic}/summary")]
public async Task<IActionResult> GetProsumerSummary(string nic)
{

```

```

        try
        {
            var summary = await _dashboardService.GetProsumerSummaryAsync(nic);
            return Ok(summary);
        }
        catch (ValidationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }

    // GET /api/dashboard/operator/summary - today's confirmed/completed counts
    // plus pending-by-station breakdown. GridOperator only.
    [HttpGet("operator/summary")]
    [Microsoft.AspNetCore.Authorization.Authorize(
        Roles = "GridOperator,Backoffice")]
    public async Task<IActionResult> GetOperatorSummary()
    {
        try
        {
            var summary = await _dashboardService.GetOperatorSummaryAsync();
            return Ok(summary);
        }
        catch (ValidationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
    }
}

```

StationService.cs

```

using System.Text.RegularExpressions; using MongoDB.Bson; using
MongoDB.Bson.Serialization.Attributes; using MongoDB.Driver; using
MongoDB.Driver.GeoJsonObjectModel; using SmartMicrogrid.Api.Data; using
SmartMicrogrid.Api.Models;

namespace SmartMicrogrid.Api.Services;

public class StationService { private readonly MongoDBContext _context;

    public StationService(MongoDbContext context)
    {
        _context = context;
    }
}

```

```

// Ensures the 2dsphere index on SolarStations.location exists; safe to call
on every startup
// since MongoDB no-ops index creation when an equivalent index already
exists.
public async Task EnsureIndexesAsync()
{
    var indexKeys = Builders<Station>.IndexKeys.Geo2DSphere(s => s.Location);
    var indexModel = new CreateIndexModel<Station>(indexKeys, new
CreateIndexOptions { Name = "location_2dsphere" });
    await _context.SolarStations.Indexes.CreateOneAsync(indexModel);
}

// Builds an optional name-search + status filter, then returns matching
stations as summaries.
public async Task<List<StationSummaryResponse>> GetAllAsync(string? search,
string? status)
{
    var filterBuilder = Builders<Station>.Filter;
    var filter = filterBuilder.Empty;

    if (!string.IsNullOrEmpty(search))
    {
        filter &= filterBuilder.Regex(s => s.Name, new
BsonRegularExpression(Regex.Escape(search), "i"));
    }

    if (!string.IsNullOrEmpty(status))
    {
        if (!Enum.TryParse<StationStatus>(status, ignoreCase: true, out var
parsedStatus))
        {
            throw new ValidationException($"Invalid status filter '{status}'.
Expected 'Active' or 'Inactive'.");
        }
        filter &= filterBuilder.Eq(s => s.Status, parsedStatus);
    }

    var stations = await _context.SolarStations.Find(filter).SortBy(s =>
s.Name).ToListAsync();
    return stations.Select(MapToSummary).ToList();
}

// Loads a station by id and attaches its slots for the full detail view;
returns null if not found.
public async Task<StationDetailResponse?> GetByIdAsync(string id)

```

```

{
    if (!ObjectId.TryParse(id, out _))
    {
        return null;
    }

    var station = await _context.SolarStations.Find(s => s.Id ==
id).FirstOrDefaultAsync();
    if (station is null)
    {
        return null;
    }

    var slots = await _context.EnergyBookingSlots.Find(sl => sl.StationId ==
id).OrderBy(sl => sl.SlotNumber).ToListAsync();

    return new StationDetailResponse
    {
        Id = station.Id!,
        Name = station.Name,
        Location = MapLocation(station.Location),
        CapacityKWh = station.CapacityKWh,
        TotalBatterySlots = station.TotalBatterySlots,
        OperatingSchedule = new OperatingScheduleDto
        {
            OpensAt = station.OperatingSchedule.OpensAt,
            ClosesAt = station.OperatingSchedule.ClosesAt
        },
        Status = station.Status.ToString(),
        CreatedByUserId = station.CreatedByUserId,
        CreatedAt = station.CreatedAt,
        UpdatedAt = station.UpdatedAt,
        Slots = slots.Select(MapSlot).ToList()
    };
}

```

// Validates the incoming station fields, then inserts a new SolarStations document.

```

public async Task<string> CreateAsync(CreateStationRequest request, string?
createdByUserId)
{
    ValidateName(request.Name);
    ValidateCoordinates(request.Lat, request.Lng);
    ValidateCapacity(request.CapacityKWh);
    ValidateTotalBatterySlots(request.TotalBatterySlots);
    ValidateOperatingSchedule(request.OperatingSchedule);
}

```

```

var now = DateTime.UtcNow;
var station = new Station
{
    Name = request.Name.Trim(),
    Location = new GeoJsonPoint<GeoJson2DGeographicCoordinates>(
        new GeoJson2DGeographicCoordinates(request.Lng, request.Lat)),
    CapacityKWh = request.CapacityKWh,
    TotalBatterySlots = request.TotalBatterySlots,
    OperatingSchedule = new OperatingSchedule
    {
        OpensAt = request.OperatingSchedule.OpensAt,
        ClosesAt = request.OperatingSchedule.ClosesAt
    },
    Status = StationStatus.Active,
    CreatedByUserId = createdByUserId,
    CreatedAt = now,
    UpdatedAt = now
};

await _context.SolarStations.InsertOneAsync(station);
return station.Id!;
}

// Applies only the fields present on the request to an existing station,
// validating each one that's present.
public async Task UpdateAsync(string id, UpdateStationRequest request)
{
    if (!ObjectId.TryParse(id, out _))
    {
        throw new NotFoundException($"Station '{id}' not found.");
    }

    var updateBuilder = Builders<Station>.Update;
    var updates = new List<UpdateDefinition<Station>>();

    if (request.Name is not null)
    {
        ValidateName(request.Name);
        updates.Add(updateBuilder.Set(s => s.Name, request.Name.Trim()));
    }

    if (request.Lat.HasValue || request.Lng.HasValue)
    {
        if (!request.Lat.HasValue || !request.Lng.HasValue)
        {

```

```

        throw new ValidationException("Both 'lat' and 'lng' must be
provided together to update location.");
    }
    ValidateCoordinates(request.Lat.Value, request.Lng.Value);
    updates.Add(updateBuilder.Set(
        s => s.Location,
        new GeoJsonPoint<GeoJson2DGeographicCoordinates>(
            new GeoJson2DGeographicCoordinates(request.Lng.Value,
request.Lat.Value))));
    }

    if (request.CapacityKWh.HasValue)
    {
        ValidateCapacity(request.CapacityKWh.Value);
        updates.Add(updateBuilder.Set(s => s.CapacityKWh,
request.CapacityKWh.Value));
    }

    if (request.TotalBatterySlots.HasValue)
    {
        ValidateTotalBatterySlots(request.TotalBatterySlots.Value);
        updates.Add(updateBuilder.Set(s => s.TotalBatterySlots,
request.TotalBatterySlots.Value));
    }

    if (request.OperatingSchedule is not null)
    {
        ValidateOperatingSchedule(request.OperatingSchedule);
        updates.Add(updateBuilder.Set(s => s.OperatingSchedule, new
OperatingSchedule
        {
            OpensAt = request.OperatingSchedule.OpensAt,
            ClosesAt = request.OperatingSchedule.ClosesAt
        }));
    }

    if (updates.Count == 0)
    {
        throw new ValidationException("No fields provided to update.");
    }

    updates.Add(updateBuilder.Set(s => s.UpdatedAt, DateTime.UtcNow));

    var result = await _context.SolarStations.UpdateOneAsync(s => s.Id == id,
updateBuilder.Combine(updates));
    if (result.MatchedCount == 0)

```

```

    {
        throw new NotFoundException($"Station '{id}' not found.");
    }
}

// Flips a station to Inactive, but only once it's confirmed there are no
// active slots or
// reservations tying it up (architecture.md §3: PUT
// /stations/{id}/deactivate, 409 if any Slot
// is Reserved or any Reservation is Confirmed at this station).
public async Task DeactivateAsync(string id)
{
    if (!ObjectId.TryParse(id, out _))
    {
        throw new NotFoundException($"Station '{id}' not found.");
    }

    var station = await _context.SolarStations.Find(s => s.Id ==
id).FirstOrDefaultAsync();
    if (station is null)
    {
        throw new NotFoundException($"Station '{id}' not found.");
    }

    var hasReservedSlots = await HasReservedSlotsAsync(id);
    var hasConfirmedReservations = await HasConfirmedReservationsAsync(id);

    if (hasReservedSlots || hasConfirmedReservations)
    {
        var blockers = new List<string>();
        if (hasReservedSlots)
        {
            blockers.Add("one or more of its slots are currently Reserved");
        }
        if (hasConfirmedReservations)
        {
            blockers.Add("one or more of its reservations are currently
Confirmed");
        }
        throw new ConflictException($"Cannot deactivate station '{id}':
{string.Join(", and ", blockers)}");
    }

    var update = Builders<Station>.Update
        .Set(s => s.Status, StationStatus.Inactive)
        .Set(s => s.UpdatedAt, DateTime.UtcNow);

```



```

        await _context.SolarStations.UpdateOneAsync(s => s.Id == id, update);
    }

    // Flips a station back to Active. Unlike deactivation, reactivating never
    // conflicts with
    // anything - it only makes the station bookable again, so there's no
    // equivalent check to run.
    public async Task ActivateAsync(string id)
    {
        if (!ObjectId.TryParse(id, out _))
        {
            throw new NotFoundException($"Station '{id}' not found.");
        }

        var update = Builders<Station>.Update
            .Set(s => s.Status, StationStatus.Active)
            .Set(s => s.UpdatedAt, DateTime.UtcNow);
        var result = await _context.SolarStations.UpdateOneAsync(s => s.Id == id,
update);
        if (result.MatchedCount == 0)
        {
            throw new NotFoundException($"Station '{id}' not found.");
        }
    }

    // Returns the read-only "who's booked what" overview for a station, per
    // architecture.md §3
    // GET /stations/{id}/reservations-overview. Reads Dinil's Reservations
    // collection (write-side
    // owner: Dinil; Shalon reads it - see §4).
    public async Task<List<ReservationOverviewResponse>>
    GetReservationsOverviewAsync(string stationId)
    {
        if (!ObjectId.TryParse(stationId, out _))
        {
            throw new NotFoundException($"Station '{stationId}' not found.");
        }

        var stationExists = await _context.SolarStations.Find(s => s.Id ==
stationId).AnyAsync();
        if (!stationExists)
        {
            throw new NotFoundException($"Station '{stationId}' not found.");
        }

        var reservations = await _context.Reservations

```

```

        .Find(r => r.StationId == stationId)
        .SortByDescending(r => r.ScheduledAt)
        .ToListAsync();

return reservations.Select(r => new ReservationOverviewResponse
{
    SlotId = r.SlotId,
    Status = r.Status,
    ProsumerNic = r.ProsumerNic,
    ScheduledAt = r.ScheduledAt
}).ToList();
}

// Finds active stations within radiusKm of (lat, lng), closest first, via a
$geoNear
// aggregation against the 2dsphere index on SolarStations.location. Moved
from Migara to
// Shalon 2026-09-21 - see architecture.md §3/§4.
public async Task<List<NearbyStationResponse>> GetNearbyAsync(double lat,
double lng, double radiusKm)
{
    ValidateCoordinates(lat, lng);
    if (radiusKm <= 0)
    {
        throw new ValidationException("radiusKm must be greater than 0.");
    }

    var geoNearStage = new BsonDocument("$geoNear", new BsonDocument
    {
        { "near", new BsonDocument { { "type", "Point" }, { "coordinates",
new BsonArray { lng, lat } } } },
        { "distanceField", "distanceMeters" },
        { "maxDistance", radiusKm * 1000 },
        { "spherical", true },
        { "query", new BsonDocument("status",
StationStatus.Active.ToString()) }
    });
    // Joins EnergyBookingSlots to count each station's Available slots in
the same pipeline,
    // so the mobile map's info window (name/capacity/available slots)
doesn't need a second
    // round trip per marker tap. Added 2026-09-24 alongside capacityKWh
below - both are
    // backward-compatible additions beyond architecture.md §3's original
    // { id, name, location, distanceKm } shape, documented there same as the
earlier

```

```

// totalBatterySlots addition to GET /stations.
var lookupStage = new BsonDocument("$lookup", new BsonDocument
{
    { "from", "EnergyBookingSlots" },
    { "localField", "_id" },
    { "foreignField", "stationId" },
    { "as", "slots" }
});
var addFieldsStage = new BsonDocument("$addFields", new BsonDocument
{
    {
        "availableSlots",
        new BsonDocument("$size", new BsonDocument("$filter", new
BsonDocument
        {
            { "input", "$slots" },
            { "as", "slot" },
            { "cond", new BsonDocument("$eq", new BsonArray {
"$slot.status", "Available" }) }
        }
    })
    }
});
var projectStage = new BsonDocument("$project", new BsonDocument
{
    { "_id", 1 },
    { "name", 1 },
    { "location", 1 },
    { "distanceMeters", 1 },
    { "capacityKWh", 1 },
    { "availableSlots", 1 }
});

var pipeline = PipelineDefinition<Station,
NearbyAggregationResult>.Create(
    new[] { geoNearStage, lookupStage, addFieldsStage, projectStage });
var results = await
_context.SolarStations.Aggregate(pipeline).ToListAsync();

return results.Select(r => new NearbyStationResponse
{
    Id = r.Id,
    Name = r.Name,
    Location = MapLocation(r.Location),
    CapacityKWh = r.CapacityKWh,
    AvailableSlots = r.AvailableSlots,
    DistanceKm = Math.Round(r.DistanceMeters / 1000, 3)

```

```

    }).ToList();
}

// Checks EnergyBookingSlots (Shalon's own collection) for any Reserved slot
// at this station.
private async Task<bool> HasReservedSlotsAsync(string stationId)
{
    return await _context.EnergyBookingSlots
        .Find(sl => sl.StationId == stationId && sl.Status ==
SlotStatus.Reserved)
        .AnyAsync();
}

// Checks Dinil's Reservations collection (write-side owner: Dinil; Shalon
// reads it - see §4)
// for any Confirmed reservation at this station.
private async Task<bool> HasConfirmedReservationsAsync(string stationId)
{
    return await _context.Reservations
        .Find(r => r.StationId == stationId && r.Status == "Confirmed")
        .AnyAsync();
}

// Maps a stored Station document to the GET /stations list-row shape.
private static StationSummaryResponse MapToSummary(Station station) => new()
{
    Id = station.Id!,
    Name = station.Name,
    Location = MapLocation(station.Location),
    CapacityKWh = station.CapacityKWh,
    TotalBatterySlots = station.TotalBatterySlots,
    Status = station.Status.ToString()
};

// Maps a stored Slot document to its client-facing response shape.
private static SlotResponse MapSlot(Slot slot) => new()
{
    Id = slot.Id!,
    StationId = slot.StationId,
    SlotNumber = slot.SlotNumber,
    Type = slot.Type.ToString(),
    CapacityKWh = slot.CapacityKWh,
    Status = slot.Status.ToString(),
    UpdatedAt = slot.UpdatedAt
};

```

```

// Converts the stored GeoJSON point back into the flat {type, coordinates}
shape clients receive.
private static LocationResponse
MapLocation(GeoJsonPoint<GeoJson2DGeographicCoordinates> location) => new()
{
    Type = "Point",
    Coordinates = new[] { location.Coordinates.Longitude,
location.Coordinates.Latitude }
};

// Ensures the station name is present and not just whitespace.
private static void ValidateName(string name)
{
    if (string.IsNullOrEmpty(name))
    {
        throw new ValidationException("Station name is required.");
    }
}

// Ensures latitude/longitude are within valid GPS bounds (numeric-ness is
already enforced by the double type).
private static void ValidateCoordinates(double lat, double lng)
{
    if (lat < -90 || lat > 90)
    {
        throw new ValidationException("Latitude must be a number between -90
and 90.");
    }
    if (lng < -180 || lng > 180)
    {
        throw new ValidationException("Longitude must be a number between -
180 and 180.");
    }
}

// Ensures capacity is a positive number.
private static void ValidateCapacity(double capacityKWh)
{
    if (capacityKWh <= 0)
    {
        throw new ValidationException("capacityKWh must be greater than 0.");
    }
}

// Ensures the declared slot count is a positive number.
private static void ValidateTotalBatterySlots(int totalBatterySlots)

```

```

{
    if (totalBatterySlots <= 0)
    {
        throw new ValidationException("totalBatterySlots must be greater than
0.");
    }
}

// Ensures both operating-hours fields are present.
private static void ValidateOperatingSchedule(OperatingScheduleDto schedule)
{
    if (string.IsNullOrEmpty(schedule.OpensAt) ||
string.IsNullOrEmpty(schedule.ClosesAt))
    {
        throw new ValidationException("operatingSchedule.opensAt and closesAt
are required.");
    }
}

// Shape the $geoNear + $project pipeline in GetNearbyAsync projects into -
kept to exactly
// these fields so deserialization doesn't collide with Station's own class
map (which has no
// "distanceMeters" field and would otherwise reject it as an unmapped
element).
private class NearbyAggregationResult
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; } = string.Empty;

    [BsonElement("name")]
    public string Name { get; set; } = string.Empty;

    [BsonElement("location")]
    public GeoJsonPoint<GeoJson2DGeographicCoordinates> Location { get; set;
} = null!;

    [BsonElement("distanceMeters")]
    public double DistanceMeters { get; set; }

    [BsonElement("capacityKWh")]
    public double CapacityKWh { get; set; }

    [BsonElement("availableSlots")]
    public int AvailableSlots { get; set; }
}

```

```
}
```

```
}
```

DashboardService.cs

```
using MongoDB.Driver; using SmartMicrogrid.Api.Data; using SmartMicrogrid.Api.Models;
```

```
namespace SmartMicrogrid.Api.Services;
```

```
public class DashboardService { private const int RecentHistoryLimit = 10;
```

```
private readonly MongoDBContext _context;
```

```
// Injects the shared Mongo context this service reads Reservations from.
```

```
public DashboardService(MongoDbContext context)
```

```
{
```

```
    _context = context;
```

```
}
```

```
// Builds a prosumer's dashboard summary: how many reservations are active,  
// how many of those
```

```
// are past their scheduled time and awaiting operator check-in, how many are  
// still upcoming,
```

```
// and their most recent bookings.
```

```
//
```

```
// Field semantics (architecture.md §3 only ever named these fields - nobody  
// had implemented
```

```
// this endpoint before, so these definitions are set here for the first  
// time):
```

```
// - activeCount: every reservation with status == Confirmed (booked, not  
// yet completed or
```

```
// cancelled).
```

```
// - pendingCount: Confirmed AND scheduledAt <= now - i.e. its slot time  
// has arrived/passed
```

```
// but a Grid Operator hasn't scanned/completed it yet.
```

```
// - approvedFutureCount: Confirmed AND scheduledAt > now - upcoming  
// approved bookings, per
```

```
// the explicit "count of approved (Confirmed) future reservations"  
// requirement.
```

```
// pendingCount + approvedFutureCount always equals activeCount, since  
// they're a time-based
```

```
// partition of the same Confirmed set.
```

```
public async Task<ProsumerDashboardSummaryResponse>
```

```
GetProsumerSummaryAsync(string nic)
```

```
{
```

```

ValidateNic(nic);

var now = DateTime.UtcNow;
var filterBuilder = Builders<Reservation>.Filter;
var byNic = filterBuilder.Eq(r => r.ProsumerNic, nic);
var confirmed = byNic & filterBuilder.Eq(r => r.Status, "Confirmed");

var activeCount = (int)await
_context.Reservations.CountDocumentsAsync(confirmed);
var pendingCount = (int)await _context.Reservations
    .CountDocumentsAsync(confirmed & filterBuilder.Lte(r =>
r.ScheduledAt, now));
var approvedFutureCount = (int)await _context.Reservations
    .CountDocumentsAsync(confirmed & filterBuilder.Gt(r => r.ScheduledAt,
now));

var recentHistory = await _context.Reservations
    .Find(byNic)
    .SortByDescending(r => r.ScheduledAt)
    .Limit(RecentHistoryLimit)
    .ToListAsync();

return new ProsumerDashboardSummaryResponse
{
    ActiveCount = activeCount,
    PendingCount = pendingCount,
    ApprovedFutureCount = approvedFutureCount,
    RecentHistory = recentHistory.Select(MapToHistoryItem).ToList()
};
}

```

```

// =====
// Dinil addition to DashboardService – operator summary
// aggregation. Reads the Reservations collection (architecture.md
// 2.4, owner: Dinil) for confirmed/completed-today counts and
// pending-by-station breakdown. Called by DashboardController's
// GET /api/dashboard/operator/summary.
// =====

// Returns today's confirmed/completed reservation counts plus a
// per-station breakdown of pending (Confirmed but not yet scanned)
// reservations. All times computed in UTC; the client renders them.
// =====
// OPERATOR DASHBOARD SUMMARY
// =====

```



```

public async Task<OperatorSummaryResponse> GetOperatorSummaryAsync()
{
    var now = DateTime.UtcNow;

    var startOfTodayUtc = now.Date;
    var startOfTomorrowUtc = startOfTodayUtc.AddDays(1);

    var reservationFilter = Builders<Reservation>.Filter;

    // -----
    // Confirmed reservations created today
    // -----

    var confirmedTodayFilter =
        reservationFilter.Gte(
            r => r.CreatedAt,
            startOfTodayUtc
        )
        &
        reservationFilter.Lt(
            r => r.CreatedAt,
            startOfTomorrowUtc
        );

    var confirmedTodayCount =
        (int)await _context.Reservations
            .CountDocumentsAsync(confirmedTodayFilter);

    // -----
    // Completed reservations today
    // -----

    var completedTodayFilter =
        reservationFilter.Eq(
            r => r.Status,
            "Completed"
        )
        &
        reservationFilter.Gte(
            r => r.CompletedAt,
            startOfTodayUtc
        )
        &
        reservationFilter.Lt(
            r => r.CompletedAt,
            startOfTomorrowUtc
        );
}

```

```

    );

    var completedTodayCount =
        (int)await _context.Reservations
            .CountDocumentsAsync(completedTodayFilter);

    // -----
    // Pending reservations by station
    // -----
    //
    // Pending = Confirmed reservations that haven't been
    // completed/scanned yet.
    //

    var pendingFilter =
        reservationFilter.Eq(
            r => r.Status,
            "Confirmed"
        );

    var pendingReservations =
        await _context.Reservations
            .Find(pendingFilter)
            .ToListAsync();

    var groupedByStation = pendingReservations
        .GroupBy(r => r.StationId)
        .Select(g => new
        {
            StationId = g.Key,
            Count = g.Count()
        })
        .ToList();

    // -----
    // Get station names
    // -----

    var stationIds = groupedByStation
        .Select(g => g.StationId)
        .Where(id => !string.IsNullOrEmpty(id))
        .Distinct()
        .ToList();

    var stationLookup = new Dictionary<string, string>();

```

```

if (stationIds.Count > 0)
{
    // IMPORTANT:
    // Change SolarStations to the actual collection property
    // in MongoDBContext if your project uses another name.

    var stationFilter =
        Builders<Station>.Filter.In(
            s => s.Id,
            stationIds
        );

    var stations =
        await _context.SolarStations
            .Find(stationFilter)
            .ToListAsync();

    foreach (var station in stations)
    {
        if (!string.IsNullOrEmpty(station.Id))
        {
            stationLookup[station.Id] = station.Name;
        }
    }
}

// -----
// Build response
// -----

var pendingByStation = groupedByStation
    .Select(g => new PendingByStation
    {
        StationId = g.StationId,
        StationName =
            stationLookup.TryGetValue(
                g.StationId,
                out var stationName
            )
            ? stationName
            : "",

        PendingCount = g.Count
    })
    .ToList();

```

```

    return new OperatorSummaryResponse
    {
        ConfirmedTodayCount = confirmedTodayCount,
        CompletedTodayCount = completedTodayCount,
        PendingByStation = pendingByStation
    };
}

// Ensures a NIC was actually supplied – this endpoint reads another person's
reservations by
// NIC, so an empty value would otherwise silently match nothing rather than
fail loudly.
private static void ValidateNic(string nic)
{
    if (string.IsNullOrEmpty(nic))
    {
        throw new ValidationException("nic is required.");
    }
}

// Maps a stored Reservation to the compact shape used in a dashboard's
recent-history list.
private static ReservationHistoryItemResponse MapToHistoryItem(Reservation
reservation) => new()
{
    Id = reservation.Id!,
    StationId = reservation.StationId,
    SlotId = reservation.SlotId,
    ScheduledAt = reservation.ScheduledAt,
    Status = reservation.Status
};

}

```

StationsPage.jsx

```

import { useCallback, useEffect, useMemo, useState } from 'react'; import { Link } from 'react-router-
dom'; import api from '../services/api.js'; import { Table, Th, Td } from
'../components/common/Table.jsx'; import Button from '../components/common/Button.jsx';
import Input from '../components/common/Input.jsx'; import Badge from
'../components/common/Badge.jsx'; import Toast from '../components/common/Toast.jsx'; import
ConfirmDialog from '../components/common/ConfirmDialog.jsx'; import StatCard from
'../components/common/StatCard.jsx'; import Icon from '../components/common/Icon.jsx'; import
StationForm from './StationForm.jsx';

```

```

// Formats a GeoJSON [lng, lat] coordinate pair as a fixed-width "lat, lng" string. function
formatCoordinates(location) { const [lng, lat] = location?.coordinates ?? []; if (lat === undefined || lng
=== undefined) { return '-'; } return `${lat.toFixed(4)}, ${lng.toFixed(4)}; }

export default function StationsPage() { const [stations, setStations] = useState([]); const [loading,
setLoading] = useState(true); const [loadError, setLoadError] = useState(""); const [search, setSearch] =
useState(""); const [statusFilter, setStatusFilter] = useState('All'); const [viewModel, setViewModel] =
useState('table'); // 'table' | 'grid'

const [isFormOpen, setIsFormOpen] = useState(false); const [editingStation, setEditingStation] =
useState(null); const [submitting, setSubmitting] = useState(false); const [formError, setFormError] =
useState(""); const [formKey, setFormKey] = useState(0);

const [deactivatingId, setDeactivatingId] = useState(null); const [confirmingStation,
setConfirmingStation] = useState(null); const [activatingId, setActivatingId] = useState(null); const [toast,
setToast] = useState({ message: '', tone: 'error' });

const [currentPage, setCurrentPage] = useState(1); const [pageSize, setPageSize] = useState(6);

// Quick-glance summary row, derived from the already-loaded list const summary = useMemo( () => ({
total: stations.length, active: stations.filter((s) => s.status === 'Active').length, inactive: stations.filter((s)
=> s.status !== 'Active').length, capacityKWh: stations.reduce((sum, s) => sum + (s.capacityKWh ?? 0), 0),
slots: stations.reduce((sum, s) => sum + (s.batterySlots?.length ?? s.totalBatterySlots ?? 0), 0), }),
[stations], );

const maxCapacity = useMemo( () => Math.max(1, ...(stations.map((s) => s.capacityKWh ?? 0))),
[stations], );

// Filtered station list based on status tab const filteredStations = useMemo(() => { if (statusFilter ===
'All') return stations; return stations.filter((s) => s.status === statusFilter); }, [stations, statusFilter]);

// Total pages and paginated slice of stations const totalPages = useMemo( () => Math.max(1,
Math.ceil(filteredStations.length / pageSize)), [filteredStations, pageSize], );

const paginatedStations = useMemo(() => { const start = (currentPage - 1) * pageSize; return
filteredStations.slice(start, start + pageSize); }, [filteredStations, currentPage, pageSize]);

// Reset to page 1 whenever filter, search or pageSize changes useEffect(() => { setCurrentPage(1); },
[search, statusFilter, pageSize]);

// Loads the station list from the real API const loadStations = useCallback(async (term) => {
setLoading(true); setLoadError(""); try { const { data } = await api.get('/stations', { params: term ? {
search: term } : undefined }); setStations(data); } catch (err) { setLoadError(err.response?.data?.message
|| 'Failed to load stations. '); } finally { setLoading(false); }, [], );

```

```

// Re-fetches the list 300ms after the user stops typing useEffect(() => { const timeoutId = setTimeout(()
=> loadStations(search), 300); return () => clearTimeout(timeoutId); }, [search, loadStations]);

// Opens the modal in "create" mode const handleOpenCreate = () => { setEditingStation(null);
setFormError(""); setFormKey((key) => key + 1); setIsFormOpen(true); };

// Opens the modal in "edit" mode const handleOpenEdit = async (stationId) => { try { const { data } =
await api.get(/stations/${stationId}); setEditingStation(data); setFormError(""); setFormKey((key)
=> key + 1); setIsFormOpen(true); } catch (err) { setLoadError(err.response?.data?.message || 'Failed to
load station details.');
```

```

    } };

// Submits create/edit form const handleSubmit = async (values) => { setSubmitting(true);
setFormError(""); try { if (editingStation) { await api.put(/stations/${editingStation.id}, values);
setToast({ message: 'Station updated successfully.', tone: 'success' }); } else { await api.post(/stations',
values); setToast({ message: 'New station created successfully.', tone: 'success' }); }
setIsFormOpen(false); await loadStations(search); } catch (err) {
setFormError(err.response?.data?.message || 'Something went wrong. Please try again.');
```

```

    } finally {
setSubmitting(false); } };

const handleRequestDeactivate = (station) => { setConfirmingStation(station); };

const handleConfirmDeactivate = async () => { const station = confirmingStation; if (!station) return;
setDeactivatingId(station.id); try { await api.put(/stations/${station.id}/deactivate);
setToast({ message: 'Station deactivated.', tone: 'success' }); setConfirmingStation(null); await
loadStations(search); } catch (err) { setToast({ message: err.response?.data?.message || 'Failed to
deactivate station.', tone: 'error' }); setConfirmingStation(null); } finally { setDeactivatingId(null); } };

const handleActivate = async (station) => { setActivatingId(station.id); try { await
api.put(/stations/${station.id}/activate); setToast({ message: 'Station reactivated.', tone:
'success' }); await loadStations(search); } catch (err) { setToast({ message: err.response?.data?.message
|| 'Failed to reactivate station.', tone: 'error' }); } finally { setActivatingId(null); } };

return (
  

```

 /* — Page Header ————— */
 <div className="flex flex-col sm:flex-row sm:items-center sm:justify-
between gap-4 pb-2 border-b border-line/60">
 <div>
 <div className="flex items-center gap-2">
 <h1 className="text-2xl font-bold tracking-tight text-ink">Microgrid
Hubs</h1>
 <span className="inline-flex items-center rounded-full bg-primary/10
px-2.5 py-0.5 text-xs font-semibold text-primary">
 {summary.total} nodes

 </div>
 </div>

```


```

```

        <p className="mt-1 text-[13px] text-muted">
            Solar microgrid stations, capacity specifications, and battery slot
            provisioning.
        </p>
    </div>
    <Button variant="primary" onClick={handleOpenCreate} className="shadow-
card">
        <span className="flex items-center gap-1.5">
            <span>+</span>
            <span>New Station</span>
        </span>
    </Button>
</div>

{/* — Summary Stat Bar ————— */}
<div className="grid grid-cols-2 gap-3.5 sm:grid-cols-4">
    <StatCard
        icon="hubs"
        label="Total Hubs"
        value={loading ? '...' : summary.total}
        hint="Registered grid nodes"
    />
    <StatCard
        icon="pulse"
        label="Active Nodes"
        value={loading ? '...' : summary.active}
        hint={` ${summary.inactive} currently inactive`}
    />
    <StatCard
        icon="battery"
        label="Total Capacity"
        value={loading ? '...' : `${summary.capacityKWh.toLocaleString()} kWh`}
        hint="Across all solar stations"
    />
    <StatCard
        icon="bolt"
        label="Battery Slots"
        value={loading ? '...' : summary.slots}
        hint="Total declared slots"
    />
</div>

{/* — Controls Toolbar: Search, Filter Tabs & View Toggle — */}
<div className="flex flex-col sm:flex-row sm:items-center sm:justify-
between gap-3 bg-surface p-2.5 rounded-xl border border-line shadow-card">
    {/* Search */}

```

```

<div className="relative w-full sm:max-w-xs">
  <Input
    placeholder="Search by station name..."
    aria-label="Search stations"
    value={search}
    onChange={(event) => setSearch(event.target.value)}
    className="w-full pl-3 pr-8"
  />
  {search && (
    <button
      type="button"
      onClick={() => setSearch('')}
      className="absolute right-2.5 top-1/2 -translate-y-1/2 text-muted
hover:text-ink text-xs"
    >
      ×
    </button>
  )}
</div>

{/* Status Filter Tabs & Layout Switcher */}
<div className="flex items-center justify-between sm:justify-end gap-3 w-
full sm:w-auto">
  {/* Status Tabs */}
  <div className="inline-flex rounded-lg bg-surface-alt p-1 border
border-line text-xs font-medium">
    {[ 'All', 'Active', 'Inactive' ].map((tab) => (
      <button
        key={tab}
        type="button"
        onClick={() => setStatusFilter(tab)}
        className={`px-3 py-1 rounded-md transition-all ${
          statusFilter === tab
            ? 'bg-surface text-ink font-semibold shadow-xs'
            : 'text-muted hover:text-body'
        }`}
      >
        {tab}
        <span className="ml-1.5 text-[10px] opacity-70">
          {tab === 'All' ? summary.total : tab === 'Active' ?
summary.active : summary.inactive}
        </span>
      </button>
    )})
  </div>

```



```

    { /* View Toggle */ }
    <div className="inline-flex rounded-lg bg-surface-alt p-1 border
border-line text-xs">
      <button
        type="button"
        onClick={() => setViewMode('table')}
        title="Table view"
        className={`p-1.5 rounded-md transition-all ${
          viewMode === 'table' ? 'bg-surface text-primary shadow-xs' :
'text-muted hover:text-body'
        }`>
        >
          <svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M4 6h16M4 12h16M4 18h16" />
          </svg>
        </button>
        <button
          type="button"
          onClick={() => setViewMode('grid')}
          title="Grid card view"
          className={`p-1.5 rounded-md transition-all ${
            viewMode === 'grid' ? 'bg-surface text-primary shadow-xs' :
'text-muted hover:text-body'
          }`>
          >
            <svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
              <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M4 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0 01-
2-2V6zM14 6a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V6zM
16a2 2 0 012-2h2a2 2 0 012 2v2a2 2 0 01-2 2H6a2 2 0 01-2-2v-2zM14 16a2 2 0
012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2v-2z" />
            </svg>
          </button>
        </div>
      </div>
    </div>

    {loadError && (
      <div className="rounded-xl border border-error/30 bg-error-soft px-4 py-3
text-[13px] font-medium text-error flex items-center justify-between">
        <span>{loadError}</span>
        <button type="button" onClick={() => loadStations(search)}
className="underline hover:no-underline">

```

```

        Retry
      </button>
    </div>
  )}

  {/* — Main View: Table or Grid ————— */}
  {viewModel === 'table' ? (
    <div className="rounded-xl border border-line bg-surface shadow-card
overflow-hidden">
      <Table>
        <thead>
          <tr className="bg-surface-alt/70 border-b border-line text-[11px]
font-semibold uppercase tracking-wider text-muted">
            <Th>Hub Station</Th>
            <Th>GPS Coordinates</Th>
            <Th className="min-w-[170px]">Capacity Spec</Th>
            <Th className="text-right">Slots</Th>
            <Th>Status</Th>
            <Th className="text-right">Actions</Th>
          </tr>
        </thead>
        <tbody className="divide-y divide-line/80">
          {loading && (
            <tr>
              <Td colSpan={6} className="py-12 text-center text-[13px] text-
muted">
                Loading stations...
              </Td>
            </tr>
          )}
          {!loading && filteredStations.length === 0 && (
            <tr>
              <Td colSpan={6} className="py-12 text-center text-[13px] text-
muted">
                {search ? `No stations matching "${search}"` : 'No stations
registered.'}
              </Td>
            </tr>
          )}
          {!loading &&
            paginatedStations.map((station) => {
              const percent = Math.min(100, Math.round(((station.capacityKWh
?? 0) / maxCapacity) * 100));
              return (
                <tr key={station.id} className="transition-colors hover:bg-
surface-alt/50">

```

```

<Td className="font-medium py-3.5">
  <Link
    to={` /stations/${station.id}`}
    className="font-semibold text-ink hover:text-primary
transition-colors block"
  >
    {station.name}
  </Link>
  <span className="text-[11px] text-muted font-mono">ID:
{station.id?.slice(-8)}</span>
</Td>
<Td className="font-mono text-[12px] text-muted py-3.5">
  <span className="inline-flex items-center gap-1.5 rounded
bg-surface-alt px-2 py-0.5 border border-line text-[11px]">
    <Icon name="mapPin" className="h-3 w-3 text-muted" />
    <span>{formatCoordinates(station.location)}</span>
  </span>
</Td>
<Td className="py-3.5">
  <div className="flex items-center justify-between text-xs
mb-1">
    <span className="font-semibold text-ink
tnum">{station.capacityKWh} kWh</span>
    <span className="text-muted text-
[10px]">{percent}%</span>
  </div>
  <div className="h-1.5 w-full rounded-full bg-surface-alt
overflow-hidden">
    <div className="h-full rounded-full bg-emerald-500"
style={{ width: `${percent}%` }} />
  </div>
</Td>
<Td className="tnum text-right py-3.5">
  <span className="inline-flex items-center justify-center
rounded-md bg-surface-alt px-2.5 py-0.5 text-xs text-body font-semibold">
    {station.batterySlots?.length ??
station.totalBatterySlots ?? '-'}
  </span>
</Td>
<Td className="py-3.5">
  <span className={`inline-flex items-center rounded-md px-
2.5 py-0.5 text-[11px] font-medium border ${
station.status === 'Active'
? 'bg-emerald-50 text-emerald-700 border-emerald-
200/80'
: 'bg-slate-100 text-slate-600 border-slate-200'

```

```

    }`}>
      {station.status}
    </span>
  </Td>
  <Td className="text-right py-3.5">
    <div className="flex justify-end items-center gap-1.5">
      <Link
        to={` /stations/${station.id}`}
        className="inline-flex items-center rounded-md border
border-line bg-surface px-2.5 py-1 text-xs font-medium text-ink hover:bg-
surface-alt hover:text-primary transition-colors"
      >
        Details
      </Link>
      <Button variant="secondary" size="sm" onClick={() =>
handleOpenEdit(station.id)}>
        Edit
      </Button>
      {station.status === 'Active' ? (
        <Button
          variant="danger-outline"
          size="sm"
          disabled={deactivatingId === station.id}
          onClick={() => handleRequestDeactivate(station)}
        >
          {deactivatingId === station.id ? '...' :
'Deactivate'}
        </Button>
      ) : (
        <Button
          variant="secondary"
          size="sm"
          disabled={activatingId === station.id}
          onClick={() => handleActivate(station)}
        >
          {activatingId === station.id ? '...' : 'Activate'}
        </Button>
      )}
    </div>
  </Td>
</tr>
);
}}
</tbody>
</Table>
</div>

```

```

) : (
  /* — Grid Card View — */
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
    {loading && (
      <div className="col-span-full py-16 text-center text-muted text-sm">
        Loading station cards...
      </div>
    )}
    {!loading && filteredStations.length === 0 && (
      <div className="col-span-full py-16 text-center text-muted text-sm">
        {search ? `No stations matching "${search}"` : 'No stations
registered.'}
      </div>
    )}
    {!loading &&
      paginatedStations.map((station) => {
        const percent = Math.min(100, Math.round(((station.capacityKWh ??
0) / maxCapacity) * 100));
        const slotsCount = station.batterySlots?.length ??
station.totalBatterySlots ?? 0;
        return (
          <div
            key={station.id}
            className="rounded-xl border border-line bg-surface p-5 shadow-
card hover:shadow-panel transition-all flex flex-col justify-between"
          >
            <div>
              <div className="flex items-start justify-between gap-2">
                <div>
                  <Link
                    to={`/${stations}/${station.id}`}
                    className="font-bold text-ink hover:text-primary
transition-colors text-base"
                  >
                    {station.name}
                  </Link>
                  <p className="text-[11px] text-muted font-mono mt-
0.5">ID: {station.id?.slice(-8)}</p>
                </div>
                <span className={`inline-flex items-center rounded-md px-
2.5 py-0.5 text-[11px] font-medium border ${
station.status === 'Active'
? 'bg-emerald-50 text-emerald-700 border-emerald-
200/80'
: 'bg-slate-100 text-slate-600 border-slate-200'
} `>
              </span>
            </div>
          </div>
        )
      )
    }
  )
}

```

```

        {station.status}
      </span>
    </div>

    <div className="mt-4 pt-3 border-t border-line/60 space-y-
2.5">
      <div className="flex items-center justify-between text-xs">
        <span className="text-muted">Coordinates</span>
        <span className="font-mono text-body font-
medium">{formatCoordinates(station.location)}</span>
      </div>
      <div>
        <div className="flex items-center justify-between text-xs
mb-1">
          <span className="text-muted">Capacity Spec</span>
          <span className="font-bold text-ink
tnum">{station.capacityKWh} kWh</span>
        </div>
        <div className="h-1.5 w-full rounded-full bg-surface-alt
overflow-hidden">
          <div className="h-full rounded-full bg-emerald-500"
style={{ width: `${percent}%` }} />
        </div>
      </div>
      <div className="flex items-center justify-between text-xs
pt-1">
        <span className="text-muted">Battery Slots</span>
        <span className="inline-flex items-center rounded-md bg-
surface-alt px-2 py-0.5 font-bold text-ink">
          {slotsCount} slots
        </span>
      </div>
    </div>

    <div className="mt-5 pt-3 border-t border-line/60 flex items-
center justify-between gap-2">
      <Link
        to={` /stations/${station.id}`}
        className="inline-flex items-center gap-1 text-xs font-
semibold text-primary hover:underline"
      >
        Manage Node →
      </Link>
      <div className="flex items-center gap-1.5">
        <Button variant="secondary" size="sm" onClick={() =>

```

```

handleOpenEdit(station.id))>
    Edit
</Button>
{station.status === 'Active' ? (
    <Button
        variant="danger-outline"
        size="sm"
        disabled={deactivatingId === station.id}
        onClick={() => handleRequestDeactivate(station)}
    >
        {deactivatingId === station.id ? '...' : 'Deactivate'}
    </Button>
) : (
    <Button
        variant="secondary"
        size="sm"
        disabled={activatingId === station.id}
        onClick={() => handleActivate(station)}
    >
        {activatingId === station.id ? '...' : 'Activate'}
    </Button>
)}
</div>
</div>
</div>
);
}}
</div>
)}

{/* — Pagination Controls ————— */}
{!loading && filteredStations.length > 0 && (
    <div className="flex flex-col sm:flex-row items-center justify-between
gap-4 pt-2 border-t border-line/60">
        <div className="flex items-center gap-3 text-xs text-muted">
            <span>
                Showing <span className="font-semibold text-ink">{(currentPage - 1)
* pageSize + 1}</span> to{' '}
                <span className="font-semibold text-ink">
                    {Math.min(currentPage * pageSize, filteredStations.length)}
                </span>{' '}
                of <span className="font-semibold text-
ink">{filteredStations.length}</span> hubs
            </span>
            <span className="text-line">|</span>
            <div className="flex items-center gap-1.5">

```

```

        <span>Per page:</span>
        <select
          aria-label="Stations per page"
          value={pageSize}
          onChange={(e) => setPageSize(Number(e.target.value))}
          className="rounded-md border border-line bg-surface px-2 py-1
text-xs font-medium text-ink focus:outline-none focus:ring-1 focus:ring-
primary"
        >
          <option value={6}>6</option>
          <option value={12}>12</option>
          <option value={24}>24</option>
        </select>
      </div>
    </div>

    <div className="flex items-center gap-1.5">
      <Button
        variant="secondary"
        size="sm"
        onClick={() => setCurrentPage((p) => Math.max(1, p - 1))}
        disabled={currentPage === 1}
        className="px-2.5"
      >
        ← Prev
      </Button>
      <div className="flex items-center gap-1">
        {Array.from({ length: totalPages }, (_, i) => i + 1).map((pageNum)
=> (
          <button
            key={pageNum}
            type="button"
            onClick={() => setCurrentPage(pageNum)}
            className={`min-w-[32px] h-8 rounded-md text-xs font-semibold
transition-all ${
              currentPage === pageNum
                ? 'bg-primary text-white shadow-xs'
                : 'border border-line bg-surface text-ink hover:bg-surface-
alt'
            }`}
          >
            {pageNum}
          </button>
        ))}
      </div>
      <Button

```



```

        variant="secondary"
        size="sm"
        onClick={() => setCurrentPage((p) => Math.min(totalPages, p + 1))}
        disabled={currentPage === totalPages}
        className="px-2.5"
      >
        Next →
      </Button>
    </div>
  </div>
)}

{/* — Modals & Dialogs ————— */}
<StationForm
  key={formKey}
  open={isFormOpen}
  station={editingStation}
  submitting={submitting}
  serverError={formError}
  onClose={() => setIsFormOpen(false)}
  onSubmit={handleSubmit}
/>

<ConfirmDialog
  open={Boolean(confirmingStation)}
  title="Deactivate station node?"
  description={
    confirmingStation
      ? ` "${confirmingStation.name}" will be marked inactive and taken
offline for new reservations. Historical bookings will remain intact.`
      : ''
  }
  confirmLabel="Deactivate Node"
  tone="danger"
  confirming={deactivatingId === confirmingStation?.id}
  onConfirm={handleConfirmDeactivate}
  onClose={() => setConfirmingStation(null)}
/>

<Toast message={toast.message} tone={toast.tone} onDismiss={() =>
setToast({ message: '', tone: 'error' })} />
</div>

);}

```

StationDetailPage.jsx

```
import { useEffect, useState } from 'react'; import { Link, useParams } from 'react-router-dom'; import
api from '../services/api.js'; import { Table, Th, Td } from '../components/common/Table.jsx'; import
Button from '../components/common/Button.jsx'; import Input from
'../components/common/Input.jsx'; import Badge from '../components/common/Badge.jsx'; import
Toast from '../components/common/Toast.jsx'; import ConfirmDialog from
'../components/common/ConfirmDialog.jsx'; import Icon from '../components/common/Icon.jsx';
import SlotForm from './SlotForm.jsx';
```

```
// Formats a GeoJSON [lng, lat] coordinate pair as a readable "lat, lng" string. function
formatCoordinates(location) { const [lng, lat] = location?.coordinates ?? []; if (lat === undefined || lng
=== undefined) { return '-'; } return `${lat.toFixed(4)}, ${lng.toFixed(4)}; }
```

```
// Editable opening/closing hours function ScheduleEditor({ station, submitting, onSave }) { const
[opensAt, setOpensAt] = useState(station.operatingSchedule?.opensAt || '08:00'); const [closesAt,
setClosesAt] = useState(station.operatingSchedule?.closesAt || '20:00');
```

```
const handleSubmit = (event) => { event.preventDefault(); onSave({ opensAt, closesAt }); };
```

```
return (
```

```
<Input label="Opens at" type="time" required value={opensAt} onChange={(event) =>
setOpensAt(event.target.value)} />
<Input label="Closes at" type="time" required value={closesAt} onChange={(event) =>
setClosesAt(event.target.value)} />
{submitting ? 'Saving...' : 'Update Schedule'} ); }
```

```
export default function StationDetailPage() { const { id } = useParams();
```

```
const [station, setStation] = useState(null); const [loading, setLoading] = useState(true); const [loadError,
setLoadError] = useState(""); const [scheduleSubmitting, setScheduleSubmitting] = useState(false); const
[deactivating, setDeactivating] = useState(false); const [confirmDeactivateOpen,
setConfirmDeactivateOpen] = useState(false); const [activating, setActivating] = useState(false);
```

```
const [reservations, setReservations] = useState([]); const [reservationsLoading, setReservationsLoading]
= useState(true); const [reservationsError, setReservationsError] = useState("");
```

```
const [isSlotFormOpen, setIsSlotFormOpen] = useState(false); const [editingSlot, setEditingSlot] =
useState(null); const [slotFormKey, setSlotFormKey] = useState(0); const [slotSubmitting,
setSlotSubmitting] = useState(false); const [slotFormError, setSlotFormError] = useState(""); const
[slotStatusUpdatingId, setSlotStatusUpdatingId] = useState(null);
```

```
const [toast, setToast] = useState({ message: "", tone: 'error' }); const [refreshToken, setRefreshToken] =
useState(0);
```

```
// Load station details useEffect(() => { let cancelled = false;

async function loadStation() {
  setLoading(true);
  setLoadError('');
  try {
    const { data } = await api.get(`/stations/${id}`);
    if (!cancelled) {
      setStation(data);
    }
  } catch (err) {
    if (!cancelled) {
      setLoadError(err.response?.data?.message || 'Failed to load station.');
```

```
    } finally {
      if (!cancelled) {
        setLoading(false);
      }
    }
  }
}

loadStation();
return () => {
  cancelled = true;
};
```

```
}, [id, refreshToken]);

// Load reservations for station useEffect(() => { let cancelled = false;
```

```
async function loadReservations() {
  setReservationsLoading(true);
  setReservationsError('');
  try {
    const { data } = await api.get(`/stations/${id}/reservations-overview`);
    if (!cancelled) {
      setReservations(data ?? []);
    }
  } catch (err) {
    if (!cancelled) {
      setReservationsError(err.response?.data?.message || 'Failed to load
reservations overview.');
```

```

    }
  }
}

```

```

loadReservations();
return () => {
  cancelled = true;
};

```

```

}, [id]);

```

```

const handleSaveSchedule = async ({ opensAt, closesAt }) => { setScheduleSubmitting(true); try { await
api.put(/stations/${id}, { operatingSchedule: { opensAt, closesAt } }); setRefreshToken((token) =>
token + 1); setToast({ message: 'Operating schedule updated.', tone: 'success' }); } catch (err) { setToast({
message: err.response?.data?.message || 'Failed to update schedule.', tone: 'error' }); } finally {
setScheduleSubmitting(false); } };

```

```

const handleDeactivate = async () => { setDeactivating(true); try { await
api.put(/stations/${id}/deactivate); setRefreshToken((token) => token + 1); setToast({ message:
'Station deactivated.', tone: 'success' }); } catch (err) { setToast({ message: err.response?.data?.message
|| 'Failed to deactivate station.', tone: 'error' }); } finally { setDeactivating(false);
setConfirmDeactivateOpen(false); } };

```

```

const handleActivate = async () => { setActivating(true); try { await
api.put(/stations/${id}/activate); setRefreshToken((token) => token + 1); setToast({ message:
'Station reactivated.', tone: 'success' }); } catch (err) { setToast({ message: err.response?.data?.message
|| 'Failed to reactivate station.', tone: 'error' }); } finally { setActivating(false); } };

```

```

const handleOpenCreateSlot = () => { setEditingSlot(null); setSlotFormError(""); setSlotFormKey((key) =>
key + 1); setIsSlotFormOpen(true); };

```

```

const handleOpenEditSlot = (slot) => { setEditingSlot(slot); setSlotFormError(""); setSlotFormKey((key) =>
key + 1); setIsSlotFormOpen(true); };

```

```

const handleSubmitSlot = async (values) => { setSlotSubmitting(true); setSlotFormError(""); try { if
(editingSlot) { await api.put(/slots/${editingSlot.id}, values); } else { await
api.post(/stations/${id}/slots, values); } setIsSlotFormOpen(false); setRefreshToken((token) =>
token + 1); setToast({ message: editingSlot ? 'Slot updated.' : 'Slot created.', tone: 'success' }); } catch
(err) { setSlotFormError(err.response?.data?.message || 'Something went wrong. Please try again. '); }
finally { setSlotSubmitting(false); } };

```

```

const handleSlotStatusChange = async (slotId, status) => { setSlotStatusUpdatingId(slotId); try { await
api.put(/slots/${slotId}, { status }); setRefreshToken((token) => token + 1); setToast({ message:
'Slot status updated.', tone: 'success' }); } catch (err) { setToast({ message: err.response?.data?.message
|| 'Failed to update slot status.', tone: 'error' }); } finally { setSlotStatusUpdatingId(null); } };

```

```

const slotNumberFor = (slotId) => station?.slots?.find((slot) => slot.id === slotId)?.slotNumber ?? slotId;

if (loading) { return (

Loading station details...
); }

if (loadError || !station) { return (

← Back to Microgrid Hubs
{loadError || 'Station not found.'}
); }

return (

  { /* — Breadcrumb & Back Link ————— */ }
  <div>
    <Link
      to="/stations"
      className="inline-flex items-center gap-1.5 text-xs font-semibold text-
muted hover:text-primary transition-colors mb-3"
    >
      <span>←</span>
      <span>Back to Microgrid Hubs</span>
    </Link>
  </div>

  { /* — Station Hero Card ————— */ }
  <div className="rounded-xl border border-line bg-surface p-6 shadow-card">
    <div className="flex flex-col sm:flex-row sm:items-start sm:justify-
between gap-4">
      <div>
        <div className="flex items-center gap-2.5">
          <h1 className="text-2xl font-bold tracking-tight text-
ink">{station.name}</h1>
          <span className={`inline-flex items-center rounded-md px-2.5 py-0.5
text-xs font-medium border ${
            station.status === 'Active'
              ? 'bg-emerald-50 text-emerald-700 border-emerald-200/80'
              : 'bg-slate-100 text-slate-600 border-slate-200'
            }`}>
            {station.status}
          </span>
        </div>
        <p className="mt-1.5 font-mono text-[12px] text-muted flex items-
center gap-1.5">
          <span className="inline-flex items-center gap-1">

```

```

        <Icon name="mapPin" className="h-3.5 w-3.5 text-muted" />
        <span>GPS: {formatCoordinates(station.location)}</span>
    </span>
    <span>•</span>
    <span>ID: {station.id}</span>
</p>

<div className="mt-6 flex flex-wrap gap-8 pt-4 border-t border-
line/60">
    <div>
        <p className="text-[11px] font-semibold uppercase tracking-wider
text-muted">Total Capacity</p>
        <p className="tnum mt-1 text-2xl font-bold text-ink">
            {station.capacityKWh}
            <span className="ml-1 text-[13px] font-normal text-
muted">kWh</span>
        </p>
    </div>
    <div>
        <p className="text-[11px] font-semibold uppercase tracking-wider
text-muted">Declared Slots</p>
        <p className="tnum mt-1 text-2xl font-bold text-
ink">{station.totalBatterySlots ?? '-'}</p>
    </div>
    <div>
        <p className="text-[11px] font-semibold uppercase tracking-wider
text-muted">Active Slot Configs</p>
        <p className="tnum mt-1 text-2xl font-bold text-
ink">{station.slots?.length ?? 0}</p>
    </div>
</div>

<div className="flex items-center gap-2">
    {station.status === 'Active' ? (
        <Button
            variant="danger-outline"
            onClick={() => setConfirmDeactivateOpen(true)}
            disabled={deactivating}
            className="shadow-xs"
        >
            {deactivating ? 'Deactivating...' : 'Deactivate Station'}
        </Button>
    ) : (
        <Button
            variant="secondary"

```

```

        onClick={handleActivate}
        disabled={activating}
        className="shadow-xs"
      >
        {activating ? 'Reactivating...' : 'Reactivate Station'}
      </Button>
    )}
  </div>
</div>
</div>

{/* — Operating Schedule Section ————— */}
<section className="rounded-xl border border-line bg-surface p-6 shadow-
card space-y-4">
  <div>
    <h2 className="text-base font-semibold tracking-tight text-
ink">Operating Schedule</h2>
    <p className="text-[12px] text-muted mt-0.5">Opening and closing window
for prosumer battery reservation check-ins.</p>
  </div>
  <div className="pt-2">
    <ScheduleEditor
      key={station.updatedAt}
      station={station}
      submitting={scheduleSubmitting}
      onSave={handleSaveSchedule}
    />
  </div>
</section>

{/* — Battery Slots Management ————— */}
<section className="space-y-4">
  <div className="flex items-center justify-between">
    <div>
      <h2 className="text-base font-semibold tracking-tight text-
ink">Battery Slots Provisioning</h2>
      <p className="text-[12px] text-muted mt-0.5">Individual battery slots
and live availability state.</p>
    </div>
    <Button variant="primary" onClick={handleOpenCreateSlot}
className="shadow-card">
      + New Slot
    </Button>
  </div>

  <div className="rounded-xl border border-line bg-surface shadow-card

```

```

overflow-hidden">
  <Table>
    <thead>
      <tr className="bg-surface-alt/70 border-b border-line text-[11px]
font-semibold uppercase tracking-wider text-muted">
        <Th className="text-right">Slot #</Th>
        <Th>Battery Type</Th>
        <Th className="text-right">Capacity</Th>
        <Th>Status</Th>
        <Th className="text-right">Actions</Th>
      </tr>
    </thead>
    <tbody className="divide-y divide-line/80">
      {( !station.slots || station.slots.length === 0 ) && (
        <tr>
          <Td colSpan={5} className="py-8 text-center text-[13px] text-
muted">
            No battery slots defined for this hub yet. Click "+ New Slot"
above to provision one.
          </Td>
        </tr>
      )}
      {station.slots?.map((slot) => (
        <tr key={slot.id} className="transition-colors hover:bg-surface-
alt/50">
          <Td className="tnum text-right font-bold text-ink py-3.5">
            Slot {slot.slotNumber}
          </Td>
          <Td className="text-body font-medium py-3.5">{slot.type}</Td>
          <Td className="tnum text-right font-semibold text-ink py-3.5">
            {slot.capacityKWh}
            <span className="ml-1 text-[11px] font-normal text-
muted">kWh</span>
          </Td>
          <Td className="py-3.5">
            <select
              value={slot.status}
              disabled={slotStatusUpdatingId === slot.id}
              onChange={(event) => handleSlotStatusChange(slot.id,
event.target.value)}
              className="h-8 rounded-md border border-line bg-surface px-
2.5 text-xs font-medium text-ink focus:border-primary focus:outline-none
focus:ring-2 focus:ring-primary/20 disabled:opacity-50 cursor-pointer shadow-
xs"
            >
              <option value="Available">Available</option>

```



```

        <option value="Reserved">Reserved</option>
        <option value="Maintenance">Maintenance</option>
    </select>
</Td>
<Td className="text-right py-3.5">
    <Button variant="secondary" size="sm" onClick={() =>
handleOpenEditSlot(slot)}>
        Edit Slot
    </Button>
</Td>
</tr>
    )})
</tbody>
</Table>
</div>
</section>

{ /* — Active Reservations Overview ————— */ }
<section className="space-y-4">
    <div>
        <h2 className="text-base font-semibold tracking-tight text-ink">Active
Reservations Overview</h2>
        <p className="text-[12px] text-muted mt-0.5">Live schedule of prosumers
booked for this microgrid hub.</p>
    </div>

    <div className="rounded-xl border border-line bg-surface shadow-card
overflow-hidden">
        <Table>
            <thead>
                <tr className="bg-surface-alt/70 border-b border-line text-[11px]
font-semibold uppercase tracking-wider text-muted">
                    <Th className="text-right">Slot #</Th>
                    <Th>Prosumer NIC</Th>
                    <Th>Scheduled Time</Th>
                    <Th>Status</Th>
                </tr>
            </thead>
            <tbody className="divide-y divide-line/80">
                {reservationsLoading && (
                    <tr>
                        <Td colSpan={4} className="py-8 text-center text-[13px] text-
muted">
                            Loading reservation overview...
                        </Td>
                    </tr>
                )}
            </tbody>
        </Table>
    </div>

```

```

    })
    {!reservationsLoading && reservationsError && (
      <tr>
        <Td colSpan={4} className="py-8 text-center text-[13px] text-
error">
          {reservationsError}
        </Td>
      </tr>
    })
    {!reservationsLoading && !reservationsError && reservations.length
=== 0 && (
      <tr>
        <Td colSpan={4} className="py-8 text-center text-[13px] text-
muted">
          No active reservations for this station.
        </Td>
      </tr>
    })
    {!reservationsLoading &&
      !reservationsError &&
      reservations.map((reservation) => (
        <tr key={` ${reservation.slotId}-${reservation.scheduledAt}`}
className="hover:bg-surface-alt/50 transition-colors">
          <Td className="tnum text-right font-medium py-3.5">Slot
{slotNumberFor(reservation.slotId)}</Td>
          <Td className="font-mono text-[12px] text-body py-
3.5">{reservation.prosumerNic ?? '-'}</Td>
          <Td className="tnum text-body py-3.5">
            {reservation.scheduledAt ? new
Date(reservation.scheduledAt).toLocaleString() : '-'}
          </Td>
          <Td className="py-3.5">
            <Badge tone={reservation.status === 'Confirmed' ? 'info' :
'neutral'}>{reservation.status}</Badge>
          </Td>
        </tr>
      ))
    </tbody>
  </Table>
</div>
</section>

{ /* — Dialogs & Modals ————— */ }
<SlotForm
  key={slotFormKey}
  open={isSlotFormOpen}

```

```

    slot={editingSlot}
    submitting={slotSubmitting}
    serverError={slotFormError}
    onClose={() => setIsSlotFormOpen(false)}
    onSubmit={handleSubmitSlot}
  />

  <ConfirmDialog
    open={confirmDeactivateOpen}
    title="Deactivate station node?"
    description={`"${station.name}" will be marked inactive and taken offline
for new reservations.`}
    confirmLabel="Deactivate Node"
    tone="danger"
    confirming={deactivating}
    onConfirm={handleDeactivate}
    onClose={() => setConfirmDeactivateOpen(false)}
  />

  <Toast message={toast.message} tone={toast.tone} onDismiss={() =>
setToast({ message: '', tone: 'error' })} />
</div>

);}

```

StationForm.jsx

```

import { useState } from 'react'; import Modal from '../components/common/Modal.jsx'; import Input
from '../components/common/Input.jsx'; import Button from '../components/common/Button.jsx';
import MapPicker from '../components/common/MapPicker.jsx';

// Small inline pin glyph for the "Pick on map" trigger - avoids pulling in an icon library // for a single
icon. function PinIcon() { return ( ); }

const EMPTY_VALUES = { name: "", lat: "", lng: "", capacityKWh: "", totalBatterySlots: "", opensAt: "",
closesAt: "", };

// Converts a loaded station (list-row or full-detail shape) into the form's flat field values. function
toFormValues(station) { if (!station) { return EMPTY_VALUES; } const [lng, lat] =
station.location?.coordinates ?? []; return { name: station.name ?? "", lat: lat ?? "", lng: lng ?? "",
capacityKWh: station.capacityKWh ?? "", totalBatterySlots: station.totalBatterySlots ?? "", opensAt:
station.operatingSchedule?.opensAt ?? "", closesAt: station.operatingSchedule?.closesAt ?? "", }; }

// Create/edit modal for a station; renders the shared Modal shell with a form inside. // The parent
remounts this component (via a changing key) each time it opens, so field // values are simply derived

```

```

once from station at mount - no effect-based reset needed. export default function StationForm({
open, station, submitting, serverError, onClose, onSubmit }) { const [values, setValues] = useState(() =>
toFormValues(station)); const [isMapPickerOpen, setIsMapPickerOpen] = useState(false); // Bumped
every time the map picker opens so it remounts fresh (see MapPicker.jsx). const [mapPickerKey,
setMapPickerKey] = useState(0); const isEdit = Boolean(station);

// Updates a single field's value as the user types. const handleChange = (field) => (event) => {
setValues((prev) => ({ ...prev, [field]: event.target.value })); };

// Writes the map picker's confirmed coordinates into the same lat/lng fields manual entry // uses - both
paths converge on the same string-valued form state, so nothing downstream // needs to know which
one was used. Values arrive as plain decimal numbers (see MapPicker's // roundCoordinate), matching
the backend's lat/lng number contract. const handleMapConfirm = ({ lat, lng }) => { setValues((prev) => ({
...prev, lat: String(lat), lng: String(lng) })); setIsMapPickerOpen(false); };

// Builds the request payload from form state and hands it to the parent's submit handler. const
handleSubmit = (event) => { event.preventDefault(); onSubmit({ name: values.name, lat:
Number(values.lat), lng: Number(values.lng), capacityKWh: Number(values.capacityKWh),
totalBatterySlots: Number(values.totalBatterySlots), operatingSchedule: { opensAt: values.opensAt,
closesAt: values.closesAt }, }); };

return ( <Modal open={open} onClose={onClose} title={isEdit ? 'Edit station' : 'New station'}
description="GPS position, capacity specs and operating hours for this grid node."
suppressClose={isMapPickerOpen} >

<Input label="Station name" required value={values.name} onChange={handleChange('name')} />
  <div>
    <div className="mb-1.5 flex items-center justify-between">
      <span className="text-[13px] font-medium text-ink">Location</span>
      <Button
        type="button"
        variant="ghost"
        size="sm"
        onClick={() => {
          setMapPickerKey((key) => key + 1);
          setIsMapPickerOpen(true);
        }}
      >
      <span className="flex items-center gap-1.5 text-primary">
        <PinIcon />
        Pick on map
      </span>
    </Button>
  </div>
  {/* Paired inputs sit in a balanced 2-column grid, stacking on narrow
screens. Values

```

typed here or written by the map picker are the exact same state -
both stay

```
    fully editable regardless of which one was used last. */}
<div className="grid grid-cols-1 gap-4 sm:grid-cols-2">
  <Input
    label="Latitude"
    type="number"
    step="any"
    required
    placeholder="6.9271"
    className="font-mono"
    value={values.lat}
    onChange={handleChange('lat')}
  />
  <Input
    label="Longitude"
    type="number"
    step="any"
    required
    placeholder="79.8612"
    className="font-mono"
    value={values.lng}
    onChange={handleChange('lng')}
  />
</div>
</div>

<div className="grid grid-cols-1 gap-4 sm:grid-cols-2">
  <Input
    label="Capacity (kWh)"
    type="number"
    step="any"
    required
    className="tnum"
    value={values.capacityKWh}
    onChange={handleChange('capacityKWh')}
  />
  <Input
    label="Total battery slots"
    type="number"
    step="1"
    required
    className="tnum"
    value={values.totalBatterySlots}
    onChange={handleChange('totalBatterySlots')}
  />
```

```

    </div>

    <div className="grid grid-cols-1 gap-4 sm:grid-cols-2">
      <Input label="Opens at" type="time" required value={values.opensAt}
onChange={handleChange('opensAt')} />
      <Input label="Closes at" type="time" required value={values.closesAt}
onChange={handleChange('closesAt')} />
    </div>

    {serverError && (
      <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2
text-[13px] font-medium text-error">
        {serverError}
      </p>
    )}

    <div className="-mx-6 -mb-5 mt-1 flex justify-end gap-2 border-t border-
line bg-surface-alt px-6 py-4">
      <Button type="button" variant="secondary" onClick={onClose}
disabled={submitting}>
        Cancel
      </Button>
      <Button type="submit" variant="primary" disabled={submitting}>
        {submitting ? 'Saving...' : isEdit ? 'Save changes' : 'Create station'}
      </Button>
    </div>
  </form>

  <MapPicker
    key={mapPickerKey}
    open={isMapPickerOpen}
    initialLat={values.lat === '' ? NaN : Number(values.lat)}
    initialLng={values.lng === '' ? NaN : Number(values.lng)}
    onClose={() => setIsMapPickerOpen(false)}
    onConfirm={handleMapConfirm}
  />
</Modal>

); }

```

MapPicker.jsx

```

import { useEffect, useState } from 'react'; import { MapContainer, TileLayer, Marker, useMap,
useMapEvents } from 'react-leaflet'; import L from 'leaflet'; import 'leaflet/dist/leaflet.css'; import
Modal from './Modal.jsx'; import Button from './Button.jsx'; import Input from './Input.jsx';

```

```

const DEFAULT_CENTER = { lat: 6.9271, lng: 79.8612 }; // Sri Lanka
const DEFAULT_ZOOM = 8;
const SELECTED_ZOOM = 13;

// Inline SVG pin so we don't depend on Leaflet's default marker image assets, which don't // resolve
// correctly out of the box under Vite's bundler without extra asset config.
const PIN_ICON = L.divIcon({
  className: "",
  html: `<svg width="28" height="36" viewBox="0 0 28 36"
  xmlns="http://www.w3.org/2000/svg">
    <path d="M14 0C6.27 0 0 6.27 0 14c0 10.5 14 22 14 22s14-11.5 14-22C28 6.27 21.73 0 14 0z" fill="#0F172A"/>
    <circle cx="14" cy="14" r="5.5" fill="white"/>
  </svg>`,
  iconSize: [28, 36],
  iconAnchor: [14, 36],
});

// Places the marker wherever the user clicks on the map.
function ClickHandler({ onPick }) {
  useMapEvents({
    click(event) {
      onPick({ lat: event.latlng.lat, lng: event.latlng.lng });
    },
  });
  return null;
}

// Recenters the map imperatively whenever target changes (e.g. after a search result is // picked) -
// MapContainer's own center prop only applies on first mount, not on updates.
function Recenter({ target, zoom }) {
  const map = useMap();
  useEffect(() => {
    if (target) {
      map.setView([target.lat, target.lng], zoom);
    }
  }, [target, zoom, map]);
  return null;
}

// Rounds to 6 decimal places (~0.1m precision) as a plain number, matching the backend's // lat/lng
// number contract (not a formatted string).
function roundCoordinate(value) {
  return Math.round(value * 1e6) / 1e6;
}

// Derives the marker's starting position from the form's current lat/lng, falling back to the // default Sri
// Lanka center when they're empty/invalid.
function toStartPosition(initialLat, initialLng) {
  const hasValidStart = Number.isFinite(initialLat) && Number.isFinite(initialLng);
  return hasValidStart ? { lat: initialLat, lng: initialLng } : DEFAULT_CENTER;
}

// Modal for visually picking a station's GPS position; confirms back a { lat, lng } number pair.
// The parent (StationForm) remounts this component via a changing key each time it opens, so // state below
// is simply derived once from props at mount - no effect-based reset needed.
export default function MapPicker({ open, initialLat, initialLng, onClose, onConfirm }) {
  const [position, setPosition] = useState(() => toStartPosition(initialLat, initialLng));
  const [recenterTarget, setRecenterTarget] = useState(() => toStartPosition(initialLat, initialLng));
  const [searchQuery, setSearchQuery] = useState("");
  const [searchResults, setSearchResults] = useState([]);
  const [searching, setSearching] = useState(false);
  const [searchError, setSearchError] = useState("");

  // Looks up a place name via OpenStreetMap's free Nominatim search (no API key required).
  // Runs only on explicit submit, not per keystroke, per Nominatim's fair-use policy.
  const handleSearch = async (event) => {
    event.preventDefault();
    const query = searchQuery.trim();
    if (!query) {
      return;
    }
    setSearching(true);
    setSearchError("");
    setSearchResults([]);
    try {
      const response = await fetch(
https://nominatim.openstreetmap.org/search?format=json&limit=5&q=\${encodeURIComponent\(query\)}
      );
      if (!response.ok) {
        throw new Error("Search request failed.");
      }
      const results = await response.json();
      setSearchResults(results);
      if (results.length === 0) {
        setSearchError("No locations found for that search.");
      }
    } catch {
      setSearchError("Could not reach the location search service.");
    }
    finally {
      setSearching(false);
    }
  };
}

```

```

// Selects a search result: moves the marker and recenters the map there. const handleSelectResult =
(result) => { const next = { lat: Number(result.lat), lng: Number(result.lng) }; setPosition(next);
setRecenterTarget(next); setSearchResults([]); setSearchQuery(result.display_name); };

// Confirms the current marker position back to the parent form as rounded plain numbers. const
handleConfirm = () => { onConfirm({ lat: roundCoordinate(position.lat), lng:
roundCoordinate(position.lng) }); };

if (!open) { return null; }

return ( <Input placeholder="Search for a place or address" aria-label="Search for a location"
value={searchQuery} onChange={(event) => setSearchQuery(event.target.value)} className="flex-1" />
{searching ? 'Searching...' : 'Search'}

{searchError && <p className="mb-3 text-[13px] text-
error">{searchError}</p>}

{searchResults.length > 0 && (
  <ul className="mb-3 max-h-36 divide-y divide-line overflow-y-auto
rounded-md border border-line bg-surface-alt">
    {searchResults.map((result) => (
      <li key={result.place_id}>
        <button
          type="button"
          onClick={() => handleSelectResult(result)}
          className="block w-full px-3 py-2 text-left text-[13px] text-body
transition-colors hover:bg-surface hover:text-ink"
        >
          {result.display_name}
        </button>
      </li>
    ))}
  </ul>
)}

<div className="h-80 w-full overflow-hidden rounded-md border border-line
sm:h-96">
  <MapContainer
    center={[position.lat, position.lng]}
    zoom={Number.isFinite(initialLat) && Number.isFinite(initialLng) ?
SELECTED_ZOOM : DEFAULT_ZOOM}
    style={{ height: '100%', width: '100%' }}
  >
    <TileLayer
      attribution='&copy; <a
href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>

```



```

contributors'
    url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
  />
  <Marker
    position={[position.lat, position.lng]}
    icon={PIN_ICON}
    draggable
    eventHandlers={{
      dragend: (event) => {
        const latlng = event.target.getLatLng();
        setPosition({ lat: latlng.lat, lng: latlng.lng });
      },
    }}
  />
  <ClickHandler onPick={setPosition} />
  <Recenter target={recenterTarget} zoom={SELECTED_ZOOM} />
</MapContainer>
</div>

<div className="mt-3 flex items-center justify-between rounded-md border
border-line bg-surface-alt px-3 py-2">
  <span className="font-mono text-[13px] text-ink">
    {position.lat.toFixed(6)}, {position.lng.toFixed(6)}
  </span>
  <span className="text-[11px] text-muted">Click or drag the pin to
adjust</span>
</div>

<div className="-mx-6 -mb-5 mt-5 flex justify-end gap-2 border-t border-
line bg-surface-alt px-6 py-4">
  <Button type="button" variant="secondary" onClick={onClose}>
    Cancel
  </Button>
  <Button type="button" variant="primary" onClick={handleConfirm}>
    Confirm location
  </Button>
</div>
</Modal>

);}

```

AppShell.jsx

```

import { useState } from 'react'; import { Outlet, useNavigate } from 'react-router-dom'; import {
useAuth } from '../context/AuthContext.jsx'; import Sidebar from './Sidebar.jsx'; import ScrollArea
from '../common/ScrollArea.jsx'; import Icon from '../common/Icon.jsx';

// Nav sections shown in the sidebar. icon is a name from components/common/Icon.jsx. const
NAV_SECTIONS = [ { title: 'Overview', items: [{ label: 'Dashboard', to: '/dashboard', icon: 'grid' }], }, { title:
'Node management', items: [{ label: 'Microgrid hubs', to: '/stations', icon: 'hubs' }], }, // Dinil -
reservation oversight (web admin view of the reservations collection). { title: 'Reservations', items: [{
label: 'Reservations', to: '/reservations', icon: 'calendar' }], }, // Rukshan - prosumer account
management (Backoffice only). { title: 'Prosumer management', items: [ { label: 'Prosumers', to:
'/admin/prosumers', icon: 'prosumer', end: true }, { label: 'Pending Prosumers', to:
'/admin/prosumers/pending', icon: 'clock' }, ], }, // Migara - user management (Backoffice/GridOperator
accounts). { title: 'Administration', items: [{ label: 'Users', to: '/admin/users', icon: 'user' }], }, ];

// Renders the shared shell (sidebar/content panel) around whichever page route is active. export
default function AppShell() { const { fullName, role, profilePicture, logout } = useAuth(); const navigate =
useNavigate(); const [sidebarOpen, setSidebarOpen] = useState(false);

// Clears the session and returns to the public landing page. const handleLogout = () => { logout();
navigate('/'); };

return (

<Sidebar sections={NAV_SECTIONS} open={sidebarOpen} onClose={() => setSidebarOpen(false)}
fullName={fullName} role={role} profilePicture={profilePicture} onLogout={handleLogout} />
  { /* Mobile sidebar toggle button */ }
  <button
    type="button"
    onClick={() => setSidebarOpen(true)}
    className="fixed top-3 left-3 z-20 flex h-9 w-9 items-center justify-
center rounded-lg bg-surface border border-line text-body shadow-card
transition-colors hover:bg-surface-alt hover:text-ink lg:hidden"
    aria-label="Open navigation"
  >
    <Icon name="menu" className="h-5 w-5" />
  </button>

  <div className="flex min-w-0 flex-1 flex-col h-full">
    <main className="flex-1 min-h-0 bg-surface border-l border-line overflow-
hidden">
      <ScrollArea className="h-full w-full">
        <Outlet />
      </ScrollArea>
    </main>
  </div>

```

```
</div>
```

```
); }
```

Sidebar.jsx

```
import { NavLink } from 'react-router-dom'; import JouleMark from '../common/JouleMark.jsx'; import
Icon from '../common/Icon.jsx';
```

```
// Initials for the avatar chip, e.g. "Shalon Fernando" -> "SF". function initialsOf(name) { return name
.split(/\s+/).filter(Boolean).slice(0, 2).map((part) => part[0].toUpperCase()).join(""); }
```

```
// Renders one nav section's items as icon links, no title header. function NavSection({ section,
onNavigate }) { return (
```

```
{section.items.map((item) => ( <NavLink key={item.to} to={item.to} end={item.end}
onClick={onNavigate} className={({ isActive }) => flex items-center gap-2.5 rounded-lg px-
3 py-2 text-[13px] font-medium transition-colors ${ isActive ? 'bg-surface
text-ink shadow-card ring-1 ring-line' : 'text-body hover:bg-surface/70
hover:text-ink' } }) > {{{ isActive }} => ( <> <Icon name={item.icon} className={h-4 w-4
${isActive ? 'text-primary' : 'text-muted'} /> {item.label} </> ) }}}
); }
```

```
// Renders the sidebar: brand, nav sections, and the user/sign-out footer. On small screens it's a // fixed
drawer driven by open; on desktop (lg:) it's a static rail. export default function Sidebar({ sections, open,
onClose, fullName, role, profilePicture, onLogout }) { return ( <> {open &&
```

```
} <aside className={fixed inset-y-0 left-0 z-40 flex w-64 flex-col bg-canvas px-3
py-4 shadow-panel transition-transform duration-200 lg:static lg:translate-x-
0 lg:shadow-none ${ open ? 'translate-x-0' : '-translate-x-full' } }>
```

```
Joule
```

```
  <nav className="flex-1 overflow-y-auto">
    {sections.map((section) => (
      <NavSection key={section.title} section={section}
onNavigate={onClose} />
    ))}
  </nav>
```

```
  <div className="mt-4 border-t border-line pt-4">
    <NavLink
      to="/profile"
      onClick={onClose}
      className={({ isActive }) =>
        `flex items-center gap-2.5 rounded-xl p-2 transition-colors ${
          isActive
```

```

        ? 'bg-surface text-ink ring-1 ring-line shadow-card'
        : 'hover:bg-surface/70'
      }`
    }
  >
  {profilePicture ? (
    <img
      src={profilePicture}
      alt={fullName || 'Avatar'}
      className="h-8 w-8 shrink-0 rounded-full object-cover ring-1
ring-line"
    />
  ) : (
    <span className="flex h-8 w-8 shrink-0 items-center justify-center
rounded-full bg-primary-soft text-[12px] font-semibold text-primary">
      {fullName ? initialsOf(fullName) : '-'}
    </span>
  )}
  <div className="min-w-0 flex-1">
    <p className="truncate text-[13px] font-medium text-ink">{fullName
?? 'Not signed in'}</p>
    {role && <p className="truncate text-[11px] text-muted">{role}</p>}
  </div>
  </NavLink>
  <button
    type="button"
    onClick={onLogout}
    className="mt-2 flex w-full items-center gap-2.5 rounded-lg px-3 py-2
text-[13px] font-medium text-body transition-colors hover:bg-surface/70
hover:text-ink"
  >
    <Icon name="logout" className="h-4 w-4 text-muted" />
    Sign out
  </button>
  </div>
</aside>
</>

);}

```

TopBar.jsx

```
import { Link } from 'react-router-dom'; import Icon from '../common/Icon.jsx';
```

// Renders the hamburger (mobile only) and the breadcrumb trail. Every crumb but the last links back.
export default function TopBar({ onToggleSidebar, crumbs }) { return (

```

    <nav aria-label="Breadcrumb" className="flex min-w-0 items-center gap-1.5
text-[13px]">
      {crumbs.map((crumb, index) => {
        const isLast = index === crumbs.length - 1;
        return (
          <span key={crumb.label} className="flex min-w-0 items-center gap-
1.5">
            {index > 0 && <Icon name="chevronRight" className="h-3.5 w-3.5
shrink-0 text-line-strong" />}
            {isLast || !crumb.to ? (
              <span className={`truncate ${isLast ? 'font-medium text-ink' :
'text-muted'}`} >{crumb.label}</span>
            ) : (
              <Link to={crumb.to} className="truncate text-muted transition-
colors hover:text-ink">
                {crumb.label}
              </Link>
            )}
          </span>
        );
      })}
    </nav>
  </header>

```

```
); }
```

DashboardPage.jsx

```

import { useEffect, useMemo, useState } from 'react'; import { Link } from 'react-router-dom'; import {
AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, BarChart, Bar, } from
'recharts'; import { useAuth } from '../context/AuthContext.jsx'; import api from '../services/api.js';
import Icon from '../components/common/Icon.jsx';

```

```

// — Helpers —————
function formatRelative(iso) { if (!iso) return '-'; const diff = Date.now() - new Date(iso).getTime(); const
m = Math.floor(diff / 60000); if (m < 1) return 'Just now'; if (m < 60) return `${m}m ago; const h =
Math.floor(m / 60); if (h < 24) return `${h}h ago; return `${Math.floor(h / 24)}d ago; }

function formatDate(iso) { if (!iso) return '-'; try { const d = new Date(iso); return
d.toLocaleDateString(undefined, { month: 'short', day: 'numeric', hour: '2-digit', minute: '2-digit' }); }
catch (_) { return iso; } }

```

```

const STATUS_CONFIG = { Confirmed: { bg: 'bg-amber-50 text-amber-700 border-amber-200/80', dot:
'bg-amber-500', }, Completed: { bg: 'bg-emerald-50 text-emerald-700 border-emerald-200/80', dot: 'bg-
emerald-500', }, Cancelled: { bg: 'bg-rose-50 text-rose-700 border-rose-200/80', dot: 'bg-rose-500', },
Active: { bg: 'bg-emerald-50 text-emerald-700 border-emerald-200/80', dot: 'bg-emerald-500', },
Inactive: { bg: 'bg-slate-100 text-slate-600 border-slate-200', dot: 'bg-slate-400', }, };

// Build 7-day reservation volume trend from live reservations function
buildReservationTrend(reservations) { const days = []; for (let i = 6; i >= 0; i--) { const d = new Date();
d.setDate(d.getDate() - i); days.push(d.toISOString().slice(0, 10)); } const counts = {}; days.forEach(d => {
counts[d] = 0; }); reservations.forEach(r => { const day = r.scheduledAt?.slice(0, 10); if (day &&
counts[day] !== undefined) counts[day]++; }); return days.map(d => ({ fullDate: d, day: new
Date(d).toLocaleDateString(undefined, { weekday: 'short', month: 'numeric', day: 'numeric' }), shortDay:
new Date(d).toLocaleDateString(undefined, { weekday: 'short' }), count: counts[d], )); }

// Build per-station capacity chart data function buildStationChart(stations) { return (stations ||
[]).slice(0, 6).map(s => ({ id: s.id, name: s.name || Station ${s.id?.slice(-4)}, shortName:
s.name?.length > 12 ? ${s.name.slice(0, 10)}... : (s.name || s.id?.slice(-4)), kWh: s.capacityKWh
?? 0, slots: s.batterySlots?.length ?? s.totalBatterySlots ?? 0, })); }

// — Custom Chart Tooltip ————— function
CustomChartTooltip({ active, payload, label, unit = 'Bookings' }) { if (!active || !payload ||
!payload.length) return null; const data = payload[0]; return (

{label || data.payload?.name || data.payload?.day}

<span className="h-2 w-2 rounded-full" style={{ backgroundColor: data.color || '#635bff' }} /> {unit}:
{data.value?.toLocaleString()}
); }

// — Refined Subcomponents ————— function
PrimaryKpiCard({ label, value, hint, icon, accent = 'primary', loading }) { const accentStyles = { primary:
'bg-primary/10 text-primary border-primary/20', emerald: 'bg-emerald-500/10 text-emerald-600
border-emerald-500/20', amber: 'bg-amber-500/10 text-amber-600 border-amber-500/20', cyan: 'bg-
sky-500/10 text-sky-600 border-sky-500/20', };

return (

{label} <span className={flex h-9 w-9 items-center justify-center rounded-lg border
transition-transform duration-200 group-hover:scale-105
${accentStyles[accent] || accentStyles.primary}}>

{loading ? '...' : (value ?? '-')}

{hint && (

{hint}


```

```

    })
  ); }

function SecondaryKpiStrip({ label, value, icon, hint, loading }) { return (

  {label}

  {loading ? '...' : (value ?? '-')}

  {hint && {hint}}
); }

function SectionHeader({ title, subtitle, badge, action }) { return (

  {title}
  {badge && ( {badge} )}
  {subtitle &&

  {subtitle}

  }
  {action}
); }

function ActivityItem({ reservation }) { const status = reservation.status || 'Confirmed'; const conf =
STATUS_CONFIG[status] || STATUS_CONFIG.Confirmed;

return (

  • NIC {reservation.prosumerNic}

  · Station {reservation.stationId?.slice(-6) ?? '-'}

  {formatDate(reservation.scheduledAt)}

  <span className={inline-flex items-center px-2.5 py-0.5 rounded-md text-[11px]
font-medium border ${conf.bg}}> {status} {formatRelative(reservation.scheduledAt)}
  ); }

function QuickActionCard({ label, description, icon, to, color = 'bg-primary-soft text-primary' }) { return (

  <span className={flex h-10 w-10 shrink-0 items-center justify-center rounded-xl
transition-transform duration-200 group-hover:scale-105 ${color}}>

  {label}

  {description}

```

```
→ ); }
```

```
// — Main Page Component ————— export
default function DashboardPage() { const { fullName, role } = useAuth(); const isBackoffice = role ===
'Backoffice';
```

```
const [stations, setStations] = useState(null); const [reservations, setReservations] = useState(null);
const [users, setUsers] = useState(null); const [loading, setLoading] = useState(true);
```

```
useEffect(() => { async function fetchAll() { try { const [stRes, rvRes] = await Promise.all([
api.get('/stations'), api.get('/reservations'), ]); setStations(stRes.data ?? []); setReservations(rvRes.data
?? []);
```

```
    if (isBackoffice) {
      const uRes = await api.get('/users');
      setUsers(uRes.data ?? []);
    }
  } catch (_) {
    setStations(prev => prev ?? []);
    setReservations(prev => prev ?? []);
  } finally {
    setLoading(false);
  }
}
fetchAll();
```

```
}, [isBackoffice]);
```

```
// Derived KPIs const activeStations = useMemo(() => stations?.filter(s => s.status === 'Active').length ??
0, [stations]); const totalCapacity = useMemo(() => stations?.reduce((acc, s) => acc + (s.capacityKWh ??
0), 0) ?? 0, [stations]); const maxStationCapacity = useMemo(() => Math.max(1, ...(stations?.map(s =>
s.capacityKWh ?? 0) ?? [1])), [stations]); const confirmedRes = useMemo(() => reservations?.filter(r =>
r.status === 'Confirmed').length ?? 0, [reservations]); const completedRes = useMemo(() =>
reservations?.filter(r => r.status === 'Completed').length ?? 0, [reservations]); const cancelledRes =
useMemo(() => reservations?.filter(r => r.status === 'Cancelled').length ?? 0, [reservations]); const
totalSlots = useMemo(() => stations?.reduce((acc, s) => acc + (s.batterySlots?.length ??
s.totalBatterySlots ?? 0), 0) ?? 0, [stations]);
```

```
// Chart data const reservationTrend = useMemo(() => reservations ?
buildReservationTrend(reservations) : [], [reservations]); const stationChart = useMemo(() => stations ?
buildStationChart(stations) : [], [stations]);
```

```
const recentActivity = useMemo(() => reservations?.slice().sort((a, b) => new Date(b.scheduledAt) - new
Date(a.scheduledAt)).slice(0, 7) ?? [], [reservations] );
```



```

const hour = new Date().getHours(); const greeting = hour < 12 ? 'Good morning' : hour < 17 ? 'Good
afternoon' : 'Good evening'; const firstName = fullName?.split(' ')[0] ?? 'there';

return (

  {/* — Welcome & Status Header ————— */}
  <div className="flex flex-col md:flex-row md:items-center md:justify-
between gap-4 pb-2 border-b border-line/60">
    <div>
      <div className="flex items-center gap-2">
        <h1 className="text-2xl font-bold tracking-tight text-ink">
          {greeting}, {firstName}
        </h1>
      </div>
      <p className="mt-1 text-[13px] text-muted flex items-center gap-2">
        <span>{new Date().toLocaleDateString(undefined, { weekday: 'long',
year: 'numeric', month: 'long', day: 'numeric' })}</span>
        <span>•</span>
        <span className="font-semibold text-body bg-surface-alt px-2 py-0.5
rounded border border-line text-[11px]">{role}</span>
      </p>
    </div>

    <div className="flex items-center gap-3">
      <Link
        to="/stations"
        className="inline-flex items-center gap-1.5 rounded-lg border border-
line bg-surface px-3.5 py-2 text-[13px] font-medium text-ink shadow-card
hover:bg-surface-alt transition-colors"
      >
        <Icon name="hubs" className="h-4 w-4 text-primary" />
        <span>Manage Hubs</span>
      </Link>
      <Link
        to="/reservations"
        className="inline-flex items-center gap-1.5 rounded-lg bg-primary px-
3.5 py-2 text-[13px] font-medium text-on-primary shadow-card hover:bg-
primary-hover transition-colors"
      >
        <Icon name="calendar" className="h-4 w-4" />
        <span>All Bookings</span>
      </Link>
    </div>
  </div>

  {/* — Primary KPI Grid (4 Hero Cards) ————— */}

```

```

<div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-4">
  <PrimaryKpiCard
    label="Active Hubs"
    value={activeStations}
    hint={` ${activeStations} of ${stations?.length ?? 0} operational
nodes` }
    icon="hubs"
    accent="primary"
    loading={loading}
  />
  <PrimaryKpiCard
    label="Total Capacity"
    value={` ${totalCapacity.toLocaleString()} kWh` }
    hint="Across all solar stations"
    icon="battery"
    accent="emerald"
    loading={loading}
  />
  <PrimaryKpiCard
    label="Confirmed Bookings"
    value={confirmedRes}
    hint="Awaiting slot check-in"
    icon="calendar"
    accent="amber"
    loading={loading}
  />
  <PrimaryKpiCard
    label="Completed Transfers"
    value={completedRes}
    hint="Successfully dispatched energy"
    icon="bolt"
    accent="cyan"
    loading={loading}
  />
</div>

```

```

{/* — Secondary Metrics Strip (For Backoffice) — */}
{isBackoffice && (
  <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">
    <SecondaryKpiStrip
      label="Total Battery Slots"
      value={totalSlots}
      icon="pulse"
      hint="Provisioned"
      loading={loading}
    />
  </div>
)}

```

```

    <SecondaryKpiStrip
      label="Staff Accounts"
      value={users?.length ?? '-'}
      icon="user"
      hint="Operators & Admins"
      loading={loading}
    />
    <SecondaryKpiStrip
      label="Cancelled Bookings"
      value={cancelledRes}
      icon="check"
      hint="Total period"
      loading={loading}
    />
  </div>
)}

{/* — Interactive Charts Row ————— */}
<div className="grid grid-cols-1 gap-6 lg:grid-cols-2">
  {/* Booking Trend Chart */}
  <div className="rounded-xl border border-line bg-surface p-5 shadow-card">
    <SectionHeader
      title="Booking Volume"
      subtitle="Reservations scheduled each day"
      badge="Last 7 Days"
    />
    <div className="h-52 w-full">
      <ResponsiveContainer width="100%" height="100%">
        <AreaChart data={reservationTrend} margin={{ top: 8, right: 8,
left: -24, bottom: 0 }}>
          <defs>
            <linearGradient id="bookingGrad" x1="0" y1="0" x2="0" y2="1">
              <stop offset="0%" stopColor="#635bff" stopOpacity={0.25} />
              <stop offset="100%" stopColor="#635bff" stopOpacity={0.0} />
            </linearGradient>
          </defs>
          <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0"
vertical={false} />
          <XAxis dataKey="shortDay" tick={{ fontSize: 11, fill: '#64748b'
}} axisLine={false} tickLine={false} />
          <YAxis tick={{ fontSize: 11, fill: '#64748b' }} axisLine={false}
tickLine={false} allowDecimals={false} />
          <Tooltip content={<CustomChartTooltip unit="Reservations" />} />
          <Area
            type="monotone"

```

```

        dataKey="count"
        stroke="#635bff"
        strokeWidth={2.5}
        fill="url(#bookingGrad)"
        activeDot={{ r: 5, fill: '#635bff', stroke: '#ffffff',
strokeWidth: 2 }}
      />
    </AreaChart>
  </ResponsiveContainer>
</div>
</div>

{/* Station Capacity Chart */}
<div className="rounded-xl border border-line bg-surface p-5 shadow-
card">
  <SectionHeader
    title="Station Capacity"
    subtitle="Declared kWh per solar microgrid hub"
    badge="Top Hubs"
  />
  <div className="h-52 w-full">
    {loading ? (
      <div className="flex h-full items-center justify-center text-muted
text-xs">Loading station metrics...</div>
    ) : stationChart.length === 0 ? (
      <div className="flex h-full items-center justify-center text-muted
text-xs">No registered stations</div>
    ) : (
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={stationChart} margin={{ top: 8, right: 8, left: -
20, bottom: 0 }}>
          <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0"
vertical={false} />
          <XAxis dataKey="shortName" tick={{ fontSize: 10, fill:
'#64748b' }} axisLine={false} tickLine={false} />
          <YAxis tick={{ fontSize: 11, fill: '#64748b' }}
axisLine={false} tickLine={false} />
          <Tooltip content={<CustomChartTooltip unit="Capacity (kWh)" />}
/>
          <Bar dataKey="kWh" fill="#059669" radius={[4, 4, 0, 0]}
maxBarSize={36} />
        </BarChart>
      </ResponsiveContainer>
    )}
  </div>
</div>

```

```

</div>

{/* — Activity Feed & Quick Actions ————— */}
<div className="grid grid-cols-1 gap-6 lg:grid-cols-3">
  {/* Left 2 Cols: Activity Feed */}
  <div className="lg:col-span-2 rounded-xl border border-line bg-surface p-
5 shadow-card flex flex-col justify-between">
    <div>
      <SectionHeader
        title="Recent Reservations"
        subtitle="Live reservation events across grid nodes"
        action={
          <Link to="/reservations" className="text-[12px] font-semibold
text-primary hover:text-primary-hover flex items-center gap-1 transition-
colors">
            View all →
          </Link>
        }
      />
      {loading ? (
        <div className="flex items-center justify-center h-40 text-muted
text-xs">Loading activity feed...</div>
      ) : recentActivity.length === 0 ? (
        <div className="flex flex-col items-center justify-center h-40
text-center py-6">
          <span className="flex h-10 w-10 items-center justify-center
rounded-full bg-surface-alt text-muted mb-2">
            <Icon name="calendar" className="h-5 w-5" />
          </span>
          <p className="text-[13px] font-medium text-ink">No reservations
recorded yet</p>
          <p className="text-[12px] text-muted mt-0.5">New bookings will
appear here in real time.</p>
        </div>
      ) : (
        <ul className="space-y-0.5">
          {recentActivity.slice(0, 5).map(r => (
            <ActivityItem key={r.id} reservation={r} />
          ))}
        </ul>
      )}
    </div>

    {!loading && (reservations?.length ?? 0) > 0 && (
      <div className="pt-3 mt-3 border-t border-line/60 flex items-center
justify-between text-xs">

```

```

        <span className="text-muted">
            Showing recent <span className="font-semibold text-
ink">{Math.min(5, reservations.length)}</span> of{' '}
            <span className="font-semibold text-
ink">{reservations.length}</span> total reservations
        </span>
        <Link
            to="/reservations"
            className="inline-flex items-center gap-1 font-semibold text-
primary hover:text-primary-hover transition-colors"
        >
            <span>View in Reservations Page</span>
            <span>></span>
        </Link>
    </div>
)}
</div>

{/* Right 1 Col: Quick Navigation Modules */}
<div className="space-y-3.5">
    <SectionHeader title="Quick Actions" subtitle="Frequently accessed
workflows" />
    <QuickActionCard
        label="Microgrid Hubs"
        description="Manage solar grid nodes, specs & battery slots"
        icon="hubs"
        to="/stations"
        color="bg-primary-soft text-primary"
    />
    <QuickActionCard
        label="Reservation Log"
        description="Inspect all energy charging bookings"
        icon="calendar"
        to="/reservations"
        color="bg-sky-50 text-sky-600"
    />
    {isBackoffice && (
        <QuickActionCard
            label="Staff Directory"
            description="Provision Operator and Backoffice accounts"
            icon="user"
            to="/admin/users"
            color="bg-emerald-50 text-emerald-600"
        />
    )}
</div>

```

```

</div>

{ /* — Station Overview Table ————— */}
<div className="rounded-xl border border-line bg-surface shadow-card
overflow-hidden">
  <div className="px-5 py-4 border-b border-line flex items-center justify-
between bg-surface">
    <div>
      <h2 className="text-[15px] font-semibold tracking-tight text-
ink">Microgrid Stations Overview</h2>
      <p className="text-[12px] text-muted mt-0.5">Summary of capacity and
slot allocations per node</p>
    </div>
    <Link
      to="/stations"
      className="text-[12px] font-semibold text-primary hover:text-primary-
hover transition-colors"
    >
      Manage all stations →
    </Link>
  </div>
  <div className="overflow-x-auto">
    <table className="w-full text-left text-[13px]">
      <thead>
        <tr className="bg-surface-alt/70 border-b border-line text-[11px]
font-semibold uppercase tracking-wider text-muted">
          <th className="px-5 py-3">Station Node</th>
          <th className="px-5 py-3">Capacity Utilization</th>
          <th className="px-5 py-3 text-right">Slots</th>
          <th className="px-5 py-3">Status</th>
          <th className="px-5 py-3 text-right">Action</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-line/80">
        {loading && (
          <tr><td colSpan={5} className="px-5 py-8 text-center text-muted
text-xs">Loading station data...</td></tr>
        )}
        {!loading && (!stations || stations.length === 0) && (
          <tr><td colSpan={5} className="px-5 py-8 text-center text-muted
text-xs">No stations registered yet.</td></tr>
        )}
        {!loading && stations?.slice(0, 5).map(s => {
          const percent = Math.min(100, Math.round(((s.capacityKWh ?? 0) /
maxStationCapacity) * 100));
          return (

```

```

<tr key={s.id} className="hover:bg-surface-alt/50 transition-
colors">
  <td className="px-5 py-3.5">
    <div className="font-semibold text-ink">{s.name}</div>
    <div className="text-[11px] text-muted font-mono">ID:
{s.id?.slice(-8)}</div>
  </td>
  <td className="px-5 py-3.5 min-w-[180px]">
    <div className="flex items-center justify-between text-
[12px] mb-1">
      <span className="font-semibold text-ink
tnum">{(s.capacityKWh ?? 0).toLocaleString()} kWh</span>
      <span className="text-muted text-
[11px]">{percent}%</span>
    </div>
    <div className="h-1.5 w-full rounded-full bg-surface-alt
overflow-hidden">
      <div className="h-full rounded-full bg-emerald-500"
style={{ width: `${percent}%` }} />
    </div>
  </td>
  <td className="px-5 py-3.5 text-right font-medium text-ink
tnum">
    <span className="inline-flex items-center justify-center
rounded-md bg-surface-alt px-2 py-0.5 text-xs text-body font-semibold">
      {s.batterySlots?.length ?? s.totalBatterySlots ?? 0}
    </span>
  </td>
  <td className="px-5 py-3.5">
    <span className={`inline-flex items-center rounded-md px-
2.5 py-0.5 text-[11px] font-medium border ${
s.status === 'Active'
? 'bg-emerald-50 text-emerald-700 border-emerald-
200/80'
: 'bg-slate-100 text-slate-600 border-slate-200'
}`}>
      {s.status}
    </span>
  </td>
  <td className="px-5 py-3.5 text-right">
    <Link
to={` /stations/${s.id}`}
className="inline-flex items-center gap-1 text-[12px]
font-medium text-primary hover:underline"
>
      Details

```



```

                </Link>
            </td>
        </tr>
    );
    }
}
</tbody>
</table>
</div>

{!loading && (stations?.length ?? 0) > 0 && (
    <div className="px-5 py-3 border-t border-line/60 bg-surface-alt/30
flex flex-col sm:flex-row items-center justify-between gap-2 text-xs">
        <span className="text-muted">
            Displaying <span className="font-semibold text-ink">{Math.min(5,
stations.length)}</span> of{' '}
            <span className="font-semibold text-ink">{stations.length}</span>
registered microgrid nodes
        </span>
        <Link
            to="/stations"
            className="inline-flex items-center gap-1.5 font-semibold text-
primary hover:text-primary-hover transition-colors"
        >
            <span>View in Microgrid Hubs Page</span>
            <span>→</span>
        </Link>
    </div>
)}
</div>

</div>

); }

```

MapScreen.kt

```
package com.smartmicrogrid.ui.operator
```

```
import android.Manifest import android.annotation.SuppressLint import
android.content.pm.PackageManager import
androidx.activity.compose.rememberLauncherForActivityResult import
androidx.activity.result.contract.ActivityResultContracts import
androidx.compose.animation.AnimatedVisibility import androidx.compose.animation.expandVertically
import androidx.compose.animation.fadeIn import androidx.compose.animation.fadeOut import
androidx.compose.animation.shrinkVertically import androidx.compose.foundation.BorderStroke

```

```

import androidx.compose.foundation.background import androidx.compose.foundation.border import
import androidx.compose.foundation.clickable import
import androidx.compose.foundation.interaction.MutableInteractionSource import
import androidx.compose.foundation.layout.* import androidx.compose.foundation.shape.CircleShape import
import androidx.compose.foundation.shape.RoundedCornerShape import androidx.compose.material3.*
import androidx.compose.runtime.* import androidx.compose.ui.Alignment import
import androidx.compose.ui.Modifier import androidx.compose.ui.draw.clip import
import androidx.compose.ui.graphics.Color import androidx.compose.ui.platform.LocalContext import
import androidx.compose.ui.text.font.FontWeight import androidx.compose.ui.text.style.TextAlign import
import androidx.compose.ui.unit.dp import androidx.compose.ui.unit.sp import
import androidx.core.content.ContextCompat import com.google.android.gms.location.LocationServices import
import com.google.android.gms.maps.model.BitmapDescriptorFactory import
import com.google.android.gms.maps.model.CameraPosition import
import com.google.android.gms.maps.model.LatLng import com.google.maps.android.compose.GoogleMap
import com.google.maps.android.compose.MapProperties import
import com.google.maps.android.compose.MapUiSettings import com.google.maps.android.compose.Marker
import com.google.maps.android.compose.MarkerInfoWindowContent import
import com.google.maps.android.compose.rememberCameraPositionState import
import com.google.maps.android.compose.rememberMarkerState import
import com.smartmicrogrid.data.remote.ApiClient import com.smartmicrogrid.ui.components.IconTile import
import com.smartmicrogrid.ui.components.JouleIcons import com.smartmicrogrid.ui.home.UserRole import
import com.smartmicrogrid.ui.theme.* import kotlinx.coroutines.Dispatchers import
import kotlinx.coroutines.withContext

```

```

// Sri Lanka Center default private val DEFAULT_CENTER = LatLng(6.9271, 79.8612) private const val
DEFAULT_ZOOM = 11f private const val SEARCH_RADIUS_ALL_KM = 500.0 // Load all stations by default

```

```

@Composable fun MapScreen( userRole: UserRole = UserRole.Prosumer, onStationSelect: ((String) ->
Unit)? = null, ) { val context = LocalContext.current val api = remember { ApiClient.service }

```

```

var hasLocationPermission by remember {
    mutableStateOf(
        ContextCompat.checkSelfPermission(context,
Manifest.permission.ACCESS_FINE_LOCATION) ==
        PackageManager.PERMISSION_GRANTED,
    )
}
var showRationaleDialog by remember { mutableStateOf(!hasLocationPermission) }
var userLocation by remember { mutableStateOf<LatLng?>(null) }
var center by remember { mutableStateOf<LatLng?>(null) }
var usingDeviceLocation by remember { mutableStateOf(false) }
var allStations by remember { mutableStateOf<List<NearbyStation>?>(null) }
var mapError by remember { mutableStateOf<String?>(null) }
var mapRefreshToken by remember { mutableIntStateOf(0) }

```

```

// Filter states
var isFilterExpanded by remember { mutableStateOf(false) }
var searchQuery by remember { mutableStateOf("") }
var filterAvailableOnly by remember { mutableStateOf(false) }
var filterMinCapacityKWh by remember { mutableDoubleStateOf(0.0) }

val permissionLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.RequestPermission(),
) { granted ->
    hasLocationPermission = granted
    if (!granted) {
        userLocation = null
        center = DEFAULT_CENTER
        usingDeviceLocation = false
    }
}

LaunchedEffect(hasLocationPermission) {
    if (!hasLocationPermission) {
        if (center == null) {
            userLocation = null
            center = DEFAULT_CENTER
            usingDeviceLocation = false
        }
        return@LaunchedEffect
    }
    val resolved = readLastKnownLocation(context)
    userLocation = resolved
    center = resolved ?: DEFAULT_CENTER
    usingDeviceLocation = resolved != null
}

LaunchedEffect(center, mapRefreshToken) {
    val current = center ?: return@LaunchedEffect
    allStations = null
    mapError = null
    runCatching { api.getNearbyStations(current.latitude, current.longitude,
SEARCH_RADIUS_ALL_KM) }
        .onSuccess { allStations = it }
        .onFailure { mapError = it.message ?: "Couldn't reach the server." }
}

if (showRationaleDialog) {
    LocationRationaleDialog(
        onAllow = {

```

```

        showRationaleDialog = false

permissionLauncher.launch(Manifest.permission.ACCESS_FINE_LOCATION)
    },
    onDismiss = {
        showRationaleDialog = false
        center = DEFAULT_CENTER
        usingDeviceLocation = false
    },
)
}

// Filter computation
val filteredStations = remember(allStations, searchQuery,
filterAvailableOnly, filterMinCapacityKWh) {
    allStations?.filter { s ->
        val matchesSearch = searchQuery.isBlank() ||
s.name.contains(searchQuery, ignoreCase = true)
        val matchesSlots = !filterAvailableOnly || s.availableSlots > 0
        val matchesCapacity = filterMinCapacityKWh <= 0.0 || s.capacityKWh >=
filterMinCapacityKWh
        matchesSearch && matchesSlots && matchesCapacity
    }
}

val activeFilterCount = (if (searchQuery.isNotBlank()) 1 else 0) +
    (if (filterAvailableOnly) 1 else 0) +
    (if (filterMinCapacityKWh > 0.0) 1 else 0)

Box(modifier = Modifier.fillMaxSize().background(StripeCanvas)) {
    when {
        mapError != null -> ErrorState(message = mapError!!, onRetry = {
mapRefreshToken++ })
        center == null || allStations == null -> LoadingState()
        filteredStations != null && filteredStations.isEmpty() -> {
            StationsMap(
                center = center!!,
                userLocation = userLocation,
                userRole = userRole,
                hasLocationPermission = hasLocationPermission,
                stations = emptyList(),
                onStationSelect = onStationSelect,
            )
        }
    }
    Box(
        modifier = Modifier
            .fillMaxSize()

```

```

        .padding(top = 130.dp, start = 24.dp, end = 24.dp),
        contentAlignment = Alignment.Center,
    ) {
        Surface(
            shape = RoundedCornerShape(16.dp),
            color = StripeSurface,
            border = BorderStroke(1.dp, StripeBorder),
            shadowElevation = 6.dp,
        ) {
            Column(
                modifier = Modifier.padding(24.dp),
                horizontalAlignment = Alignment.CenterHorizontally,
            ) {
                Text("No stations match filters", style =
MaterialTheme.typography.titleMedium, fontWeight = FontWeight.Bold, color =
StripeInk)

                Spacer(modifier = Modifier.height(6.dp))
                Text("Try adjusting or resetting your filter
criteria.", style = MaterialTheme.typography.bodySmall, color = StripeBody,
textAlign = TextAlign.Center)

                Spacer(modifier = Modifier.height(12.dp))
                OutlinedButton(
                    onClick = {
                        searchQuery = ""
                        filterAvailableOnly = false
                        filterMinCapacityKWh = 0.0
                    },
                    shape = RoundedCornerShape(8.dp),
                ) {
                    Text("Reset Filters")
                }
            }
        }
    }
}
else -> StationsMap(
    center = center!!,
    userLocation = userLocation,
    userRole = userRole,
    hasLocationPermission = hasLocationPermission,
    stations = filteredStations ?: allStations!!,
    onStationSelect = onStationSelect,
)
}

// Floating Filter & Status Header

```

```

Column(
    modifier = Modifier
        .fillMaxWidth()
        .align(Alignment.TopCenter)
        .padding(12.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp),
) {
    // Main Top Bar Card
    Surface(
        shape = RoundedCornerShape(16.dp),
        color = StripeSurface,
        border = BorderStroke(1.dp, StripeBorder),
        shadowElevation = 6.dp,
        modifier = Modifier.fillMaxWidth(),
    ) {
        Row(
            modifier = Modifier.padding(horizontal = 14.dp, vertical =
10.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween,
        ) {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(10.dp),
                modifier = Modifier.weight(1f),
            ) {
                Icon(
                    imageVector = JouleIcons.MapPin,
                    contentDescription = null,
                    tint = StripePrimary,
                    modifier = Modifier.size(22.dp),
                )
                Column {
                    Row(
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement =
Arrangement.spacedBy(6.dp),
                    ) {
                        Text(
                            text = if (usingDeviceLocation) {
                                if (userRole == UserRole.GridOperator)
"Operator Location" else "Prosumer Location"
                            } else "All Microgrid Hubs",
                            style = MaterialTheme.typography.titleSmall,
                            fontWeight = FontWeight.Bold,
                            color = StripeInk,

```

```

        )
        Surface(
            shape = RoundedCornerShape(4.dp),
            color = if (userRole ==
UserRole.GridOperator) StripeBrandVioletSoft else StripeSurfaceAlt,
            border = BorderStroke(1.dp, if (userRole ==
UserRole.GridOperator) StripePrimary.copy(alpha = 0.4f) else StripeBorder),
        ) {
            Text(
                text = if (userRole ==
UserRole.GridOperator) "Operator" else "Prosumer",
                style =
MaterialTheme.typography.labelSmall,
                fontSize = 9.sp,
                fontWeight = FontWeight.Bold,
                color = if (userRole ==
UserRole.GridOperator) StripePrimary else StripeBody,
                modifier = Modifier.padding(horizontal =
5.dp, vertical = 1.dp),
            )
        }
    }
    Text(
        text = if (filteredStations != null &&
allStations != null) {
            "${filteredStations.size} of
${allStations!!.size} stations shown"
        } else {
            "Loading stations..."
        },
        style = MaterialTheme.typography.bodySmall,
        color = StripeMuted,
        fontSize = 11.sp,
    )
}

// Filter Expand Toggle Button
Surface(
    shape = RoundedCornerShape(50),
    color = if (isFilterExpanded || activeFilterCount > 0)
StripeBrandVioletSoft else StripeSurfaceAlt,
    border = BorderStroke(
        1.dp,
        if (activeFilterCount > 0) StripePrimary else
StripeBorder,

```

```

        ),
        modifier = Modifier.clickable { isFilterExpanded =
!isFilterExpanded },
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(horizontal = 12.dp,
vertical = 6.dp),
            horizontalArrangement = Arrangement.spacedBy(6.dp),
        ) {
            Icon(
                imageVector = JouleIcons.Filter,
                contentDescription = "Filter",
                tint = if (activeFilterCount > 0) StripePrimary
else StripeInk,
                modifier = Modifier.size(14.dp),
            )
            Text(
                text = if (activeFilterCount > 0) "Filter
($activeFilterCount)" else "Filter",
                style = MaterialTheme.typography.labelMedium,
                fontWeight = FontWeight.SemiBold,
                color = if (activeFilterCount > 0) StripePrimary
else StripeInk,
            )
        }
    }
}

// Expandable Filter Panel
AnimatedVisibility(
    visible = isFilterExpanded,
    enter = fadeIn() + expandVertically(),
    exit = fadeOut() + shrinkVertically(),
) {
    Surface(
        shape = RoundedCornerShape(16.dp),
        color = StripeSurface,
        border = BorderStroke(1.dp, StripeBorder),
        shadowElevation = 8.dp,
        modifier = Modifier.fillMaxWidth(),
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),

```



```

    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Text(
                text = "Filter Stations",
                style = MaterialTheme.typography.titleSmall,
                fontWeight = FontWeight.Bold,
                color = StripeInk,
            )
            if (activeFilterCount > 0) {
                TextButton(
                    onClick = {
                        searchQuery = ""
                        filterAvailableOnly = false
                        filterMinCapacityKWh = 0.0
                    },
                    contentPadding = PaddingValues(0.dp),
                ) {
                    Text("Reset all", style =
MaterialTheme.typography.labelSmall, color = StripePrimary)
                }
            }
        }

        // Search by name
        OutlinedTextField(
            value = searchQuery,
            onChange = { searchQuery = it },
            placeholder = { Text("Search station name or city...",
fontSize = 13.sp) },
            singleLine = true,
            leadingIcon = {
                Icon(JouleIcons.Search, contentDescription =
null, tint = StripeMuted, modifier = Modifier.size(16.dp))
            },
            shape = RoundedCornerShape(10.dp),
            colors = OutlinedTextFieldDefaults.colors(
                unfocusedBorderColor = StripeBorder,
                focusedBorderColor = StripePrimary,
                unfocusedContainerColor = StripeSurfaceAlt,
                focusedContainerColor = StripeSurfaceAlt,
            ),
            modifier = Modifier.fillMaxWidth(),

```

```

    )

    // Availability filter chips
    Column(verticalArrangement = Arrangement.spacedBy(6.dp))
{
    Text("Slot Availability", style =
MaterialTheme.typography.labelSmall, color = StripeMuted, fontWeight =
FontWeight.Medium)
    Row(horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
        FilterOptionChip(
            label = "All Stations",
            selected = !filterAvailableOnly,
            onSelect = { filterAvailableOnly = false },
        )
        FilterOptionChip(
            label = "Available Slots Only",
            selected = filterAvailableOnly,
            onSelect = { filterAvailableOnly = true },
        )
    }
}

    // Minimum Capacity filter chips
    Column(verticalArrangement = Arrangement.spacedBy(6.dp))
{
    Text("Minimum Capacity", style =
MaterialTheme.typography.labelSmall, color = StripeMuted, fontWeight =
FontWeight.Medium)
    Row(horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
        listOf(
            0.0 to "Any",
            50.0 to "≥ 50 kW",
            100.0 to "≥ 100 kW",
            200.0 to "≥ 200 kW",
        ).forEach { (cap, label) ->
            FilterOptionChip(
                label = label,
                selected = filterMinCapacityKWh == cap,
                onSelect = { filterMinCapacityKWh = cap
},
            )
        }
    }
}

```



```

= Modifier.fillMaxWidth(), ) { Column( modifier = Modifier.padding(horizontal = 24.dp, vertical = 32.dp),
horizontalAlignment = Alignment.CenterHorizontally, ) { IconTile(icon = JouleIcons.Bolt, size = 48.dp)
Text( text = "Couldn't load stations", style = MaterialTheme.typography.titleMedium, fontWeight =
FontWeight.SemiBold, color = StripeInk, modifier = Modifier.padding(top = 16.dp), ) Text( text =
message, style = MaterialTheme.typography.bodyMedium, color = StripeBody, textAlign =
TextAlign.Center, modifier = Modifier.padding(top = 6.dp), ) OutlinedButton(onClick = onRetry, modifier
= Modifier.padding(top = 16.dp)) { Text("Retry") } } } } }

```

```

@Composable private fun StationsMap( center: LatLng, userLocation: LatLng?, userRole: UserRole,
hasLocationPermission: Boolean, stations: List, onStationSelect: ((String) -> Unit)? = null, ) { val
cameraPositionState = rememberCameraPositionState { position =
CameraPosition.fromLatLngZoom(center, DEFAULT_ZOOM) }

```

```

GoogleMap(
    modifier = Modifier.fillMaxSize(),
    cameraPositionState = cameraPositionState,
    properties = MapProperties(
        isMyLocationEnabled = hasLocationPermission,
    ),
    uiSettings = MapUiSettings(
        zoomControlsEnabled = true,
        mapToolbarEnabled = true,
        myLocationButtonEnabled = hasLocationPermission,
    ),
) {
    // Render Role-Specific Current Location Marker
    if (userLocation != null) {
        val isOperator = userRole == UserRole.GridOperator
        val title = if (isOperator) "You (Grid Operator)" else "You
(Prosumer)"
        val snippet = if (isOperator) "Field Dispatch Unit Position" else
"Current Solar Prosumer Location"
        val hue = if (isOperator) BitmapDescriptorFactory.HUE_VIOLET else
BitmapDescriptorFactory.HUE_AZURE

        Marker(
            state = rememberMarkerState(position = userLocation),
            title = title,
            snippet = snippet,
            icon = BitmapDescriptorFactory.defaultMarker(hue),
            zIndex = 3.0f,
        )
    }

    // Render Microgrid Hub Station Markers
    stations.forEach { station ->

```

```

        val isAvailable = station.availableSlots > 0
        val stationHue = if (isAvailable) BitmapDescriptorFactory.HUE_GREEN
    else BitmapDescriptorFactory.HUE_RED

```

```

        MarkerInfoWindowContent(
            state = rememberMarkerState(
                position = LatLng(station.location.lat,
station.location.lng),
            ),
            title = station.name,
            icon = BitmapDescriptorFactory.defaultMarker(stationHue),
            onInfoWindowClick = {
                onStationSelect?.invoke(station.id)
            },
        ) {
            StationInfoWindow(
                station = station,
                onBookClick = if (onStationSelect != null) {
                    { onStationSelect(station.id) }
                } else null
            )
        }
    }
}

}

```

```

@Composable private fun StationInfoWindow(station: NearbyStation, onBookClick: (() -> Unit)? = null) {
    Surface( shape = RoundedCornerShape(16.dp), color = StripeSurface, shadowElevation = 8.dp, border =
    BorderStroke(1.dp, StripeBorder), modifier = Modifier.widthIn(min = 260.dp, max = 320.dp), ) {
        Column(modifier = Modifier.padding(14.dp)) { Row( modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.SpaceBetween, )
        { Text( text = station.name, style = MaterialTheme.typography.titleSmall, fontWeight = FontWeight.Bold,
        color = StripeInk, modifier = Modifier.weight(1f), ) Spacer(modifier = Modifier.width(8.dp)) Surface(
        shape = RoundedCornerShape(50), color = StripeBrandVioletSoft, ) { Text( text = "%.1f
        km".format(station.distanceKm), style = MaterialTheme.typography.labelSmall, color = StripePrimary,
        modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp), ) } }
    }
}

```

```

        Spacer(modifier = Modifier.height(10.dp))
    }
}

```

```

        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
        ) {
            InfoChip(label = "Capacity", value = "${station.capacityKWh}
        )
    }
}

```


ProsumerDashboardScreen.kt

```
package com.smartmicrogrid.ui.dashboard
```

```
import androidx.compose.animation.AnimatedVisibility import androidx.compose.foundation.Image
import androidx.compose.foundation.background import androidx.compose.foundation.border import
import androidx.compose.foundation.clickable import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ExperimentalLayoutApi import
import androidx.compose.foundation.layout.FlowRow import androidx.compose.foundation.layout.IntrinsicSize
import androidx.compose.foundation.layout.Row import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight import
import androidx.compose.foundation.layout.fillMaxSize import
import androidx.compose.foundation.layout.fillMaxWidth import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding import
import androidx.compose.foundation.layout.padding import androidx.compose.foundation.layout.size import
import androidx.compose.foundation.layout.width import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape import
import androidx.compose.foundation.shape.RoundedCornerShape import
import androidx.compose.foundation.verticalScroll import androidx.compose.material3.DatePicker import
import androidx.compose.material3.DatePickerDialog import
import androidx.compose.material3.ExperimentalMaterial3Api import androidx.compose.material3.Icon import
import androidx.compose.material3.MaterialTheme import androidx.compose.material3.ModalBottomSheet
import androidx.compose.material3.Text import androidx.compose.material3.TextButton import
import androidx.compose.material3.rememberDatePickerState import
import androidx.compose.material3.rememberModalBottomSheetState import
import androidx.compose.runtime.Composable import androidx.compose.runtime.LaunchedEffect import
import androidx.compose.runtime.getValue import androidx.compose.runtime.mutableIntStateOf import
import androidx.compose.runtime.mutableStateOf import androidx.compose.runtime.remember import
import androidx.compose.runtime.setValue import androidx.compose.ui.Alignment import
import androidx.compose.ui.Modifier import androidx.compose.ui.draw.clip import
import androidx.compose.ui.graphics.Brush import androidx.compose.ui.graphics.Color import
import androidx.compose.ui.platform.LocalContext import androidx.compose.ui.res.painterResource import
import androidx.compose.ui.semantics.Role import androidx.compose.ui.text.style.TextOverflow import
import androidx.compose.ui.unit.dp import com.smartmicrogrid.R import
import com.smartmicrogrid.data.local.AppDbHelper import
import com.smartmicrogrid.data.local.DashboardCacheDao import
import com.smartmicrogrid.data.local.ProsumerSessionDao import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.ReservationResponse import
import com.smartmicrogrid.ui.components.ControlShape import com.smartmicrogrid.ui.components.DateBlock
import com.smartmicrogrid.ui.components.EmptyState import
import com.smartmicrogrid.ui.components.ErrorBanner import com.smartmicrogrid.ui.components.Hairline
import com.smartmicrogrid.ui.components.JouleCard import
import com.smartmicrogrid.ui.components.JouleChip import
import com.smartmicrogrid.ui.components.JouleIconButton import
```

```

com.smartmicrogrid.ui.components.JouleIcons import
com.smartmicrogrid.ui.components.JoulePrimaryButton import
com.smartmicrogrid.ui.components.JouleSecondaryButton import
com.smartmicrogrid.ui.components.JouleTextField import
com.smartmicrogrid.ui.components.MetricTile import com.smartmicrogrid.ui.components.PillShape
import com.smartmicrogrid.ui.components.SectionHeader import
com.smartmicrogrid.ui.components.StatusPill import com.smartmicrogrid.ui.components.softShadow
import com.smartmicrogrid.ui.theme.StripeBody import com.smartmicrogrid.ui.theme.StripeBorder
import com.smartmicrogrid.ui.theme.StripeBrandVioletSoft import
com.smartmicrogrid.ui.theme.StripeCanvas import com.smartmicrogrid.ui.theme.StripeCyan import
com.smartmicrogrid.ui.theme.StripeCyanContainer import com.smartmicrogrid.ui.theme.StripeError
import com.smartmicrogrid.ui.theme.StripeErrorContainer import
com.smartmicrogrid.ui.theme.StripeInk import com.smartmicrogrid.ui.theme.StripeMuted import
com.smartmicrogrid.ui.theme.StripeOnErrorContainer import
com.smartmicrogrid.ui.theme.StripePrimary import com.smartmicrogrid.ui.theme.StripeSurface import
com.smartmicrogrid.ui.theme.StripeWarning import
com.smartmicrogrid.ui.theme.StripeWarningContainer import kotlinx.coroutines.Dispatchers import
kotlinx.coroutines.delay import kotlinx.coroutines.withContext import java.time.Duration import
java.time.Instant import java.time.LocalDate import java.time.LocalTime import java.time.ZoneId import
java.time.ZoneOffset import java.time.format.DateTimeFormatter import java.util.Locale

private val STATUS_FILTERS = listOf("All", "Confirmed", "Completed", "Cancelled") private val
DATE_INPUT_PATTERN = Regex("''''^\\d{4}-\\d{2}-\\d{2}\\$''''") private const val HISTORY_PREVIEW = 5

// The signature gradient stop from the JouleMark - used only as a thin accent stripe. private val
SignatureCyan = Color(0xFF11EFE3)

@OptIn(ExperimentalMaterial3Api::class, ExperimentalLayoutApi::class) @Composable fun
ProsumerDashboardScreen( onBookSlot: () -> Unit = {}, onOpenBookings: () -> Unit = {},
onNavigateToSignIn: () -> Unit = {}, ) { val context = LocalContext.current val api = remember {
ApiClient.service } val session = remember { ProsumerSessionDao(context).getSession() } val
prosumerNic = remember { session?.nic ?: FIXTURE_PROSUMER_NIC }

var summary by remember { mutableStateOf<ProsumerDashboardSummary?>(null) }
var loadError by remember { mutableStateOf<String?>(null) }
var refreshToken by remember { mutableIntStateOf(0) }
var refreshing by remember { mutableStateOf(false) }

var reservations by remember {
mutableStateOf<List<ReservationResponse>>(emptyList()) }
var reservationsError by remember { mutableStateOf<String?>(null) }
var stationNames by remember { mutableStateOf<Map<String,
String>>(emptyMap()) }

var statusFilter by remember { mutableStateOf("All") }

```



```

var stationFilter by remember { mutableStateOf("") }
var fromFilter by remember { mutableStateOf("") }
var toFilter by remember { mutableStateOf("") }
var showFilterSheet by remember { mutableStateOf(false) }
var showAllHistory by remember { mutableStateOf(false) }

// Cache-first, then refresh (see file header). A manual refresh skips the
// cache re-read.
LaunchedEffect(refreshToken) {
    loadError = null
    if (refreshToken == 0) {
        val cached = runCatching {
            withContext(Dispatchers.IO) {
                AppDbHelper(context).readableDatabase.use { db ->
                    DashboardCacheDao.read(db, prosumerNic) }
            }
        }.getOrNull()
        if (cached != null) summary = cached
    }

    runCatching { api.getProsumerDashboardSummary(prosumerNic) }
        .onSuccess { fresh ->
            summary = fresh
            runCatching {
                withContext(Dispatchers.IO) {
                    AppDbHelper(context).writableDatabase.use { db ->
                        DashboardCacheDao.write(db, prosumerNic, fresh) }
                }
            }
        }
        .onFailure { throwable ->
            if (throwable is retrofit2.HttpException && (throwable.code() ==
401 || throwable.code() == 403)) {
                onNavigateToSignIn()
            } else {
                loadError = if (summary == null) {
                    "Cannot connect to server. Check your network connection
or sign in again."
                } else {
                    "Couldn't refresh dashboard - showing last saved data."
                }
            }
        }
    refreshing = false
}

```

```

// Display-only lookup so rows show station names instead of raw ids; failure
just falls back
// to the short id, it never blocks the screen.
LaunchedEffect(refreshToken) {
    runCatching { api.getStations(search = null, status = null) }
        .onSuccess { list -> stationNames = list.associate { it.id to it.name } }
}
}

val fromDateValid = fromFilter.isBlank() ||
DATE_INPUT_PATTERN.matches(fromFilter)
val toDateValid = toFilter.isBlank() || DATE_INPUT_PATTERN.matches(toFilter)

// Debounced GET /reservations - "All" is a UI-only sentinel, sent as no
status param.
LaunchedEffect(statusFilter, stationFilter, fromFilter, toFilter,
refreshToken) {
    delay(300)
    reservationsError = null
    runCatching {
        api.searchReservations(
            nic = prosumerNic,
            stationId = stationFilter.ifBlank { null },
            status = statusFilter.takeIf { it != "All" },
            from = fromFilter.takeIf { fromDateValid && it.isNotBlank() },
            to = toFilter.takeIf { toDateValid && it.isNotBlank() },
        )
    }.onSuccess { reservations = it }
        .onFailure { throwable ->
            if (throwable is retrofit2.HttpException && (throwable.code() ==
401 || throwable.code() == 403)) {
                onNavigateToSignIn()
            } else {
                reservationsError = "Couldn't load bookings. Pull down to try
again."
            }
        }
}

val now = Instant.now().toString()
val current = reservations.filter { it.status == "Confirmed" }.sortedBy {
it.scheduledAt }
val needsCheckIn = current.filter { it.scheduledAt <= now }
val upcoming = current.filter { it.scheduledAt > now }
val history = reservations.filter { it.status == "Completed" || it.status ==
"Cancelled" }

```

```

        .sortedByDescending { it.scheduledAt }
val nextBooking = upcoming.firstOrNull() ?: needsCheckIn.firstOrNull()
val advancedFilterCount = listOf(stationFilter, fromFilter, toFilter).count {
    it.isNotBlank() }

fun nameFor(stationId: String) = stationNames[stationId] ?: "Station
${stationId.takeLast(6)}"

// If there is no cached or live data and a connection/network error
occurred, show a prominent recovery screen
if (summary == null && loadError != null) {
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(StripeCanvas)
            .padding(24.dp),
        contentAlignment = Alignment.Center,
    ) {
        JouleCard(modifier = Modifier.fillMaxWidth()) {
            Column(
                modifier = Modifier.padding(24.dp),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.spacedBy(14.dp),
            ) {
                Box(
                    modifier = Modifier
                        .size(54.dp)
                        .clip(CircleShape)
                        .background(StripeErrorContainer),
                    contentAlignment = Alignment.Center,
                ) {
                    Icon(
                        JouleIcons.Pulse,
                        contentDescription = null,
                        tint = StripeError,
                        modifier = Modifier.size(26.dp),
                    )
                }
                Text(
                    text = "Connection Error",
                    style = MaterialTheme.typography.titleMedium,
                    color = StripeInk,
                    fontWeight =
androidx.compose.ui.text.font.FontWeight.Bold,
                )
                Text(

```

```

        text = "Unable to reach the smart grid server. Check your
network connection (file sharing / Wi-Fi / USB tethering) or return to sign
in.",
        style = MaterialTheme.typography.bodyMedium,
        color = StripeBody,
        textAlign =
androidx.compose.ui.text.style.TextAlign.Center,
    )
    Spacer(Modifier.height(6.dp))
    JoulePrimaryButton(
        text = "Go to Sign In",
        onClick = onNavigateToSignIn,
        modifier = Modifier.fillMaxWidth(),
    )
    JouleSecondaryButton(
        text = "Retry Connection",
        onClick = { refreshing = true; refreshToken++ },
        modifier = Modifier.fillMaxWidth(),
    )
}
}
}
return
}

```

```

Box(modifier = Modifier.fillMaxSize().background(StripeCanvas)) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(horizontal = 20.dp)
            .padding(top = 12.dp, bottom = 28.dp),
        verticalArrangement = Arrangement.spacedBy(20.dp),
    ) {
        GreetingHeader(
            name = session?.fullName,
            onRefresh = { refreshing = true; refreshToken++ },
        )

        loadError?.let {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .clip(ControlShape)
                    .background(StripeErrorContainer)
                    .border(1.dp, StripeError.copy(alpha = 0.2f),

```

```

ControlShape)
                .padding(start = 14.dp, end = 8.dp, top = 8.dp, bottom =
8.dp),
                verticalAlignment = Alignment.CenterVertically,
            ) {
                Text(
                    it,
                    style = MaterialTheme.typography.bodySmall,
                    color = StripeOnErrorContainer,
                    modifier = Modifier.weight(1f),
                )
                TextButton(onClick = onNavigateToSignIn) {
                    Text("Sign In", color = StripeError, style =
MaterialTheme.typography.labelSmall)
                }
            }
        }
        reservationsError?.let { ErrorBanner(message = it, onDismiss = {
reservationsError = null }) }

        NextBookingHero(
            booking = nextBooking,
            stationName = nextBooking?.let { nameFor(it.stationId) },
            onBookSlot = onBookSlot,
            onOpenBookings = onOpenBookings,
        )

        Row(
            modifier = Modifier
                .fillMaxWidth()
                .height(IntrinsicSize.Max),
            horizontalArrangement = Arrangement.spacedBy(10.dp),
        ) {
            MetricTile(
                label = "Active",
                value = summary?.activeCount,
                icon = JouleIcons.Pulse,
                accent = StripePrimary,
                accentSoft = StripeBrandVioletSoft,
                modifier = Modifier
                    .weight(1f)
                    .fillMaxHeight(),
            )
            MetricTile(
                label = "Needs check-in",
                value = summary?.pendingCount,
            )
        }
    }
}

```

```

        icon = JouleIcons.Bolt,
        accent = StripeWarning,
        accentSoft = StripeWarningContainer,
        modifier = Modifier
            .weight(1f)
            .fillMaxHeight(),
    )
    MetricTile(
        label = "Upcoming",
        value = summary?.approvedFutureCount,
        icon = JouleIcons.Calendar,
        accent = StripeCyan,
        accentSoft = StripeCyanContainer,
        modifier = Modifier
            .weight(1f)
            .fillMaxHeight(),
    )
}

// Filters: status chips always visible; station/date live in a
bottom sheet.
Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
    SectionHeader(title = "Your bookings", subtitle = "Filter by
status, station or date")
    FlowRow(
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp),
        modifier = Modifier.fillMaxWidth(),
    ) {
        STATUS_FILTERS.forEach { status ->
            JouleChip(label = status, selected = statusFilter ==
status, onClick = { statusFilter = status })
        }
        FiltersChip(count = advancedFilterCount, onClick = {
showFilterSheet = true })
    }
    if (advancedFilterCount > 0) {
        ActiveFilterSummary(
            station = stationFilter,
            from = fromFilter,
            to = toFilter,
            onClear = { stationFilter = ""; fromFilter = ""; toFilter
= "" },
        )
    }
}
}

```

```

AnimatedVisibility(visible = needsCheckIn.isNotEmpty()) {
    BookingGroup(
        title = "Needs check-in",
        subtitle = "Past start time - show your QR at the hub",
        items = needsCheckIn,
        nameFor = ::nameFor,
        highlightFirst = true,
    )
}

BookingGroup(
    title = "Upcoming",
    subtitle = null,
    items = upcoming,
    nameFor = ::nameFor,
    emptyTitle = "Nothing scheduled",
    emptyBody = "Confirmed bookings that match your filters appear
here.",
    onEmptyAction = onBookSlot,
    emptyActionLabel = "Book a slot",
)

BookingGroup(
    title = "History",
    subtitle = null,
    items = if (showAllHistory) history else
history.take(HISTORY_PREVIEW),
    nameFor = ::nameFor,
    emptyTitle = "No history yet",
    emptyBody = "Completed and cancelled bookings will show up
here.",
    actionLabel = when {
        history.size <= HISTORY_PREVIEW -> null
        showAllHistory -> "Show less"
        else -> "Show all ${history.size}"
    },
    onAction = { showAllHistory = !showAllHistory },
)
}
}

if (showFilterSheet) {
    FilterSheet(
        station = stationFilter,
        onStationChange = { stationFilter = it },

```

```

        from = fromFilter,
        onFromChange = { fromFilter = it },
        to = toFilter,
        onToChange = { toFilter = it },
        onReset = { stationFilter = ""; fromFilter = ""; toFilter = "" },
        onDismiss = { showFilterSheet = false },
    )
}

}

// ----- Header -----

private val EYEBROW_FORMAT = DateTimeFormatter.ofPattern("EEE, d MMM", Locale.ENGLISH)

@Composable private fun GreetingHeader(name: String?, onRefresh: () -> Unit) { val greeting = when
(LocalTime.now().hour) { in 5..11 -> "Good morning" in 12..16 -> "Good afternoon" else -> "Good
evening" } val firstName = name?.trim()?.substringBefore(' ')??.takeIf { it.isNotBlank() }
Row(verticalAlignment = Alignment.CenterVertically) { Column(modifier = Modifier.weight(1f)) { Text(
text = "${LocalDate.now().format(EYEBROW_FORMAT)} · Prosumer".uppercase(), style =
MaterialTheme.typography.labelSmall, color = StripeMuted, ) Spacer(Modifier.height(4.dp)) Text( text =
if (firstName != null) "$greeting, $firstName" else greeting, style =
MaterialTheme.typography.headlineMedium, color = StripeInk, maxLines = 2, ) } JouleIconButton(icon =
JouleIcons.Refresh, contentDescription = "Refresh dashboard", onClick = onRefresh) } }

// ----- Hero -----

private val HERO_DATE_FORMAT: DateTimeFormatter = DateTimeFormatter.ofPattern("EEE, d MMM ·
HH:mm", Locale.ENGLISH).withZone(ZoneId.systemDefault())

private fun relativeLabel(iso: String): String { val instant = runCatching { Instant.parse(iso) }.getOrNull() ?:
return "" val minutes = Duration.between(Instant.now(), instant).toMinutes() return when { minutes < 0
-> "Check-in due" minutes < 60 -> "In ${minutes}m" minutes < 60 * 24 -> "In ${minutes / 60}h" else -> "In
${minutes / (60 * 24)}d" } }

@Composable private fun NextBookingHero( booking: ReservationResponse?, stationName: String?,
onBookSlot: () -> Unit, onOpenBookings: () -> Unit, ) { Box( modifier = Modifier .fillMaxWidth()
.softShadow(RoundedCornerShape(20.dp), 14.dp) .clip(RoundedCornerShape(20.dp))
.background(StripeInk), ) { // Thin signature stripe along the top edge - a narrow accent, never a page-
wide gradient. Box( Modifier .fillMaxWidth() .height(3.dp)
.background(Brush.horizontalGradient(listOf(StripePrimary, SignatureCyan))), ) Image( painter =
painterResource(R.drawable.home), contentDescription = null, modifier = Modifier
.align(Alignment.TopEnd) .padding(top = 18.dp, end = 12.dp) .size(104.dp), ) Column(modifier =
Modifier.padding(start = 20.dp, end = 20.dp, top = 22.dp, bottom = 20.dp)) { Text( text = if (booking !=
null) "NEXT BOOKING" else "NO UPCOMING BOOKINGS", style = MaterialTheme.typography.labelSmall,

```



```

color = Color.White.copy(alpha = 0.6f), ) Spacer(Modifier.height(8.dp)) if (booking != null) { Text( text =
stationName.orEmpty(), style = MaterialTheme.typography.titleLarge, color = Color.White, maxLines = 2,
overflow = TextOverflow.Ellipsis, modifier = Modifier.fillMaxWidth(0.62f), )
Spacer(Modifier.height(10.dp)) Row(verticalAlignment = Alignment.CenterVertically) {
Icon(JouleIcons.Clock, contentDescription = null, tint = Color.White.copy(alpha = 0.7f), modifier =
Modifier.size(15.dp)) Spacer(Modifier.width(6.dp)) Text( text = runCatching {
HERO_DATE_FORMAT.format(Instant.parse(booking.scheduledAt))
}.getOrElse(booking.scheduledAt), style = MaterialTheme.typography.bodyMedium, color =
Color.White.copy(alpha = 0.85f), ) } Spacer(Modifier.height(10.dp)) Box( modifier = Modifier
.clip(PillShape) .background(Color.White.copy(alpha = 0.12f)) .padding(horizontal = 10.dp, vertical =
5.dp), ) { Text(relativeLabel(booking.scheduledAt), style = MaterialTheme.typography.labelSmall, color =
SignatureCyan) } Spacer(Modifier.height(18.dp)) OnDarkButton(text = "View bookings", onClick =
onOpenBookings) } else { Text( text = "Reserve a battery slot at a nearby hub", style =
MaterialTheme.typography.titleLarge, color = Color.White, modifier = Modifier.fillMaxWidth(0.62f), )
Spacer(Modifier.height(18.dp)) JoulePrimaryButton(text = "Book a slot", onClick = onBookSlot,
leadingIcon = JouleIcons.Plus, modifier = Modifier.height(46.dp)) } } }

```

```

@Composable private fun OnDarkButton(text: String, onClick: () -> Unit) { Row( modifier = Modifier
.clip(PillShape) .background(Color.White) .clickable(role = Role.Button, onClick = onClick)
.padding(horizontal = 18.dp, vertical = 12.dp), verticalAlignment = Alignment.CenterVertically, ) {
Text(text, style = MaterialTheme.typography.labelLarge, color = StripeInk) Spacer(Modifier.width(6.dp))
Icon(JouleIcons.ChevronRight, contentDescription = null, tint = StripeInk, modifier =
Modifier.size(16.dp)) } }

```

// ----- Filters -----

```

@Composable private fun FiltersChip(count: Int, onClick: () -> Unit) { Row( modifier = Modifier
.clip(PillShape) .background(if (count > 0) StripeBrandVioletSoft else StripeSurface) .border(1.dp, if
(count > 0) StripePrimary.copy(alpha = 0.35f) else StripeBorder, PillShape) .clickable(role = Role.Button,
onClick = onClick) .padding(horizontal = 12.dp, vertical = 8.dp), verticalAlignment =
Alignment.CenterVertically, ) { Icon(JouleIcons.Sliders, contentDescription = null, tint = StripePrimary,
modifier = Modifier.size(16.dp)) Spacer(Modifier.width(6.dp)) Text(if (count > 0) "Filters · $count" else
"Filters", style = MaterialTheme.typography.labelMedium, color = StripePrimary) } }

```

```

@Composable private fun ActiveFilterSummary(station: String, from: String, to: String, onClear: () ->
Unit) { val parts = buildList { if (station.isNotBlank()) add("Station \"$station\"") if (from.isNotBlank() ||
to.isNotBlank()) add("${from.ifBlank { \"...\" }} → ${to.ifBlank { \"...\" }}") } Row(verticalAlignment =
Alignment.CenterVertically) { Text(parts.joinToString(" · "), style =
MaterialTheme.typography.bodySmall, color = StripeBody, modifier = Modifier.weight(1f))
TextButton(onClick = onClear) { Text("Clear", color = StripePrimary, style =
MaterialTheme.typography.labelMedium) } } }

```

```

private val ISO_DATE: DateTimeFormatter = DateTimeFormatter.ISO_LOCAL_DATE

```

```

@OptIn(ExperimentalMaterial3Api::class) @Composable private fun FilterSheet( station: String,
onStationChange: (String) -> Unit, from: String, onFromChange: (String) -> Unit, to: String, onToChange:
(String) -> Unit, onReset: () -> Unit, onDismiss: () -> Unit, ) { val sheetState =
rememberModalBottomSheetState(skipPartiallyExpanded = true) var pickingFrom by remember {
mutableStateOf(false) } var pickingTo by remember { mutableStateOf(false) }

ModalBottomSheet(onDismissRequest = onDismiss, sheetState = sheetState,
containerColor = StripeSurface) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .navigationBarsPadding()
            .padding(horizontal = 20.dp)
            .padding(bottom = 20.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
    ) {
        Text("Filter bookings", style = MaterialTheme.typography.titleLarge,
color = StripeInk)
        JouleTextField(
            value = station,
            onValueChange = onStationChange,
            label = "Station ID",
            placeholder = "Contains...",
            leadingIcon = JouleIcons.Search,
        )
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            DateField(label = "From", value = from, onClick = { pickingFrom =
true }, modifier = Modifier.weight(1f))
            DateField(label = "To", value = to, onClick = { pickingTo = true
}, modifier = Modifier.weight(1f))
        }
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            JouleSecondaryButton(text = "Reset", onClick = onReset, modifier
= Modifier.weight(1f))
            JoulePrimaryButton(text = "Done", onClick = onDismiss, modifier =
Modifier.weight(1f))
        }
    }
}

if (pickingFrom) DatePick(initial = from, onPicked = { onFromChange(it);
pickingFrom = false }, onDismiss = { pickingFrom = false })
if (pickingTo) DatePick(initial = to, onPicked = { onToChange(it); pickingTo
= false }, onDismiss = { pickingTo = false })

```

```
}
```

```
@Composable private fun DateField(label: String, value: String, onClick: () -> Unit, modifier: Modifier = Modifier) { Column(modifier = modifier) { Text(label, style = MaterialTheme.typography.labelMedium, color = StripeInk) Spacer(Modifier.height(6.dp)) Row( modifier = Modifier.fillMaxWidth().height(52.dp).clip(RoundedCornerShape(12.dp)).border(1.dp, StripeBorder, RoundedCornerShape(12.dp)).clickable(role = Role.Button, onClick = onClick).padding(horizontal = 14.dp), verticalAlignment = Alignment.CenterVertically, ) { Icon(JouleIcons.Calendar, contentDescription = null, tint = StripeMuted, modifier = Modifier.size(18.dp)) Spacer(Modifier.width(10.dp)) Text(value.ifBlank { "Any date" }, style = MaterialTheme.typography.bodyMedium, color = if (value.isBlank()) StripeMuted else StripeInk) } } }
```

```
@OptIn(ExperimentalMaterial3Api::class) @Composable private fun DatePick(initial: String, onPicked: (String) -> Unit, onDismiss: () -> Unit) { val initialMillis = runCatching { LocalDate.parse(initial).atStartOfDay(ZoneOffset.UTC).toInstant().toEpochMilli() }.getOrNull() val state = rememberDatePickerState(initialSelectedDateMillis = initialMillis) DatePickerDialog( onDismissRequest = onDismiss, confirmButton = { TextButton(onClick = { state.selectedDateMillis?.let { onPicked(Instant.ofEpochMilli(it).atZone(ZoneOffset.UTC).toLocalDate().format(ISO_DATE)) } } : onDismiss() ) } { Text("Apply", color = StripePrimary) } }, dismissButton = { TextButton(onClick = onDismiss) { Text("Cancel", color = StripeBody) } }, ) { DatePicker(state = state) } }
```

```
// ----- Booking lists -----
```

```
private val ROW_TIME_FORMAT: DateTimeFormatter = DateTimeFormatter.ofPattern("EEE · HH:mm", Locale.ENGLISH).withZone(ZoneId.systemDefault())
```

```
@Composable private fun BookingGroup( title: String, subtitle: String?, items: List, nameFor: (String) -> String, highlightFirst: Boolean = false, emptyTitle: String = "", emptyBody: String = "", emptyActionLabel: String? = null, onEmptyAction: (() -> Unit)? = null, actionLabel: String? = null, onAction: (() -> Unit)? = null, ) { Column(verticalArrangement = Arrangement.spacedBy(10.dp)) { SectionHeader( title = if (items.isNotEmpty()) "$title · ${items.size}" else title, subtitle = subtitle, actionLabel = actionLabel, onAction = onAction, ) JouleCard(contentPadding = androidx.compose.foundation.layout.PaddingValues(horizontal = 16.dp, vertical = 4.dp)) { if (items.isEmpty()) { EmptyState( icon = JouleIcons.Calendar, title = emptyTitle, body = emptyBody, actionLabel = emptyActionLabel, onAction = onEmptyAction, ) } else { items.forEachIndexed { index, reservation -> if (index > 0) Hairline() BookingRow(reservation, nameFor(reservation.stationId), highlight = highlightFirst && index == 0) } } } }
```

```
@Composable private fun BookingRow(reservation: ReservationResponse, stationName: String, highlight: Boolean) { Row( modifier = Modifier.fillMaxWidth().padding(vertical = 12.dp), verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(14.dp), ) { DateBlock(iso = reservation.scheduledAt, highlight = highlight) Column(modifier = Modifier.weight(1f)) { Text(stationName, style = MaterialTheme.typography.titleSmall, color = StripeInk, maxLines = 1, overflow = TextOverflow.Ellipsis) Spacer(Modifier.height(2.dp)) Text( text = "${runCatching { ROW_TIME_FORMAT.format(Instant.parse(reservation.scheduledAt)) }.getOrDefault("")} · Slot
```

```
 ${reservation.slotId.takeLast(4)}", style = MaterialTheme.typography.bodySmall, color = StripeMuted,
 maxLines = 1, overflow = TextOverflow.Ellipsis, ) } StatusPill(reservation.status) } }
```

OnboardingActivity.kt

```
package com.smartmicrogrid.ui.onboarding
```

```
import android.content.Intent import android.os.Bundle import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent import androidx.activity.enableEdgeToEdge import
androidx.compose.animation.core.animateDpAsState import androidx.compose.foundation.Image
import androidx.compose.foundation.background import
androidx.compose.foundation.layout.Arrangement import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize import
androidx.compose.foundation.layout.fillMaxWidth import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding import
androidx.compose.foundation.layout.safeDrawingPadding import
androidx.compose.foundation.layout.width import
androidx.compose.foundation.pager.HorizontalPager import
androidx.compose.foundation.pager.rememberPagerState import
androidx.compose.foundation.rememberScrollState import
androidx.compose.foundation.shape.RoundedCornerShape import
androidx.compose.foundation.verticalScroll import androidx.compose.material3.Button import
androidx.compose.material3.ButtonDefaults import androidx.compose.material3.MaterialTheme import
androidx.compose.material3.Text import androidx.compose.material3.TextButton import
androidx.compose.runtime.Composable import androidx.compose.runtime.getValue import
androidx.compose.runtime.rememberCoroutineScope import androidx.compose.ui.Alignment import
androidx.compose.ui.Modifier import androidx.compose.ui.draw.clip import
androidx.compose.ui.graphics.Brush import androidx.compose.ui.graphics.Color import
androidx.compose.ui.layout.ContentScale import androidx.compose.ui.res.painterResource import
androidx.compose.ui.text.font.FontWeight import androidx.compose.ui.text.style.TextAlign import
androidx.compose.ui.unit.dp import com.smartmicrogrid.R import com.smartmicrogrid.MainActivity
import com.smartmicrogrid.ui.components.JouleMark import
com.smartmicrogrid.ui.theme.SmartMicrogridTheme import com.smartmicrogrid.ui.theme.StripeInk
import kotlinx.coroutines.launch
```

```
// One onboarding slide's background photo (null = brand-gradient background instead) and copy.
private data class OnboardingSlide(val imageRes: Int?, val eyebrow: String, val title: String, val body:
String)
```

```
private val SLIDES = listOf( OnboardingSlide( imageRes = R.drawable.onboarding_solar, eyebrow =
"Discover", title = "Find microgrid hubs near you", body = "Browse solar energy hubs on the map, each
with live battery slot availability.", ), OnboardingSlide( imageRes = R.drawable.onboarding_battery,
eyebrow = "Reserve", title = "Reserve an energy slot", body = "Book a charging or discharging slot in a
few taps, within a simple 7-day window.", ), OnboardingSlide( imageRes = null, eyebrow = "Track", title =
```

"Track every transfer", body = "Get a QR code for each booking and follow your energy transfer history from your dashboard.",),)

```
class OnboardingActivity : ComponentActivity() {

    // Shows the swipeable slide flow; both Skip and "Get started" mark
    onboarding as shown.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            OnboardingScreen(onFinished = ::finishOnboarding)
        }
    }

    // Persists that onboarding has been shown, then routes into the app and
    clears this
    // activity (and Splash) from the back stack.
    private fun finishOnboarding() {
        OnboardingPreferences.markOnboardingShown(this)
        startActivity(Intent(this, MainActivity::class.java))
        finish()
    }

}

// Renders the full-bleed background for the active slide, the brand row + Skip, the slide pager, // the pill
indicator, and Next/Get started - all overlaid on top of that background rather than // confined to a plain
surface, so the photo/gradient reaches every edge of the screen. Still uses // SmartMicrogridTheme for
the shared type scale, but every color on this screen is set explicitly // (white text, ink button content)
rather than theme-relative - this screen is intentionally // always dark-on-photo regardless of the
device's light/dark setting. @Composable private fun OnboardingScreen(onFinished: () -> Unit) {
    SmartMicrogridTheme { val pagerState = rememberPagerState(pageCount = { SLIDES.size }) val
    coroutineScope = rememberCoroutineScope() val isLastPage = pagerState.currentPage ==
    SLIDES.lastIndex

    Box(modifier = Modifier.fillMaxSize()) {
        SlideBackground(slide = SLIDES[pagerState.currentPage])

        Column(
            modifier = Modifier
                .fillMaxSize()
                .safeDrawingPadding(),
        ) {
            Row(
```

```

        modifier = Modifier
            .fillMaxWidth()
            .padding(start = 20.dp, end = 12.dp, top = 8.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        JouleMark(modifier = Modifier.width(26.dp), monochrome =
Color.White)
        Text(
            text = "Joule",
            style = MaterialTheme.typography.titleMedium,
            fontWeight = FontWeight.SemiBold,
            color = Color.White,
            modifier = Modifier
                .padding(start = 8.dp)
                .weight(1f),
        )
        TextButton(onClick = onFinish, colors =
ButtonDefaults.textButtonColors(contentColor = Color.White)) {
            Text("Skip")
        }
    }

    HorizontalPager(
        state = pagerState,
        modifier = Modifier.weight(1f),
    ) { page ->
        OnboardingSlideContent(SLIDES[page])
    }

    PageIndicator(
        pageCount = SLIDES.size,
        currentPage = pagerState.currentPage,
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 20.dp),
    )

    Button(
        onClick = {
            if (isLastPage) {
                onFinish()
            } else {
                coroutineScope.launch {
                    pagerState.animateScrollToPage(pagerState.currentPage
+ 1)
                }
            }
        }
    )

```

```

        },
        colors = ButtonDefaults.buttonColors(containerColor =
Color.White, contentColor = StripeInk),
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 24.dp)
            .padding(bottom = 20.dp)
            .height(52.dp),
    ) {
        Text(if (isLastPage) "Get started" else "Next", style =
MaterialTheme.typography.labelLarge)
    }
}

}

}

```

```

// The active slide's full-screen backdrop: a photo with a bottom-heavy dark scrim (so white text // stays
legible without hiding the image), or the Joule brand gradient when there's no photo. @Composable
private fun SlideBackground(slide: OnboardingSlide) { if (slide.imageRes != null) { Image( painter =
painterResource(slide.imageRes), contentDescription = null, contentScale = ContentScale.Crop, modifier =
Modifier.fillMaxSize(), ) Box( modifier = Modifier.fillMaxSize() .background( Brush.verticalGradient( Of
to StripeInk.copy(alpha = 0.35f), 0.55f to StripeInk.copy(alpha = 0.75f), 1f to StripeInk.copy(alpha =
0.96f), ), ), ) } else { Box( modifier = Modifier.fillMaxSize()
.background(Brush.linearGradient(listOf(Color(0xFF533AFD), Color(0xFF2E2B8C), Color(0xFF11EFE3))))), )
}
}

```

```

// Renders a single slide's eyebrow, title and body, anchored toward the bottom of the pager area //
(over the scrim) rather than centered - a Stories-style layout that keeps the photo's upper // two-thirds
uncluttered. Scrollable so long body copy on a short/landscape screen clips into a // scroll instead of
overflowing off-screen. @Composable private fun OnboardingSlideContent(slide: OnboardingSlide) {
Column( modifier = Modifier.fillMaxSize() .verticalScroll(rememberScrollState()) .padding(horizontal =
28.dp), verticalArrangement = Arrangement.Bottom, horizontalAlignment = Alignment.Start, ) { Text(
text = slide.eyebrow.uppercase(), style = MaterialTheme.typography.labelMedium, color =
Color(0xFF9AF2EC), modifier = Modifier.padding(bottom = 10.dp), ) Text( text = slide.title, style =
MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.SemiBold, color = Color.White,
textAlign = TextAlign.Start, ) Text( text = slide.body, style = MaterialTheme.typography.bodyLarge, color
= Color.White.copy(alpha = 0.8f), textAlign = TextAlign.Start, modifier = Modifier.padding(top = 10.dp,
bottom = 24.dp), ) }
}

```

```

// Renders pager progress: the current page is an elongated pill, the rest are small dots. @Composable
private fun PageIndicator(pageCount: Int, currentPage: Int, modifier: Modifier = Modifier) { Row(
modifier = modifier.padding(horizontal = 28.dp), horizontalArrangement = Arrangement.Start, ) {
repeat(pageCount) { index -> val isSelected = index == currentPage val width by

```

```
animateDpAsState(targetValue = if (isSelected) 22.dp else 6.dp, label = "indicatorWidth") Box( modifier =
Modifier .padding(end = 6.dp) .height(6.dp) .width(width) .clip(RoundedCornerShape(50))
.background(if (isSelected) Color.White else Color.White.copy(alpha = 0.35f)), ) } } }
```

SplashActivity.kt

```
package com.smartmicrogrid.ui.onboarding
```

```
import android.content.Intent import android.os.Bundle import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent import androidx.activity.enableEdgeToEdge import
androidx.compose.foundation.layout.Box import androidx.compose.foundation.layout.Column import
androidx.compose.foundation.layout.fillMaxSize import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.safeDrawingPadding import
androidx.compose.foundation.layout.width import androidx.compose.material3.MaterialTheme import
androidx.compose.material3.Surface import androidx.compose.material3.Text import
androidx.compose.runtime.Composable import androidx.compose.runtime.LaunchedEffect import
androidx.compose.ui.Alignment import androidx.compose.ui.Modifier import
androidx.compose.ui.graphics.Color import androidx.compose.ui.text.font.FontWeight import
androidx.compose.ui.unit.dp import com.smartmicrogrid.MainActivity import
com.smartmicrogrid.data.local.ProsumerSessionDao import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.ui.auth.StaffSessionPreferences import
com.smartmicrogrid.ui.components.JouleMark import com.smartmicrogrid.ui.home.HomeActivity
import com.smartmicrogrid.ui.home.UserRole import com.smartmicrogrid.ui.theme.StripeInk import
com.smartmicrogrid.ui.theme.SmartMicrogridTheme import kotlinx.coroutines.delay
```

```
private const val SPLASH_DELAY_MS = 900L
```

```
class SplashActivity : ComponentActivity() {
```

```
// Shows the splash content, waits briefly, then routes to onboarding,
straight into the app
```

```
// (if a session is already saved), or Sign In.
```

```
override fun onCreate(savedInstanceState: Bundle?) {
```

```
    super.onCreate(savedInstanceState)
```

```
    enableEdgeToEdge()
```

```
    setContent {
```

```
        SmartMicrogridTheme {
```

```
            SplashScreenContent(onTimeout = ::navigateNext)
```

```
        }
```

```
    }
```

```
}
```

```
// Decides the next screen: onboarding first, then an existing session (skip
Sign In
```

```
// entirely), then Sign In. Finishes this activity so it never sits in the
back stack.
```



```

private fun navigateNext() {
    if (!OnboardingPreferences.hasSeenOnboarding(this)) {
        startActivity(Intent(this, OnboardingActivity::class.java))
        finish()
        return
    }

    val prosumerSession = ProsumerSessionDao(this).getSession()
    val staffSession = StaffSessionPreferences.read(this)
    val intent = when {
        prosumerSession != null -> {
            ApiClient.authToken = prosumerSession.token
            HomeActivity.intentFor(this, UserRole.Prosumer)
        }
        staffSession != null -> {
            ApiClient.authToken = staffSession.token
            HomeActivity.intentFor(this,
UserRole.fromClaimValue(staffSession.role))
        }
        else -> Intent(this, MainActivity::class.java)
    }
    startActivity(intent)
    finish()
}

}

```

```

// Renders the branded splash UI and fires onTimeout once after a short delay. @Composable private
fun SplashScreenContent(onTimeout: () -> Unit) { LaunchedEffect(Unit) { delay(SPLASH_DELAY_MS)
onTimeout() } Surface(modifier = Modifier.fillMaxSize(), color = Stripelink) { // enableEdgeToEdge() draws
this Surface behind the system status/nav bars - safeDrawingPadding() // keeps the wordmark clear of
them instead of risking it sitting under a status bar cutout. Box( modifier = Modifier .fillMaxSize()
.safeDrawingPadding(), contentAlignment = Alignment.Center, ) { Column(horizontalAlignment =
Alignment.CenterHorizontally) { JouleMark(modifier = Modifier.width(96.dp)) Text( text = "Joule", color
= Color.White, style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.SemiBold,
modifier = Modifier.padding(top = 20.dp), ) Text( text = "Power, exchanged precisely.", color =
Color.White.copy(alpha = 0.6f), style = MaterialTheme.typography.bodyMedium, modifier =
Modifier.padding(top = 6.dp), ) } } } }

```

5.2 A.G.L.M Wijesinghe (IT23264670)

5.2.1 Description

Overall Contribution:

I was responsible for developing the core User Management features for the Web Application and the complete Prosumer Reservation & QR Dispatch lifecycle for the Mobile Application.

- **Web Application:** I implemented the full CRUD (Create, Read, Update, Deactivate) lifecycle for system staff (Backoffice and Grid Operators). I enhanced the User Management page by adding real-time search and filtering (by Role and Status), and ensured Prosumer data is strictly excluded from this view. I also resolved a critical MongoDB serialization crash (FormatException) by implementing schema flexibility in the backend models
- **Mobile Application:** I built the end-to-end booking flow for Prosumers. This includes integrating the Retrofit API to fetch and manage reservations, building a polished Jetpack Compose UI for viewing and creating bookings (with a 7-day scheduling window), and implementing a custom QrCodeHelper utility using the ZXing library to securely generate and display transaction QR codes. Additionally, I managed the mobile build configurations (resolving AGP/Gradle version conflicts) and handled complex Git merges to integrate team updates.

5.2.2 Code Files

WebService (Backend)

SmartMicrogrid.Api/Models/User.cs

```
// =====  
// File: User.cs  
// Purpose: Defines the MongoDB document structure for system users  
//          (Backoffice and Grid Operators).  
// Author: Migara  
// =====  
  
using MongoDB.Bson;  
using MongoDB.Bson.Serialization.Attributes;  
  
namespace SmartMicrogrid.Api.Models  
{  
    // [BsonIgnoreExtraElements] tells MongoDB to ignore fields like 'nic'  
    // that might exist in the database but aren't in this C# class, preventing crashes.  
    [BsonIgnoreExtraElements]  
    public class User  
    {  
        [BsonId]  
        [BsonRepresentation(BsonType.ObjectId)]  
        public string Id { get; set; } = null!;  
  
        [BsonElement("role")]  
        public string Role { get; set; } = null!;  
    }  
}
```

```

        [BsonElement("username")]
        public string Username { get; set; } = null!;

        [BsonElement("passwordHash")]
        public string PasswordHash { get; set; } = null!;

        [BsonElement("fullName")]
        public string FullName { get; set; } = null!;

        [BsonElement("email")]
        public string Email { get; set; } = null!;

        [BsonElement("status")]
        public string Status { get; set; } = null!;

        [BsonElement("createdAt")]
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        [BsonElement("updatedAt")]
        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    }

    // DTO for creating a user
    public class CreateUserDto
    {
        public string Username { get; set; } = null!;
        public string Password { get; set; } = null!;
        public string Role { get; set; } = null!;
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
    }

    // DTO for updating a user
    public class UpdateUserDto
    {
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string Role { get; set; } = null!;
    }

    // DTO for login request
    public class LoginDto
    {
        public string Username { get; set; } = null!;
        public string Password { get; set; } = null!;
    }

    // DTO for login response
    public class LoginResponseDto
    {
        public string Token { get; set; } = null!;
        public string Role { get; set; } = null!;
        public string FullName { get; set; } = null!;
        public string UserId { get; set; } = null!;
    }

    // DTO for returning user data (excludes password hash for security)
    public class UserResponseDto
    {
        public string Id { get; set; } = null!;
    }

```

```

        public string Username { get; set; } = null!;
        public string Role { get; set; } = null!;
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string Status { get; set; } = null!;
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
    }
}

```

WebApp (Frontend)

WebApp/src/pages/auth/LoginPage.jsx

```

// =====
// File: LoginPage.jsx
// Purpose: Handles user authentication UI with side-by-side branding
//           and role-based redirection (Backoffice -> /stations,
//           GridOperator -> /reservations).
// Author: Migara
// =====

import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../../../context/AuthContext';
import api from '../../../services/api';
import Button from '../../../components/common/Button';
import Input from '../../../components/common/Input';
import JouleMark from '../../../components/common/JouleMark';

export default function LoginPage() {
    const [username, setUsername] = useState('');
    const [password, setPassword] = useState('');
    const [error, setError] = useState('');
    const [isLoading, setIsLoading] = useState(false);

    const { login } = useAuth();
    const navigate = useNavigate();

    const handleSubmit = async (e) => {
        e.preventDefault();
        setError('');
        setIsLoading(true);

        try {
            // 1. Call the backend API (FAT Service Pattern: server validates credentials)
            const response = await api.post('/auth/login', { username, password });
            const { token, role, fullName } = response.data;

            // 2. Update global auth state via AuthContext (persists to localStorage)
            login({ token, role, fullName });

            // 3. All roles land on the Dashboard (architecture.md §6 – central overview first)
            if (role === 'Backoffice' || role === 'GridOperator') {
                navigate('/dashboard');
            } else {
                navigate('/');
            }
        } catch (err) {

```

```

    // Surface exact API error message verbatim
    setError(err.response?.data?.message || 'Login failed. Check credentials.');
```

```

  } finally {
    setIsLoading(false);
  }
};

return (
  <div className="flex min-h-screen w-full bg-canvas">
    { /* Left Column: Visual branding & tagline (visible on lg screens) */ }
    <div className="relative hidden w-1/2 flex-col justify-between overflow-hidden bg-slate-950 p-12 text-white lg:flex">
      { /* Background Image with layered gradient */ }
      <div
        className="absolute inset-0 bg-cover bg-center opacity-45 mix-blend-luminosity
        filter transition-transform duration-700 hover:scale-105"
        style={{ backgroundImage: `url('/images/solar-farm-hero.jpg')` }}
      />
      <div className="absolute inset-0 bg-gradient-to-t from-slate-950 via-slate-950/75 to-slate-900/60" />

      { /* Brand header */ }
      <div className="relative z-10">
        <Link to="/" className="inline-flex items-center gap-3">
          <JouleMark id="login-hero-joule-mark" className="h-7 w-8" />
          <span className="text-xl font-bold tracking-tight text-white">Joule</span>
        </Link>
      </div>

      { /* Center/Bottom Headline & Tagline */ }
      <div className="relative z-10 max-w-lg space-y-6">
        <div className="inline-flex items-center gap-2 rounded-full border border-white/10 bg-white/5 px-3 py-1 text-xs font-medium tracking-wide text-cyan-300 backdrop-blur-md">
          Smart Solar Microgrid System
        </div>

        <div className="space-y-2">
          <h2 className="text-3xl font-extrabold tracking-tight text-white sm:text-4xl">
            Power, exchanged precisely.
          </h2>
          <p className="text-sm leading-relaxed text-slate-300">
            Coordinated energy reservation, distributed battery slot management,
            and real-time grid dispatch for clean microgrid communities.
          </p>
        </div>

        { /* Feature Highlights */ }
        <div className="grid grid-cols-2 gap-3 pt-2">
          <div className="rounded-xl border border-white/10 bg-white/5 p-3.5 backdrop-blur-sm">
            <p className="text-xs font-medium text-slate-400">Microgrid Nodes</p>
            <p className="mt-1 text-sm font-semibold text-white">Automated Capacity</p>
          </div>
          <div className="rounded-xl border border-white/10 bg-white/5 p-3.5 backdrop-blur-sm">
            <p className="text-xs font-medium text-slate-400">Oversight</p>
            <p className="mt-1 text-sm font-semibold text-white">Real-Time Validation</p>
          </div>
        </div>
      </div>
    </div>
  </div>
);

```

```

    { /* Footer info */ }
    <div className="relative z-10 flex items-center justify-between text-xs text-slate-
400 border-t border-white/10 pt-4">

        </div>
    </div>

    { /* Right Column: Sign In Form */ }
    <div className="flex flex-1 flex-col justify-between px-6 py-10 sm:px-12 lg:w-1/2
lg:px-16 xl:px-24">
        { /* Top bar */ }
        <div className="flex items-center justify-between">
            <div className="lg:hidden">
                <Link to="/" className="flex items-center gap-2">
                    <JouleMark id="login-mobile-joule-mark" className="h-6 w-7" />
                    <span className="text-[17px] font-bold tracking-tight text-ink">Joule</span>
                </Link>
            </div>
            <Link
                to="/"
                className="ml-auto inline-flex items-center gap-1.5 text-xs font-medium text-
muted hover:text-ink transition-colors"
            >
                ← Back to home
            </Link>
        </div>

        { /* Main Sign In Box */ }
        <div className="mx-auto w-full max-w-sm my-auto py-8">
            <div className="mb-6">
                <h1 className="text-2xl font-bold tracking-tight text-ink">Staff sign in</h1>
                <p className="mt-1.5 text-[13px] text-muted">
                    Sign in with your Backoffice or Grid Operator credentials.
                </p>
            </div>

            {error && (
                <div className="mb-5 rounded-lg border border-error/30 bg-error-soft px-3.5 py-
2.5 text-[13px] font-medium text-error">
                    {error}
                </div>
            )}

            <form onSubmit={handleSubmit} className="flex flex-col gap-4">
                <Input
                    label="Username"
                    type="text"
                    value={username}
                    onChange={(e) => setUsername(e.target.value)}
                    required
                    placeholder="e.g. operator1"
                    autoFocus
                />
                <Input
                    label="Password"
                    type="password"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    required
                    placeholder="....."
                />
            </form>
        </div>
    </div>

```

```

        <Button
          type="submit"
          variant="primary"
          className="mt-3 w-full justify-center py-2.5 text-sm font-medium shadow-sm
transition-all"
          disabled={isLoading}
        >
          {isLoading ? 'Signing in...' : 'Sign in'}
        </Button>
      </form>

      {/* Security & Access note */}
      <div className="mt-8 rounded-lg border border-line bg-surface-alt/70 p-3.5 text-
center text-xs text-muted">
        Staff and operator accounts are provisioned exclusively by Backoffice
administrators. Prosumers use the mobile application.
      </div>
    </div>

    {/* Bottom copyright */}
    <div className="text-center text-xs text-muted">
      &copy; {new Date().getFullYear()} Joule Smart Microgrid System. All rights
reserved.
    </div>
  </div>
</div>
);
}

```

WebApp/src/pages/admin/UsersPage.jsx

```

// =====
// File: UsersPage.jsx
// Purpose: Handles CRUD operations for system users (Backoffice/
//          GridOperator). Fetches users, displays them in a table,
//          provides modals for creation/updating, and includes
//          search and filter capabilities. Excludes Prosumers.
// Author: Migara (updated with Search, Filters, and Prosumer exclusion)
// =====

import { useMemo, useState, useEffect, useCallback } from 'react';
import { useAuth } from '../../../context/AuthContext';
import api from '../../../services/api';
import Toast from '../../../components/common/Toast';
import Button from '../../../components/common/Button';
import Input from '../../../components/common/Input';
import Badge from '../../../components/common/Badge';
import Modal from '../../../components/common/Modal';
import ConfirmDialog from '../../../components/common/ConfirmDialog';
import DataTable from '../../../components/common/Table';
import StatCard from '../../../components/common/StatCard';

const EMPTY_FORM = { username: '', password: '', role: 'GridOperator', fullName: '', email:
'' };

export default function UsersPage() {
  const { role } = useAuth();

```

```

const [users, setUsers] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [toast, setToast] = useState({ message: '', tone: 'error' });

const [isModalOpen, setIsModalOpen] = useState(false);
const [editingUser, setEditingUser] = useState(null);
const [formData, setFormData] = useState(EMPTY_FORM);
const [creating, setCreating] = useState(false);
const [updating, setUpdating] = useState(false);

const [deactivateTarget, setDeactivateTarget] = useState(null);
const [deactivating, setDeactivating] = useState(false);

// --- NEW: State for Search and Filters ---
const [searchQuery, setSearchQuery] = useState('');
const [roleFilter, setRoleFilter] = useState('All');
const [statusFilter, setStatusFilter] = useState('All');

const fetchUsers = useCallback(async () => {
  setIsLoading(true);
  try {
    const response = await api.get('/users');
    setUsers(response.data);
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to load users', tone:
'error' });
  } finally {
    setIsLoading(false);
  }
}, []);

useEffect(() => {
  if (role === 'Backoffice') {
    fetchUsers();
  }
}, [role, fetchUsers]);

// --- NEW: Filter logic (Excludes Prosumers, applies search/filters) ---
const filteredUsers = useMemo(() => {
  return users.filter((u) => {
    // 1. Exclude Prosumers entirely
    if (u.role === 'Prosumer') return false;

    // 2. Search by username (case-insensitive)
    if (searchQuery && !u.username.toLowerCase().includes(searchQuery.toLowerCase())) {
      return false;
    }

    // 3. Filter by role
    if (roleFilter !== 'All' && u.role !== roleFilter) {
      return false;
    }

    // 4. Filter by status
    if (statusFilter !== 'All' && u.status !== statusFilter) {
      return false;
    }

    return true;
  });
}, [users, searchQuery, roleFilter, statusFilter]);

```



```

// Summary now reflects the currently filtered view
const summary = useMemo(
  () => ({
    total: filteredUsers.length,
    active: filteredUsers.filter((u) => u.status === 'Active').length,
    backoffice: filteredUsers.filter((u) => u.role === 'Backoffice').length,
    operators: filteredUsers.filter((u) => u.role === 'GridOperator').length,
  }),
  [filteredUsers],
);

const handleOpenCreate = () => {
  setEditingUser(null);
  setFormData(EMPTY_FORM);
  setIsModalOpen(true);
};

const handleOpenEdit = (user) => {
  setEditingUser(user);
  setFormData({
    username: user.username,
    password: '',
    role: user.role,
    fullName: user.fullName,
    email: user.email,
  });
  setIsModalOpen(true);
};

const handleCloseModal = () => {
  setIsModalOpen(false);
  setEditingUser(null);
  setFormData(EMPTY_FORM);
};

const handleCreateUser = async (e) => {
  e.preventDefault();
  setCreating(true);
  try {
    await api.post('/users', formData);
    setToast({ message: 'User created successfully!', tone: 'success' });
    handleCloseModal();
    await fetchUsers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to create user', tone:
'error' });
  } finally {
    setCreating(false);
  }
};

const handleUpdateUser = async (e) => {
  e.preventDefault();
  if (!editingUser) return;
  setUpdating(true);
  try {
    const updatePayload = {
      fullName: formData.fullName,
      email: formData.email,
      role: formData.role,

```

```

    });
    await api.put(`/users/${editingUser.id}`, updatePayload);
    setToast({ message: 'User updated successfully!', tone: 'success' });
    handleCloseModal();
    await fetchUsers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to update user', tone:
'error' });
  } finally {
    setUpdating(false);
  }
};

const handleConfirmDeactivate = async () => {
  if (!deactivateTarget) return;
  setDeactivating(true);
  try {
    await api.put(`/users/${deactivateTarget.id}/deactivate`);
    setToast({ message: 'User deactivated successfully!', tone: 'success' });
    setDeactivateTarget(null);
    await fetchUsers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to deactivate user', tone:
'error' });
    setDeactivateTarget(null);
  } finally {
    setDeactivating(false);
  }
};

const columns = useMemo(
  () => [
    { key: 'username', header: 'Username', render: (u) => <span className="font-medium
text-ink">{u.username}</span> },
    { key: 'fullName', header: 'Full name' },
    { key: 'email', header: 'Email' },
    {
      key: 'role',
      header: 'Role',
      render: (u) => <Badge tone={u.role === 'Backoffice' ? 'info' :
'neutral'}>{u.role}</Badge>,
    },
    {
      key: 'status',
      header: 'Status',
      render: (u) => <Badge tone={u.status === 'Active' ? 'success' :
'neutral'}>{u.status}</Badge>,
    },
    {
      key: 'actions',
      header: '',
      className: 'text-right',
      render: (u) => (
        <div className="flex justify-end gap-2">
          {u.status === 'Active' ? (
            <>
              <Button variant="secondary" size="sm" onClick={() => handleOpenEdit(u)}>
                Edit
              </Button>
              <Button variant="danger-outline" size="sm" onClick={() =>
setDeactivateTarget(u)}>

```

```

                Deactivate
            </Button>
        </>
    ) : (
        <span className="text-xs text-muted italic">No actions available</span>
    )}
</div>
),
],
[handleOpenEdit],
);

if (role !== 'Backoffice') {
    return (
        <div className="mx-auto max-w-6xl px-6 py-8">
            <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2 text-[13px] font-medium text-error">
                Access denied – only Backoffice users can manage system users.
            </p>
        </div>
    );
}

return (
    <div className="mx-auto max-w-6xl px-6 py-8">
        <div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
            <div>
                <h1 className="text-xl font-semibold tracking-tight text-ink">User management</h1>
                <p className="mt-1 text-[13px] text-muted">Backoffice and Grid Operator staff accounts.</p>
            </div>
            <Button variant="primary" onClick={handleOpenCreate}>
                New user
            </Button>
        </div>

        <div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-4">
            <StatCard icon="user" label="Total staff" value={summary.total} hint="Filtered staff accounts" />
            <StatCard
                icon="pulse"
                label="Active"
                value={summary.active}
                hint={` ${summary.total - summary.active} deactivated`
            />
            <StatCard icon="hubs" label="Backoffice" value={summary.backoffice} hint="Admin accounts" />
            <StatCard icon="bolt" label="Grid operators" value={summary.operators} hint="Field accounts" />
        </div>

        { /* --- NEW: Search and Filter Controls --- */ }
        <div className="mb-4 flex flex-col gap-3 sm:flex-row sm:items-end">
            <div className="flex-1">
                <Input
                    label="Search by username"
                    placeholder="e.g., migara_admin"
                    value={searchQuery}
                    onChange={(e) => setSearchQuery(e.target.value)}

```

```

    />
  </div>
  <div className="w-full sm:w-48">
    <Input
      label="Filter by Role"
      as="select"
      value={roleFilter}
      onChange={(e) => setRoleFilter(e.target.value)}
    >
      <option value="All">All Roles</option>
      <option value="Backoffice">Backoffice</option>
      <option value="GridOperator">Grid Operator</option>
    </Input>
  </div>
  <div className="w-full sm:w-48">
    <Input
      label="Filter by Status"
      as="select"
      value={statusFilter}
      onChange={(e) => setStatusFilter(e.target.value)}
    >
      <option value="All">All Statuses</option>
      <option value="Active">Active</option>
      <option value="Deactivated">Deactivated</option>
    </Input>
  </div>
</div>

<DataTable
  columns={columns}
  data={filteredUsers}
  loading={isLoading}
  emptyMessage="No matching staff users found."
  rowKey={(u) => u.id}
/>

<Modal
  open={isModalOpen}
  onClose={handleCloseModal}
  title={editingUser ? "Edit user" : "Create new user"}
  description={editingUser ? "Update user details." : "Provisions a Backoffice or Grid
Operator staff account."}
>
  <form onSubmit={editingUser ? handleUpdateUser : handleCreateUser} className="flex
flex-col gap-4">
    <{!editingUser && (
      <>
        <Input
          label="Username"
          required
          value={formData.username}
          onChange={(e) => setFormData({ ...formData, username: e.target.value })}
        />
        <Input
          label="Password"
          type="password"
          required
          value={formData.password}
          onChange={(e) => setFormData({ ...formData, password: e.target.value })}
        />
      </>
    )}
  </form>
</div>

```

```

    })
    <Input
        label="Full name"
        required
        value={formData.fullName}
        onChange={(e) => setFormData({ ...formData, fullName: e.target.value })}
    />
    <Input
        label="Email"
        type="email"
        required
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
    />
    <Input
        label="Role"
        as="select"
        value={formData.role}
        onChange={(e) => setFormData({ ...formData, role: e.target.value })}
    >
        <option value="Backoffice">Backoffice</option>
        <option value="GridOperator">Grid Operator</option>
    </Input>
    <div className="mt-2 flex justify-end gap-2">
        <Button type="button" variant="secondary" onClick={handleCloseModal}
disabled={creating || updating}>
            Cancel
        </Button>
        <Button type="submit" variant="primary" disabled={creating || updating}>
            {creating ? 'Creating...' : updating ? 'Updating...' : (editingUser ? 'Update user'
: 'Create user')}
        </Button>
    </div>
</form>
</Modal>

    <ConfirmDialog
        open={Boolean(deactivateTarget)}
        title="Deactivate user?"
        description={deactivateTarget ? `${deactivateTarget.username}" will no longer be
able to sign in.` : ''}
        confirmLabel="Deactivate"
        tone="danger"
        confirming={deactivating}
        onConfirm={handleConfirmDeactivate}
        onClose={() => setDeactivateTarget(null)}
    />

    <Toast message={toast.message} tone={toast.tone} onDismiss={() => setToast({ message:
'', tone: 'error' })} />
</div>
);
}

```

MobileApp (Android)

Core Feature Files:

app/src/main/java/com/smartmicrogrid/util/QrCodeHelper.kt

```
// =====
// File: QrCodeHelper.kt
// Purpose: Utility class to generate QR code ImageBitmaps from
//          text strings (e.g., backend-generated qrTokens) using
//          the ZXing library. Designed specifically for rendering
//          in Jetpack Compose UIs without external UI dependencies.
// Author: Migara
// =====
package com.smartmicrogrid.util

import android.graphics.Bitmap
import android.graphics.Color
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import java.util.HashMap

object QrCodeHelper {
    fun generateQrCodeBitmap(text: String, size: Int = 512): ImageBitmap? {
        return try {
            val writer = QRCodeWriter()

            // Use java.util.HashMap explicitly to avoid IDE confusion
            val hints = HashMap<EncodeHintType, Any>()
            hints[EncodeHintType.MARGIN] = 1

            val bitMatrix = writer.encode(text, BarcodeFormat.QR_CODE, size, size, hints)
            val width = bitMatrix.width
            val height = bitMatrix.height

            val bmp = Bitmap.createBitmap(width, height, Bitmap.Config.RGB_565)

            // Use basic while loops to completely avoid the '..' vs 'until' IDE bug
            var x = 0
            while (x < width) {
                var y = 0
                while (y < height) {
                    bmp.setPixel(x, y, if (bitMatrix[x, y]) Color.BLACK else Color.WHITE)
                    y++
                }
                x++
            }

            bmp.asImageBitmap()
        } catch (e: Exception) {
            e.printStackTrace()
            null
        }
    }
}
```

app/src/main/java/com/smartmicrogrid/ui/prosumer/BookingsScreen.kt

```
// =====
// File: BookingsScreen.kt
// Purpose: Enhanced Prosumer bookings list — polished cards with
```

```

// collapsible QR code sections, status badges, formatted
// timestamps, and the station name pulled from the response.
// Author: Migara (enhanced by Shalon 2026-09-29)
// =====
package com.smartmicrogrid.ui.prosumer

import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.expandVertically
import androidx.compose.animation.shrinkVertically
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Dialog
import androidx.lifecycle.viewmodel.compose.viewModel
import com.smartmicrogrid.data.remote.dto.CreateReservationRequest
import com.smartmicrogrid.data.remote.dto.ReservationResponse
import com.smartmicrogrid.ui.components.JouleIcons
import com.smartmicrogrid.ui.theme.*
import com.smartmicrogrid.util.QrCodeHelper
import java.time.Instant
import java.time.ZoneId
import java.time.format.DateTimeFormatter

// Formats "2026-10-01T14:00:00Z" → "Oct 1, 2026 · 2:00 PM"
private fun formatScheduledAt(iso: String?): String {
    if (iso.isNullOrBlank()) return "—"
    return try {
        val instant = Instant.parse(iso)
        val zdt = instant.atZone(ZoneId.systemDefault())
        val datePart = DateTimeFormatter.ofPattern("MMM d, yyyy").format(zdt)
        val timePart = DateTimeFormatter.ofPattern("h:mm a").format(zdt)
        "$datePart · $timePart"
    } catch (_: Exception) {
        iso
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BookingsScreen(
    prosumerNic: String? = null,
    onNavigateToCreate: (() -> Unit)? = null,
    viewModel: BookingsViewModel = viewModel()
) {
    val context = androidx.compose.ui.platform.LocalContext.current

```

```

val effectiveNic = remember(prosumerNic) {
    prosumerNic ?: com.smartmicrogrid.data.local.ProsumerSessionDao(context).getSession()?.nic ?: "TEST-NIC-123"
}

val uiState by viewModel.uiState.collectAsState()
val actionResult by viewModel.actionResult.collectAsState()
var showCreateDialog by remember { mutableStateOf(false) }
var stationId by remember { mutableStateOf("") }
var slotId by remember { mutableStateOf("") }
var scheduledAt by remember { mutableStateOf("") }

LaunchedEffect(effectiveNic) {
    viewModel.fetchReservations(effectiveNic)
}

Scaffold(
    topBar = {
        TopAppBar(
            title = {
                Text(
                    "My Bookings",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.SemiBold,
                    color = StripeInk,
                )
            },
            windowInsets = WindowInsets(0, 0, 0, 0),
            colors = TopAppBarDefaults.topAppBarColors(
                containerColor = StripeSurface,
                actionIconContentColor = StripePrimary,
            ),
            actions = {
                IconButton(onClick = {
                    if (onNavigateToCreate != null) {
                        onNavigateToCreate()
                    } else {
                        showCreateDialog = true
                    }
                }) {
                    Icon(
                        imageVector = JouleIcons.Calendar,
                        contentDescription = "New booking",
                        tint = StripePrimary,
                    )
                }
            },
        )
    },
    containerColor = StripeCanvas,
) { paddingValues ->
    Box(modifier = Modifier.fillMaxSize().padding(paddingValues)) {
        when (val state = uiState) {
            is BookingsUiState.Loading -> {
                CircularProgressIndicator(
                    modifier = Modifier.align(Alignment.Center),
                    color = StripePrimary,
                    strokeWidth = 3.dp,
                )
            }
        }
    }
}

```



```

}
is BookingsUiState.Error -> {
  Column(
    modifier = Modifier.align(Alignment.Center).padding(24.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(12.dp),
  ) {
    Text(
      "Couldn't load bookings",
      style = MaterialTheme.typography.titleMedium,
      fontWeight = FontWeight.SemiBold,
      color = StripeInk,
    )
    Text(
      state.message,
      style = MaterialTheme.typography.bodySmall,
      color = StripeMuted,
      textAlign = TextAlign.Center,
    )
    Button(
      onClick = { viewModel.fetchReservations(effectiveNic) },
      colors = ButtonDefaults.buttonColors(containerColor = StripePrimary),
      shape = RoundedCornerShape(50),
    ) { Text("Retry") }
  }
}
is BookingsUiState.Success -> {
  if (state.reservations.isEmpty()) {
    Column(
      modifier = Modifier.align(Alignment.Center).padding(32.dp),
      horizontalAlignment = Alignment.CenterHorizontally,
      verticalArrangement = Arrangement.spacedBy(12.dp),
    ) {
      Box(
        modifier = Modifier
          .size(64.dp)
          .clip(RoundedCornerShape(16.dp))
          .background(StripeBrandVioletSoft),
        contentAlignment = Alignment.Center,
      ) {
        Icon(
          imageVector = JouleIcons.Calendar,
          contentDescription = null,
          tint = StripePrimary,
          modifier = Modifier.size(30.dp),
        )
      }
      Text(
        "No bookings yet",
        style = MaterialTheme.typography.titleMedium,
        fontWeight = FontWeight.SemiBold,
        color = StripeInk,
      )
      Text(
        "Tap the calendar icon above to reserve an energy slot at a nearby station.",
        style = MaterialTheme.typography.bodySmall,
        color = StripeMuted,
        textAlign = TextAlign.Center,
      )
    }
  }
}

```

```

    )
  }
} else {
  LazyColumn(
    modifier = Modifier.fillMaxSize(),
    contentPadding = PaddingValues(horizontal = 16.dp, vertical = 16.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp),
  ) {
    items(state.reservations) { reservation ->
      ReservationCard(
        reservation = reservation,
        onCancel = { viewModel.cancelReservation(reservation.id) },
      )
    }
  }
}
}
}

actionResult?.let { message ->
  Snackbar(
    modifier = Modifier.align(Alignment.BottomCenter).padding(16.dp),
    action = {
      TextButton(onClick = { viewModel.clearActionResult() }) {
        Text("Dismiss", color = StripePrimary)
      }
    },
    containerColor = StripeInk,
    contentColor = Color.White,
  ) { Text(message) }
}
}

// CREATE BOOKING DIALOG (legacy fallback — normal path now navigates to CreateBookingScreen)
if (showCreateDialog) {
  Dialog(onDismissRequest = { showCreateDialog = false }) {
    Surface(
      shape = RoundedCornerShape(16.dp),
      color = StripeSurface,
      modifier = Modifier.fillMaxWidth().padding(16.dp),
      tonalElevation = 4.dp,
    ) {
      Column(modifier = Modifier.padding(20.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        Text(
          "Create New Booking",
          style = MaterialTheme.typography.titleLarge,
          fontWeight = FontWeight.Bold,
          color = StripeInk,
        )
        OutlinedTextField(value = stationId, onValueChange = { stationId = it }, label = { Text("Station ID") }, modifier =
Modifier.fillMaxWidth(), singleLine = true)
        OutlinedTextField(value = slotId, onValueChange = { slotId = it }, label = { Text("Slot ID") }, modifier =
Modifier.fillMaxWidth(), singleLine = true)
        OutlinedTextField(value = scheduledAt, onValueChange = { scheduledAt = it }, label = { Text("Time (e.g., 2026-10-
01T14:00:00Z)") }, modifier = Modifier.fillMaxWidth(), singleLine = true)
        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.End) {
          TextButton(onClick = { showCreateDialog = false }) { Text("Cancel", color = StripeMuted) }

```

```

        Spacer(modifier = Modifier.width(8.dp))
        Button(
            onClick = {
                if (stationId.isNotBlank() && slotId.isNotBlank() && scheduledAt.isNotBlank()) {
                    viewModel.createReservation(CreateReservationRequest(stationId, slotId, scheduledAt))
                    showCreateDialog = false; stationId = ""; slotId = ""; scheduledAt = ""
                }
            },
            colors = ButtonDefaults.buttonColors(containerColor = StripePrimary),
            shape = RoundedCornerShape(50),
        ) { Text("Confirm") }
    }
}
}
}
}

// Status-to-color mapping
private data class StatusStyle(val bg: Color, val text: Color, val label: String)
private fun statusStyle(status: String): StatusStyle = when (status) {
    "Confirmed" -> StatusStyle(StripeSuccessContainer, StripeSuccess, "Confirmed")
    "Completed" -> StatusStyle(StripeCyanContainer, StripeOnCyanContainer, "Completed")
    "Cancelled" -> StatusStyle(StripeErrorContainer, StripeError, "Cancelled")
    else -> StatusStyle(StripeTray, StripeMuted, status)
}

@Composable
fun ReservationCard(reservation: ReservationResponse, onCancel: () -> Unit) {
    var qrExpanded by remember { mutableStateOf(false) }
    val style = statusStyle(reservation.status)
    val shortId = reservation.id.takeLast(6)

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .border(1.dp, StripeBorder, RoundedCornerShape(16.dp)),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = StripeSurface),
        elevation = CardDefaults.cardElevation(defaultElevation = 1.dp),
    ) {
        Column(modifier = Modifier.fillMaxWidth()) {

            // — Header —————
            Row(
                modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 14.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically,
            ) {
                Column(modifier = Modifier.weight(1f)) {
                    Text(
                        text = "Booking #\$shortId",
                        style = MaterialTheme.typography.titleSmall,
                        fontWeight = FontWeight.Bold,
                        color = StripeInk,
                    )
                    Spacer(modifier = Modifier.height(2.dp))
                    Text(

```

```

        text = formatScheduledAt(reservation.scheduledAt),
        style = MaterialTheme.typography.bodySmall,
        color = StripeMuted,
    )
}
Spacer(modifier = Modifier.width(8.dp))
// Status Pill
Box(
    modifier = Modifier
        .clip(RoundedCornerShape(50))
        .background(style.bg)
        .padding(horizontal = 10.dp, vertical = 4.dp),
) {
    Text(
        text = style.label,
        style = MaterialTheme.typography.labelSmall,
        fontWeight = FontWeight.SemiBold,
        color = style.text,
    )
}
}

HorizontalDivider(color = StripeBorder)

// — Detail Row —————
Row(
    modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 12.dp),
    horizontalArrangement = Arrangement.spacedBy(16.dp),
) {
    DetailChip(label = "Station", value = reservation.stationId.takeLast(8), modifier = Modifier.weight(1f))
    DetailChip(label = "Slot", value = reservation.slotId?.takeLast(6) ?: "—", modifier = Modifier.weight(1f))
}

// — QR Code toggle (Confirmed only) —————
if (reservation.status == "Confirmed" && !reservation.qrToken.isNullOrEmpty()) {
    HorizontalDivider(color = StripeBorder)
    // Toggle row
    TextButton(
        onClick = { qrExpanded = !qrExpanded },
        modifier = Modifier.fillMaxWidth().padding(horizontal = 8.dp),
        contentPadding = PaddingValues(horizontal = 8.dp, vertical = 4.dp),
    ) {
        Icon(
            imageVector = if (qrExpanded) JouleIcons.Check else JouleIcons.Scan,
            contentDescription = null,
            tint = StripePrimary,
            modifier = Modifier.size(16.dp),
        )
        Spacer(modifier = Modifier.width(6.dp))
        Text(
            text = if (qrExpanded) "Hide QR Code" else "Show QR Code for Scan",
            style = MaterialTheme.typography.labelMedium,
            color = StripePrimary,
            fontWeight = FontWeight.SemiBold,
        )
        Spacer(modifier = Modifier.weight(1f))
        Text(
            text = if (qrExpanded) "▲" else "▼",

```

```

        style = MaterialTheme.typography.labelSmall,
        color = StripeMuted,
    )
}

// Expandable QR Section
AnimatedVisibility(
    visible = qrExpanded,
    enter = expandVertically(),
    exit = shrinkVertically(),
){
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .background(StripeCanvas)
            .padding(vertical = 20.dp, horizontal = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(12.dp),
    ){
        Text(
            "Present at the station node",
            style = MaterialTheme.typography.labelMedium,
            fontWeight = FontWeight.SemiBold,
            color = StripeInk,
        )
        // QR frame card
        Box(
            modifier = Modifier
                .clip(RoundedCornerShape(12.dp))
                .background(Color.White)
                .border(1.dp, StripeBorder, RoundedCornerShape(12.dp))
                .padding(16.dp),
            contentAlignment = Alignment.Center,
        ){
            val qrBitmap = remember(reservation.qrToken) {
                QRCodeHelper.generateQRCodeBitmap(reservation.qrToken, size = 220)
            }
            if (qrBitmap != null) {
                androidx.compose.foundation.Image(
                    bitmap = qrBitmap,
                    contentDescription = "Reservation QR Code",
                    modifier = Modifier.size(220.dp),
                )
            } else {
                Text("QR unavailable", color = StripeMuted, style = MaterialTheme.typography.bodySmall)
            }
        }
        Text(
            "Token: ...${reservation.qrToken.takeLast(12)}",
            style = MaterialTheme.typography.labelSmall,
            color = StripeMuted,
            fontFamily = androidx.compose.ui.text.font.FontFamily.Monospace,
        )
    }
}

// — Cancel Button —————

```

```

        if (reservation.status == "Confirmed") {
            HorizontalDivider(color = StripeBorder)
            TextButton(
                onClick = onCancel,
                modifier = Modifier.fillMaxWidth().padding(horizontal = 8.dp, vertical = 2.dp),
                colors = ButtonDefaults.textButtonColors(contentColor = StripeError),
            ) {
                Text(
                    "Cancel Booking",
                    style = MaterialTheme.typography.labelMedium,
                    fontWeight = FontWeight.Medium,
                )
            }
        }
    }
}
}

@Composable
private fun DetailChip(label: String, value: String, modifier: Modifier = Modifier) {
    Column(
        modifier = modifier
            .clip(RoundedCornerShape(8.dp))
            .background(StripeTray)
            .padding(horizontal = 10.dp, vertical = 8.dp),
    ) {
        Text(label, style = MaterialTheme.typography.labelSmall, color = StripeMuted)
        Spacer(modifier = Modifier.height(2.dp))
        Text(
            value,
            style = MaterialTheme.typography.bodySmall,
            fontWeight = FontWeight.SemiBold,
            color = StripeInk,
            fontFamily = androidx.compose.ui.text.font.FontFamily.Monospace,
        )
    }
}

```

app/src/main/java/com/smartmicrogrid/ui/prosumer/CreateBookingScreen.k

```

// =====
// File: CreateBookingScreen.kt
// Purpose: Prosumer UI to book an energy charging/discharging slot.
// Features:
// - Dropdown station selector populated via GET /api/stations
// - Auto-selection if opened from map pin (Bonus Flow)
// - Dynamic slot selection (chips) via GET /api/stations/{id}
// - Native Date & Time picker enforcing the 7-day scheduling window
// - Zero double-padding on top bar
// Author: Migara
// =====
package com.smartmicrogrid.ui.prosumer

import android.app.DatePickerDialog
import android.app.TimePickerDialog

```

```

import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.CreateReservationRequest
import com.smartmicrogrid.data.remote.dto.SlotResponse
import com.smartmicrogrid.data.remote.dto.StationSummaryResponse
import com.smartmicrogrid.ui.components.JouleIcons
import com.smartmicrogrid.ui.theme.StripeAccent
import com.smartmicrogrid.ui.theme.StripeBrandVioletSoft
import com.smartmicrogrid.ui.theme.StripeInk
import com.smartmicrogrid.ui.theme.StripePrimary
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.time.Instant
import java.time.ZoneOffset
import java.time.format.DateTimeFormatter
import java.util.Calendar

@OptIn(ExperimentalMaterial3Api::class, ExperimentalLayoutApi::class)
@Composable
fun CreateBookingScreen(
    initialStationId: String? = null,
    onNavigateBack: () -> Unit,
    viewModel: BookingsViewModel = viewModel()
) {
    val context = LocalContext.current
    val coroutineScope = rememberCoroutineScope()

    // Station list & selection state
    var stations by remember { mutableStateOf<List<StationSummaryResponse>>(emptyList()) }
    var isLoadingStations by remember { mutableStateOf(true) }
    var stationDropdownExpanded by remember { mutableStateOf(false) }
    var selectedStation by remember { mutableStateOf<StationSummaryResponse?>(null) }
    var isStationLocked by remember { mutableStateOf(initialStationId != null) }

    // Slots state
    var slots by remember { mutableStateOf<List<SlotResponse>>(emptyList()) }
    var isLoadingSlots by remember { mutableStateOf(false) }
    var selectedSlotId by remember { mutableStateOf("") }

    // Date & Time state
    var selectedDateTimeDisplay by remember { mutableStateOf("") }
    var scheduledIsoString by remember { mutableStateOf("") }

```

```

var isSubmitting by remember { mutableStateOf(false) }
val actionResult by viewModel.actionResult.collectAsState()

// 1. Fetch all active stations
LaunchedEffect(Unit) {
    isLoadingStations = true
    runCatching {
        withContext(Dispatchers.IO) {
            ApiClient.service.getStations(status = "Active")
        }
    }.onSuccess { list ->
        stations = list
        if (!initialStationId.isNullOrBlank()) {
            val matched = list.firstOrNull { it.id == initialStationId }
            if (matched != null) {
                selectedStation = matched
            }
        }
    }
    isLoadingStations = false
}

// 2. Fetch slots whenever the selected station changes
LaunchedEffect(selectedStation) {
    val station = selectedStation
    selectedSlotId = ""
    slots = emptyList()
    if (station != null) {
        isLoadingSlots = true
        runCatching {
            withContext(Dispatchers.IO) {
                ApiClient.service.getStationDetail(station.id)
            }
        }.onSuccess { detail ->
            slots = detail.slots
        }
        isLoadingSlots = false
    }
}

// 3. Handle booking result
LaunchedEffect(actionResult) {
    if (actionResult != null) {
        isSubmitting = false
        if (actionResult == "Booking confirmed!") {
            onNavigateBack()
        }
    }
}

// 4. Function to open Date & Time Pickers with 7-day rule
fun openDateTimePicker() {
    val calendar = Calendar.getInstance()
    val currentYear = calendar.get(Calendar.YEAR)
    val currentMonth = calendar.get(Calendar.MONTH)
    val currentDay = calendar.get(Calendar.DAY_OF_MONTH)

    val datePickerDialog = DatePickerDialog(

```



```

context,
{_, year, month, dayOfMonth ->
  val timePickerDialog = TimePickerDialog(
    context,
    {_, hourOfDay, minute ->
      val chosenCal = Calendar.getInstance().apply {
        set(year, month, dayOfMonth, hourOfDay, minute, 0)
        set(Calendar.MILLISECOND, 0)
      }

      // ISO 8601 formatting for backend (e.g., 2026-10-01T14:00:00Z)
      val instant = chosenCal.toInstant()
      scheduledIsoString = DateTimeFormatter.ISO_INSTANT.format(instant)

      // Human-readable format for UI display
      val friendlyFormat = DateTimeFormatter.ofPattern("EEE, dd MMM yyyy · HH:mm")
        .withZone(ZoneOffset.systemDefault())
      selectedDateTimeDisplay = friendlyFormat.format(instant)
    },
    calendar.get(Calendar.HOUR_OF_DAY) + 1,
    0,
    true // 24-hour format
  )
  timePickerDialog.setTitle("Select Time")
  timePickerDialog.show()
},
currentYear,
currentMonth,
currentDay
)

// Enforce the 7-day window rule:
// minDate = Now
// maxDate = 7 days from now
datePickerDialog.datePicker.minDate = System.currentTimeMillis() - 1000L
datePickerDialog.datePicker.maxDate = System.currentTimeMillis() + (7L * 24 * 60 * 60 * 1000L)
datePickerDialog.setTitle("Select Date (Within 7 Days)")
datePickerDialog.show()
}

Scaffold(
  containerColor = MaterialTheme.colorScheme.background,
  topBar = {
    TopAppBar(
      title = { Text("New Booking", fontWeight = FontWeight.SemiBold) },
      windowInsets = WindowInsets(0, 0, 0, 0), // Prevents double status bar padding!
      navigationIcon = {
        IconButton(onClick = onNavigateBack) {
          Icon(JouleIcons.Back, contentDescription = "Back")
        }
      }
    )
  }
) { paddingValues ->
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(paddingValues)
  )
}

```

```

        .padding(horizontal = 20.dp, vertical = 12.dp)
        .verticalScroll(rememberScrollState()),
        verticalArrangement = Arrangement.spacedBy(20.dp)
    ){
        Text(
            text = "Reserve a battery charging or discharging slot at your preferred microgrid station.",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )

        // =====
        // SECTION 1: STATION SELECTION DROPDOWN
        // =====
        Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ){
                Text(
                    text = "Microgrid Station Hub",
                    style = MaterialTheme.typography.titleSmall,
                    fontWeight = FontWeight.SemiBold
                )
                if (isStationLocked && selectedStation != null) {
                    Surface(
                        shape = RoundedCornerShape(50),
                        color = StripeBrandVioletSoft,
                        modifier = Modifier.clickable { isStationLocked = false }
                    ){
                        Row(
                            modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
                            verticalAlignment = Alignment.CenterVertically
                        ){
                            Text(" ⓘ Map Selected (Change)", style = MaterialTheme.typography.labelSmall, color = StripePrimary)
                        }
                    }
                }
            }
        }

        if (isLoadingStations) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 12.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(12.dp)
            ){
                CircularProgressIndicator(modifier = Modifier.size(20.dp), strokeWidth = 2.dp)
                Text("Loading nearby hubs...", style = MaterialTheme.typography.bodySmall)
            }
        } else {
            ExposedDropdownMenuBox(
                expanded = stationDropdownExpanded,
                onExpandedChange = {
                    if (!isStationLocked) stationDropdownExpanded = !stationDropdownExpanded
                },
                modifier = Modifier.fillMaxWidth()
            )
        }
    }
}

```

```

    ){
        OutlinedTextField(
            value = selectedStation?.name ?: "",
            onChange = {},
            readOnly = true,
            label = { Text("Select Station Hub") },
            placeholder = { Text("Choose a microgrid station") },
            trailingIcon = {
                ExposedDropdownMenuDefaults.TrailingIcon(expanded = stationDropdownExpanded)
            },
            shape = RoundedCornerShape(12.dp),
            colors = OutlinedTextFieldDefaults.colors(
                focusedBorderColor = StripePrimary,
                unfocusedBorderColor = MaterialTheme.colorScheme.outline
            ),
            modifier = Modifier
                .fillMaxWidth()
                .menuAnchor()
        )

        ExposedDropdownMenu(
            expanded = stationDropdownExpanded,
            onDismissRequest = { stationDropdownExpanded = false }
        ){
            stations.forEach { station ->
                DropdownMenuItem(
                    text = {
                        Column {
                            Text(station.name, fontWeight = FontWeight.SemiBold)
                            Text(
                                "Capacity: ${station.capacityKWh} kWh · ${station.totalBatterySlots} battery slots",
                                style = MaterialTheme.typography.bodySmall,
                                color = MaterialTheme.colorScheme.onSurfaceVariant
                            )
                        }
                    },
                    onClick = {
                        selectedStation = station
                        isStationLocked = false
                        stationDropdownExpanded = false
                    }
                )
            }
        }
    }
}

// =====
// SECTION 2: BATTERY SLOT SELECTION
// =====
Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
    Text(
        text = "Battery Slot",
        style = MaterialTheme.typography.titleSmall,
        fontWeight = FontWeight.SemiBold
    )
}

```

```

if (selectedStation == null) {
    Surface(
        shape = RoundedCornerShape(12.dp),
        color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            text = "Please choose a station hub above to see available slots.",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            modifier = Modifier.padding(14.dp)
        )
    }
} else if (isLoadingSlots) {
    Row(
        modifier = Modifier.padding(vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(10.dp)
    ) {
        CircularProgressIndicator(modifier = Modifier.size(20.dp), strokeWidth = 2.dp)
        Text("Fetching slot availability...", style = MaterialTheme.typography.bodySmall)
    }
} else if (slots.isEmpty()) {
    Text(
        text = "No battery slots found for this station.",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.error
    )
} else {
    FlowRow(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        slots.forEach { slot ->
            val isAvailable = slot.status.equals("Available", ignoreCase = true)
            val isSelected = selectedSlotId == slot.id

            FilterChip(
                selected = isSelected,
                onClick = {
                    if (isAvailable) {
                        selectedSlotId = slot.id
                    }
                },
                enabled = isAvailable,
                label = {
                    Text(
                        text = "Slot #${slot.slotNumber} · ${slot.type} (${slot.capacityKWh.toInt()} kWh)" +
                            if (!isAvailable) " [${slot.status}]" else ""
                    )
                },
                shape = RoundedCornerShape(50),
                colors = FilterChipDefaults.filterChipColors(
                    selectedContainerColor = StripePrimary,
                    selectedLabelColor = MaterialTheme.colorScheme.onPrimary,
                    disabledContainerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.4f),
                    disabledLabelColor = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.5f)
                )
            )
        }
    }
}

```

```

    )
  )
}
}
}
}

// =====
// SECTION 3: SCHEDULED TIME (7-DAY RULE PICKER)
// =====
Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
  Text(
    text = "Scheduled Reservation Time",
    style = MaterialTheme.typography.titleSmall,
    fontWeight = FontWeight.SemiBold
  )

  Surface(
    shape = RoundedCornerShape(12.dp),
    border = BorderStroke(1.dp, if (scheduledIsoString.isNotBlank()) StripePrimary else
MaterialTheme.colorScheme.outline),
    color = MaterialTheme.colorScheme.surface,
    modifier = Modifier
      .fillMaxWidth()
      .clickable { openDateTimePicker() }
  ) {
    Row(
      modifier = Modifier.padding(horizontal = 16.dp, vertical = 14.dp),
      verticalAlignment = Alignment.CenterVertically,
      horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
      Icon(
        imageVector = JouleIcons.Calendar,
        contentDescription = "Date and Time",
        tint = if (scheduledIsoString.isNotBlank()) StripePrimary else MaterialTheme.colorScheme.onSurfaceVariant
      )
      Column(modifier = Modifier.weight(1f)) {
        Text(
          text = if (selectedDateTimeDisplay.isNotBlank()) selectedDateTimeDisplay else "Tap to choose Date & Time",
          style = MaterialTheme.typography.bodyLarge,
          fontWeight = if (selectedDateTimeDisplay.isNotBlank()) FontWeight.SemiBold else FontWeight.Normal,
          color = if (selectedDateTimeDisplay.isNotBlank()) StripeInk else
MaterialTheme.colorScheme.onSurfaceVariant
        )
        Text(
          text = "Permitted window: Today up to 7 days in advance",
          style = MaterialTheme.typography.bodySmall,
          color = MaterialTheme.colorScheme.onSurfaceVariant
        )
      }
    }
  }
}

// =====
// SECTION 4: ERROR / SUBMISSION
// =====
actionResult?.let { message ->

```

```

        if (message != "Booking confirmed!") {
            Surface(
                shape = RoundedCornerShape(8.dp),
                color = MaterialTheme.colorScheme.errorContainer,
                modifier = Modifier.fillMaxWidth()
            ) {
                Text(
                    text = message,
                    color = MaterialTheme.colorScheme.onErrorContainer,
                    style = MaterialTheme.typography.bodySmall,
                    modifier = Modifier.padding(12.dp)
                )
            }
        }
    }

    val canSubmit = selectedStation != null &&
        selectedSlotId.isNotBlank() &&
        scheduledIsoString.isNotBlank() &&
        !isSubmitting

    Button(
        onClick = {
            if (canSubmit) {
                isSubmitting = true
                viewModel.createReservation(
                    CreateReservationRequest(
                        stationId = selectedStation!!.id,
                        slotId = selectedSlotId,
                        scheduledAt = scheduledIsoString
                    )
                )
            }
        },
        modifier = Modifier
            .fillMaxWidth()
            .height(52.dp),
        shape = RoundedCornerShape(50),
        colors = ButtonDefaults.buttonColors(containerColor = StripePrimary),
        enabled = canSubmit
    ) {
        if (isSubmitting) {
            CircularProgressIndicator(
                color = MaterialTheme.colorScheme.onPrimary,
                modifier = Modifier.size(24.dp),
                strokeWidth = 2.dp
            )
        } else {
            Text("Confirm Booking", style = MaterialTheme.typography.titleSmall)
        }
    }
}

DisposableEffect(Unit) {
    onDispose {
        viewModel.clearActionResult()
    }
}

```

```
}  
}
```

app/src/main/java/com/smartmicrogrid/ui/prosumer/BookingsViewModel.kt

```
// =====  
// File: BookingsViewModel.kt  
// Purpose: Manages state and API calls for the Prosumer Bookings screen.  
// Fetches reservations, handles creation, and cancellation.  
// Author: Migara  
// =====  
package com.smartmicrogrid.ui.prosumer  
  
import androidx.lifecycle.ViewModel  
import androidx.lifecycle.viewModelScope  
import com.smartmicrogrid.data.remote.ApiClient  
import com.smartmicrogrid.data.remote.ApiService  
import com.smartmicrogrid.data.remote.dto.CancelReservationRequest  
import com.smartmicrogrid.data.remote.dto.CreateReservationRequest  
import com.smartmicrogrid.data.remote.dto.ReservationResponse  
import kotlinx.coroutines.flow.MutableStateFlow  
import kotlinx.coroutines.flow.StateFlow  
import kotlinx.coroutines.flow.asStateFlow  
import kotlinx.coroutines.launch  
  
sealed class BookingsUiState {  
    data object Loading : BookingsUiState()  
    data class Success(val reservations: List<ReservationResponse>) : BookingsUiState()  
    data class Error(val message: String) : BookingsUiState()  
}  
  
class BookingsViewModel @JvmOverloads constructor(  
    private val apiService: ApiService = ApiClient.service  
) : ViewModel() {  
  
    private val _uiState = MutableStateFlow<BookingsUiState>(BookingsUiState.Loading)  
    val uiState: StateFlow<BookingsUiState> = _uiState.asStateFlow()  
  
    private val _actionResult = MutableStateFlow<String?>(null)  
    val actionResult: StateFlow<String?> = _actionResult.asStateFlow()  
  
    private var currentNic: String? = null  
  
    fun fetchReservations(nic: String) {  
        currentNic = nic  
        viewModelScope.launch {  
            _uiState.value = BookingsUiState.Loading  
            try {  
                val response = apiService.searchReservations(  
                    nic = nic, status = null, from = null, to = null, stationId = null  
                )  
                _uiState.value = BookingsUiState.Success(response)  
            } catch (e: Exception) {  
                _uiState.value = BookingsUiState.Error(e.message ?: "Failed to load reservations")  
            }  
        }  
    }  
}
```

```

    }
}

fun createReservation(request: CreateReservationRequest) {
    viewModelScope.launch {
        try {
            val response = apiService.createReservation(request)
            if (response.isSuccessful) {
                _actionResult.value = "Booking confirmed!"
                currentNic?.let { fetchReservations(it) }
            } else {
                _actionResult.value = "Failed to create booking"
            }
        } catch (e: Exception) {
            _actionResult.value = e.message ?: "Network error"
        }
    }
}

fun cancelReservation(id: String, reason: String? = "Cancelled by user") {
    viewModelScope.launch {
        try {
            val response = apiService.cancelReservation(id, CancelReservationRequest(reason))
            if (response.isSuccessful) {
                _actionResult.value = "Reservation cancelled."
                currentNic?.let { fetchReservations(it) }
            } else {
                _actionResult.value = "Failed to cancel"
            }
        } catch (e: Exception) {
            _actionResult.value = e.message ?: "Network error"
        }
    }
}

fun clearActionResult() {
    _actionResult.value = null
}
}

```

Testing & Configuration Files:

app/src/main/java/com/smartmicrogrid/ui/prosumer/TestBookingsActivity.kt

```

// =====
// File: TestBookingsActivity.kt
// Purpose: Temporary launcher to capture UI screenshots of the
//          Bookings Screen using mock data. Bypasses login and
//          network requirements. Does NOT touch real workflow files.
// Author: Migara (Temporary Test File)
// =====
package com.smartmicrogrid.ui.prosumer

import android.os.Bundle

```



```

import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.smartmicrogrid.data.remote.dto.ReservationResponse
import com.smartmicrogrid.ui.theme.SmartMicrogridTheme

class TestBookingsActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SmartMicrogridTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    MockBookingsScreen()
                }
            }
        }
    }
}

@Composable
fun MockBookingsScreen() {
    // Hardcoded mock data to show the UI perfectly without network/backend
    val mockReservations = listOf(
        ReservationResponse(
            id = "64f1a2b3c4d5e6f7g8h9i0j1",
            prosumerNic = "991234567V",
            stationId = "station-alpha-001",
            slotId = "slot-05",
            scheduledAt = "2026-10-05T14:00:00Z",
            status = "Confirmed",
            qrToken = "MOCK_QR_TOKEN_FOR_TESTING_12345",
            createdAt = "2026-09-28T10:00:00Z",
            completedAt = null,
            cancelReason = null
        ),
        ReservationResponse(
            id = "99z8y7x6w5v4u3t2s1r0q9p8",
            prosumerNic = "991234567V",
            stationId = "station-beta-002",
            slotId = "slot-12",
            scheduledAt = "2026-10-10T09:30:00Z",
            status = "Completed",
            qrToken = null,
            createdAt = "2026-09-20T08:00:00Z",
            completedAt = "2026-10-10T10:00:00Z",
            cancelReason = null
        )
    )
    Column(

```

```

        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ){
        Text(
            "My Bookings (Test Mode)",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold
        )
        Text(
            "Showing mock data for UI screenshots",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )

        // Reusing your REAL ReservationCard component to show exactly how it looks!
        mockReservations.forEach { reservation ->
            ReservationCard(
                reservation = reservation,
                onCancel = { /* Do nothing in test mode */ }
            )
        }
    }
}

```

5.2.3 UI (Screenshots)

1. User Management Dashboard:

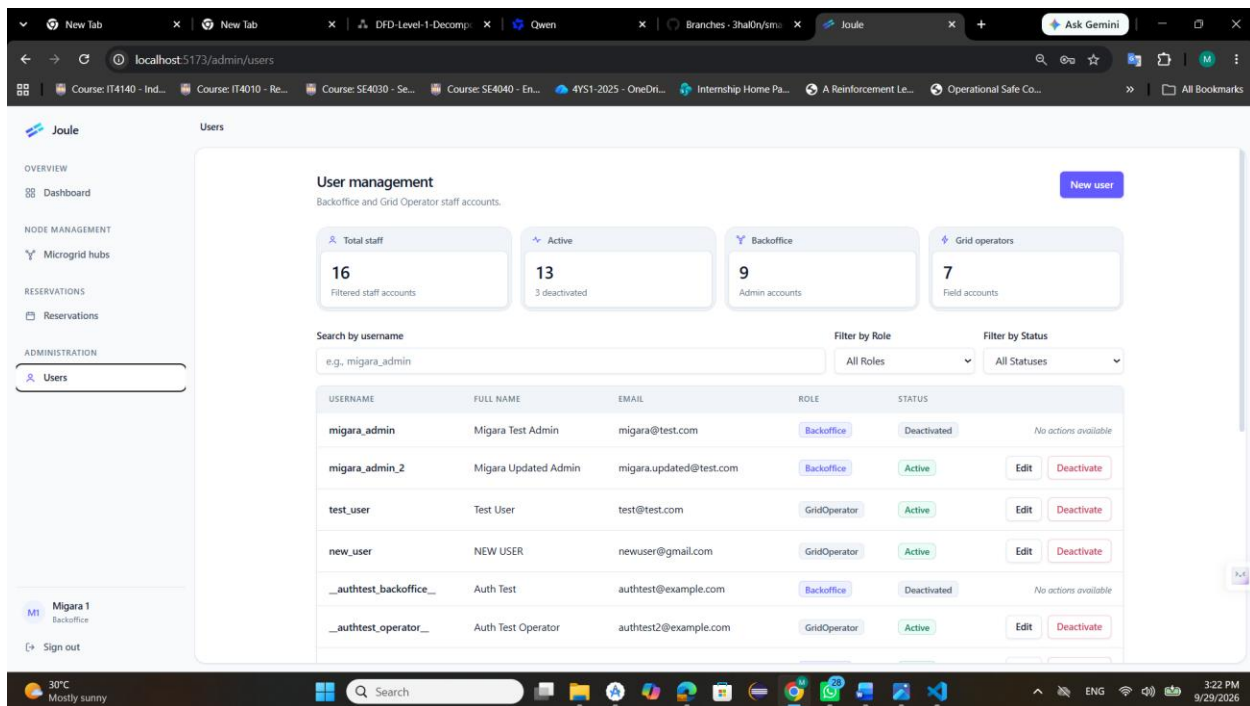


Figure 20 : User Management Dashboard

2. Filtered Table View

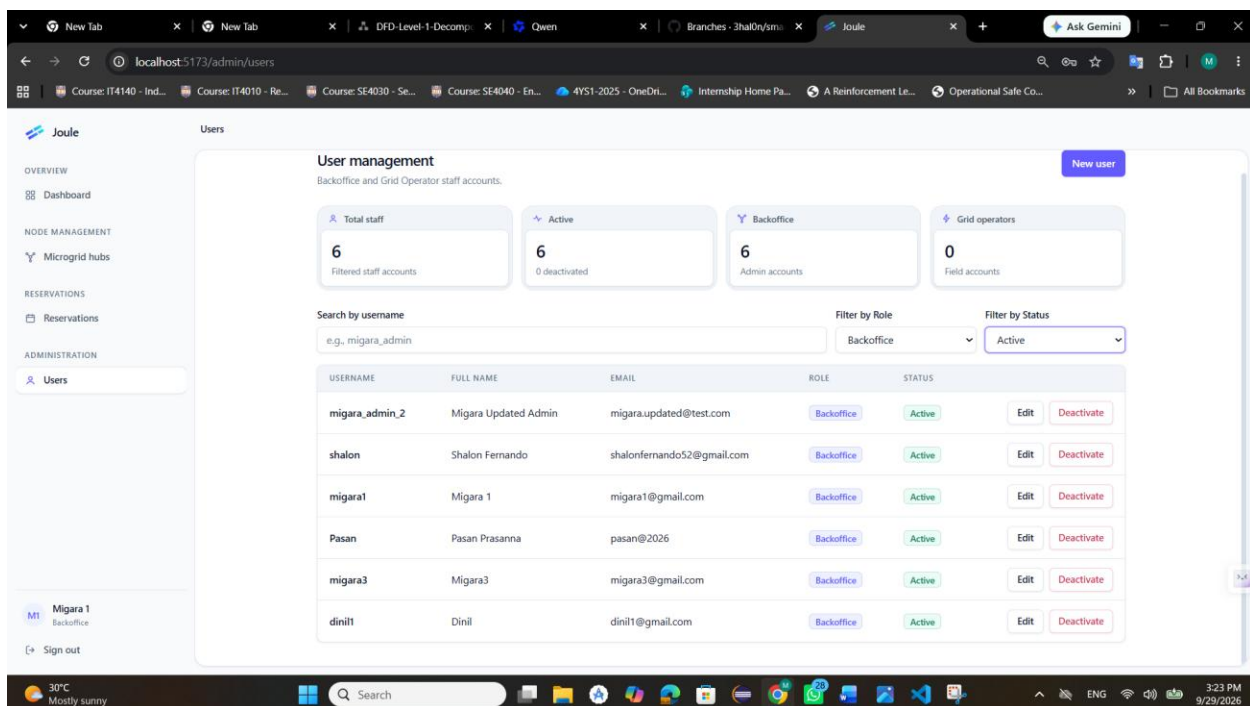


Figure 21 : Filtered Table View

3. Create/Edit User Modal:

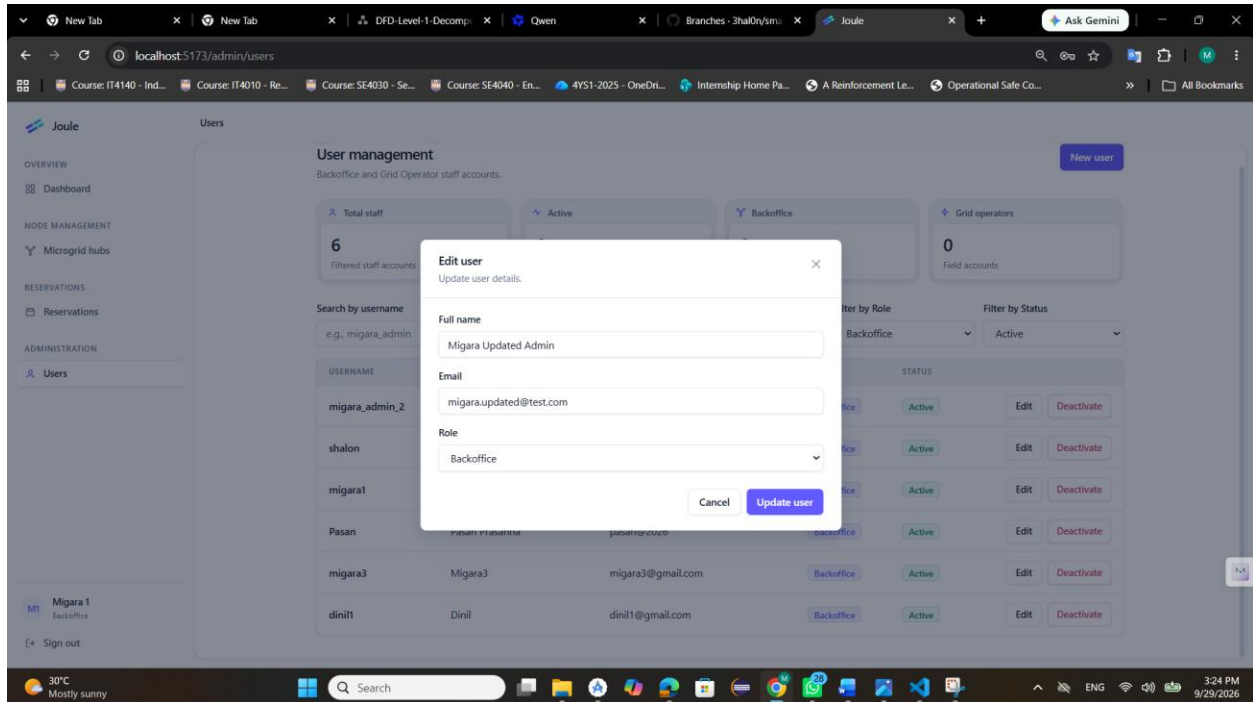


Figure 22 : Edit User Modal

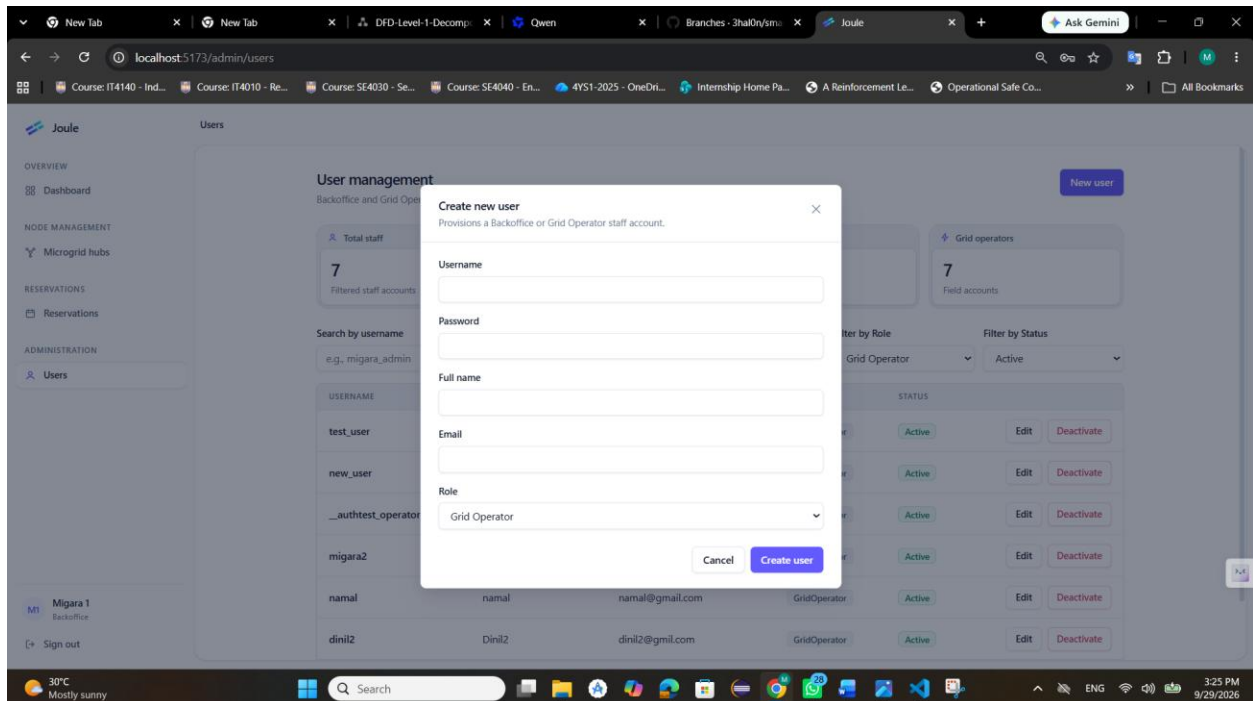


Figure 23 : Create User Modal

Mobile Application UI

1. Prosumer Bookings List (Empty State)

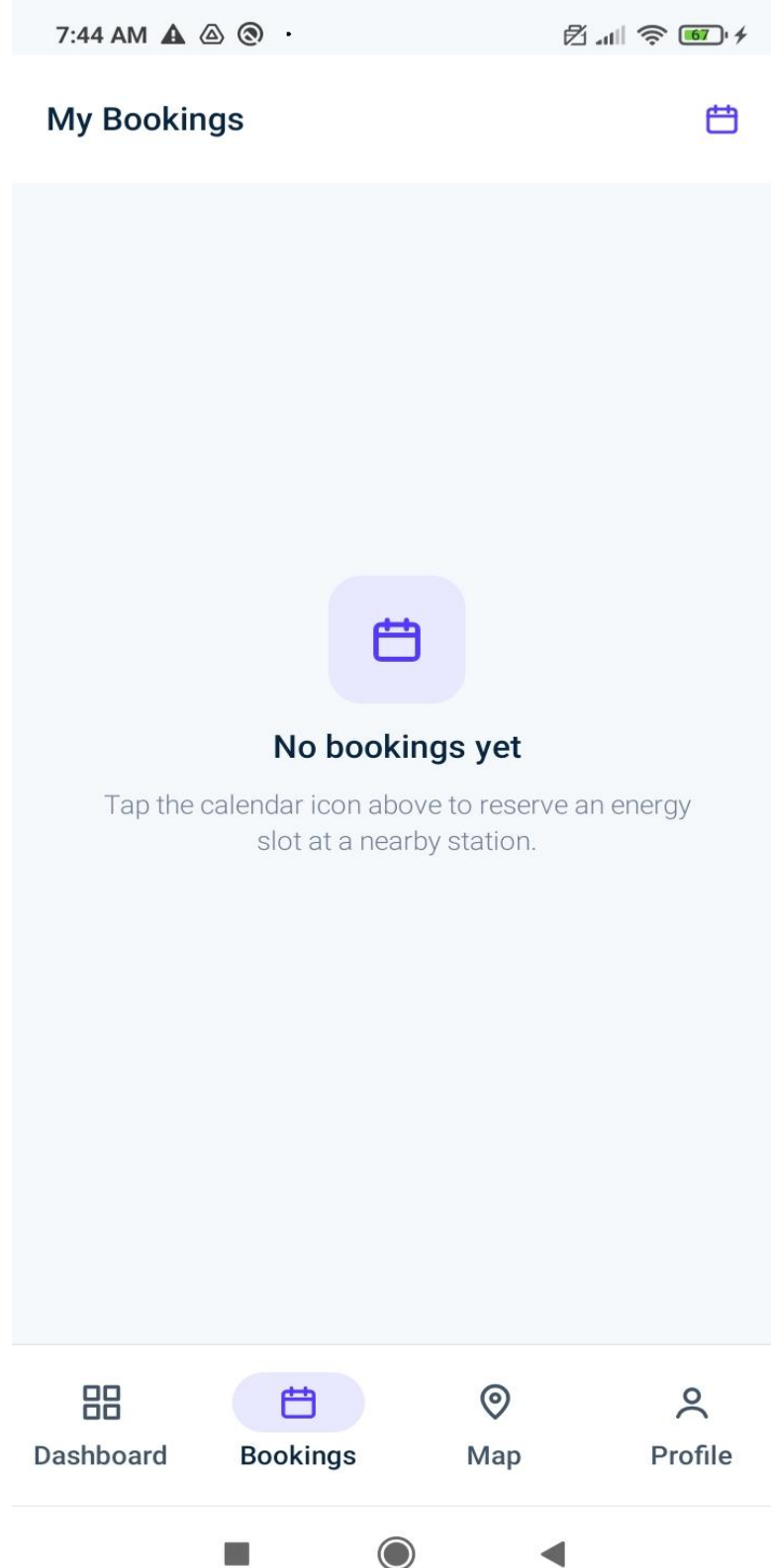


Figure 24 : Prosumer Bookings List (Empty State)

2. Create New Booking Screen

9:39 AM    ...

   82% 

< **New Booking**

Reserve a battery charging or discharging slot at your preferred microgrid station.

Microgrid Station Hub

Select Station Hub

Test Station Beta

Battery Slot

Slot #1 · Charging (100 kWh)

Scheduled Reservation Time

 **Wed, 30 Sep 2026 · 10:00**
Permitted window: Today up to 7 days in advance

Confirm Booking

 Dashboard

 Bookings

 Map

 Profile

Figure 25 : Create New Booking Screen

3. Booking Card (Confirmed)

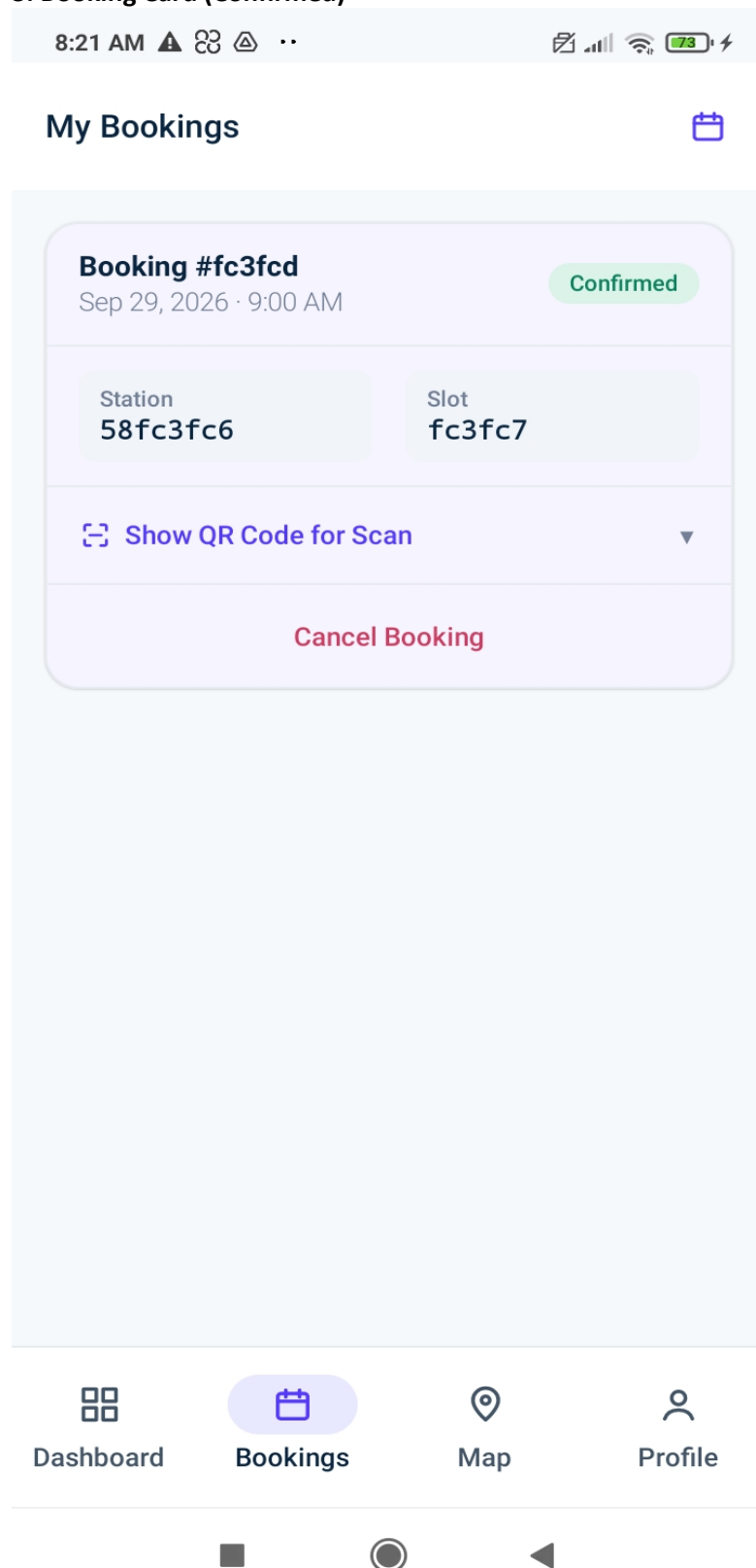


Figure 26 : Booking Card (Confirmed)

4. Generated QR Code

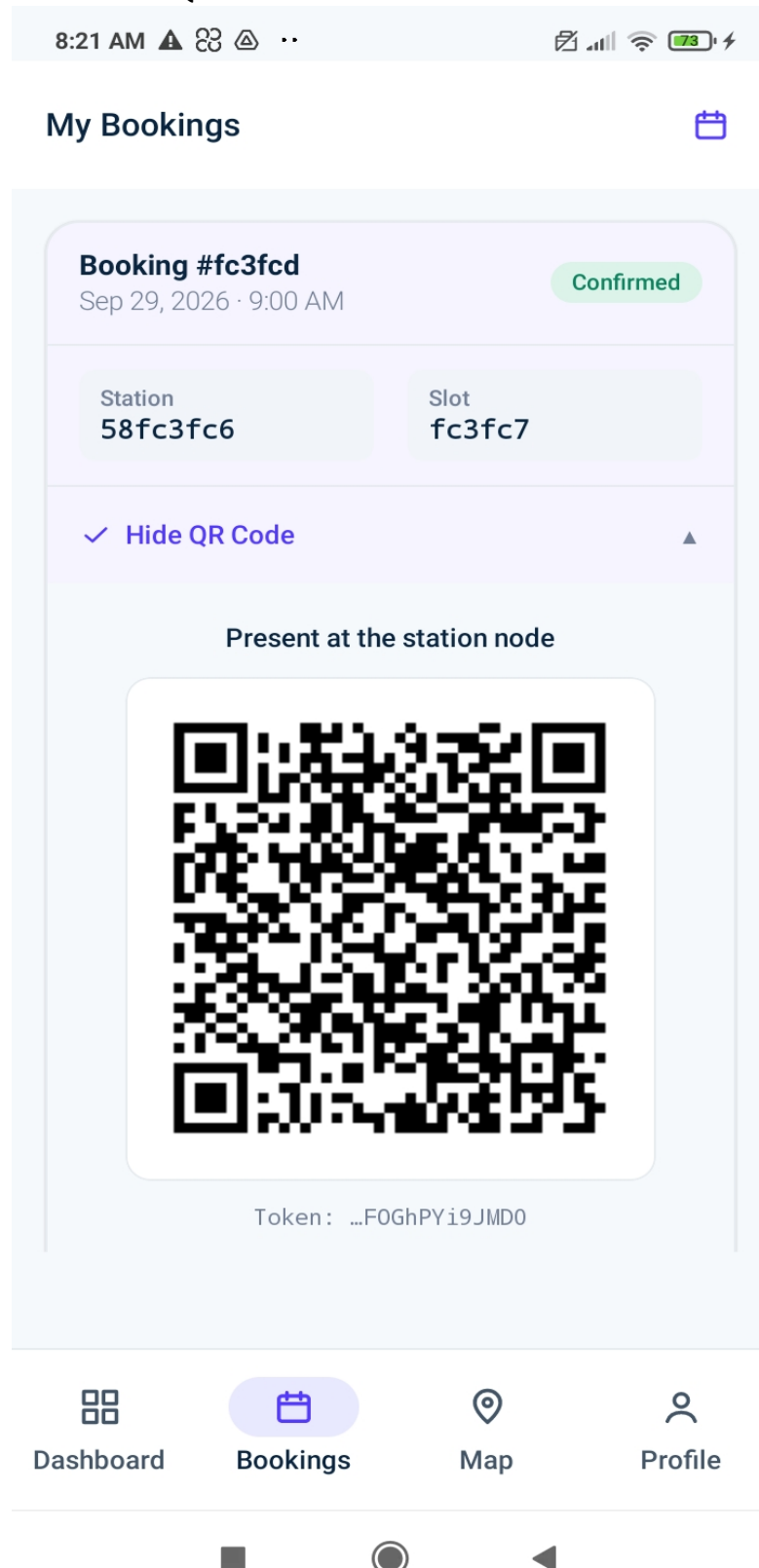


Figure 27 : Generated QR Code

5. Cancel Booking Action

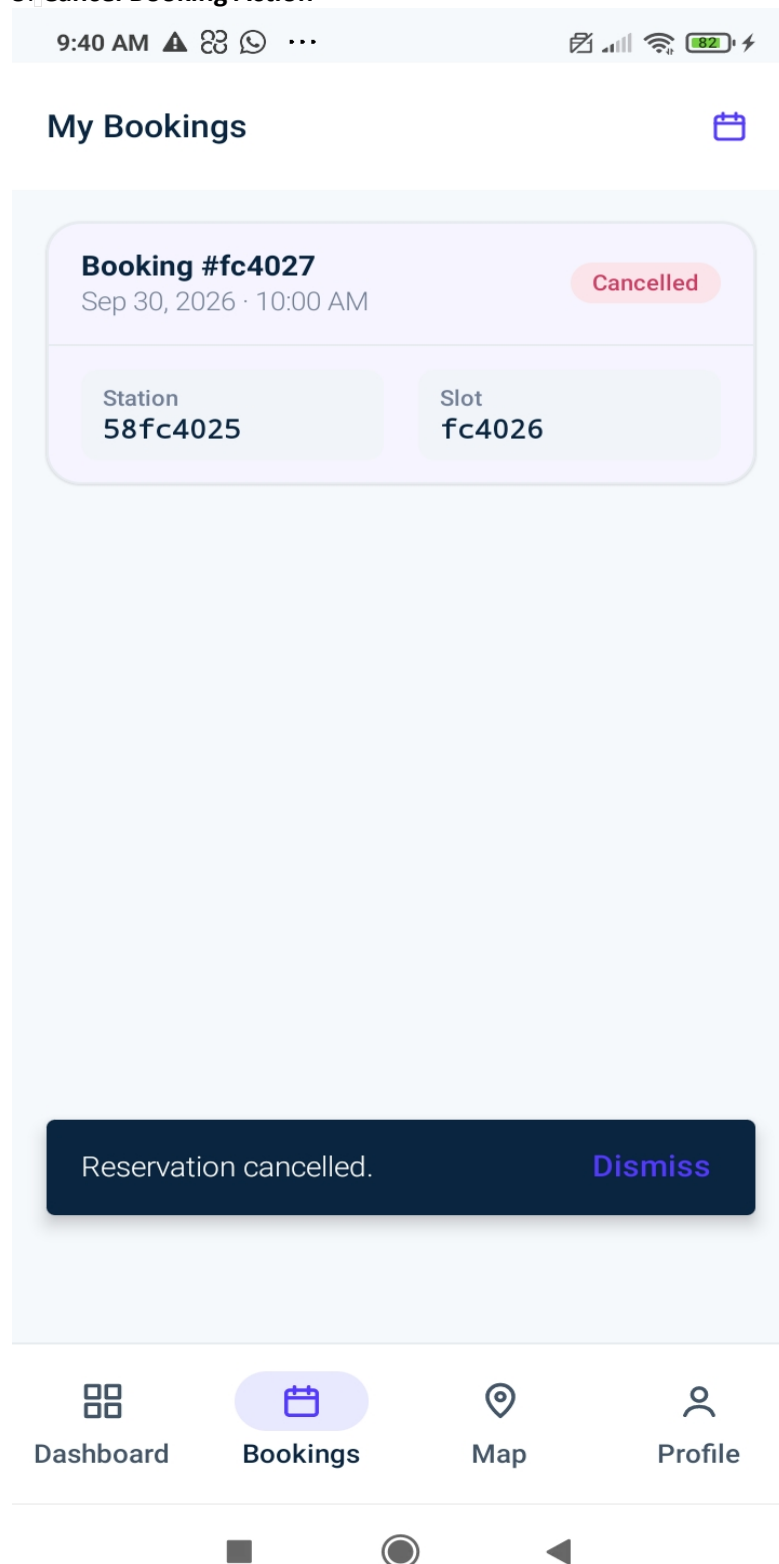


Figure 28 : Cancel Booking Action

5.3 Rukshan E A Y (IT23263512)

5.3.1 Description

Overall Contribution:

I was responsible for the end-to-end Prosumer Account Management and Authentication lifecycle across the Backend, Web, and Mobile applications. My primary focus was ensuring strict adherence to the FAT Service pattern, where all business rules are centralized, and explicitly implementing the full CRUD (Create, Read, Update, Deactivate, Reactivate) lifecycle for Prosumer accounts using the National Identity Card (NIC) as the primary key and ensuring only Backoffice officers can reactivate deactivated accounts.

- **Web Application:** I conceptualized and implemented the core Prosumer management logic, ensuring that all data operations strictly revolve around the NIC as the unique identifier. I designed the system to support a complete CRUD workflow: creating new Prosumer profiles, reading account details, updating profile information, and deactivating accounts. A key architectural idea was enforcing role-based access control at the service level, ensuring that while Prosumers can request deactivation, the critical "Reactivate" operation is exclusively restricted to Backoffice officers, maintaining strict system integrity and data security.
- **Web Application:** I developed the Backoffice administrative interfaces to provide a centralized hub for managing Prosumer lifecycles. The core idea was to give administrators full, intuitive CRUD control over user accounts. I engineered a responsive dashboard that allows Backoffice officers to seamlessly Create new accounts, Read and filter existing Prosumer lists, Update profile details, and Deactivate or Reactivate accounts based on their current status. To enhance usability and data integrity, I integrated real-time search, structured pagination, and strict client-side form validation (using Regex for NIC and email formats) to ensure only clean, valid data enters the system.
- **Mobile Application:** I built the Prosumer self-service account control flow using pure native Android (Kotlin) and Jetpack Compose. The central idea was to empower users with secure, self-managed CRUD operations without compromising system rules. Prosumers can Create their account via NIC-based registration, Read their profile details, Update their contact information, and request to Deactivate their account. Crucially, I fulfilled the "SQLite local persistence" rubric requirement by designing a local caching

mechanism for session state, ensuring a seamless, uninterrupted user experience while keeping all actual CRUD data mutations strictly delegated to the central web service.

5.3.2 Code Files

Web Service (Backend)

SmartMicrogrid.Api/Models/ Prosumer.cs

```
// =====  
// File: Prosumer.cs  
// Purpose: Defines the MongoDB document structure for Prosumers.  
//     NIC is used as the primary business key for lookups.  
// Author: Rukshan  
// =====  
  
using MongoDB.Bson;  
using MongoDB.Bson.Serialization.Attributes;  
  
namespace SmartMicrogrid.Api.Models  
{  
    [BsonIgnoreExtraElements]  
    public class Prosumer  
    {  
        [BsonId]  
        [BsonRepresentation(BsonType.ObjectId)]  
        public string Id { get; set; } = null!;  
  
        [BsonElement("role")]  
        public string Role { get; set; } = "Prosumer";  
  
        [BsonElement("nic")]  
        public string Nic { get; set; } = null!; // Primary Business Key  
  
        [BsonElement("fullName")]  
        public string FullName { get; set; } = null!;  
  
        [BsonElement("email")]  
        public string Email { get; set; } = null!;  
  
        [BsonElement("phone")]  
        public string? Phone { get; set; }  
  
        [BsonElement("address")]  
        public string? Address { get; set; }  
  
        [BsonElement("profilePicture")]  
        public string? ProfilePicture { get; set; }  
  
        [BsonElement("passwordHash")]  
        public string PasswordHash { get; set; } = null!;  
  
        [BsonElement("status")]
```

```

    public string Status { get; set; } = "PendingActivation"; // Active, PendingActivation, Deactivated

    [BsonElement("deactivationRequestedAt")]
    public DateTime? DeactivationRequestedAt { get; set; }

    [BsonElement("createdAt")]
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    [BsonElement("updatedAt")]
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
}

```

SmartMicrogrid.Api/Models/ ProsumerDtos.cs

```

// =====
// File: ProsumerDtos.cs
// Purpose: Data Transfer Objects for Prosumer management operations.
// Author: Rukshan
// =====

namespace SmartMicrogrid.Api.Models
{
    // DTO for Mobile App Registration or Backoffice Manual Creation
    public class ProsumerRegistrationDto
    {
        public string Nic { get; set; } = null!;
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public string Password { get; set; } = null!;
    }

    // DTO for Backoffice/Admin or Self Updates
    public class UpdateProsumerDto
    {
        public string? FullName { get; set; }
        public string? Email { get; set; }
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public string? ProfilePicture { get; set; }
    }

    // DTO for API Responses (Hides password hash for security)
    public class ProsumerResponseDto
    {
        public string Id { get; set; } = null!;
        public string Nic { get; set; } = null!;
        public string FullName { get; set; } = null!;
        public string Email { get; set; } = null!;
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public string? ProfilePicture { get; set; }
    }
}

```

```

        public string Status { get; set; } = null!;
        public DateTime? DeactivationRequestedAt { get; set; }
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
    }
}

```

SmartMicrogrid.Api/ Controller/ AdminProsumersController.cs

```

// =====
// File: AdminProsumersController.cs
// Purpose: Handles Backoffice administration of Prosumer profiles.
// Enforces that ONLY Backoffice can create, deactivate,
// activate, and reactivate accounts.
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/admin/prosumers")]
    [Authorize(Roles = "Backoffice")] // STRICT: Only Backoffice can access these endpoints
    public class AdminProsumersController : ControllerBase
    {
        private readonly ProsumerService _prosumerService;

        public AdminProsumersController(ProsumerService prosumerService)
        {
            _prosumerService = prosumerService;
        }

        // GET: api/admin/prosumers/pending
        // Returns only prosumers awaiting Backoffice activation (for the web app pending view)
        [HttpGet("pending")]
        public async Task<ActionResult<List<ProsumerResponseDto>>> GetPendingProsumers()
        {
            var allProsumers = await _prosumerService.GetAllProsumersAsync();
            var pending = allProsumers.Where(p => p.Status == "PendingActivation").ToList();
            return Ok(pending);
        }

        // GET: api/admin/prosumers
        [HttpGet]
        public async Task<ActionResult<List<ProsumerResponseDto>>> GetAllProsumers()
        {
            var prosumers = await _prosumerService.GetAllProsumersAsync();
            return Ok(prosumers);
        }

        // GET: api/admin/prosumers/{nic}

```

```

[HttpGet("{nic}")]
public async Task<ActionResult<ProsumerResponseDto>> GetProsumerByNic(string nic)
{
    var prosumer = await _prosumerService.GetByNicAsync(nic);
    if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

    return Ok(prosumer);
}

// POST: api/admin/prosumers
// Allows Backoffice to manually create a prosumer profile
[HttpPost]
public async Task<ActionResult> CreateProsumer([FromBody] ProsumerRegistrationDto dto)
{
    var success = await _prosumerService.CreateManuallyAsync(dto);
    if (!success) return Conflict(new { message = "A prosumer with this NIC already exists." });

    return CreatedAtAction(nameof(GetProsumerByNic), new { nic = dto.Nic }, new { message = "Prosumer created successfully." });
}

// PUT: api/admin/prosumers/{nic}
[HttpPut("{nic}")]
public async Task<ActionResult> UpdateProsumer(string nic, [FromBody] UpdateProsumerDto dto)
{
    var success = await _prosumerService.UpdateAsync(nic, dto);
    if (!success) return NotFound(new { message = "Prosumer not found or no changes made." });

    return NoContent(); // 204 Success
}

// PUT: api/admin/prosumers/{nic}/deactivate
[HttpPut("{nic}/deactivate")]
public async Task<ActionResult> DeactivateProsumer(string nic)
{
    var success = await _prosumerService.DeactivateAsync(nic);
    if (!success) return NotFound(new { message = "Prosumer not found." });

    return NoContent(); // 204 Success
}

// PUT: api/admin/prosumers/{nic}/activate
// Activates a prosumer who is currently in "PendingActivation" status
[HttpPut("{nic}/activate")]
public async Task<ActionResult> ActivateProsumer(string nic)
{
    var prosumer = await _prosumerService.GetByNicAsync(nic);
    if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

    if (prosumer.Status != "PendingActivation")
    {
        return BadRequest(new { message = "Only pending accounts can be activated." });
    }

    var success = await _prosumerService.ActivateAsync(nic);
    if (!success) return StatusCode(500, new { message = "Failed to activate prosumer." });

    return NoContent(); // 204 Success
}

```

```

    }

    // PUT: api/admin/prosumers/{nic}/reactivate
    // CRITICAL RULE: Deactivated accounts can ONLY be reactivated by a Backoffice officer.
    [HttpPut("{nic}/reactivate")]
    public async Task<ActionResult> ReactivateProsumer(string nic)
    {
        var prosumer = await _prosumerService.GetByNicAsync(nic);
        if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

        if (prosumer.Status != "Deactivated")
        {
            return BadRequest(new { message = "Only deactivated accounts can be reactivated." });
        }

        var success = await _prosumerService.ReactivateAsync(nic);
        if (!success) return StatusCode(500, new { message = "Failed to reactivate prosumer." });

        return NoContent(); // 204 Success
    }
}

```

SmartMicrogrid.Api/ Controller/ ProsumerAuthController.cs

```

// =====
// File: ProsumerAuthController.cs
// Purpose: Handles Prosumer authentication (Login) for the Mobile App.
//          Calls ProsumerService to enforce business rules (Status checks).
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Mvc;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers
{
    [ApiController]
    [Route("api/auth/prosumer")]
    public class ProsumerAuthController : ControllerBase
    {
        private readonly ProsumerService _prosumerService;
        private readonly JwtService _jwtService;

        public ProsumerAuthController(ProsumerService prosumerService, JwtService jwtService)
        {
            _prosumerService = prosumerService;
            _jwtService = jwtService;
        }

        // POST: api/auth/prosumer/login
        [HttpPost("login")]
        public async Task<ActionResult> Login([FromBody] LoginProsumerDto dto)

```

```

{
    // 1. Call Service to handle business logic (Status checks, Password verify)
    var result = await _prosumerService.AuthenticateAsync(dto.Nic, dto.Password);

    // 2. Handle Failure Cases
    if (!result.Success)
    {
        // Return 403 Forbidden for Pending/Deactivated, 401 Unauthorized for wrong credentials
        if (result.Message.Contains("pending") || result.Message.Contains("deactivated"))
        {
            return StatusCode(403, new { message = result.Message });
        }
        return Unauthorized(new { message = result.Message });
    }

    // 3. Generate JWT Token on Success
    var prosumer = result.Prosumer!;
    var token = _jwtService.GenerateToken(prosumer.Id, prosumer.Role, prosumer.FullName, prosumer.Nic);

    // 4. Return Success Response
    return Ok(new
    {
        token = token,
        role = prosumer.Role,
        fullName = prosumer.FullName,
        nic = prosumer.Nic,
        status = prosumer.Status
    });
}

// DTO for Prosumer Login Request
public class LoginProsumerDto
{
    public string Nic { get; set; } = null!;
    public string Password { get; set; } = null!;
}
}

```

SmartMicrogrid.Api/ Controller/ ProsumerController.cs

```

// =====
// File: ProsumerController.cs
// Purpose: Handles Prosumer self-service actions (Register, Edit, Request Deactivation).
//         Used by the Native Android Mobile Application.
// Author: Rukshan
// =====

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;
using SmartMicrogrid.Api.Models;
using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Controllers

```



```

{
    [ApiController]
    [Route("api/prosumers")]
    public class ProsumerController : ControllerBase
    {
        private readonly ProsumerService _prosumerService;

        public ProsumerController(ProsumerService prosumerService)
        {
            _prosumerService = prosumerService;
        }

        // POST: api/prosumers/register
        // AllowAnonymous so the mobile app can register without a token
        [AllowAnonymous]
        [HttpPost("register")]
        public async Task<ActionResult> Register([FromBody] ProsumerRegistrationDto dto)
        {
            var success = await _prosumerService.RegisterAsync(dto);
            if (!success) return Conflict(new { message = "A prosumer with this NIC already exists." });

            return CreatedAtAction(nameof(GetProfile), new { nic = dto.Nic }, new { message = "Registration successful. Pending Backoffice activation." });
        }

        // GET: api/prosumers/{nic}
        [Authorize]
        [HttpGet("{nic}")]
        public async Task<ActionResult<ProsumerResponseDto>> GetProfile(string nic)
        {
            // Security: Ensure the user can only fetch their own profile (or let Backoffice fetch any)
            var userNic = User.FindFirstValue("nic");
            var role = User.FindFirstValue(ClaimTypes.Role);

            if (role != "Backoffice" && userNic != nic)
            {
                return Forbid(); // 403 Forbidden
            }

            var prosumer = await _prosumerService.GetByNicAsync(nic);
            if (prosumer == null) return NotFound(new { message = "Prosumer not found." });

            return Ok(prosumer);
        }

        // PUT: api/prosumers/{nic}
        [Authorize]
        [HttpPut("{nic}")]
        public async Task<ActionResult> UpdateProfile(string nic, [FromBody] UpdateProsumerDto dto)
        {
            var userNic = User.FindFirstValue("nic");
            if (userNic != nic) return Forbid();

            var success = await _prosumerService.UpdateAsync(nic, dto);
            if (!success) return NotFound(new { message = "Prosumer not found." });

            return NoContent();
        }
    }
}

```

```

// GET: api/prosumers?status=Active
// Returns the list of Prosumers (optionally filtered by status).
// Used by the Web App's reservation form so operators can pick a
// Prosumer from a dropdown instead of typing an NIC by hand.
// Backoffice and GridOperator may call this.
//[Authorize(Roles = "Backoffice,GridOperator")]
[HttpGet]
public async Task<ActionResult<IEnumerable<ProsumerResponseDto>>> List(
    [FromQuery] string? status)
{
    var prosumers = await _prosumerService.ListAsync(status);
    return Ok(prosumers);
}

// PUT: api/prosumers/{nic}/request-deactivation
[Authorize]
[HttpPut("{nic}/request-deactivation")]
public async Task<ActionResult> RequestDeactivation(string nic)
{
    var userNic = User.FindFirstValue("nic");
    if (userNic != nic) return Forbid();

    var success = await _prosumerService.RequestDeactivationAsync(nic);
    if (!success) return NotFound(new { message = "Prosumer not found." });

    return NoContent();
}
}
}

```

SmartMicrogrid.Api/ Services/ ProsumerService.cs

```

// =====
// File: ProsumerService.cs
// Purpose: Handles all business logic for Prosumer management.
// Enforces rules like reactivation only by Backoffice.
// Author: Rukshan
// =====

using MongoDB.Driver;
using SmartMicrogrid.Api.Data;
using SmartMicrogrid.Api.Models;

namespace SmartMicrogrid.Api.Services
{
    public class ProsumerService
    {
        private readonly IMongoCollection<Prosumer> _prosumers;

        public ProsumerService(MongoDbContext context)
        {
            // Explicitly targeting the unified "Users" collection as per architecture.md
            _prosumers = context.Database.GetCollection<Prosumer>("Users");
        }
    }
}

```

```

}

// Get all prosumers (for Backoffice dashboard)
public async Task<List<ProsumerResponseDto>> GetAllProsumersAsync()
{
    var filter = Builders<Prosumer>.Filter.Eq(p => p.Role, "Prosumer");
    var prosumers = await _prosumers.Find(filter).ToListAsync();

    return prosumers.Select(p => new ProsumerResponseDto
    {
        Id = p.Id,
        Nic = p.Nic,
        FullName = p.FullName,
        Email = p.Email,
        Phone = p.Phone,
        Address = p.Address,
        ProfilePicture = p.ProfilePicture,
        Status = p.Status,
        DeactivationRequestedAt = p.DeactivationRequestedAt,
        CreatedAt = p.CreatedAt,
        UpdatedAt = p.UpdatedAt
    }).ToList();
}

// Returns all prosumers, optionally filtered by status.
// Uses a projection so MongoDB only returns the fields the DTO needs -
// this prevents deserialization 500s on documents missing optional
// fields (e.g. legacy test data with no CreatedAt, no Phone, etc.).
public async Task<List<ProsumerResponseDto>> ListAsync(string? status)
{
    var builder = Builders<Prosumer>.Filter;
    var filter = builder.Eq(p => p.Role, "Prosumer");

    if (!string.IsNullOrEmpty(status))
    {
        filter &= builder.Eq(p => p.Status, status);
    }

    return await _prosumers
        .Find(filter)
        .SortByDescending(p => p.CreatedAt)
        .Project<ProsumerResponseDto>(Builders<Prosumer>.Projection
            .Expression(p => new ProsumerResponseDto
            {
                Id = p.Id,
                Nic = p.Nic,
                FullName = p.FullName,
                Email = p.Email,
                Phone = p.Phone,
                Address = p.Address,
                Status = p.Status,
                DeactivationRequestedAt = p.DeactivationRequestedAt,
                CreatedAt = p.CreatedAt,
                UpdatedAt = p.UpdatedAt
            })
        ).ToListAsync();
}

```

```

// Get a single prosumer by NIC
public async Task<ProsumerResponseDto?> GetByNicAsync(string nic)
{
    var filter = Builders<Prosumer>.Filter.And(
        Builders<Prosumer>.Filter.Eq(p => p.Role, "Prosumer"),
        Builders<Prosumer>.Filter.Eq(p => p.Nic, nic)
    );
    var prosumer = await _prosumers.Find(filter).FirstOrDefaultAsync();

    if (prosumer == null) return null;

    return new ProsumerResponseDto
    {
        Id = prosumer.Id,
        Nic = prosumer.Nic,
        FullName = prosumer.FullName,
        Email = prosumer.Email,
        Phone = prosumer.Phone,
        Address = prosumer.Address,
        ProfilePicture = prosumer.ProfilePicture,
        Status = prosumer.Status,
        DeactivationRequestedAt = prosumer.DeactivationRequestedAt,
        CreatedAt = prosumer.CreatedAt,
        UpdatedAt = prosumer.UpdatedAt
    };
}

// Create prosumer manually (Backoffice requirement)
public async Task<bool> CreateManuallyAsync(ProsumerRegistrationDto dto)
{
    var exists = await _prosumers.Find(p => p.Nic == dto.Nic).AnyAsync();
    if (exists) return false;

    var newProsumer = new Prosumer
    {
        Nic = dto.Nic,
        FullName = dto.FullName,
        Email = dto.Email,
        Phone = dto.Phone,
        Address = dto.Address,
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),
        Status = "Active", // Backoffice creation defaults to Active
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _prosumers.InsertOneAsync(newProsumer);
    return true;
}

// Register a new prosumer (Called by Mobile App, defaults to PendingActivation)
public async Task<bool> RegisterAsync(ProsumerRegistrationDto dto)
{
    var exists = await _prosumers.Find(p => p.Nic == dto.Nic).AnyAsync();
    if (exists) return false;

    var newProsumer = new Prosumer
    {

```

```

        Nic = dto.Nic,
        FullName = dto.FullName,
        Email = dto.Email,
        Phone = dto.Phone,
        Address = dto.Address,
        PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),
        Status = "PendingActivation",
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _prosumers.InsertOneAsync(newProsumer);
    return true;
}

// Update prosumer details
public async Task<bool> UpdateAsync(string nic, UpdateProsumerDto dto)
{
    var filter = Builders<Prosumer>.Filter.Eq(p => p.Nic, nic);
    var update = Builders<Prosumer>.Update
        .Set(p => p.FullName, dto.FullName)
        .Set(p => p.Email, dto.Email)
        .Set(p => p.Phone, dto.Phone)
        .Set(p => p.Address, dto.Address)
        .Set(p => p.ProfilePicture, dto.ProfilePicture)
        .Set(p => p.UpdatedAt, DateTime.UtcNow);

    var result = await _prosumers.UpdateOneAsync(filter, update);
    return result.ModifiedCount > 0;
}

// Deactivate prosumer (Can be forced by Backoffice)
public async Task<bool> DeactivateAsync(string nic)
{
    var filter = Builders<Prosumer>.Filter.Eq(p => p.Nic, nic);
    var update = Builders<Prosumer>.Update
        .Set(p => p.Status, "Deactivated")
        .Set(p => p.UpdatedAt, DateTime.UtcNow);

    var result = await _prosumers.UpdateOneAsync(filter, update);
    return result.ModifiedCount > 0;
}

// Activate prosumer (PendingActivation -> Active)
public async Task<bool> ActivateAsync(string nic)
{
    var filter = Builders<Prosumer>.Filter.Eq(p => p.Nic, nic);
    var update = Builders<Prosumer>.Update
        .Set(p => p.Status, "Active")
        .Set(p => p.UpdatedAt, DateTime.UtcNow);

    var result = await _prosumers.UpdateOneAsync(filter, update);
    return result.ModifiedCount > 0;
}

// Reactivate prosumer (Deactivated -> Active) - STRICTLY Backoffice only
public async Task<bool> ReactivateAsync(string nic)
{

```

```

        var filter = Builders<Prosumer>.Filter.Eq(p => p.Nic, nic);
        var update = Builders<Prosumer>.Update
            .Set(p => p.Status, "Active")
            .Set(p => p.DeactivationRequestedAt, null)
            .Set(p => p.UpdatedAt, DateTime.UtcNow);

        var result = await _prosumers.UpdateOneAsync(filter, update);
        return result.ModifiedCount > 0;
    }

    // Prosumer requests deactivation (Mobile app self-service)
    public async Task<bool> RequestDeactivationAsync(string nic)
    {
        var filter = Builders<Prosumer>.Filter.Eq(p => p.Nic, nic);
        var update = Builders<Prosumer>.Update
            .Set(p => p.Status, "Deactivated")
            .Set(p => p.DeactivationRequestedAt, DateTime.UtcNow)
            .Set(p => p.UpdatedAt, DateTime.UtcNow);

        var result = await _prosumers.UpdateOneAsync(filter, update);
        return result.ModifiedCount > 0;
    }

    // Authenticate prosumer and validate business rules (Status checks)
    public async Task<(bool Success, string Message, Prosumer? Prosumer)> AuthenticateAsync(string nic, string password)
    {
        // 1. Find prosumer by NIC and Role
        var filter = Builders<Prosumer>.Filter.And(
            Builders<Prosumer>.Filter.Eq(p => p.Nic, nic),
            Builders<Prosumer>.Filter.Eq(p => p.Role, "Prosumer")
        );
        var prosumer = await _prosumers.Find(filter).FirstOrDefaultAsync();

        // 2. Check if exists
        if (prosumer == null)
            return (false, "Invalid NIC or password.", null);

        // 3. Business Rule: Reject Pending Activation
        if (prosumer.Status == "PendingActivation")
            return (false, "Account is pending activation by Backoffice.", null);

        // 4. Business Rule: Reject Deactivated accounts
        if (prosumer.Status == "Deactivated")
            return (false, "Account has been deactivated. Contact Backoffice.", null);

        // 5. Verify Password
        if (!BCrypt.Net.BCrypt.Verify(password, prosumer.PasswordHash))
            return (false, "Invalid NIC or password.", null);

        // 6. Success
        return (true, "Success", prosumer);
    }
}

```

```
// =====
// File: JwtService.cs
// Purpose: Handles JWT token generation for authenticated users.
// Adds 'nic' claim specifically for Prosumers.
// Author: Rukshan
// =====

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

namespace SmartMicrogrid.Api.Services
{
    public class JwtService
    {
        private readonly IConfiguration _configuration;

        public JwtService(IConfiguration configuration)
        {
            _configuration = configuration;
        }

        // Generates a JWT token with Role and NIC claims
        public string GenerateToken(string userId, string role, string fullName, string? nic = null)
        {
            var jwtKey = _configuration["Jwt:Key"] ?? "YourSuperSecretKeyHereAtLeast32Chars!";
            var jwtIssuer = _configuration["Jwt:Issuer"] ?? "SmartMicrogridApi";
            var jwtAudience = _configuration["Jwt:Audience"] ?? "SmartMicrogridClients";

            var claims = new List<Claim>
            {
                new Claim(JwtRegisteredClaimNames.Sub, userId),
                new Claim(ClaimTypes.Role, role),
                new Claim("fullName", fullName),
                new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
            };

            // Add NIC claim only for Prosumers (used for authorization checks)
            if (role == "Prosumer" && !string.IsNullOrEmpty(nic))
            {
                claims.Add(new Claim("nic", nic));
            }

            var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: jwtIssuer,
                audience: jwtAudience,
                claims: claims,
                expires: DateTime.UtcNow.AddDays(1), // Token valid for 1 day
                signingCredentials: creds
            );

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

```
}  
}
```

Web App (Frontend)

WebApp/src/pages/admin/ ProsumersPage.jsx

```
// =====  
// File: ProsumersPage.jsx  
// Purpose: Backoffice management of Prosumer profiles.  
//   Allows viewing, creating, updating, and deactivating  
//   prosumers. Reactivation is handled in PendingProsumersPage.  
// Author: Rukshan  
// =====  
  
import { useMemo, useState, useEffect, useCallback } from 'react';  
import { useAuth } from '../../context/AuthContext';  
import api from '../../services/api';  
import Toast from '../../components/common/Toast';  
import Button from '../../components/common/Button';  
import Input from '../../components/common/Input';  
import Badge from '../../components/common/Badge';  
import Modal from '../../components/common/Modal';  
import ConfirmDialog from '../../components/common/ConfirmDialog';  
import DataTable from '../../components/common/Table';  
import StatCard from '../../components/common/StatCard';  
  
const EMPTY_FORM = { nic: "", fullName: "", email: "", phone: "", address: "", password: "" };  
const ITEMS_PER_PAGE = 6;  
  
export default function ProsumersPage() {  
  const { role } = useAuth();  
  const [prosumers, setProsumers] = useState([]);  
  const [isLoading, setIsLoading] = useState(true);  
  const [toast, setToast] = useState({ message: "", tone: 'error' });  
  
  const [isModalOpen, setIsModalOpen] = useState(false);  
  const [editingProsumer, setEditingProsumer] = useState(null);  
  const [formData, setFormData] = useState(EMPTY_FORM);  
  const [formErrors, setFormErrors] = useState({});  
  const [creating, setCreating] = useState(false);  
  const [updating, setUpdating] = useState(false);  
  
  const [deactivateTarget, setDeactivateTarget] = useState(null);  
  const [deactivating, setDeactivating] = useState(false);  
  
  // Search and Pagination State  
  const [searchQuery, setSearchQuery] = useState("");  
  const [currentPage, setCurrentPage] = useState(1);  
  
  const fetchProsumers = useCallback(async () => {  
    setIsLoading(true);  
    try {  
      const response = await api.get('/admin/prosumers');  
      setProsumers(response.data);  
    }  
  });  
}
```



```

    } catch (err) {
      setToast({ message: err.response?.data?.message || 'Failed to load prosumers', tone: 'error' });
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    if (role === 'Backoffice') {
      fetchProsumers();
    }
  }, [role, fetchProsumers]);

  // Reset to page 1 when search query changes
  useEffect(() => {
    setCurrentPage(1);
  }, [searchQuery]);

  const summary = useMemo(
    () => ({
      total: prosumers.length,
      active: prosumers.filter((p) => p.status === 'Active').length,
      pending: prosumers.filter((p) => p.status === 'PendingActivation').length,
      deactivated: prosumers.filter((p) => p.status === 'Deactivated').length,
    }),
    [prosumers],
  );

  // Filter prosumers based on search query
  const filteredProsumers = useMemo(() => {
    if (!searchQuery) return prosumers;
    const lowerQuery = searchQuery.toLowerCase();
    return prosumers.filter(
      (p) =>
        p.nic.toLowerCase().includes(lowerQuery) ||
        p.fullName.toLowerCase().includes(lowerQuery) ||
        p.email.toLowerCase().includes(lowerQuery)
    );
  }, [prosumers, searchQuery]);

  // Pagination logic
  const totalPages = Math.ceil(filteredProsumers.length / ITEMS_PER_PAGE);
  const currentItems = filteredProsumers.slice(
    (currentPage - 1) * ITEMS_PER_PAGE,
    currentPage * ITEMS_PER_PAGE
  );

  // Form Validation Logic
  const validateForm = () => {
    const errors = {};

    if (!formData.nic.trim()) {
      errors.nic = 'NIC is required';
    } else if (!/^\d{9}[vVxX]$\d{12}$/.test(formData.nic.trim())) {
      errors.nic = 'Invalid NIC format (e.g., 200012345678 or 981234567V)';
    }

    if (!formData.fullName.trim()) {

```

```

    errors.fullName = 'Full name is required';
  }

  if (!formData.email.trim()) {
    errors.email = 'Email is required';
  } else if (!/^[\s@]+@[^\s@]+\.[^\s@]+$/ .test(formData.email.trim())) {
    errors.email = 'Invalid email format';
  }

  if (!formData.phone.trim()) {
    errors.phone = 'Phone number is required';
  } else if (!/^d{10}$/ .test(formData.phone.trim().replace(/\s+/g, ''))) {
    errors.phone = 'Phone number must be exactly 10 digits';
  }

  if (!formData.address.trim()) {
    errors.address = 'Address is required';
  }

  if (!editingProsumer && !formData.password) {
    errors.password = 'Password is required';
  } else if (!editingProsumer && formData.password.length < 6) {
    errors.password = 'Password must be at least 6 characters';
  }

  setFormErrors(errors);
  return Object.keys(errors).length === 0;
};

const handleOpenCreate = () => {
  setEditingProsumer(null);
  setFormData(EMPTY_FORM);
  setFormErrors({});
  setIsModalOpen(true);
};

const handleOpenEdit = (prosumer) => {
  setEditingProsumer(prosumer);
  setFormData({
    nic: prosumer.nic,
    fullName: prosumer.fullName,
    email: prosumer.email,
    phone: prosumer.phone || '',
    address: prosumer.address || '',
    password: '',
  });
  setFormErrors({});
  setIsModalOpen(true);
};

const handleCloseModal = () => {
  setIsModalOpen(false);
  setEditingProsumer(null);
  setFormData(EMPTY_FORM);
  setFormErrors({});
};

const handleCreateProsumer = async (e) => {

```

```

e.preventDefault();
if (!validateForm()) return;
setCreating(true);
try {
  await api.post('/admin/prosumers', formData);
  setToast({ message: 'Prosumer created successfully!', tone: 'success' });
  handleCloseModal();
  await fetchProsumers();
} catch (err) {
  setToast({ message: err.response?.data?.message || 'Failed to create prosumer', tone: 'error' });
} finally {
  setCreating(false);
}
};

const handleUpdateProsumer = async (e) => {
  e.preventDefault();
  if (!validateForm()) return;
  setUpdating(true);
  try {
    const updatePayload = {
      fullName: formData.fullName,
      email: formData.email,
      phone: formData.phone,
      address: formData.address,
    };
    await api.put(`/admin/prosumers/${editingProsumer.nic}`, updatePayload);
    setToast({ message: 'Prosumer updated successfully!', tone: 'success' });
    handleCloseModal();
    await fetchProsumers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to update prosumer', tone: 'error' });
  } finally {
    setUpdating(false);
  }
};

const handleConfirmDeactivate = async () => {
  if (!deactivateTarget) return;
  setDeactivating(true);
  try {
    await api.put(`/admin/prosumers/${deactivateTarget.nic}/deactivate`);
    setToast({ message: 'Prosumer deactivated successfully!', tone: 'success' });
    setDeactivateTarget(null);
    await fetchProsumers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to deactivate prosumer', tone: 'error' });
    setDeactivateTarget(null);
  } finally {
    setDeactivating(false);
  }
};

const columns = useMemo(
  () => [
    { key: 'nic', header: 'NIC', render: (p) => <span className="font-mono text-xs text-ink">{p.nic}</span> },
    { key: 'fullName', header: 'Full name' },
    { key: 'address', header: 'Address', className: 'hidden md:table-cell' },
  ],

```

```

{ key: 'email', header: 'Email', className: 'hidden lg:table-cell' },
{ key: 'phone', header: 'Phone', className: 'hidden lg:table-cell' },
{
  key: 'status',
  header: 'Status',
  render: (p) => (
    <Badge tone={p.status === 'Active' ? 'success' : p.status === 'PendingActivation' ? 'warning' : 'neutral'}>
      {p.status}
    </Badge>
  ),
},
{
  key: 'actions',
  header: 'Action',
  className: 'text-right',
  render: (p) => (
    <div className="flex justify-end gap-2">
      {p.status === 'Active' ? (
        <>
          <Button variant="secondary" size="sm" onClick={() => handleOpenEdit(p)}>
            Edit
          </Button>
          <Button variant="danger-outline" size="sm" onClick={() => setDeactivateTarget(p)}>
            Deactivate
          </Button>
        </>
      ) : p.status === 'Deactivated' ? (
        <span className="text-xs text-muted italic">Use Pending view</span>
      ) : (
        <span className="text-xs text-muted italic">Pending</span>
      )}
    </div>
  ),
},
],
[handleOpenEdit],
);

if (role !== 'Backoffice') {
  return (
    <div className="mx-auto max-w-6xl px-4 py-6 sm:px-6 sm:py-8">
      <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2 text-[13px] font-medium text-error">
        Access denied - only Backoffice users can manage prosumers.
      </p>
    </div>
  );
}

return (
  <div className="mx-auto max-w-6xl px-4 py-6 sm:px-6 sm:py-8">
    { /* Header Section */ }
    <div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
      <div>
        <h1 className="text-xl font-semibold tracking-tight text-ink">Prosumer management</h1>
        <p className="mt-1 text-[13px] text-muted">Manage solar prosumer accounts and profiles.</p>
      </div>
      <div className="flex flex-col gap-3 sm:flex-row sm:items-center">

```

```

        <Button variant="secondary" onClick={fetchProsumers} disabled={isLoading} className="flex items-center justify-
center gap-2">
            <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M3 12a9 9 0 0 1 9 9.75
9.75 0 0 1 6.74 2.74L21 8"/><path d="M21 3v5h-5"/><path d="M21 12a9 9 0 0 1 -9 9 9.75 9.75 0 0 1 -6.74 -2.74L3 16"/><path
d="M3 21v-5h5"/></svg>
            Refresh
        </Button>
        <Button variant="primary" onClick={handleOpenCreate}>
            New prosumer
        </Button>
    </div>
</div>

{/* Stats Cards */}
<div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-4">
    <StatCard icon="user" label="Total prosumers" value={summary.total} hint="Registered accounts" />
    <StatCard icon="pulse" label="Active" value={summary.active} hint="Currently active" />
    <StatCard icon="clock" label="Pending" value={summary.pending} hint="Awaiting activation" />
    <StatCard icon="alert" label="Deactivated" value={summary.deactivated} hint="Deactivated accounts" />
</div>

{/* Search Bar */}
<div className="mb-4">
    <Input
        placeholder="Search by NIC, Name, or Email..."
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        className="w-full sm:max-w-sm"
    />
</div>

{/* Loading State */}
{isLoading && (
    <div className="py-12 text-center text-sm text-muted">Loading prosumers...</div>
)}

{/* Empty State */}
{!isLoading && currentItems.length === 0 && (
    <div className="rounded-lg border border-line bg-surface-alt py-12 text-center">
        <p className="text-sm text-muted">
            {searchQuery ? "No matching prosumers found." : "No prosumers found."}
        </p>
    </div>
)}

{/* Desktop Table View (Hidden on Mobile) */}
{!isLoading && currentItems.length > 0 && (
    <div className="hidden md:block overflow-x-auto rounded-lg border border-line">
        <DataTable
            columns={columns}
            data={currentItems}
            loading={isLoading}
            emptyMessage={searchQuery ? "No matching prosumers found." : "No prosumers found."}
            rowKey={(p) => p.nic}
        />
    </div>
)}

```

```

{/* Mobile Cards View (Hidden on Desktop) */}
{!isLoading && currentItems.length > 0 && (
  <div className="md:hidden space-y-3">
    {currentItems.map((p) => (
      <div
        key={p.nic}
        className="rounded-lg border border-line bg-surface p-4 shadow-sm"
      >
        {/* Card Header */}
        <div className="flex items-start justify-between gap-3 mb-3">
          <div className="min-w-0 flex-1">
            <p className="font-semibold text-ink truncate">{p.fullName}</p>
            <p className="font-mono text-xs text-muted mt-0.5">{p.nic}</p>
          </div>
          <Badge tone={p.status === 'Active' ? 'success' : p.status === 'PendingActivation' ? 'warning' : 'neutral'}>
            {p.status}
          </Badge>
        </div>

        {/* Card Details */}
        <div className="space-y-2 mb-4 pb-4 border-b border-line">
          {p.email && (
            <div className="flex gap-2 text-sm">
              <span className="text-muted w-16 shrink-0">Email:</span>
              <span className="text-ink truncate">{p.email}</span>
            </div>
          )}
          {p.phone && (
            <div className="flex gap-2 text-sm">
              <span className="text-muted w-16 shrink-0">Phone:</span>
              <span className="text-ink">{p.phone}</span>
            </div>
          )}
          {p.address && (
            <div className="flex gap-2 text-sm">
              <span className="text-muted w-16 shrink-0">Address:</span>
              <span className="text-ink line-clamp-2">{p.address}</span>
            </div>
          )}
        </div>

        {/* Card Actions */}
        <div className="flex items-center justify-end gap-2">
          {p.status === 'Active' ? (
            <>
              <Button variant="secondary" size="sm" onClick={() => handleOpenEdit(p)} className="flex-1 sm:flex-none">
                Edit
              </Button>
              <Button variant="danger-outline" size="sm" onClick={() => setDeactivateTarget(p)} className="flex-1 sm:flex-
none">
                Deactivate
              </Button>
            </>
          ) : p.status === 'Deactivated' ? (
            <span className="text-xs text-muted italic w-full text-center">Use Pending view to reactivate</span>
          ) : (
            <span className="text-xs text-muted italic w-full text-center">Pending activation</span>
          )
        </div>
      </div>
    )}
  </div>
)}

```

```

    })
  </div>
</div>
  )))
</div>
})

{/* Pagination Controls */}
{!isLoading && filteredProsumers.length > 0 && (
  <div className="mt-6 flex flex-col items-center justify-between gap-4 border-t border-line pt-4 sm:flex-row">
    <p className="text-sm text-muted">
      Showing {currentItems.length > 0 ? (currentPage - 1) * ITEMS_PER_PAGE + 1 : 0} to{' '}
      {Math.min(currentPage * ITEMS_PER_PAGE, filteredProsumers.length)} of {filteredProsumers.length} results
    </p>
    <div className="flex items-center gap-2">
      <Button
        variant="secondary"
        size="sm"
        onClick={() => setCurrentPage((p) => Math.max(1, p - 1))}
        disabled={currentPage === 1}
      >
        Previous
      </Button>
      <span className="text-sm font-medium text-ink">
        Page {currentPage} of {totalPages} | 1
      </span>
      <Button
        variant="secondary"
        size="sm"
        onClick={() => setCurrentPage((p) => Math.min(totalPages, p + 1))}
        disabled={currentPage === totalPages || totalPages === 0}
      >
        Next
      </Button>
    </div>
  </div>
)}

{/* Modals and Dialogs */}
<Modal
  open={isModalOpen}
  onClose={handleCloseModal}
  title={editingProsumer ? 'Edit prosumer' : 'Create new prosumer'}
  description={editingProsumer ? 'Update prosumer profile details.' : 'Manually provision a prosumer account.'}
>
  <form onSubmit={editingProsumer ? handleUpdateProsumer : handleCreateProsumer} className="flex flex-col gap-4">
    {!editingProsumer && (
      <>
        <div>
          <Input
            label="NIC"
            required
            value={formData.nic}
            onChange={(e) => {
              setFormData({ ...formData, nic: e.target.value });
              if (formErrors.nic) setFormErrors({ ...formErrors, nic: '' });
            }}
          />
        </div>
      </>
    )}
  </form>

```

```

        placeholder="e.g., 200012345678 or 981234567V"
      />
      {formErrors.nic && <p className="mt-1 text-xs text-error">{formErrors.nic}</p>}
    </div>
    <div>
      <Input
        label="Password"
        type="password"
        required
        value={formData.password}
        onChange={(e) => {
          setFormData({ ...formData, password: e.target.value });
          if (formErrors.password) setFormErrors({ ...formErrors, password: " " });
        }}
      />
      {formErrors.password && <p className="mt-1 text-xs text-error">{formErrors.password}</p>}
    </div>
  </>
  )}
  <div>
    <Input
      label="Full name"
      required
      value={formData.fullName}
      onChange={(e) => {
        setFormData({ ...formData, fullName: e.target.value });
        if (formErrors.fullName) setFormErrors({ ...formErrors, fullName: " " });
      }}
    />
    {formErrors.fullName && <p className="mt-1 text-xs text-error">{formErrors.fullName}</p>}
  </div>
  <div>
    <Input
      label="Email"
      type="email"
      required
      value={formData.email}
      onChange={(e) => {
        setFormData({ ...formData, email: e.target.value });
        if (formErrors.email) setFormErrors({ ...formErrors, email: " " });
      }}
    />
    {formErrors.email && <p className="mt-1 text-xs text-error">{formErrors.email}</p>}
  </div>
  <div>
    <Input
      label="Phone"
      required
      value={formData.phone}
      onChange={(e) => {
        setFormData({ ...formData, phone: e.target.value });
        if (formErrors.phone) setFormErrors({ ...formErrors, phone: " " });
      }}
    />
    {formErrors.phone && <p className="mt-1 text-xs text-error">{formErrors.phone}</p>}
  </div>
  <div>
    <Input

```



```

      label="Address"
      required
      value={formData.address}
      onChange={(e) => {
        setFormData({ ...formData, address: e.target.value });
        if (formErrors.address) setFormErrors({ ...formErrors, address: " " });
      }}
    />
    {formErrors.address && <p className="mt-1 text-xs text-error">{formErrors.address}</p>}
  </div>
  <div className="mt-2 flex justify-end gap-2">
    <Button type="button" variant="secondary" onClick={handleCloseModal} disabled={creating || updating}>
      Cancel
    </Button>
    <Button type="submit" variant="primary" disabled={creating || updating}>
      {creating ? 'Creating...' : updating ? 'Updating...' : editingProsumer ? 'Update prosumer' : 'Create prosumer'}
    </Button>
  </div>
</form>
</Modal>

<ConfirmDialog
  open={Boolean(deactivateTarget)}
  title="Deactivate prosumer?"
  description={
    deactivateTarget ? `"$${deactivateTarget.fullName}" will no longer be able to use the service. Only a Backoffice officer
can reactivate this account.` : "
  }
  confirmLabel="Deactivate"
  tone="danger"
  confirming={deactivating}
  onConfirm={handleConfirmDeactivate}
  onClose={() => setDeactivateTarget(null)}
/>

<Toast message={toast.message} tone={toast.tone} onDismiss={() => setToast({ message: "", tone: 'error' })} />
</div>
);
}

```

WebApp/src/pages/admin/ PendingProsumersPage.jsx

```

// =====
// File: PendingProsumersPage.jsx
// Purpose: Backoffice view for pending activations and
//          reactivations of deactivated prosumer accounts.
//          Enforces the rule that only Backoffice can reactivate.
// Author: Rukshan
// =====

import { useMemo, useState, useEffect, useCallback } from 'react';
import { useAuth } from '../../context/AuthContext';
import api from '../../services/api';
import Toast from '../../components/common/Toast';
import Button from '../../components/common/Button';

```

```

import Badge from '../components/common/Badge';
import ConfirmDialog from '../components/common/ConfirmDialog';
import DataTable from '../components/common/Table';
import StatCard from '../components/common/StatCard';
import Input from '../components/common/Input';

const ITEMS_PER_PAGE = 6;

export default function PendingProsumersPage() {
  const { role } = useAuth();
  const [prosumers, setProsumers] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [toast, setToast] = useState({ message: '', tone: 'error' });

  const [activateTarget, setActivateTarget] = useState(null);
  const [activating, setActivating] = useState(false);

  const [reactivateTarget, setReactivateTarget] = useState(null);
  const [reactivating, setReactivating] = useState(false);

  // Search and Pagination State
  const [searchQuery, setSearchQuery] = useState('');
  const [currentPage, setCurrentPage] = useState(1);

  const fetchProsumers = useCallback(async () => {
    setIsLoading(true);
    try {
      const response = await api.get('/admin/prosumers');
      const filtered = response.data.filter(
        (p) => p.status === 'PendingActivation' || p.status === 'Deactivated'
      );
      setProsumers(filtered);
    } catch (err) {
      setToast({ message: err.response?.data?.message || 'Failed to load prosumers', tone: 'error' });
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    if (role === 'Backoffice') {
      fetchProsumers();
    }
  }, [role, fetchProsumers]);

  // Reset to page 1 when search query changes
  useEffect(() => {
    setCurrentPage(1);
  }, [searchQuery]);

  const summary = useMemo(
    () => ({
      pending: prosumers.filter((p) => p.status === 'PendingActivation').length,
      deactivated: prosumers.filter((p) => p.status === 'Deactivated').length,
    }),
    [prosumers],
  );
}

```

```

// Filter prosumers based on search query
const filteredProsumers = useMemo(() => {
  if (!searchQuery) return prosumers;
  const lowerQuery = searchQuery.toLowerCase();
  return prosumers.filter(
    (p) =>
      p.nic.toLowerCase().includes(lowerQuery) ||
      p.fullName.toLowerCase().includes(lowerQuery) ||
      p.email.toLowerCase().includes(lowerQuery)
  );
}, [prosumers, searchQuery]);

// Pagination logic
const totalPages = Math.ceil(filteredProsumers.length / ITEMS_PER_PAGE);
const currentItems = filteredProsumers.slice(
  (currentPage - 1) * ITEMS_PER_PAGE,
  currentPage * ITEMS_PER_PAGE
);

const handleConfirmActivate = async () => {
  if (!activateTarget) return;
  setActivating(true);
  try {
    await api.put(`/admin/prosumers/${activateTarget.nic}/activate`);
    setToast({ message: 'Prosumer activated successfully!', tone: 'success' });
    setActivateTarget(null);
    await fetchProsumers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to activate prosumer', tone: 'error' });
    setActivateTarget(null);
  } finally {
    setActivating(false);
  }
};

const handleConfirmReactivate = async () => {
  if (!reactivateTarget) return;
  setReactivating(true);
  try {
    await api.put(`/admin/prosumers/${reactivateTarget.nic}/reactivate`);
    setToast({ message: 'Prosumer reactivated successfully!', tone: 'success' });
    setReactivateTarget(null);
    await fetchProsumers();
  } catch (err) {
    setToast({ message: err.response?.data?.message || 'Failed to reactivate prosumer', tone: 'error' });
    setReactivateTarget(null);
  } finally {
    setReactivating(false);
  }
};

const columns = useMemo(
  () => [
    { key: 'nic', header: 'NIC', render: (p) => <span className="font-mono text-xs text-ink">{p.nic}</span> },
    { key: 'fullName', header: 'Full name' },
    { key: 'address', header: 'Address', className: 'hidden md:table-cell' },
    { key: 'email', header: 'Email', className: 'hidden lg:table-cell' },
  ]
);

```

```

    key: 'createdAt',
    header: 'Requested On',
    render: (p) => new Date(p.createdAt).toLocaleDateString(),
    className: 'hidden sm:table-cell'
  },
  {
    key: 'status',
    header: 'Status',
    render: (p) => (
      <Badge tone={p.status === 'PendingActivation' ? 'warning' : 'neutral'}>
        {p.status === 'PendingActivation' ? 'Pending Activation' : 'Deactivated'}
      </Badge>
    ),
  },
  {
    key: 'actions',
    header: 'Action',
    className: 'text-right',
    render: (p) => (
      <div className="flex justify-end gap-2">
        {p.status === 'PendingActivation' ? (
          <Button variant="primary" size="sm" onClick={() => setActivateTarget(p)}>
            Activate
          </Button>
        ) : (
          <Button variant="secondary" size="sm" onClick={() => setReactivateTarget(p)}>
            Reactivate
          </Button>
        )}
      </div>
    ),
  },
],
[]
);

if (role !== 'Backoffice') {
  return (
    <div className="mx-auto max-w-6xl px-4 py-6 sm:px-6 sm:py-8">
      <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2 text-[13px] font-medium text-error">
        Access denied - only Backoffice users can manage prosumer activations.
      </p>
    </div>
  );
}

return (
  <div className="mx-auto max-w-6xl px-4 py-6 sm:px-6 sm:py-8">
    { /* Header Section */ }
    <div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
      <div>
        <h1 className="text-xl font-semibold tracking-tight text-ink">Pending & Deactivated Prosumers</h1>
        <p className="mt-1 text-[13px] text-muted">Review and activate new registrations, or reactivate deactivated accounts.</p>
      </div>
      <Button variant="secondary" onClick={fetchProsumers} disabled={isLoading} className="flex items-center justify-center gap-2">

```

```

<svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M3 12a9 9 0 0 1 9 9.75
9.75 0 0 1 6.74 2.74L21 8"/><path d="M21 3v5h-5"/><path d="M21 12a9 9 0 0 1 9 9.75 9.75 0 0 1 6.74 2.74L3 16"/><path
d="M3 21v-5h5"/></svg>
    {isLoading ? 'Refreshing...' : 'Refresh'}
  </Button>
</div>

  {/ * Stats Cards */}
  <div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-2">
    <StatCard icon="clock" label="Pending Activation" value={summary.pending} hint="Awaiting Backoffice approval" />
    <StatCard icon="alert" label="Deactivated" value={summary.deactivated} hint="Require Backoffice reactivation" />
  </div>

  {/ * Search Bar */}
  <div className="mb-4">
    <Input
      placeholder="Search by NIC, Name, or Email..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="w-full sm:max-w-sm"
    />
  </div>

  {/ * Loading State */}
  {isLoading && (
    <div className="py-12 text-center text-sm text-muted">Loading prosumers...</div>
  )}

  {/ * Empty State */}
  {!isLoading && currentItems.length === 0 && (
    <div className="rounded-lg border border-line bg-surface-alt py-12 text-center">
      <p className="text-sm text-muted">
        {searchQuery ? "No matching prosumers found." : "No pending or deactivated prosumers found."}
      </p>
    </div>
  )}

  {/ * Desktop Table View (Hidden on Mobile) */}
  {!isLoading && currentItems.length > 0 && (
    <div className="hidden md:block overflow-x-auto rounded-lg border border-line">
      <DataTable
        columns={columns}
        data={currentItems}
        loading={isLoading}
        emptyMessage={searchQuery ? "No matching prosumers found." : "No pending or deactivated prosumers found."}
        rowKey={(p) => p.nic}
      />
    </div>
  )}

  {/ * Mobile Cards View (Hidden on Desktop) */}
  {!isLoading && currentItems.length > 0 && (
    <div className="md:hidden space-y-3">
      {currentItems.map((p) => (
        <div
          key={p.nic}
          className="rounded-lg border border-line bg-surface p-4 shadow-sm"

```

```

>
{ /* Card Header */}
<div className="flex items-start justify-between gap-3 mb-3">
  <div className="min-w-0 flex-1">
    <p className="font-semibold text-ink truncate">{p.fullName}</p>
    <p className="font-mono text-xs text-muted mt-0.5">{p.nic}</p>
  </div>
  <Badge tone={p.status === 'PendingActivation' ? 'warning' : 'neutral'}>
    {p.status === 'PendingActivation' ? 'Pending' : 'Deactivated'}
  </Badge>
</div>

{ /* Card Details */}
<div className="space-y-2 mb-4 pb-4 border-b border-line">
  {p.email && (
    <div className="flex gap-2 text-sm">
      <span className="text-muted w-20 shrink-0">Email:</span>
      <span className="text-ink truncate">{p.email}</span>
    </div>
  )}
  {p.address && (
    <div className="flex gap-2 text-sm">
      <span className="text-muted w-20 shrink-0">Address:</span>
      <span className="text-ink line-clamp-2">{p.address}</span>
    </div>
  )}
  {p.createdAt && (
    <div className="flex gap-2 text-sm">
      <span className="text-muted w-20 shrink-0">Requested:</span>
      <span className="text-ink">{new Date(p.createdAt).toLocaleDateString()}</span>
    </div>
  )}
</div>

{ /* Card Actions */}
<div className="flex items-center justify-end gap-2">
  {p.status === 'PendingActivation' ? (
    <Button variant="primary" size="sm" onClick={() => setActivateTarget(p)} className="w-full sm:w-auto">
      Activate Account
    </Button>
  ) : (
    <Button variant="secondary" size="sm" onClick={() => setReactivateTarget(p)} className="w-full sm:w-auto">
      Reactivate Account
    </Button>
  )}
</div>
</div>
)}}
</div>
}}

{ /* Pagination Controls */}
{!isLoading && filteredProsumers.length > 0 && (
  <div className="mt-6 flex flex-col items-center justify-between gap-4 border-t border-line pt-4 sm:flex-row">
    <p className="text-sm text-muted">
      Showing {currentItems.length > 0 ? (currentPage - 1) * ITEMS_PER_PAGE + 1 : 0} to{' '}
      {Math.min(currentPage * ITEMS_PER_PAGE, filteredProsumers.length)} of {filteredProsumers.length} results
    </p>
  </div>
)}

```

```

<div className="flex items-center gap-2">
  <Button
    variant="secondary"
    size="sm"
    onClick={() => setCurrentPage((p) => Math.max(1, p - 1))}
    disabled={currentPage === 1}
  >
    Previous
  </Button>
  <span className="text-sm font-medium text-ink">
    Page {currentPage} of {totalPages} | 1
  </span>
  <Button
    variant="secondary"
    size="sm"
    onClick={() => setCurrentPage((p) => Math.min(totalPages, p + 1))}
    disabled={currentPage === totalPages || totalPages === 0}
  >
    Next
  </Button>
</div>
</div>
)}

{/* Modals and Dialogs */}
<ConfirmDialog
  open={Boolean(activateTarget)}
  title="Activate prosumer?"
  description={
    activateTarget ? `Approve "${activateTarget.fullName}" (${activateTarget.nic}) to access the system.` : ""
  }
  confirmLabel="Activate"
  tone="success"
  confirming={activating}
  onConfirm={handleConfirmActivate}
  onClose={() => setActivateTarget(null)}
/>

<ConfirmDialog
  open={Boolean(reactivateTarget)}
  title="Reactivate prosumer?"
  description={
    reactivateTarget ? `Restore access for "${reactivateTarget.fullName}" (${reactivateTarget.nic}). Only Backoffice
officers can perform this action.` : ""
  }
  confirmLabel="Reactivate"
  tone="primary"
  confirming={reactivating}
  onConfirm={handleConfirmReactivate}
  onClose={() => setReactivateTarget(null)}
/>

<Toast message={toast.message} tone={toast.tone} onDismiss={() => setToast({ message: "", tone: 'error' })} />
</div>
);
}

```

Mobile App (Android)

app/src/main/java/com/smartmicrogrid/data/local/ ProsumerSessionDao.kt

```
// =====
// File: ProsumerSessionDao.kt
// Purpose: SQLite DAO for persisting Prosumer session data locally.
// Stores JWT token, NIC, and user info for offline access.
// Author: Rukshan
// =====
package com.smartmicrogrid.data.local

import android.content.ContentValues
import android.content.Context
import android.database.Cursor
import android.database.sqlite.SQLiteDatabase

class ProsumerSessionDao(context: Context) {

    private val dbHelper = AppDbHelper(context)

    // Save prosumer session after successful login
    fun saveSession(nic: String, fullName: String, token: String, status: String) {
        val db = dbHelper.writableDatabase
        db.delete(TABLE_NAME, null, null) // Clear old session

        val values = ContentValues().apply {
            put(COLUMN_NIC, nic)
            put(COLUMN_FULL_NAME, fullName)
            put(COLUMN_TOKEN, token)
            put(COLUMN_STATUS, status)
            put(COLUMN_LOGGED_IN_AT, System.currentTimeMillis())
        }
        db.insert(TABLE_NAME, null, values)
    }

    // Get current active session
    fun getSession(): ProsumerSession? {
        val db = dbHelper.readableDatabase
        val cursor: Cursor = db.query(TABLE_NAME, null, null, null, null, null, null)
        return if (cursor.moveToFirst()) {
            ProsumerSession(
                nic = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_NIC)),
                fullName = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_FULL_NAME)),
                token = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_TOKEN)),
                status = cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_STATUS)),
                loggedInAt = cursor.getLong(cursor.getColumnIndexOrThrow(COLUMN_LOGGED_IN_AT))
            )
        } else null
        .also { cursor.close() }
    }

    // Update session status (e.g., after deactivation request)
    fun updateStatus(newStatus: String) {
        val db = dbHelper.writableDatabase
    }
```



```

        val values = ContentValues().apply { put(COLUMN_STATUS, newStatus) }
        db.update(TABLE_NAME, values, null, null)
    }

    // Clear session (logout)
    fun clearSession() {
        val db = dbHelper.writableDatabase
        db.delete(TABLE_NAME, null, null)
    }

    // Check if user is logged in
    fun isLoggedIn(): Boolean = getSession() != null

    companion object {
        const val TABLE_NAME = "prosumer_session"
        const val COLUMN_NIC = "nic"
        const val COLUMN_FULL_NAME = "full_name"
        const val COLUMN_TOKEN = "token"
        const val COLUMN_STATUS = "status"
        const val COLUMN_LOGGED_IN_AT = "logged_in_at"

        // SQL statement to create the table (used by AppDbHelper)
        const val CREATE_TABLE = """
            CREATE TABLE IF NOT EXISTS $TABLE_NAME (
                $COLUMN_NIC TEXT PRIMARY KEY,
                $COLUMN_FULL_NAME TEXT NOT NULL,
                $COLUMN_TOKEN TEXT NOT NULL,
                $COLUMN_STATUS TEXT NOT NULL,
                $COLUMN_LOGGED_IN_AT INTEGER NOT NULL
            )
        """
    }
}

data class ProsumerSession(
    val nic: String, val fullName: String, val token: String, val status: String, val loggedInAt: Long
)

```

app/src/main/java/com/smartmicrogrid/data/remort/ ProsumerDtos.kt

```

// =====
// File: ProsumerDtos.kt
// Purpose: Data Transfer Objects for Prosumer API communication.
//         Matches the backend C# DTOs exactly (JSON contract).
// Author: Rukshan
// =====
package com.smartmicrogrid.data.remote.dto

import com.google.gson.annotations.SerializedName

// Request: Prosumer Registration
data class ProsumerRegistrationRequest(
    @SerializedName("nic") val nic: String,
    @SerializedName("fullName") val fullName: String,

```

```

        @SerializedName("email") val email: String,
        @SerializedName("phone") val phone: String?,
        @SerializedName("address") val address: String?,
        @SerializedName("password") val password: String
    )

    // Request: Prosumer Login
    data class ProsumerLoginRequest(
        @SerializedName("nic") val nic: String,
        @SerializedName("password") val password: String
    )

    // Response: Prosumer Login Success
    data class ProsumerLoginResponse(
        @SerializedName("token") val token: String,
        @SerializedName("role") val role: String,
        @SerializedName("fullName") val fullName: String,
        @SerializedName("nic") val nic: String,
        @SerializedName("status") val status: String
    )

    // Response: Prosumer Profile Details
    data class ProsumerProfileResponse(
        @SerializedName("id") val id: String,
        @SerializedName("nic") val nic: String,
        @SerializedName("fullName") val fullName: String,
        @SerializedName("email") val email: String,
        @SerializedName("phone") val phone: String?,
        @SerializedName("address") val address: String?,
        @SerializedName("profilePicture") val profilePicture: String?,
        @SerializedName("status") val status: String,
        @SerializedName("deactivationRequestedAt") val deactivationRequestedAt: String?,
        @SerializedName("createdAt") val createdAt: String,
        @SerializedName("updatedAt") val updatedAt: String
    )

    // Request: Update Prosumer Profile
    data class ProsumerUpdateRequest(
        @SerializedName("fullName") val fullName: String?,
        @SerializedName("email") val email: String?,
        @SerializedName("phone") val phone: String?,
        @SerializedName("address") val address: String?,
        @SerializedName("profilePicture") val profilePicture: String? = null
    )

```

app/src/main/java/com/smartmicrogrid/ui/prosumer/RegisterScreen.kt

```

// =====
// File: RegisterScreen.kt
// Purpose: Prosumer registration screen with signup.png image,
//         Joule branding, password eye toggle, and registration form.
// Author: Rukshan
// =====
package com.smartmicrogrid.ui.prosumer

```

```

import android.widget.Toast
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.smartmicrogrid.R
import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.ProsumerRegistrationRequest
import com.smartmicrogrid.ui.components.JouleIcons
import com.smartmicrogrid.ui.components.JouleMark
import com.smartmicrogrid.ui.theme.StripeAccent
import com.smartmicrogrid.ui.theme.StripeBody
import com.smartmicrogrid.ui.theme.StripeBorder
import com.smartmicrogrid.ui.theme.StripeCanvas
import com.smartmicrogrid.ui.theme.StripeError
import com.smartmicrogrid.ui.theme.StripeInk
import com.smartmicrogrid.ui.theme.StripeOnPrimary
import com.smartmicrogrid.ui.theme.StripePrimary
import com.smartmicrogrid.ui.theme.StripeSurface
import kotlinx.coroutines.launch

```

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RegisterScreen(onNavigateToLogin: () -> Unit) {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    var nic by remember { mutableStateOf("") }
    var fullName by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var phone by remember { mutableStateOf("") }
    var address by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var passwordVisible by remember { mutableStateOf(false) }
    var isLoading by remember { mutableStateOf(false) }
    var errorMessage by remember { mutableStateOf<String?>(null) }
}

```

```

Scaffold(containerColor = StripeCanvas) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .verticalScroll(rememberScrollState()),
    ) {
        // Hero image at top with floating back button overlay
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(200.dp),
        ) {
            Image(
                painter = painterResource(id = R.drawable.auth_register_hero),
                contentDescription = null,
                modifier = Modifier.fillMaxSize(),
                contentScale = ContentScale.Crop,
            )

            // Floating Back button over image
            Box(
                modifier = Modifier
                    .padding(start = 16.dp, top = 16.dp)
                    .size(38.dp)
                    .clip(CircleShape)
                    .background(StripeSurface.copy(alpha = 0.92f))
                    .border(1.dp, StripeBorder, CircleShape)
                    .clickable(
                        indication = null,
                        interactionSource = remember { MutableInteractionSource() },
                        onClick = onNavigateToLogin,
                    ),
                contentAlignment = Alignment.Center,
            ) {
                Icon(
                    imageVector = JouleIcons.Back,
                    contentDescription = "Back to sign in",
                    tint = StripeInk,
                    modifier = Modifier.size(18.dp),
                )
            }
        }

        Spacer(modifier = Modifier.height(14.dp))

        // Logo & Title
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            JouleMark(modifier = Modifier.width(42.dp))
            Spacer(modifier = Modifier.height(6.dp))
            Text(
                text = "Create your account",
                style = MaterialTheme.typography.titleLarge,
            )
        }
    }
}

```

```

        fontWeight = FontWeight.Bold,
        color = StripeInk,
    )
    Text(
        text = "Power, exchanged precisely.",
        style = MaterialTheme.typography.bodySmall,
        color = StripeBody,
    )
}

Spacer(modifier = Modifier.height(16.dp))

// Form container
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(horizontal = 24.dp)
        .padding(bottom = 32.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp),
) {
    AuthTextField(value = nic, onValueChange = { nic = it }, label = "NIC Number *")
    AuthTextField(value = fullName, onValueChange = { fullName = it }, label = "Full Name *")
    AuthTextField(value = email, onValueChange = { email = it }, label = "Email Address *", keyboardOptions =
KeyboardOptions(keyboardType = KeyboardType.Email))
    AuthTextField(value = phone, onValueChange = { phone = it }, label = "Phone Number", keyboardOptions =
KeyboardOptions(keyboardType = KeyboardType.Phone))
    AuthTextField(value = address, onValueChange = { address = it }, label = "Residential Address", singleLine = false,
minLines = 2)

    // Password with Eye toggle
    OutlinedTextField(
        value = password,
        onValueChange = { password = it },
        label = { Text("Password *") },
        singleLine = true,
        visualTransformation = if (passwordVisible) VisualTransformation.None else PasswordVisualTransformation(),
        trailingIcon = {
            IconButton(onClick = { passwordVisible = !passwordVisible }) {
                Icon(
                    imageVector = if (passwordVisible) JouleIcons.EyeOff else JouleIcons.Eye,
                    contentDescription = if (passwordVisible) "Hide password" else "Show password",
                    tint = StripeBody,
                    modifier = Modifier.size(20.dp),
                )
            }
        },
        shape = RoundedCornerShape(12.dp),
        colors = OutlinedTextFieldDefaults.colors(
            unfocusedBorderColor = StripeBorder,
            focusedBorderColor = StripePrimary,
            unfocusedContainerColor = StripeSurface,
            focusedContainerColor = StripeSurface,
        ),
        modifier = Modifier.fillMaxWidth(),
    )

    errorMessage?.let {
        Box(

```

```

        modifier = Modifier
            .fillMaxWidth()
            .clip(RoundedCornerShape(8.dp))
            .background(StripeError.copy(alpha = 0.1f))
            .border(1.dp, StripeError.copy(alpha = 0.3f), RoundedCornerShape(8.dp))
            .padding(horizontal = 12.dp, vertical = 8.dp),
    ){
        Text(
            text = it,
            color = StripeError,
            style = MaterialTheme.typography.bodySmall,
            fontWeight = FontWeight.Medium,
        )
    }
}

Spacer(modifier = Modifier.height(4.dp))

Button(
    onClick = {
        errorMessage = null

        // Trim all inputs for accurate validation
        val trimmedNic = nic.trim()
        val trimmedFullName = fullName.trim()
        val trimmedEmail = email.trim()
        val trimmedPhone = phone.trim()
        val trimmedAddress = address.trim()
        val trimmedPassword = password.trim()

        when {
            trimmedNic.isBlank() -> errorMessage = "NIC is required"
            !Regex("^[\\d{9}[vVxX]$|^\\d{12}$").matches(trimmedNic) ->
                errorMessage = "Invalid NIC format (e.g., 200012345678 or 981234567V)"
            trimmedFullName.isBlank() -> errorMessage = "Full name is required"
            trimmedEmail.isBlank() -> errorMessage = "Email address is required"
            !Regex("^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+\\$").matches(trimmedEmail) ->
                errorMessage = "Invalid email format"
            trimmedPhone.isNotBlank() && !Regex("^[\\d]{10}$").matches(trimmedPhone.replace("\\s+", ""))
                ->
                errorMessage = "Phone number must be exactly 10 digits"
            trimmedPassword.isBlank() -> errorMessage = "Password is required"
            trimmedPassword.length < 6 -> errorMessage = "Password must be at least 6 characters"
            else -> {
                isLoading = true
                scope.launch {
                    try {
                        val request = ProsumerRegistrationRequest(
                            nic = trimmedNic,
                            fullName = trimmedFullName,
                            email = trimmedEmail,
                            phone = trimmedPhone.ifBlank { null },
                            address = trimmedAddress.ifBlank { null },
                            password = trimmedPassword,
                        )
                        val response = ApiClient.service.registerProsumer(request)
                        if (response.isSuccessful) {

```

```

                Toast.makeText(context, "Registration successful! Pending Backoffice activation.",
Toast.LENGTH_LONG).show()
                onNavigateToLogin()
            } else {
                errorMessage = "Registration failed: ${response.errorBody()?.string() ?: response.code()}"
            }
        } catch (e: Exception) {
            errorMessage = "Network error: ${e.localizedMessage}"
        } finally {
            isLoading = false
        }
    }
}
},
enabled = !isLoading,
shape = RoundedCornerShape(50),
colors = ButtonDefaults.buttonColors(containerColor = StripePrimary, contentColor = StripeOnPrimary),
modifier = Modifier
    .fillMaxWidth()
    .height(50.dp),
){
    if (isLoading) {
        CircularProgressIndicator(modifier = Modifier.size(22.dp), color = StripeOnPrimary, strokeWidth = 2.dp)
    } else {
        Text("Register", style = MaterialTheme.typography.titleSmall, fontWeight = FontWeight.SemiBold)
    }
}

TextButton(
    onClick = onNavigateToLogin,
    modifier = Modifier.align(Alignment.CenterHorizontally),
){
    Text("Already have an account? ", color = StripeBody)
    Text("Sign in", color = StripeAccent, fontWeight = FontWeight.SemiBold)
}
}
}
}
}
}

```

app/src/main/java/com/smartmicrogrid/ui/prosumer/LoginScreen.kt

```

// =====
// File: LoginScreen.kt
// Purpose: Sign-in screen using home.png image, Joule logo,
//         tagline, password eye toggle, and role selector.
// Author: Rukshan
// =====
package com.smartmicrogrid.ui.prosumer

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border

```

```

import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.smartmicrogrid.R
import com.smartmicrogrid.data.local.ProsumerSessionDao
import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.ProsumerLoginRequest
import com.smartmicrogrid.data.remote.dto.StaffLoginRequest
import com.smartmicrogrid.ui.auth.StaffSession
import com.smartmicrogrid.ui.auth.StaffSessionPreferences
import com.smartmicrogrid.ui.components.JouleIcons
import com.smartmicrogrid.ui.components.JouleMark
import com.smartmicrogrid.ui.home.UserRole
import com.smartmicrogrid.ui.theme.StripeAccent
import com.smartmicrogrid.ui.theme.StripeBody
import com.smartmicrogrid.ui.theme.StripeBorder
import com.smartmicrogrid.ui.theme.StripeBrandVioletSoft
import com.smartmicrogrid.ui.theme.StripeCanvas
import com.smartmicrogrid.ui.theme.StripeError
import com.smartmicrogrid.ui.theme.StripeInk
import com.smartmicrogrid.ui.theme.StripeOnPrimary
import com.smartmicrogrid.ui.theme.StripePrimary
import com.smartmicrogrid.ui.theme.StripeSurface
import kotlinx.coroutines.launch

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LoginScreen(onLoginSuccess: (UserRole) -> Unit, onNavigateToRegister: () -> Unit) {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val sessionDao = remember { ProsumerSessionDao(context) }

    var role by remember { mutableStateOf(UserRole.Prosumer) }
    var identifier by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var passwordVisible by remember { mutableStateOf(false) }
    var isLoading by remember { mutableStateOf(false) }
    var errorMessage by remember { mutableStateOf<String?>(null) }

    fun submit() {
        errorMessage = null
    }

```



```

        if (identifier.isBlank() || password.isBlank()) {
            errorMessage = if (role == UserRole.Prosumer) "NIC and password are required" else "Username and password are required"
            return
        }
        isLoading = true
        scope.launch {
            try {
                if (role == UserRole.Prosumer) {
                    val response = ApiClient.service.loginProsumer(ProsumerLoginRequest(identifier.trim(), password))
                    if (response.isSuccessful && response.body() != null) {
                        val body = response.body()!!
                        ApiClient.authToken = body.token
                        sessionDao.saveSession(body.nic, body.fullName, body.token, body.status)
                        onLoginSuccess(UserRole.Prosumer)
                    } else {
                        errorMessage = when (response.code()) {
                            401 -> "Invalid NIC or password"
                            403 -> "Account is not active. Contact Backoffice."
                            else -> "Login failed: ${response.code()}"
                        }
                    }
                } else {
                    val response = ApiClient.service.loginStaff(StaffLoginRequest(identifier.trim(), password))
                    if (response.isSuccessful && response.body() != null) {
                        val body = response.body()!!
                        ApiClient.authToken = body.token
                        StaffSessionPreferences.save(context, StaffSession(body.token, identifier.trim(), body.fullName, body.role))
                        onLoginSuccess(UserRole.fromClaimValue(body.role))
                    } else {
                        errorMessage = when (response.code()) {
                            401 -> "Invalid username or password"
                            else -> "Login failed: ${response.code()}"
                        }
                    }
                }
            } catch (e: Exception) {
                errorMessage = "Network error: ${e.localizedMessage}"
            } finally {
                isLoading = false
            }
        }
    }
}

```

```

Scaffold(containerColor = StripeCanvas) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        // Dedicated login hero image
        Image(
            painter = painterResource(id = R.drawable.auth_login_hero),
            contentDescription = null,
            modifier = Modifier
                .fillMaxWidth()

```

```

        .height(200.dp),
        contentScale = ContentScale.Crop,
    )

    Spacer(modifier = Modifier.height(16.dp))

    // Joule Logo & Tagline
    JouleMark(modifier = Modifier.width(48.dp))
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = "Joule",
        style = MaterialTheme.typography.headlineMedium,
        fontWeight = FontWeight.Bold,
        color = StripeInk,
    )
    Text(
        text = "Power, exchanged precisely.",
        style = MaterialTheme.typography.bodyMedium,
        color = StripeBody,
    )

    Spacer(modifier = Modifier.height(20.dp))

    // Form container
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 24.dp)
            .padding(bottom = 32.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        RoleToggle(
            selected = role,
            onSelect = {
                role = it
                identifier = ""
                errorMessage = null
            },
        )

        Spacer(modifier = Modifier.height(16.dp))

        AuthTextField(
            value = identifier,
            onValueChange = { identifier = it },
            label = if (role == UserRole.Prosumer) "NIC Number" else "Username",
        )

        Spacer(modifier = Modifier.height(12.dp))

        // Password field with Eye toggle icon
        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text("Password") },
            singleLine = true,
            visualTransformation = if (passwordVisible) VisualTransformation.None else PasswordVisualTransformation(),
        )
    }

```

```

trailingIcon = {
  IconButton(onClick = { passwordVisible = !passwordVisible }) {
    Icon(
      imageVector = if (passwordVisible) JouleIcons.EyeOff else JouleIcons.Eye,
      contentDescription = if (passwordVisible) "Hide password" else "Show password",
      tint = StripeBody,
      modifier = Modifier.size(20.dp),
    )
  }
},
shape = RoundedCornerShape(12.dp),
colors = OutlinedTextFieldDefaults.colors(
  unfocusedBorderColor = StripeBorder,
  focusedBorderColor = StripePrimary,
  unfocusedContainerColor = StripeSurface,
  focusedContainerColor = StripeSurface,
),
modifier = Modifier.fillMaxWidth(),
)

errorMessage?.let {
  Spacer(modifier = Modifier.height(10.dp))
  Box(
    modifier = Modifier
      .fillMaxWidth()
      .clip(RoundedCornerShape(8.dp))
      .background(StripeError.copy(alpha = 0.1f))
      .border(1.dp, StripeError.copy(alpha = 0.3f), RoundedCornerShape(8.dp))
      .padding(horizontal = 12.dp, vertical = 8.dp),
  ) {
    Text(
      text = it,
      color = StripeError,
      style = MaterialTheme.typography.bodySmall,
      fontWeight = FontWeight.Medium,
    )
  }
}

Spacer(modifier = Modifier.height(20.dp))

Button(
  onClick = ::submit,
  enabled = !isLoading,
  shape = RoundedCornerShape(50),
  colors = ButtonDefaults.buttonColors(containerColor = StripePrimary, contentColor = StripeOnPrimary),
  modifier = Modifier
    .fillMaxWidth()
    .height(50.dp),
) {
  if (isLoading) {
    CircularProgressIndicator(modifier = Modifier.size(22.dp), color = StripeOnPrimary, strokeWidth = 2.dp)
  } else {
    Text("Sign In", style = MaterialTheme.typography.titleSmall, fontWeight = FontWeight.SemiBold)
  }
}

if (role == UserRole.Prosumer) {

```

```

        Spacer(modifier = Modifier.height(16.dp))
        TextButton(onClick = onNavigateToRegister) {
            Text("Don't have an account? ", color = StripeBody)
            Text("Register Now", color = StripeAccent, fontWeight = FontWeight.SemiBold)
        }
    }
}
}
}
}

// Segmented role switch between Prosumer and Grid Operator
@Composable
private fun RoleToggle(selected: UserRole, onSelect: (UserRole) -> Unit) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clip(RoundedCornerShape(50))
            .background(StripeBrandVioletSoft)
            .padding(4.dp),
    ) {
        listOf(UserRole.Prosumer to "Prosumer", UserRole.GridOperator to "Grid Operator").forEach { (value, label) ->
            val isSelected = selected == value
            Box(
                modifier = Modifier
                    .weight(1f)
                    .clip(RoundedCornerShape(50))
                    .background(if (isSelected) StripeAccent else Color.Transparent)
                    .clickable(
                        indication = null,
                        interactionSource = remember { MutableInteractionSource() },
                    ) { onSelect(value) },
                padding(vertical = 10.dp),
                contentAlignment = Alignment.Center,
            ) {
                Text(
                    label,
                    color = if (isSelected) StripeOnPrimary else StripeInk,
                    style = MaterialTheme.typography.labelLarge,
                    fontWeight = if (isSelected) FontWeight.SemiBold else FontWeight.Normal,
                )
            }
        }
    }
}

// Reusable text field
@Composable
internal fun AuthTextField(
    value: String,
    onValueChange: (String) -> Unit,
    label: String,
    visualTransformation: VisualTransformation = VisualTransformation.None,
    keyboardOptions: androidx.compose.foundation.text.KeyboardOptions =
        androidx.compose.foundation.text.KeyboardOptions.Default,
    singleLine: Boolean = true,
    minLines: Int = 1,
) {

```

```

OutlinedTextField(
    value = value,
    onValueChange = onValueChange,
    label = { Text(label) },
    singleLine = singleLine,
    minLines = minLines,
    visualTransformation = visualTransformation,
    keyboardOptions = keyboardOptions,
    shape = RoundedCornerShape(12.dp),
    colors = OutlinedTextFieldDefaults.colors(
        unfocusedBorderColor = StripeBorder,
        focusedBorderColor = StripePrimary,
        unfocusedContainerColor = StripeSurface,
        focusedContainerColor = StripeSurface,
    ),
    modifier = Modifier.fillMaxWidth(),
)
}

```

app/src/main/java/com/smartmicrogrid/ui/prosumer/ProfileScreen.kt

```

// =====
// File: ProfileScreen.kt
// Purpose: Prosumer profile screen - avatar with gallery photo upload,
//          editable fields, account status, deactivation, and logout.
//          No extra top spacing; content starts directly below insets.
// Author: Rukshan
// =====
package com.smartmicrogrid.ui.prosumer

import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.util.Base64
import android.widget.Toast
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale

```

```

import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.smartmicrogrid.data.local.ProsumerSessionDao
import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.ProsumerProfileResponse
import com.smartmicrogrid.data.remote.dto.ProsumerUpdateRequest
import com.smartmicrogrid.ui.components.JouleIcons
import com.smartmicrogrid.ui.theme.*
import kotlinx.coroutines.launch
import java.io.ByteArrayOutputStream

// Generate avatar initials from name
private fun initials(name: String?): String {
    if (name.isNullOrEmpty()) return "?"
    return name.trim().split("\\s+").toRegex()
        .filter { it.isNotEmpty() }
        .take(2)
        .joinToString("") { it[0].uppercase() }
}

// Decode base64 data string to Compose ImageBitmap
private fun decodeBase64ToBitmap(base64Str: String?): ImageBitmap? {
    if (base64Str.isNullOrEmpty()) return null
    return try {
        val clean = if (base64Str.contains(",")) base64Str.substringAfter(",") else base64Str
        val decoded = Base64.decode(clean, Base64.DEFAULT)
        BitmapFactory.decodeByteArray(decoded, 0, decoded.size)?.asImageBitmap()
    } catch (_: Exception) {
        null
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProfileScreen(onLogout: () -> Unit) {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val sessionDao = remember { ProsumerSessionDao(context) }
    val session = remember { sessionDao.getSession() }

    var profile by remember { mutableStateOf<ProsumerProfileResponse?>(null) }
    var avatarBase64 by remember { mutableStateOf<String?>(null) }
    var isLoading by remember { mutableStateOf(true) }
    var isEditing by remember { mutableStateOf(false) }
    var editFullName by remember { mutableStateOf("") }
    var editEmail by remember { mutableStateOf("") }
    var editPhone by remember { mutableStateOf("") }
    var editAddress by remember { mutableStateOf("") }
    var showDeactivateDialog by remember { mutableStateOf(false) }

    // Image picker launcher for photo upload
    val imagePickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.GetContent(),
    ) { uri ->
        uri?.let {

```

```

    try {
        val inputStream = context.contentResolver.openInputStream(it)
        val bitmap = BitmapFactory.decodeStream(inputStream)
        val scaled = Bitmap.createScaledBitmap(bitmap, 256, 256, true)
        val outputStream = ByteArrayOutputStream()
        scaled.compress(Bitmap.CompressFormat.JPEG, 85, outputStream)
        val base64 = "data:image/jpeg;base64," + Base64.encodeToString(outputStream.toByteArray(),
Base64.NO_WRAP)
        avatarBase64 = base64

        session?.let { s ->
            scope.launch {
                try {
                    val req = ProsumerUpdateRequest(
                        fullName = null,
                        email = null,
                        phone = null,
                        address = null,
                        profilePicture = base64,
                    )
                    ApiClient.service.updateProsumerProfile(s.nic, req)
                    Toast.makeText(context, "Profile picture updated", Toast.LENGTH_SHORT).show()
                } catch (_: Exception) {
                    Toast.makeText(context, "Failed to save picture", Toast.LENGTH_SHORT).show()
                }
            }
        }
    } catch (_: Exception) {
        Toast.makeText(context, "Failed to load selected photo", Toast.LENGTH_SHORT).show()
    }
}

LaunchedEffect(Unit) {
    session?.let {
        try {
            val response = ApiClient.service.getProsumerProfile(it.nic)
            if (response.isSuccessful && response.body() != null) {
                val p = response.body()!!
                profile = p
                avatarBase64 = p.profilePicture
                editFullName = p.fullName
                editEmail = p.email
                editPhone = p.phone ?: ""
                editAddress = p.address ?: ""
            }
        } catch (_: Exception) {}
        isLoading = false
    }
}

Scaffold(
    containerColor = StripeCanvas,
) { padding ->
    if (isLoading) {
        Box(modifier = Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
            CircularProgressIndicator(color = StripePrimary, strokeWidth = 3.dp)
        }
    }
}

```

```

} else {
    profile?.let { p ->
        val avatarBitmap = decodeBase64ToBitmap(avatarBase64)

        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .verticalScroll(rememberScrollState())
                .padding(bottom = 32.dp),
        ) {
            // Header section (No extra top spacing)
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(top = 16.dp, bottom = 12.dp),
                horizontalAlignment = Alignment.CenterHorizontally,
            ) {
                // Clickable Avatar with unclipped Photo Upload badge
                Box(
                    modifier = Modifier
                        .size(96.dp)
                        .clickable(
                            indication = null,
                            interactionSource = remember { MutableInteractionSource() },
                            onClick = { imagePickerLauncher.launch("image/*") },
                        ),
                    contentAlignment = Alignment.Center,
                ) {
                    // Avatar circle
                    Box(
                        modifier = Modifier
                            .size(88.dp)
                            .clip(CircleShape)
                            .background(StripeBrandVioletSoft)
                            .border(3.dp, StripeSurface, CircleShape),
                        contentAlignment = Alignment.Center,
                    ) {
                        if (avatarBitmap != null) {
                            Image(
                                bitmap = avatarBitmap,
                                contentDescription = "Profile Picture",
                                modifier = Modifier.fillMaxSize(),
                                contentScale = ContentScale.Crop,
                            )
                        } else {
                            Text(
                                text = initials(p.fullName),
                                style = MaterialTheme.typography.headlineMedium.copy(
                                    fontWeight = FontWeight.Bold,
                                    fontSize = 28.sp,
                                ),
                                color = StripePrimary,
                            )
                        }
                    }
                }
            }

            // Camera badge (unclipped)

```



```

Box(
    modifier = Modifier
        .align(Alignment.BottomEnd)
        .size(28.dp)
        .clip(CircleShape)
        .background(StripePrimary)
        .border(2.5.dp, StripeSurface, CircleShape),
    contentAlignment = Alignment.Center,
) {
    Icon(
        imageVector = JouleIcons.Camera,
        contentDescription = "Change photo",
        tint = Color.White,
        modifier = Modifier.size(14.dp),
    )
}

Spacer(modifier = Modifier.height(10.dp))

Text(
    text = p.fullName,
    style = MaterialTheme.typography.titleLarge,
    fontWeight = FontWeight.Bold,
    color = StripeInk,
)
Spacer(modifier = Modifier.height(2.dp))
Text(
    text = p.email,
    style = MaterialTheme.typography.bodySmall,
    color = StripeBody,
)
Spacer(modifier = Modifier.height(8.dp))

// Status pill
Surface(
    shape = RoundedCornerShape(50),
    color = if (p.status == "Active") StripeBrandVioletSoft else StripeErrorContainer,
) {
    Text(
        text = p.status,
        style = MaterialTheme.typography.labelSmall,
        fontWeight = FontWeight.SemiBold,
        color = if (p.status == "Active") StripePrimary else StripeError,
        modifier = Modifier.padding(horizontal = 12.dp, vertical = 4.dp),
    )
}

// Profile fields
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(horizontal = 20.dp, vertical = 8.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp),
) {
    if (isEditing) {

```

```

        Text("Edit Profile", style = MaterialTheme.typography.titleSmall, fontWeight = FontWeight.SemiBold, color
= StripeInk)
        OutlinedTextField(value = editFullName, onValueChange = { editFullName = it }, label = { Text("Full Name")
}, modifier = Modifier.fillMaxWidth(), shape = RoundedCornerShape(12.dp))
        OutlinedTextField(value = editEmail, onValueChange = { editEmail = it }, label = { Text("Email") }, modifier =
Modifier.fillMaxWidth(), shape = RoundedCornerShape(12.dp))
        OutlinedTextField(value = editPhone, onValueChange = { editPhone = it }, label = { Text("Phone") }, modifier
= Modifier.fillMaxWidth(), shape = RoundedCornerShape(12.dp))
        OutlinedTextField(value = editAddress, onValueChange = { editAddress = it }, label = { Text("Address") },
modifier = Modifier.fillMaxWidth(), minLines = 2, shape = RoundedCornerShape(12.dp))

        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            OutlinedButton(onClick = { isEditing = false }, modifier = Modifier.weight(1f), shape =
RoundedCornerShape(10.dp)) { Text("Cancel") }
            Button(
                onClick = {
                    scope.launch {
                        try {
                            val req = ProsumerUpdateRequest(
                                fullName = editFullName.ifBlank { null },
                                email = editEmail.ifBlank { null },
                                phone = editPhone.ifBlank { null },
                                address = editAddress.ifBlank { null },
                                profilePicture = avatarBase64,
                            )
                            val res = ApiClient.service.updateProsumerProfile(p.nic, req)
                            if (res.isSuccessful || res.code() == 204) {
                                isEditing = false
                                profile = p.copy(fullName = editFullName, email = editEmail, phone = editPhone, address =
editAddress)
                                Toast.makeText(context, "Profile updated", Toast.LENGTH_SHORT).show()
                            }
                        } catch (_: Exception) {}
                    }
                },
                modifier = Modifier.weight(1f),
                shape = RoundedCornerShape(10.dp),
                colors = ButtonDefaults.buttonColors(containerColor = StripePrimary, contentColor = StripeOnPrimary),
            ) { Text("Save") }
        }
    } else {
        // Info card
        Surface(
            shape = RoundedCornerShape(16.dp),
            color = StripeSurface,
            tonalElevation = 1.dp,
            border = androidx.compose.foundation.BorderStroke(1.dp, StripeBorder),
            modifier = Modifier.fillMaxWidth(),
        ) {
            Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
                ProfileRow(icon = JouleIcons.User, label = "NIC", value = p.nic)
                HorizontalDivider(color = StripeBorder)
                ProfileRow(icon = JouleIcons.MapPin, label = "Phone", value = p.phone ?: "Not provided")
                HorizontalDivider(color = StripeBorder)
                ProfileRow(icon = JouleIcons.MapPin, label = "Address", value = p.address ?: "Not provided")
            }
        }
    }
}

```

```

        Button(
            onClick = { isEditing = true },
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(12.dp),
            colors = ButtonDefaults.buttonColors(containerColor = StripePrimary, contentColor = StripeOnPrimary),
        ) {
            Icon(JouleIcons.Check, contentDescription = null, modifier = Modifier.size(16.dp))
            Spacer(modifier = Modifier.width(6.dp))
            Text("Edit Profile")
        }
    }

    Spacer(modifier = Modifier.height(4.dp))

    // Danger zone
    if (p.status != "Deactivated") {
        OutlinedButton(
            onClick = { showDeactivateDialog = true },
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(12.dp),
            colors = ButtonDefaults.outlinedButtonColors(contentColor = StripeError),
        ) { Text("Request Account Deactivation") }
    } else {
        Surface(shape = RoundedCornerShape(12.dp), color = StripeErrorContainer) {
            Text(
                "Your account is deactivated. Contact Backoffice to reactivate.",
                modifier = Modifier.padding(16.dp),
                color = StripeOnErrorContainer,
                style = MaterialTheme.typography.bodySmall,
            )
        }
    }

    // Logout
    OutlinedButton(
        onClick = { sessionDao.clearSession(); ApiClient.authToken = null; onLogout() },
        modifier = Modifier.fillMaxWidth().height(48.dp),
        shape = RoundedCornerShape(12.dp),
    ) {
        Icon(JouleIcons.Logout, contentDescription = null, modifier = Modifier.size(16.dp))
        Spacer(modifier = Modifier.width(6.dp))
        Text("Logout")
    }

    Spacer(modifier = Modifier.height(36.dp))
}

}?: Box(modifier = Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
    Text("Couldn't load profile.", color = StripeBody)
}
}
}

if (showDeactivateDialog) {
    AlertDialog(
        onDismissRequest = { showDeactivateDialog = false },
        title = { Text("Deactivate Account?") },

```

```

text = { Text("Are you sure? You will not be able to login until a Backoffice officer reactivates it.") },
confirmButton = {
    TextButton(onClick = {
        showDeactivateDialog = false
        scope.launch {
            try {
                val res = ApiClient.service.requestDeactivation(session!!.nic)
                if (res.isSuccessful || res.code() == 204) {
                    sessionDao.updateStatus("Deactivated")
                    sessionDao.clearSession()
                    ApiClient.authToken = null
                    onLogout()
                }
            } catch (_: Exception) {}
        }
    }) { Text("Deactivate", color = StripeError) }
},
dismissButton = { TextButton(onClick = { showDeactivateDialog = false }) { Text("Cancel") } },
)
}
}

@Composable
private fun ProfileRow(icon: androidx.compose.ui.graphics.vector.ImageVector, label: String, value: String) {
    Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        Icon(icon, contentDescription = null, tint = StripePrimary, modifier = Modifier.size(18.dp))
        Column {
            Text(label, style = MaterialTheme.typography.labelSmall, color = StripeBody)
            Text(value, style = MaterialTheme.typography.bodyMedium, fontWeight = FontWeight.Medium, color = StripeInk)
        }
    }
}
}

```

5.3.3 UI (Screenshots)

Web Application UI's

1. Prosumers Management Dashboard:

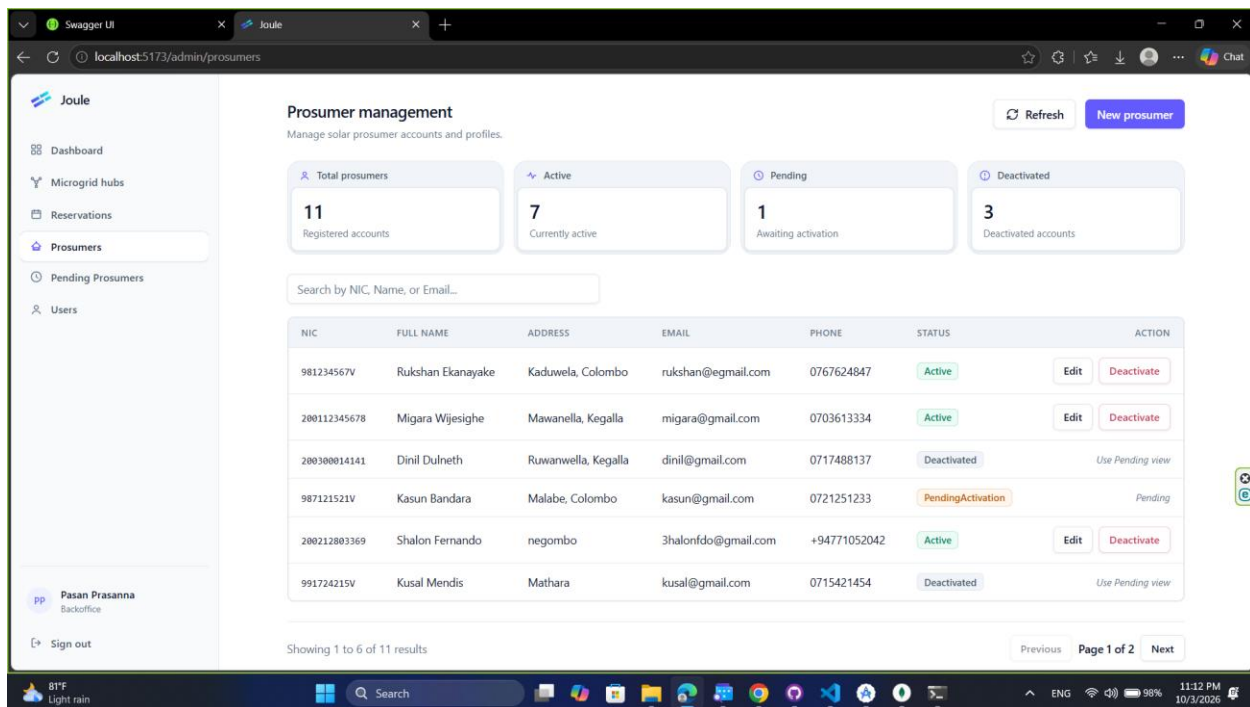


Figure : Prosumer Management Dashboard

2. Pending and Deactivated Prosumers Dashboard:

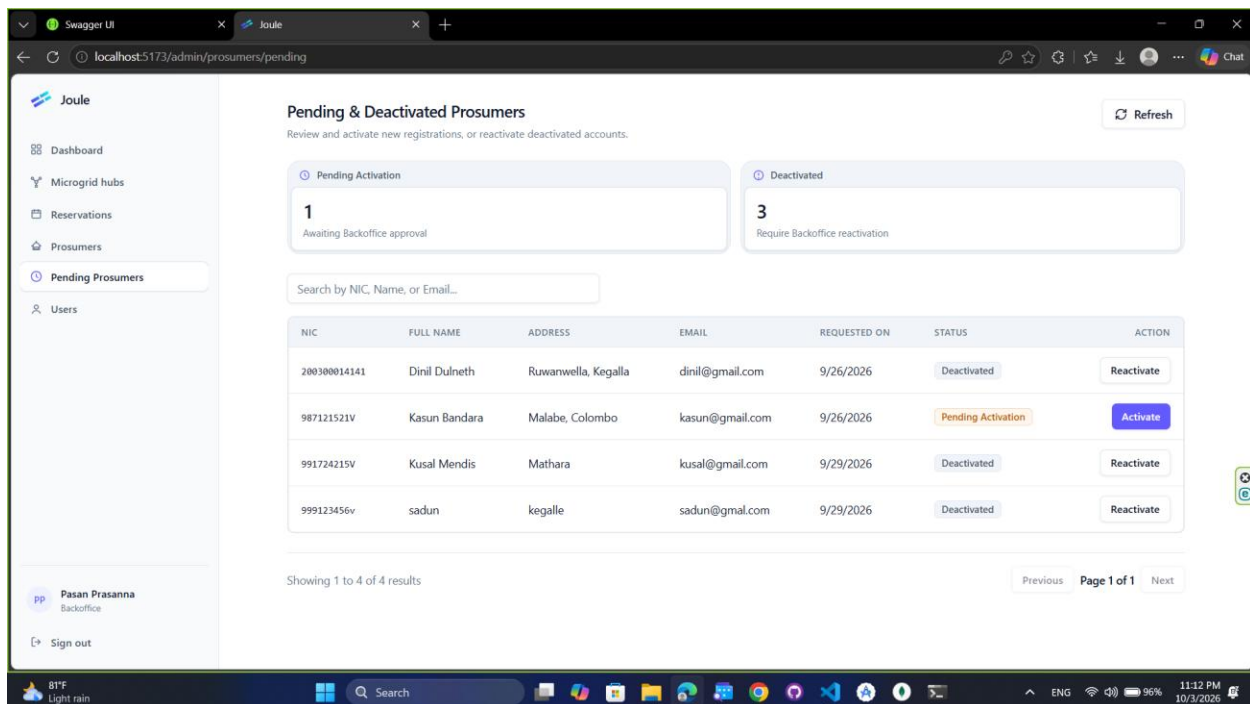


Figure : Pending and Deactivated Prosumers Dashboard

3. Create/Edit Pages

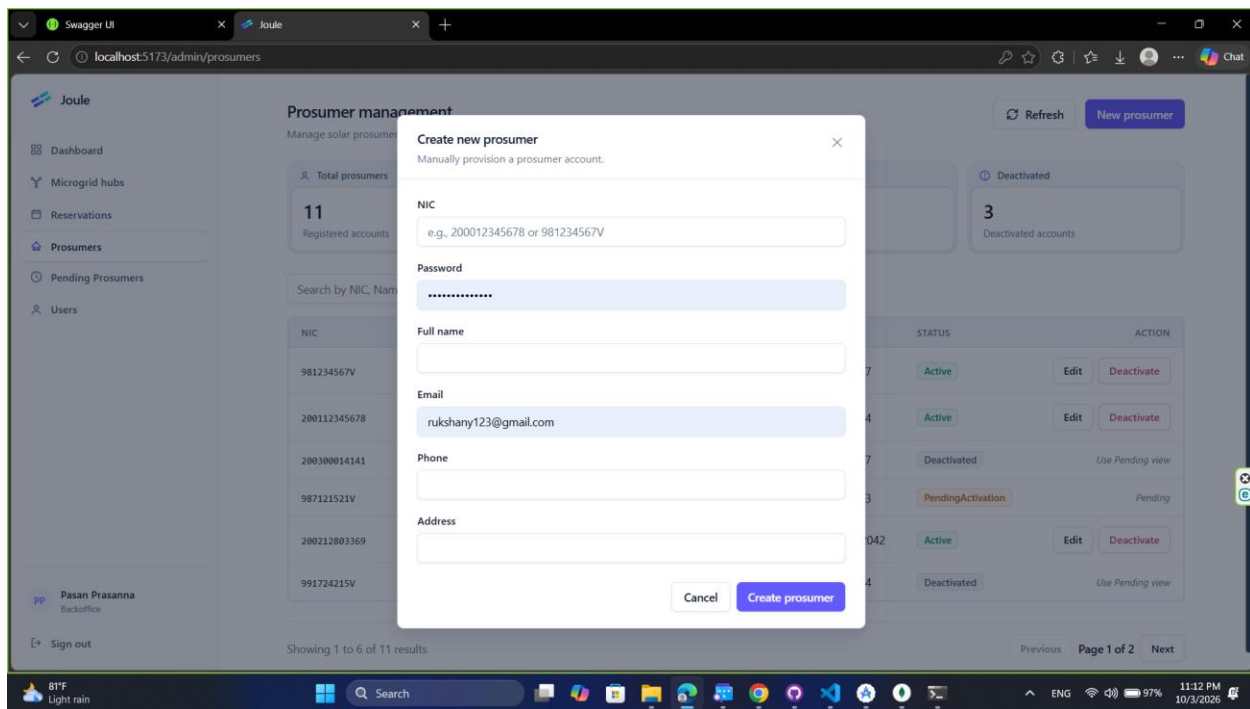


Figure : Create Page

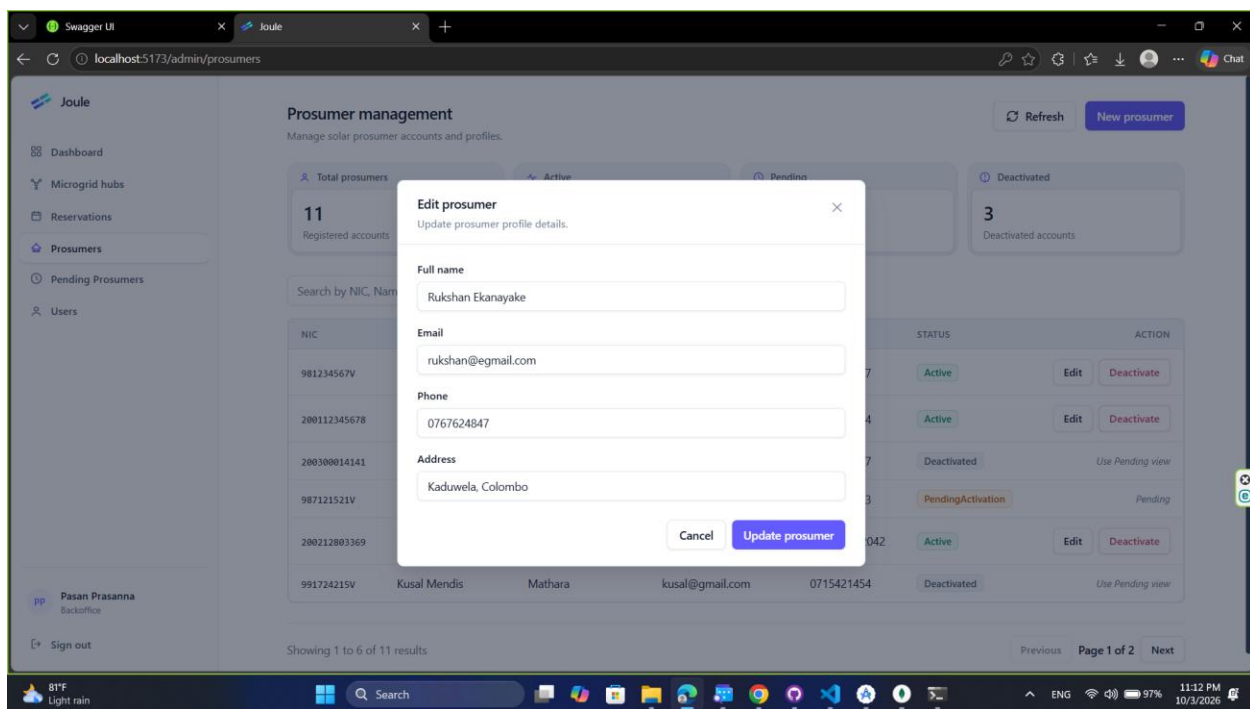


Figure : Edit Page

Mobile Application UI's

1. Prosumer Registration Screens

11:09 PM 55





Create your account

Power, exchanged precisely.



Figure : Prosumer Registration Screen 01

11:09 PM 55



Create your account

Power, exchanged precisely.

NIC Number *

Full Name *

Email Address *

Phone Number

Residential Address

Password * 

Register

Already have an account? [Sign in](#)

Figure : Prosumer Registration Screen 02

2. Prosumer Login Screen

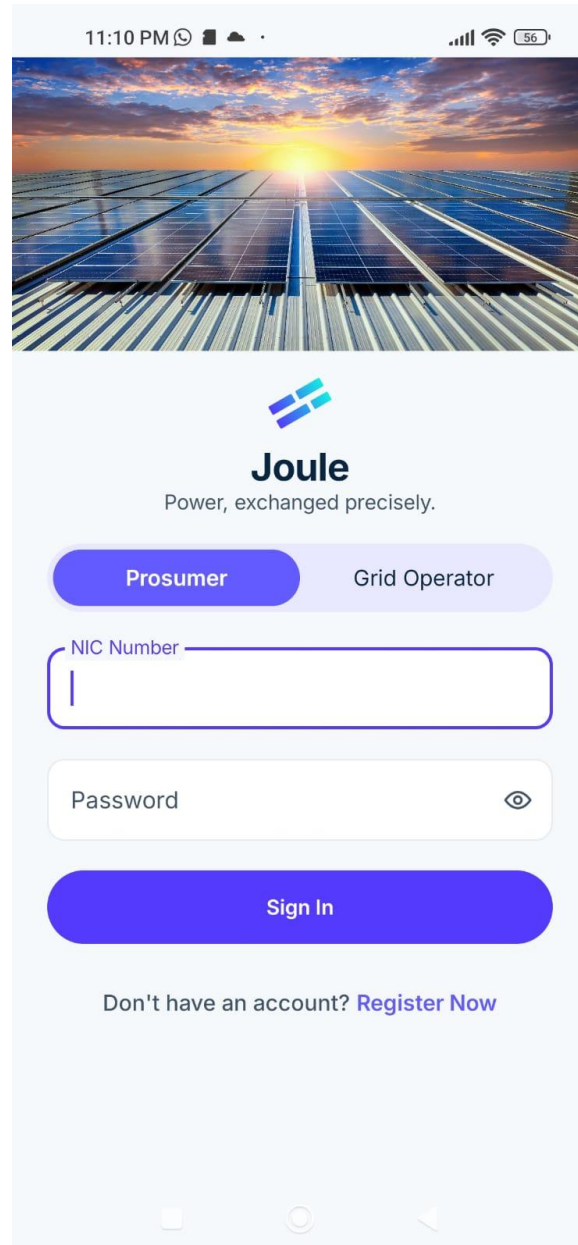


Figure : Prosumer Login Screen

3. Prosumer Profile Screen

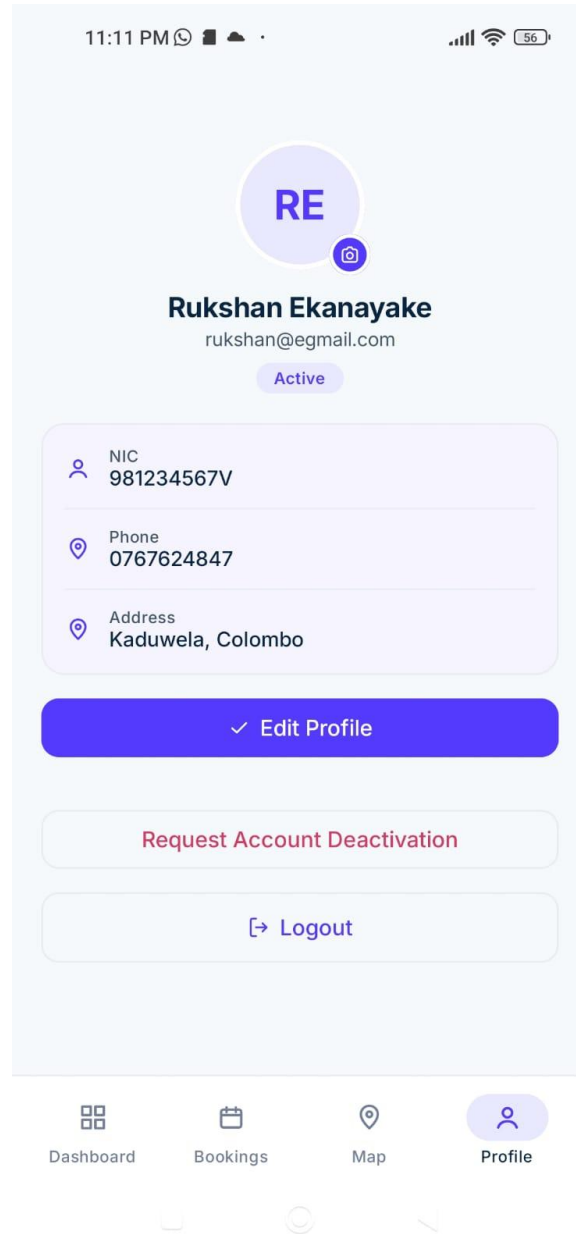


Figure : Prosumer Profile Screen

4. Prosumer Edit Profile Screen

11:11 PM

RE

Rukshan Ekanayake
rukshan@gmail.com

Active

Edit Profile

Full Name
Rukshan Ekanayake

Email
rukshan@gmail.com

Phone
0767624847

Address
Kaduwela, Colombo

Cancel Save

Dashboard Bookings Map Profile

Figure : Prosumer Edit Profile Screen

5. Prosumer Request Account Deactivation Screens

11:11 PM 5G

Full Name
Rukshan Ekanayake

Email
rukshan@gmail.com

Phone
0767624847

Address
Kaduwela, Colombo

Cancel Save

Request Account Deactivation

Logout

Dashboard Bookings Map Profile

Figure : Prosumer Request Account Deactivation Screen 01

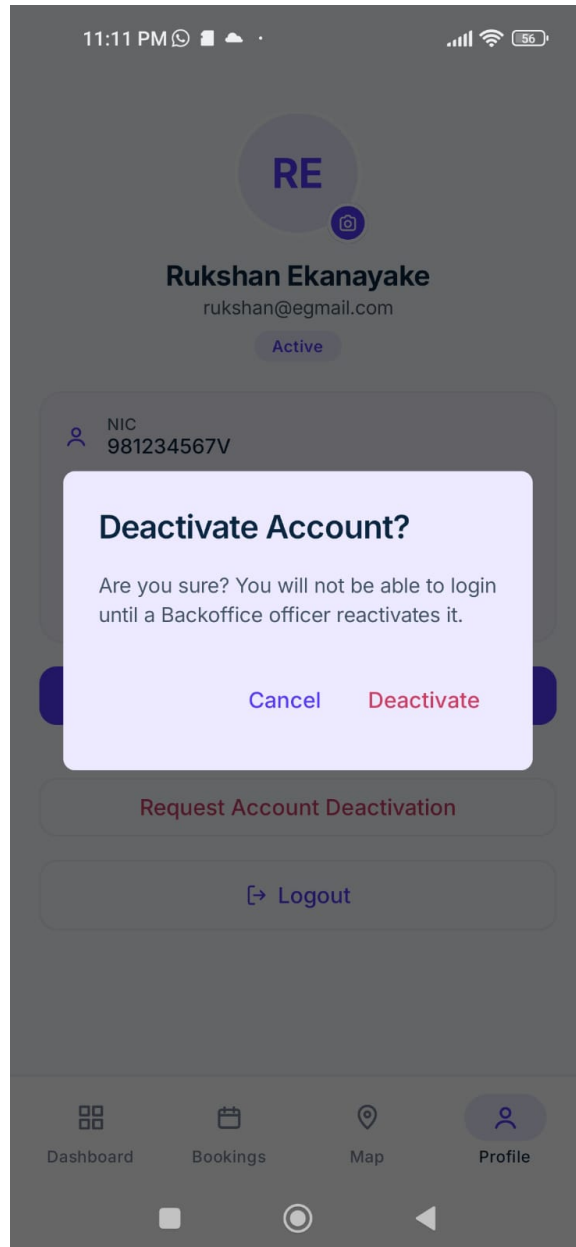


Figure : Prosumer Request Account Deactivation Screen 02

5.4 Dinil LiyanaArachchi (IT23268494)

I owned the **Energy Slot Reservation module** end-to-end. My slice spans all three layers rather than being confined to one:

- **Database:** the MongoDB Reservations collection schema
- **Service:** the C# Web API endpoints enforcing the system's two hardest business rules
- **Web client:** a React admin oversight page
- **Mobile client:** the native Android operator QR scanner

Backend (C# Web API, deployed on IIS)

Data model

Designed the Reservations MongoDB collection schema, including:

- prosumerNic
- stationId
- slotId
- scheduledAt
- status (Confirmed / Completed / Cancelled)
- qrToken
- qrTokenExpiresAt
- lifecycle timestamps

These field names are locked in the architecture document and are consumed by Migara's dashboard aggregations and Shalon's station deactivation check.

Business logic — ReservationService.cs

Every rule for the reservation lifecycle lives in this one service, following the FAT-service pattern:

- **7-day booking window.** A reservation's scheduledAt must be strictly in the future and within seven days of "now." Enforced at both creation and any later reschedule.

- **12-hour notice rule.** Updates and cancellations are rejected if the reservation's *currently stored* `scheduledAt` is less than 12 hours away. This is checked before any change is applied, so a rejected update never mutates state.
- **Slot state coordination.** Every reservation writes through Shalon's `SlotService.MarkReserved / MarkAvailable` methods, never directly to the `EnergyBookingSlots` collection. This is what keeps the two modules consistent despite living in different files.

API surface — `ReservationsController.cs`

Six endpoints:

- `POST /api/reservations`
- `PUT /api/reservations/{id}`
- `PUT /api/reservations/{id}/cancel`
- `GET /api/reservations/{id}`
- `GET /api/reservations` (with filters)
- `POST /api/reservations/verify-qr`

The controller is deliberately thin: it extracts JWT claims, calls the service, and maps `ServiceException` to HTTP statuses via the shared middleware. No validation is duplicated in the controller.

QR token service — `QrTokenService.cs` (pair-built with Migara)

HMAC-SHA256 signing over `reservationId|expiryUnix` with a server-only secret, base64url-encoded. Issue is called at reservation creation; Verify is called by the operator scan flow. The token is stored on the reservation document and never regenerated.

A signature check alone doesn't prevent replay, so `VerifyQrAsync` additionally rejects any token whose reservation status is not `Confirmed`. After one successful scan the reservation moves to `Completed`, and any subsequent scan returns **409** `RESERVATION_NOT_CONFIRMED`.

Web (React + Tailwind)

`ReservationsAdminPage.jsx`

A read-only oversight view for Backoffice and Grid Operators. Lists all reservations with filters for status, prosumer NIC, and station ID. Summary chips show total / confirmed / completed / cancelled counts. Each Confirmed reservation exposes a Cancel action, gated behind a confirmation modal that captures a reason.

API rejections — notably a 12-hour-rule violation — are surfaced **verbatim** in the toast, not re-interpreted on the client. A rule violation is a rule violation, and the UI is just a mirror of the server's decision.

ReservationDetailPage.jsx

Single-reservation view showing lifecycle timestamps (createdAt, completedAt) and the QR token as **plain text**. The web app never renders or scans the token — that is explicitly a mobile/operator action. The token is shown as text purely for developer debugging during the integration phase.

Routing

Added /reservations and /reservations/:id inside the AppShell layout route, plus a "Reservations" nav item under a new "Bookings" section in the sidebar. Extended the shared Input.jsx component with an as="select" option (backwards compatible) so the filter dropdowns reuse the same label / error / hint styling as the existing forms.

Mobile (Native Android, Jetpack Compose)

Operator QR scanner

Built ScanQrScreen.kt, ScanQrActivity.kt (thin host), and QrScannerHelper.kt. The scanner uses **CameraX** for the live preview and **ML Kit barcode-scanning** for decoding.

Flow:

1. Camera permission is requested via accompanist-permissions; the preview never binds to the lifecycle until permission is granted.
2. QrScannerHelper (an ImageAnalysis.Analyzer) fires the decoded string to the screen once per scan session — a handled flag prevents duplicate callbacks from the same frame burst.
3. The screen POSTs the token to /api/reservations/verify-qr via a Retrofit call, off the main thread using Dispatchers.IO.

4. On success, a green "✓ Verified" card shows reservation ID, prosumer, station, slot, and status. On failure, the server's error message is shown verbatim (e.g. an expired QR or a reused QR each produce specific codes the operator can act on).

SQLite persistence

ReservationCacheDao.kt stores every successful verification in a `local_verified_scans` table, so the operator's scan history survives app restarts and works without a live network. AppDbHelper.kt calls each owner's `CREATE_TABLE` constant from their own DAO file — this is what kept the mobile sides of Dinil and Migara from colliding on the same file.

Shared infrastructure

Added `ApiClient.kt` (Retrofit + OkHttp + JWT interceptor) and `ApiService.kt` (initial `verify-qr` endpoint), which other mobile owners can extend rather than re-create. The JWT token is stored in a `@Volatile` field on `ApiClient` and set by the login screen on success, so every subsequent request is automatically authenticated.

ReservationsController.cs

```
// =====  
  
// File: ReservationsController.cs  
  
// Purpose: Reservation lifecycle endpoints — create, update,  
  
//      cancel, fetch, list, and QR verification. All  
  
//      business rules delegated to ReservationService  
  
//      (FAT service pattern). No validation in clients.  
  
//      Operators/Backoffice can create reservations on  
  
//      behalf of any Prosumer (phone-in / walk-in booking).  
  
// Author: Dinil  
  
// =====
```

```
using Microsoft.AspNetCore.Authorization;
```

```

using Microsoft.AspNetCore.Mvc;

using SmartMicrogrid.Api.Models;

using SmartMicrogrid.Api.Services;


namespace SmartMicrogrid.Api.Controllers;


[ApiController]
[Route("api/reservations")]
//[Authorize]

public class ReservationsController : ControllerBase
{
    private readonly ReservationService _service;


    public ReservationsController(ReservationService service)
    {
        _service = service;
    }


    // Creates a reservation.

    // - Prosumer: creates for themselves (NIC from JWT).

    // - GridOperator / Backoffice: creates on behalf of a Prosumer (NIC from body).

    [HttpPost]

    //[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]

```

```

public async Task<IActionResult> Create([FromBody] CreateReservationRequest req)
{
    var nic = ResolveProsumerNic(req.ProsumerNic);

    var result = await _service.CreateAsync(nic, req);

    return Ok(result);
}

// Updates a reservation (12-hour rule, 7-day window re-check).
[HttpPut("{id}")]
[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]
public async Task<IActionResult> Update(string id, [FromBody] UpdateReservationRequest
req)
{
    var role = User.FindFirst("role")?.Value;

    var callerNic = User.FindFirst("nic")?.Value;

    if (role == "Prosumer")
    {
        // Prosumer can only update their own reservation.

        if (string.IsNullOrEmpty(callerNic))

            throw new ServiceException(401, "MISSING_NIC", "JWT missing nic claim.");

        await _service.UpdateAsync(id, callerNic, req);
    }
}

```

```

else

{

    // Operator/Backoffice updates without NIC ownership check.

    await _service.UpdateAsAdminAsync(id, req);

}

return NoContent();

}

// Cancels a reservation (12-hour rule, slot returned to Available).

[HttpPut("{id}/cancel")]

[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]

public async Task<ActionResult> Cancel(string id, [FromBody] CancelReservationRequest req)

{

    var role = User.FindFirst("role")?.Value;

    var callerNic = User.FindFirst("nic")?.Value;

    if (role == "Prosumer")

    {

        if (string.IsNullOrEmpty(callerNic))

            throw new ServiceException(401, "MISSING_NIC", "JWT missing nic claim.");

        await _service.CancelAsync(id, callerNic, req.Reason);

    }

}

```

```

else

{
    // Operator/Backoffice cancels on behalf — no NIC ownership check.
    await _service.CancelAsAdminAsync(id, req.Reason);
}

return NoContent();
}

// Fetches one reservation — owning Prosumer or GridOperator/Backoffice.
[HttpGet("{id}")]
[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]
public async Task<ActionResult> Get(string id)
{
    var role = User.FindFirst("role")?.Value;
    var nic = User.FindFirst("nic")?.Value;

    var reservation = (role == "Prosumer")
        ? await _service.GetOwnedAsync(id, nic!)
        : await _service.GetByIdAsync(id);

    return Ok(MapToResponse(reservation));
}

```

```

// Lists reservations with filters. Prosumers are scoped to their own NIC.

[HttpGet]

//[Authorize(Roles = "Prosumer,GridOperator,Backoffice")]

public async Task<IActionResult> List(

    [FromQuery] string? nic,

    [FromQuery] string? stationId,

    [FromQuery] string? status,

    [FromQuery] DateTime? from,

    [FromQuery] DateTime? to)

{

    var role = User.FindFirst("role")?.Value;

    var jwtNic = User.FindFirst("nic")?.Value;

    // Prosumers can only ever see their own reservations.

    var effectiveNic = (role == "Prosumer") ? jwtNic : nic;

    var list = await _service.ListAsync(effectiveNic, stationId, status, from, to);

    return Ok(list.Select(MapToResponse));

}

// Grid Operator verifies a scanned QR token, flipping the reservation to Completed.

[HttpPost("verify-qr")]

```

```

[Authorize(Roles = "GridOperator")]

public async Task<IActionResult> VerifyQr([FromBody] VerifyQrRequest req)
{
    // "sub" is the standard JWT claim for the user id; JwtService issues it via
    // JwtRegisteredClaimNames.Sub, and Program.cs clears the inbound claim map
    // so it's readable as "sub" here (not the mangled ClaimTypes.NameIdentifier URI).
    var operatorUserId = User.FindFirst("sub")?.Value

    ?? User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value
    ?? throw new ServiceException(401, "MISSING_SUB", "JWT missing sub claim.");

    var result = await _service.VerifyQrAsync(req.QrToken, operatorUserId);

    return Ok(result);
}

// ----- Helpers -----

// Resolves the NIC of the Prosumer the reservation is for.
// Prosumer: NIC must come from JWT.
// Operator/Backoffice: NIC must come from the request body.
private string ResolveProsumerNic(string? bodyNic)
{
    var role = User.FindFirst("role")?.Value;

    if (role == "Prosumer")

```



```

{
    return User.FindFirst("nic")?.Value
        ?? throw new ServiceException(401, "MISSING_NIC", "JWT missing nic claim.");
}

// Operator / Backoffice path
if (string.IsNullOrEmpty(bodyNic))
    throw new ServiceException(400, "MISSING_NIC", "Prosumer NIC is required.");

return bodyNic.Trim();
}

// Maps a Reservation model to its API response shape.
private static ReservationResponse MapToResponse(Reservation r) => new()
{
    Id = r.Id,
    ProsumerNic = r.ProsumerNic,
    StationId = r.StationId,
    SlotId = r.SlotId,
    ScheduledAt = r.ScheduledAt,
    Status = r.Status,
    QrToken = r.QrToken,
    CreatedAt = r.CreatedAt,

```

```

        CompletedAt = r.CompletedAt,

        CancelReason = r.CancelReason

    };
}

ExceptionHandlingMiddleware.cs
// =====

// File: ExceptionHandlingMiddleware.cs

// Purpose: Global exception handler. Maps ServiceException to

//     its declared HTTP status + JSON error body so clients

//     receive the exact status/code/message the service

//     threw — never a generic 500 for a business rejection.

//     Anything that isn't a known service exception is a

//     genuine 500 and gets logged with the full stack.

//     Must be registered BEFORE UseAuthentication /

//     UseAuthorization / MapControllers in Program.cs.

// Author: Dinil (shared infra)

// =====

using System.Text.Json;

using SmartMicrogrid.Api.Services;

namespace SmartMicrogrid.Api.Middleware;

```

```

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;

    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    // Captures the next middleware and the logger for unhandled-exception diagnostics.
    public ExceptionHandlingMiddleware(RequestDelegate next,
    ILogger<ExceptionHandlingMiddleware> logger)
    {
        _next = next;

        _logger = logger;
    }

    // Wraps the downstream pipeline; every request passes through here.
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }

        catch (ServiceException ex)
        {
            // The service declared its own HTTP status — honor it exactly.

```

```

// 4xx business rejections are logged at Information level so they
// don't pollute the error log.

_logger.LogInformation("ServiceException {Code}: {Message}", ex.Code, ex.Message);

await WriteErrorAsync(context, ex.StatusCode, ex.Code, ex.Message);
}

catch (NotFoundException ex)
{
    await WriteErrorAsync(context, StatusCodes.Status404NotFound, "NOT_FOUND",
ex.Message);
}

catch (ValidationException ex)
{
    await WriteErrorAsync(context, StatusCodes.Status400BadRequest,
"VALIDATION_ERROR", ex.Message);
}

catch (ConflictException ex)
{
    await WriteErrorAsync(context, StatusCodes.Status409Conflict, "CONFLICT", ex.Message);
}

catch (Exception ex)
{
    // Genuine unhandled error — log with stack trace, return a generic body
    // so internal details never leak to clients.

    _logger.LogError(ex, "Unhandled exception");
}

```

```

        await WriteErrorAsync(context, StatusCodes.Status500InternalServerError,
            "INTERNAL_ERROR", "An unexpected error occurred.");
    }
}

// Serializes { code, message } as JSON with the given HTTP status.
private static async Task WriteErrorAsync(HttpContext context, int statusCode, string code,
string message)
{
    if (context.Response.HasStarted)
    {
        // Response already partially written — can't change status. Nothing to do.
        return;
    }

    context.Response.StatusCode = statusCode;

    context.Response.ContentType = "application/json";

    var payload = JsonSerializer.Serialize(new { code, message });

    await context.Response.WriteAsync(payload);
}
}

```

Reservation.cs

// =====

// File: Reservation.cs

```

// Purpose: MongoDB model for the Reservations collection — a
//     prosumer's energy slot booking, including the QR token
//     issued at creation and its lifecycle status.
//     Write-side owner: Dinil (create/update/cancel + QR
//     issuance/verification — the 7-day window and 12-hour
//     notice rules live in his ReservationService). Shalon
//     reads this collection for the station-deactivation check,
//     the reservations-overview panel, and the
//     dashboard/search/nearby endpoints (architecture.md §3/§4)
//     — her services were updated during the 2026-09-26 merge to
//     compare Status as the plain string below (matching
//     Dinil's already-written create/update logic) rather than
//     the ReservationStatus enum her own draft of this file
//     used before the two versions were reconciled.

// Author: Dinil

// =====

using MongoDB.Bson;

using MongoDB.Bson.Serialization.Attributes;

namespace SmartMicrogrid.Api.Models;

public class Reservation
{

```

[BsonId]

[BsonRepresentation(BsonType.ObjectId)]

public string Id { get; set; } = ObjectId.GenerateNewId().ToString();

[BsonElement("prosumerNic")]

public string ProsumerNic { get; set; } = null!;

[BsonElement("stationId")]

[BsonRepresentation(BsonType.ObjectId)]

public string StationId { get; set; } = null!;

[BsonElement("slotId")]

[BsonRepresentation(BsonType.ObjectId)]

public string SlotId { get; set; } = null!;

[BsonElement("scheduledAt")]

public DateTime ScheduledAt { get; set; }

[BsonElement("status")]

public string Status { get; set; } = "Confirmed"; // Confirmed | Completed | Cancelled

[BsonElement("qrToken")]

public string QrToken { get; set; } = null!;

```

[BsonElement("qrTokenExpiresAt")]

public DateTime QrTokenExpiresAt { get; set; }


[BsonElement("createdAt")]

public DateTime CreatedAt { get; set; } = DateTime.UtcNow;


[BsonElement("updatedAt")]

public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;


[BsonElement("completedAt")]

public DateTime? CompletedAt { get; set; }


[BsonElement("completedByUserId")]

[BsonRepresentation(BsonType.ObjectId)]

public string? CompletedByUserId { get; set; }


[BsonElement("cancelReason")]

public string? CancelReason { get; set; }

}

```

ReservationDtos.cs

// =====


```
// File: ReservationDtos.cs

// Purpose: Request/response shapes for the reservation
// endpoints, keeping API contract decoupled from the
// MongoDB model and matching architecture.md §3.

// Author: Dinil

// =====

namespace SmartMicrogrid.Api.Models;

// ----- Requests -----

public class CreateReservationRequest
{
    public string StationId { get; set; } = null!;

    public string SlotId { get; set; } = null!;

    public DateTime ScheduledAt { get; set; }

    public string? ProsumerNic { get; set; } // optional for Prosumer, required for
    Operator/Backoffice
}

public class UpdateReservationRequest
{
    public string? StationId { get; set; }

    public string? SlotId { get; set; }
}
```

```

        public DateTime? ScheduledAt { get; set; }
    }

    public class CancelReservationRequest
    {
        public string? Reason { get; set; }
    }

    public class VerifyQrRequest
    {
        public string QrToken { get; set; } = null!;
    }

    // ----- Responses -----

    public class CreateReservationResponse
    {
        public string Id { get; set; } = null!;
        public string Status { get; set; } = null!;
        public string QrToken { get; set; } = null!;
        public DateTime QrTokenExpiresAt { get; set; }
    }

    public class ReservationResponse

```

```

{
    public string Id { get; set; } = null!;
    public string ProsumerNic { get; set; } = null!;
    public string StationId { get; set; } = null!;
    public string SlotId { get; set; } = null!;
    public DateTime ScheduledAt { get; set; }
    public string Status { get; set; } = null!;
    public string? QrToken { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? CompletedAt { get; set; }
    public string? CancelReason { get; set; }
}

```

```

public class VerifyQrResponse

```

```

{
    public string ReservationId { get; set; } = null!;
    public string ProsumerName { get; set; } = null!;
    public string StationName { get; set; } = null!;
    public int SlotNumber { get; set; }
    public string Status { get; set; } = null!;
}

```

```

ReservationService.cs

```

```

// =====

```

```

// File: ReservationService.cs

// Purpose: Business logic for the reservation lifecycle —
//      7-day booking window, 12-hour notice rule, slot
//      coordination through SlotService, QR issuance,
//      and operator QR verification (flips Confirmed ->
//      Completed). All validation lives here, never in
//      controllers or clients.
//
//      Admin paths (UpdateAsAdminAsync / CancelAsAdminAsync)
//      skip NIC ownership checks so Grid Operators and
//      Backoffice staff can act on behalf of Prosumers -
//      but they still enforce the 12-hour rule and the
//      7-day window.

// Author: Dinil

// =====

using MongoDB.Driver;

using SmartMicrogrid.Api.Data;

using SmartMicrogrid.Api.Models;

namespace SmartMicrogrid.Api.Services;

public class ReservationService
{

```

```

private const int BOOKING_WINDOW_DAYS = 7;

private const int NOTICE_HOURS = 12;


private readonly MongoDBContext _db;

private readonly SlotService _slotService;

private readonly StationService _stationService;

private readonly QrTokenService _qrService;


public ReservationService(
    MongoDBContext db,
    SlotService slotService,
    StationService stationService,
    QrTokenService qrService)
{
    _db = db;

    _slotService = slotService;

    _stationService = stationService;

    _qrService = qrService;
}


// Creates a reservation under the given prosumer's NIC, enforcing the 7-day
// window and slot availability; issues the QR token immediately at creation.

// The NIC is resolved by the controller — for Prosumers it comes from the

```

```

// JWT, for Operators/Backoffice it comes from the request body.

    public async Task<CreateReservationResponse> CreateAsync(string prosumerNic,
CreateReservationRequest req)
    {
        if (string.IsNullOrEmpty(prosumerNic))
            throw new ServiceException(400, "MISSING_NIC", "Prosumer NIC is required.");

        var now = DateTime.UtcNow;

        var scheduledUtc = DateTime.SpecifyKind(req.ScheduledAt, DateTimeKind.Utc);

        // 7-day window (must be in future, within BOOKING_WINDOW_DAYS days).
        if (scheduledUtc <= now)
            throw new ServiceException(400, "SCHEDULED_IN_PAST", "Scheduled time must be in the
future.");

        if (scheduledUtc > now.AddDays(BOOKING_WINDOW_DAYS))
            throw new ServiceException(400, "OUTSIDE_7_DAY_WINDOW",
                $"Reservations must be within {BOOKING_WINDOW_DAYS} days.");

        // Slot must exist and be Available.

        var slot = await _slotService.GetByIdAsync(req.SlotId)

        ?? throw new ServiceException(404, "SLOT_NOT_FOUND", "Slot does not exist.");

        if (slot.Status != SlotStatus.Available)
            throw new ServiceException(409, "SLOT_NOT_AVAILABLE", "Slot is not available.");
    }

```

```

// Station must exist and be Active.

var station = await _stationService.GetByIdAsync(req.StationId)

    ?? throw new ServiceException(404, "STATION_NOT_FOUND", "Station does not exist.");

if (station.Status != "Active")

    throw new ServiceException(409, "STATION_INACTIVE", "Station is not active.");


// Issue QR immediately — no separate approval step (see architecture.md §2.4).

var reservationId = MongoDB.Bson.ObjectId.GenerateNewId().ToString();

var qrExpiry = scheduledUtc.AddHours(24);

var qrToken = _qrService.Issue(reservationId, qrExpiry);


var reservation = new Reservation
{
    Id = reservationId,

    ProsumerNic = prosumerNic,

    StationId = req.StationId,

    SlotId = req.SlotId,

    ScheduledAt = scheduledUtc,

    Status = "Confirmed",

    QrToken = qrToken,

    QrTokenExpiresAt = qrExpiry,

    CreatedAt = now,

    UpdatedAt = now

```

```

};

await _db.Reservations.InsertOneAsync(reservation);

await _slotService.MarkReserved(req.SlotId);

return new CreateReservationResponse
{
    Id = reservation.Id,
    Status = reservation.Status,
    QrToken = reservation.QrToken,
    QrTokenExpiresAt = reservation.QrTokenExpiresAt
};
}

// Updates a reservation as the owning Prosumer — enforces ownership.
// Delegates to the shared core after the ownership check.
public async Task UpdateAsync(string id, string prosumerNic, UpdateReservationRequest req)
{
    var reservation = await GetOwnedAsync(id, prosumerNic);

    await ApplyUpdateAsync(reservation, req);
}

// Updates a reservation as a Grid Operator / Backoffice user.

```



```

// Skips the NIC ownership check but still enforces all business rules.

public async Task UpdateAsAdminAsync(string id, UpdateReservationRequest req)
{
    var reservation = await GetByIdAsync(id);

    await ApplyUpdateAsync(reservation, req);
}

// Shared update logic — used by both Prosumer and Admin paths.
// Enforces status, 12-hour notice, 7-day window, and slot coordination.

private async Task ApplyUpdateAsync(Reservation reservation, UpdateReservationRequest
req)
{
    var now = DateTime.UtcNow;

    if (reservation.Status != "Confirmed")

        throw new ServiceException(409, "RESERVATION_NOT_EDITABLE",

            "Only Confirmed reservations can be edited.");

    // 12-hour rule against current stored time.

    if (reservation.ScheduledAt - now < TimeSpan.FromHours(NOTICE_HOURS))

        throw new ServiceException(409, "TOO_LATE_TO_MODIFY",

            $"Changes require at least {NOTICE_HOURS} hours' notice.");
}

```

```

// If rescheduling, re-validate 7-day window on the new time.

if (req.ScheduledAt.HasValue)
{
    var newScheduled = DateTime.SpecifyKind(req.ScheduledAt.Value, DateTimeKind.Utc);

    if (newScheduled <= now)

        throw new ServiceException(400, "SCHEDULED_IN_PAST", "New time must be in the
future.");

    if (newScheduled > now.AddDays(BOOKING_WINDOW_DAYS))

        throw new ServiceException(400, "OUTSIDE_7_DAY_WINDOW",

            $"New time must be within {BOOKING_WINDOW_DAYS} days.");

    reservation.ScheduledAt = newScheduled;
}

// If station is changing, validate it exists and is Active.

if (!string.IsNullOrEmpty(req.StationId) && req.StationId != reservation.StationId)
{
    var newStation = await _stationService.GetByIdAsync(req.StationId)

        ?? throw new ServiceException(404, "STATION_NOT_FOUND", "New station does not
exist.");

    if (newStation.Status != "Active")

        throw new ServiceException(409, "STATION_INACTIVE", "New station is not active.");

    reservation.StationId = req.StationId;
}

```

```

// If slot is changing, validate new slot and coordinate state.
if (!string.IsNullOrEmpty(req.SlotId) && req.SlotId != reservation.SlotId)
{
    var newSlot = await _slotService.GetByIdAsync(req.SlotId)

    ?? throw new ServiceException(404, "SLOT_NOT_FOUND", "New slot does not exist.");

    if (newSlot.Status != SlotStatus.Available)

        throw new ServiceException(409, "SLOT_NOT_AVAILABLE", "New slot is not available.");

    await _slotService.MarkAvailable(reservation.SlotId);

    await _slotService.MarkReserved(req.SlotId);

    reservation.SlotId = req.SlotId;
}

reservation.UpdatedAt = now;

await _db.Reservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);
}

// Cancels a reservation as the owning Prosumer — enforces ownership.
public async Task CancelAsync(string id, string prosumerNic, string? reason)
{
    var reservation = await GetOwnedAsync(id, prosumerNic);

    await ApplyCancelAsync(reservation, reason);
}

```

```

// Cancels a reservation as a Grid Operator / Backoffice user.

// Skips the NIC ownership check but still enforces the 12-hour rule.
public async Task CancelAsAdminAsync(string id, string? reason)
{
    var reservation = await GetByIdAsync(id);

    await ApplyCancelAsync(reservation, reason);
}

// Shared cancel logic — used by both Prosumer and Admin paths.
private async Task ApplyCancelAsync(Reservation reservation, string? reason)
{
    var now = DateTime.UtcNow;

    if (reservation.Status != "Confirmed")

        throw new ServiceException(409, "RESERVATION_NOT_CANCELLABLE",
            "Only Confirmed reservations can be cancelled.");

    if (reservation.ScheduledAt - now < TimeSpan.FromHours(NOTICE_HOURS))

        throw new ServiceException(409, "TOO_LATE_TO_CANCEL",
            $"Cancellations require at least {NOTICE_HOURS} hours' notice.");

    reservation.Status = "Cancelled";
}

```

```

reservation.CancelReason = reason;

reservation.UpdatedAt = now;

await _db.Reservations.ReplaceOneAsync(r => r.Id == reservation.Id, reservation);

await _slotService.MarkAvailable(reservation.SlotId);
}

// Verifies a QR token and flips a Confirmed reservation to Completed.
// Called by Grid Operators via POST /reservations/verify-qr.
public async Task<VerifyQrResponse> VerifyQrAsync(string qrToken, string operatorUserId)
{
    // 1. Verify HMAC signature + token expiry.

    var reservationId = _qrService.Verify(qrToken)

        ?? throw new ServiceException(400, "INVALID_QR", "QR is invalid or expired.");

    // 2. Load reservation and ensure it is still Confirmed (not already used).

    var reservation = await _db.Reservations.Find(r => r.Id ==
reservationId).FirstOrDefaultAsync()

        ?? throw new ServiceException(404, "RESERVATION_NOT_FOUND", "Reservation not
found.");

    if (reservation.Status != "Confirmed")

        throw new ServiceException(409, "RESERVATION_NOT_CONFIRMED",

            $"Reservation is {reservation.Status}, cannot complete.");
}

```

```

// 3. Load station + slot for the operator-visible confirmation payload.

var station = await _stationService.GetByIdAsync(reservation.StationId)

    ?? throw new ServiceException(404, "STATION_NOT_FOUND", "Station not found.");

if (station.Status != "Active")

    throw new ServiceException(409, "STATION_INACTIVE", "Station is not active.");

var slot = await _slotService.GetByIdAsync(reservation.SlotId)

    ?? throw new ServiceException(404, "SLOT_NOT_FOUND", "Slot not found.");

// 4. Flip status and record the operator who completed it.

var now = DateTime.UtcNow;

var update = Builders<Reservation>.Update

    .Set(r => r.Status, "Completed")

    .Set(r => r.CompletedAt, now)

    .Set(r => r.CompletedByUserId, operatorUserId)

    .Set(r => r.UpdatedAt, now);

await _db.Reservations.UpdateOneAsync(r => r.Id == reservationId, update);

// 5. Free the slot again now that the energy transfer is complete.

await _slotService.MarkAvailable(reservation.SlotId);

return new VerifyQrResponse

```

```

{
    ReservationId = reservation.Id,

    ProsumerName = reservation.ProsumerNic,

    StationName = station.Name,

    SlotNumber = slot.SlotNumber,

    Status = "Completed"

};
}

// Loads a reservation and asserts the caller owns it.

public async Task<Reservation> GetOwnedAsync(string id, string prosumerNic)
{
    var reservation = await _db.Reservations.Find(r => r.Id == id).FirstOrDefaultAsync()

    ?? throw new ServiceException(404, "RESERVATION_NOT_FOUND", "Reservation not
found.");

    if (reservation.ProsumerNic != prosumerNic)

        throw new ServiceException(403, "FORBIDDEN", "Not your reservation.");

    return reservation;
}

// Loads a reservation by ID with no ownership check (operator/backoffice).

public async Task<Reservation> GetByIdAsync(string id)
{

```

```

        return await _db.Reservations.Find(r => r.Id == id).FirstOrDefaultAsync()

        ?? throw new ServiceException(404, "RESERVATION_NOT_FOUND", "Reservation not
found.");
    }

// Lists reservations with optional filters. GridOperator/Backoffice see all;
// Prosumers get restricted to their own NIC via a JWT-scoped query param.
public async Task<List<Reservation>> ListAsync(
    string? prosumerNic, string? stationId, string? status, DateTime? from, DateTime? to)
{
    var builder = Builders<Reservation>.Filter;

    var filter = builder.Empty;

    if (!string.IsNullOrEmpty(prosumerNic)) filter &= builder.Eq(r => r.ProsumerNic,
prosumerNic);

    if (!string.IsNullOrEmpty(stationId)) filter &= builder.Eq(r => r.StationId, stationId);

    if (!string.IsNullOrEmpty(status)) filter &= builder.Eq(r => r.Status, status);

    if (from.HasValue) filter &= builder.Gte(r => r.ScheduledAt, from.Value);

    if (to.HasValue) filter &= builder.Lte(r => r.ScheduledAt, to.Value);

    return await _db.Reservations.Find(filter).SortBy(r => r.ScheduledAt).ToListAsync();
}
}

ReservationDetailPage.jsx
// =====

```



```

// File: ReservationDetailPage.jsx

// Purpose: Single reservation view for Backoffice/GridOperator.

//     Shows lifecycle timestamps, QR token (as text), and
//     offers Edit + Cancel for Confirmed reservations.
//     Server enforces 7-day and 12-hour rules; rejections
//     are surfaced verbatim.
//
//     On load, fetches the Prosumer's profile via
//     GET /api/prosumers/{nic} so the operator sees the
//     name and email — not just the NIC.
//
//     The Edit modal loads Stations and (for the selected
//     station) its available Slots as dropdowns. The
//     reservation's own current slot is included in the
//     list even if its status isn't Available, so the
//     operator can leave it unchanged.

// Author: Dinil

// =====

import { useEffect, useState } from "react";

import { Link, useNavigate, useParams } from "react-router-dom";

import api from "../../services/api.js";

import Button from "../../components/common/Button.jsx";

import Badge from "../../components/common/Badge.jsx";

```

```

import Input from "../../components/common/Input.jsx";

import Modal from "../../components/common/Modal.jsx";

import Toast from "../../components/common/Toast.jsx";


// Formats an ISO timestamp for display, or an em dash for null.

function formatDateTime(iso) {

  if (!iso) return "—";

  const d = new Date(iso);

  return d.toLocaleString(undefined, {

    year: "numeric",

    month: "short",

    day: "2-digit",

    hour: "2-digit",

    minute: "2-digit"

  });

}


// Converts an ISO string to a value usable by <input type="datetime-local">.

function toLocalDateTimeInput(iso) {

  if (!iso) return "";

  const d = new Date(iso);

  const pad = (n) => String(n).padStart(2, "0");

  return (

```

```

    `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}` +
    `T${pad(d.getHours())}:${pad(d.getMinutes())}`
  );
}

```

// Minimum allowed scheduled time: now + 15 minutes.

```

function minScheduledAt() {
  const d = new Date(Date.now() + 15 * 60 * 1000);
  const pad = (n) => String(n).padStart(2, "0");
  return (
    `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}` +
    `T${pad(d.getHours())}:${pad(d.getMinutes())}`
  );
}

```

// Maximum allowed scheduled time: now + 7 days (matches server window).

```

function maxScheduledAt() {
  const d = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);
  const pad = (n) => String(n).padStart(2, "0");
  return (
    `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}` +
    `T${pad(d.getHours())}:${pad(d.getMinutes())}`
  );
}

```

```
}
```

```
function statusTone(status) {
```

```
  switch (status) {
```

```
    case "Confirmed":
```

```
      return "warning";
```

```
    case "Completed":
```

```
      return "success";
```

```
    case "Cancelled":
```

```
      return "neutral";
```

```
    default:
```

```
      return "neutral";
```

```
  }
```

```
}
```

```
export default function ReservationDetailPage() {
```

```
  const { id } = useParams();
```

```
  const navigate = useNavigate();
```

```
  const [reservation, setReservation] = useState(null);
```

```
  const [prosumer, setProsumer] = useState(null); // Fetched by NIC
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);
```

```

// Parent-owned toast state.

const [toast, setToast] = useState({ message: "", tone: "error" });


const [showCancel, setShowCancel] = useState(false);

const [cancelReason, setCancelReason] = useState("");

const [cancelBusy, setCancelBusy] = useState(false);


// Edit modal state.

const [showEdit, setShowEdit] = useState(false);

const [editForm, setEditForm] = useState(null);

const [editBusy, setEditBusy] = useState(false);


// Dropdown data for the Edit modal.

const [stations, setStations] = useState([]);

const [slots, setSlots] = useState([]);

const [dropdownsLoading, setDropdownsLoading] = useState({
  stations: false,
  slots: false
});

const [dropdownsError, setDropdownsError] = useState(null);


function show(message, tone = "error") {

```

```

    setToast({ message, tone });
  }

  // -----
  // Data load
  // -----

  async function load() {
    setLoading(true);
    setError(null);
    setProsumer(null);

    try {
      const { data } = await api.get(`/reservations/${id}`);

      setReservation(data);

      if (data?.prosumerNic) {
        try {
          const p = await api.get(`/prosumers/${data.prosumerNic}`);

          setProsumer(p.data);
        } catch {
          setProsumer(null);
        }
      }
    }
  }

```

```

} catch (e) {

  const msg =

    e?.response?.data?.message ||

    (e?.response?.status === 404 ? "Reservation not found." : e.message);

  setError(msg);

} finally {

  setLoading(false);

}
}

```

// Initial load — every setState runs after an await.

```

useEffect(() => {

  let cancelled = false;

  (async () => {

    try {

      const { data } = await api.get(`/reservations/${id}`);

      if (cancelled) return;

      setReservation(data);

      if (data?.prosumerNic) {

        try {

          const p = await api.get(`/prosumers/${data.prosumerNic}`);

```

```

        if (!cancelled) setProsumer(p.data);
    } catch {
        if (!cancelled) setProsumer(null);
    }
}

} catch (e) {
    if (cancelled) return;

    const msg =
        e?.response?.data?.message ||
        (e?.response?.status === 404 ? "Reservation not found." : e.message);

    setError(msg);
} finally {
    if (!cancelled) setLoading(false);
}

})();

return () => {
    cancelled = true;
};

// eslint-disable-next-line react-hooks/exhaustive-deps
}, [id]);

// -----

```



```

// Dropdown loaders

// -----

// Load Stations once (Active only).

useEffect(() => {

  let cancelled = false;

  (async () => {

    setDropdownsLoading((s) => ({ ...s, stations: true }));

    try {

      const { data } = await api.get("/stations", {

        params: { status: "Active" }

      });

      if (!cancelled) setStations(Array.isArray(data) ? data : []);

    } catch (e) {

      if (!cancelled) {

        setDropdownsError(

          e?.response?.data?.message || "Could not load Stations."

        );

      }

    }

  } finally {

    if (!cancelled) setDropdownsLoading((s) => ({ ...s, stations: false }));

  }

}

```

```
})();
```

```
return () => {
```

```
  cancelled = true;
```

```
};
```

```
, []);
```

```
// When edit modal is open and station is chosen, load that station's slots.
```

```
// Include the reservation's own current slot even if it's not Available,
```

```
// so opening the modal and saving without changes never errors out.
```

```
useEffect(() => {
```

```
  if (!showEdit || !editForm?.stationId) {
```

```
    setSlots([]);
```

```
    return;
```

```
  }
```

```
let cancelled = false;
```

```
(async () => {
```

```
  setDropdownsLoading((s) => ({ ...s, slots: true }));
```

```
  try {
```

```
    const { data } = await api.get(`/stations/${editForm.stationId}`);
```

```
    const all = Array.isArray(data?.slots) ? data.slots : [];
```

```

// Include Available slots + the reservation's current slot
// (which may be Reserved because this very reservation owns it).

const currentSlotId = reservation?.slotId;

const visible = all.filter(
  (s) => s.status === "Available" || s.id === currentSlotId
);

if (!cancelled) setSlots(visible);
} catch (e) {
  if (!cancelled) {
    setSlots([]);
    show(
      e?.response?.data?.message || "Could not load slots for station.",
      "error"
    );
  }
} finally {
  if (!cancelled) setDropdownsLoading((s) => ({ ...s, slots: false }));
}
})();

return () => {
  cancelled = true;

```

```

};

// eslint-disable-next-line react-hooks/exhaustive-deps
}, [showEdit, editForm?.stationId]);

// -----

// Edit

// -----

function openEdit() {
  setEditForm({
    stationId: reservation.stationId || "",
    slotId: reservation.slotId || "",
    scheduledAt: toLocalDateTimeInput(reservation.scheduledAt)
  });
  setSlots([]);
  setShowEdit(true);
}

async function submitEdit() {
  setEditBusy(true);
  try {
    const payload = {
      stationId: editForm.stationId.trim(),

```

```

        slotId: editForm.slotId.trim(),

        scheduledAt: editForm.scheduledAt

        ? new Date(editForm.scheduledAt).toISOString()

        : null

    };

    await api.put(`/reservations/${id}`, payload);

    show("Reservation updated.", "success");

    setShowEdit(false);

    await load();
} catch (e) {

    const msg =

        e?.response?.data?.message ||

        e?.response?.data?.code ||

        e.message ||

        "Update failed.";

    show(msg, "error");

} finally {

    setEditBusy(false);

}

}

// -----

// Cancel

```

```
// -----
```

```
async function confirmCancel() {  
  setCancelBusy(true);  
  try {  
    await api.put(`/reservations/${id}/cancel`, {  
      reason: cancelReason || null  
    });  
    show("Reservation cancelled.", "success");  
    setShowCancel(false);  
    setCancelReason("");  
    await load();  
  } catch (e) {  
    const msg =  
      e?.response?.data?.message ||  
      e?.response?.data?.code ||  
      e.message ||  
      "Cancel failed.";  
    show(msg, "error");  
  } finally {  
    setCancelBusy(false);  
  }  
}
```

```

// -----

// Render

// -----

if (loading) {

  return (

    <div className="mx-auto max-w-6xl px-6 py-8 text-[13px] text-muted">

      Loading reservation...

    </div>

  );

}

if (error || !reservation) {

  return (

    <div className="mx-auto max-w-6xl px-6 py-8">

      <p className="mb-4 rounded-md border border-error/30 bg-error-soft px-3 py-2 text-[13px] font-medium text-error">

        {error || "Reservation not found."}

      </p>

      <Button variant="secondary" onClick={() => navigate("/reservations")}>

        ← Back to reservations

      </Button>

    </div>

  );

}

```

```

    </div>

    );
}

const r = reservation;

const isCancellable = r.status === "Confirmed";

const isEditable = r.status === "Confirmed";

const canSave =

    editForm &&

    !editBusy &&

    editForm.stationId &&

    editForm.slotId &&

    editForm.scheduledAt;

return (

    <div className="mx-auto max-w-6xl px-6 py-8">

        { /* Breadcrumb */ }

        <div className="text-sm text-muted mb-4">

            <Link to="/reservations" className="hover:underline">

                Reservations

            </Link>

            <span className="mx-2">/</span>

```



```

<span className="font-mono text-ink">{r.id.slice(-8)}</span>

</div>

{/* Header */}

<div className="flex flex-wrap items-start justify-between gap-4 mb-6">

  <div>

    <h1 className="text-xl font-semibold tracking-tight text-ink">

      Reservation{" "}

      <span className="font-mono text-lg">{r.id.slice(-8)}</span>

    </h1>

    <p className="text-sm text-muted mt-1">

      Full lifecycle detail. Edit and cancel are available while the

      reservation is Confirmed.

    </p>

  </div>

  <div className="flex items-center gap-2">

    <Badge tone={statusTone(r.status)}>{r.status}</Badge>

    <Button variant="secondary" onClick={load}>

      Refresh

    </Button>

    {isEditable && (

      <Button variant="primary" onClick={openEdit}>

        Edit


```

```

        </Button>

    }}

    {isCancellable && (

        <Button variant="danger" onClick={() => setShowCancel(true)}>

            Cancel reservation

        </Button>

    )}

</div>

</div>

```

```

{ /* Detail grid */}

<div className="grid grid-cols-1 md:grid-cols-2 gap-6">

    { /* Prosumer card */}

    <DetailCard title="Prosumer">

        <DetailRow label="NIC" value={r.prosumerNic} mono />

        {prosumer ? (

            <>

                <DetailRow label="Name" value={prosumer.fullName} />

                <DetailRow label="Email" value={prosumer.email} />

                {prosumer.phone && (

                    <DetailRow label="Phone" value={prosumer.phone} />

                )}

            </DetailRow>

```

```

        label="Account status"

        value={<Badge tone="neutral">{prosumer.status}</Badge>}

    />

</>

): (

    <p className="text-xs text-muted pt-1">

        Profile not available (may be pending activation, deactivated, or

        lookup restricted).

    </p>

    )}

</DetailCard>

<DetailCard title="Booking">

    <DetailRow label="Station ID" value={r.stationId} mono />

    <DetailRow label="Slot ID" value={r.slotId} mono />

    <DetailRow

        label="Scheduled at"

        value={

            <span className="tnum">{formatDateTime(r.scheduledAt)}</span>

        }

    />

</DetailCard>

```

```

<DetailCard title="Lifecycle" className="md:col-span-2">

  <DetailRow

    label="Created"

    value={<span className="tnum">{formatDateTime(r.createdAt)}</span>}

  />

  <DetailRow

    label="Completed"

    value={

      <span className="tnum">{formatDateTime(r.completedAt)}</span>

    }

  />

  <DetailRow label="Cancel reason" value={r.cancelReason || "—"}/>

</DetailCard>

```

```

<DetailCard title="QR token" className="md:col-span-2">

  <p className="text-xs text-muted mb-2">

    The token is issued at creation and verified by a Grid Operator's

    mobile scan. The web app never renders or scans it — this is a text

    view only.

  </p>

  <div className="rounded-md border border-line bg-surface-alt p-3">

    <code className="text-xs font-mono break-all text-ink">

      {r.qrToken || "—"}

    </code>

  </div>

</DetailCard>

```

```

    </code>

  </div>

</DetailCard>

</div>

{/* Edit modal */}

<Modal

  open={showEdit}

  onClose={() => !editBusy && setShowEdit(false)}

  title="Edit reservation"

>

  {editForm && (

    <div className="space-y-4">

      <p className="text-sm text-body">

        Update the reservation details. The server enforces the 7-day

        window and the 12-hour modification notice.

      </p>

      {dropdownsError && (

        <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2 text-xs font-

medium text-error">

          {dropdownsError}

        </p>

```

```
}}
```

```
{/* Prosumer — read-only (ownership is not editable) */}
```

```
<div className="rounded-md border border-line bg-surface-alt px-3 py-2">
```

```
<p className="text-xs text-muted">Prosumer (not editable)</p>
```

```
<p className="text-sm font-medium text-ink">
```

```
{prosumer
```

```
  ? `${prosumer.fullName} — ${r.prosumerNic}`
```

```
  : r.prosumerNic}
```

```
</p>
```

```
</div>
```

```
{/* Station dropdown — clears slot when it changes */}
```

```
<Input
```

```
  label="Station *"
```

```
  as="select"
```

```
  value={editForm.stationId}
```

```
  onChange={(e) => {
```

```
    setEditForm({
```

```
      ...editForm,
```

```
      stationId: e.target.value,
```

```
      slotId: "" // reset slot; new list loads
```

```
    });
```

```

    setSlots([]);

  }}

  disabled={dropdownsLoading.stations}
>

  <option value="">

    {dropdownsLoading.stations

      ? "Loading stations..."

      : stations.length === 0

      ? "No active stations"

      : "Select a station..."}

  </option>

  {stations.map((s) => (

    <option key={s.id} value={s.id}>

      {s.name}

    </option>

  )))}

</Input>

{ /* Slot dropdown — includes the current slot even if not Available */ }

<Input

  label="Slot *"

  as="select"

  value={editForm.slotId}

```

```

onChange={(e) =>
  setEditForm({ ...editForm, slotId: e.target.value })
}

disabled={
  !editForm.stationId ||
  dropdownsLoading.slots ||
  slots.length === 0
}
>
<option value="">
  {!editForm.stationId
    ? "Pick a station first"
    : dropdownsLoading.slots
      ? "Loading slots..."
      : slots.length === 0
        ? "No available slots at this station"
        : "Select a slot..."}
</option>
{slots.map((s) => (
  <option key={s.id} value={s.id}>
    #{s.slotNumber} — {s.type} — {s.capacityKWh} kWh
    {s.id === r.slotId ? " (current)" : ""}
  </option>

```



```

    )}}
</Input>

{/* Scheduled at — bounded to the 7-day window */}

<Input

  label="Scheduled at *"

  type="datetime-local"

  value={editForm.scheduledAt}

  min={minScheduledAt()}

  max={maxScheduledAt()}

  onChange={(e) =>

    setEditForm({ ...editForm, scheduledAt: e.target.value })

  }

/>

<div className="flex justify-end gap-2 pt-2">

  <Button

    variant="secondary"

    onClick={() => setShowEdit(false)}

    disabled={editBusy}

  >

    Discard changes

  </Button>

```

```

<Button
  variant="primary"
  onClick={submitEdit}
  disabled={!canSave}
>
  {editBusy ? "Saving..." : "Save changes"}
</Button>
</div>
</div>
)}
</Modal>

```

```

{/* Cancel modal */}

```

```

<Modal
  open={showCancel}
  onClose={() => !cancelBusy && setShowCancel(false)}
  title="Cancel reservation"
>

```

```

<div className="space-y-4">

```

```

<p className="text-sm text-body">

```

Cancelling this reservation will free the slot. The server enforces a 12-hour notice rule — a late cancel returns a rejection message that will be shown here verbatim.

```

</p>

<Input
  label="Reason (optional)"
  placeholder="e.g. Prosumer requested via phone"
  value={cancelReason}
  onChange={(e) => setCancelReason(e.target.value)}
/>

<div className="flex justify-end gap-2 pt-2">

  <Button
    variant="secondary"
    onClick={() => setShowCancel(false)}
    disabled={cancelBusy}
  >
    Keep reservation
  </Button>

  <Button
    variant="danger"
    onClick={confirmCancel}
    disabled={cancelBusy}
  >
    {cancelBusy ? "Cancelling..." : "Confirm cancel"}
  </Button>
</div>

```

```

    </div>

  </Modal>

  { /* Toast */

  <Toast

    message={toast.message}

    tone={toast.tone}

    onDismiss={() => setToast({ message: "", tone: "error" })}

  />

</div>

);

}

// Small helper card for grouped detail fields.
function DetailCard({ title, children, className = "" }) {

  return (

    <div

      className={`rounded-lg border border-line bg-surface p-5 ${className}`}

    >

      <h2 className="text-sm font-semibold text-ink uppercase tracking-wide mb-3">

        {title}

      </h2>

      <div className="space-y-3">{children}</div>

```

```

    </div>

  );
}

```

```

function DetailRow({ label, value, mono = false }) {

  return (

    <div className="flex items-start justify-between gap-4">

      <span className="text-sm text-muted">{label}</span>

      <span

        className={`text-sm text-ink text-right break-all ${

          mono ? "font-mono text-xs" : ""

        }}

      >

        {value}

      </span>

    </div>

  );
}

```

ReservationsAdminPage.jsx

// =====

// File: ReservationsAdminPage.jsx

// Purpose: Backoffice/GridOperator admin view of all energy

```

//      reservations. Supports read, create (on behalf of
//      a Prosumer), and cancel. Update is done on the
//      detail page. All business rules (7-day window,
//      12-hour notice, status transitions) enforced
//      server-side; this page renders API responses and
//      surfaces rejection messages verbatim.
//
//      The Create modal pulls Prosumers, Stations, and
//      that station's Slots from the API and renders them
//      as dropdowns, so operators pick instead of typing
//      raw ObjectIds.
// Author: Dinil
// =====

import { useEffect, useMemo, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import api from "../../services/api.js";
import Button from "../../components/common/Button.jsx";
import Badge from "../../components/common/Badge.jsx";
import Input from "../../components/common/Input.jsx";
import { Table, Th, Td } from "../../components/common/Table.jsx";
import Modal from "../../components/common/Modal.jsx";
import Toast from "../../components/common/Toast.jsx";
import StatCard from "../../components/common/StatCard.jsx";

```

```
// Formats an ISO timestamp for display (local time, minute precision).
```

```
function formatDateTime(iso) {  
  if (!iso) return "—";  
  const d = new Date(iso);  
  return d.toLocaleString(undefined, {  
    year: "numeric",  
    month: "short",  
    day: "2-digit",  
    hour: "2-digit",  
    minute: "2-digit"  
  });  
}
```

```
// Maps a reservation status string to a Badge tone.
```

```
function statusTone(status) {  
  switch (status) {  
    case "Confirmed":  
      return "warning";  
    case "Completed":  
      return "success";  
    case "Cancelled":  
      return "neutral";  
  }  
}
```

```

    default:

        return "neutral";

    }

}

// Builds the empty form state for the create modal.

function emptyCreateForm() {

    return {

        prosumerNic: "",

        stationId: "",

        slotId: "",

        scheduledAt: ""

    };

}

export default function ReservationsAdminPage() {

    const navigate = useNavigate();

    const [reservations, setReservations] = useState([]);

    const [loading, setLoading] = useState(true);

    const [error, setError] = useState(null);

    // Toast state — parent-owned per Shalon's Toast.jsx pattern.

```



```

const [toast, setToast] = useState({ message: "", tone: "error" });

// Filters mirror the query params GET /api/reservations accepts.
const [filterStatus, setFilterStatus] = useState("");
const [filterNic, setFilterNic] = useState("");
const [filterStationId, setFilterStationId] = useState("");

// Cancel confirmation modal state.
const [cancelTarget, setCancelTarget] = useState(null);
const [cancelReason, setCancelReason] = useState("");
const [cancelBusy, setCancelBusy] = useState(false);

// Create modal state (operator-assisted booking).
const [showCreate, setShowCreate] = useState(false);
const [createForm, setCreateForm] = useState(emptyCreateForm());
const [createBusy, setCreateBusy] = useState(false);

// Dropdown data for the create modal.
// prosumers: [{ nic, fullName, status }]
// stations: [{ id, name, status }]
// slots:  [{ id, slotNumber, type, capacityKWh, status }] ← filtered by selected station
const [prosumers, setProsumers] = useState([]);
const [stations, setStations] = useState([]);

```

```

const [slots, setSlots] = useState([]);

const [dropdownsLoading, setDropdownsLoading] = useState({
  prosumers: false,
  stations: false,
  slots: false
});

const [dropdownsError, setDropdownsError] = useState(null);

// Tiny helper to fire a toast without prop-drilling.
function show(message, tone = "error") {
  setToast({ message, tone });
}

// -----
// List load
// -----

// Loads the reservation list using the current filters.
async function load() {
  setLoading(true);
  setError(null);
  try {
    const params = {};

```

```

    if (filterStatus) params.status = filterStatus;

    if (filterNic) params.nic = filterNic;

    if (filterStationId) params.stationId = filterStationId;

    const { data } = await api.get("/reservations", { params });

    setReservations(data);

  } catch (e) {

    const msg =

      e?.response?.data?.message ||

      e.message ||

      "Failed to load reservations.";

    setError(msg);

    show(msg, "error");

  } finally {

    setLoading(false);

  }

}

// Reload whenever filters change (debounced for typed fields).

useEffect(() => {

  const t = setTimeout(load, 250);

  return () => clearTimeout(t);

  // eslint-disable-next-line react-hooks/exhaustive-deps

```

```

    }, [filterStatus, filterNic, filterStationId]);

// -----

// Dropdown loaders

// -----

// Loads Prosumers + Stations once when the page first mounts.

// Kept separate from the reservation list so a dropdown failure

// never blocks the main table.

useEffect(() => {

    let cancelled = false;

    (async () => {

        setDropdownsLoading((s) => ({ ...s, prosumers: true, stations: true }));

        setDropdownsError(null);

        // Prosumers — only Active ones make sense for a new booking.

        try {

            const { data } = await api.get("/prosumers", {

                params: { status: "Active" }

            });

            if (!cancelled) setProsumers(Array.isArray(data) ? data : []);

        } catch (e) {

```

```

if (!cancelled) {

  setDropdownsError(

    e?.response?.data?.message || "Could not load Prosumers list."

  );

}

} finally {

  if (!cancelled)

    setDropdownsLoading((s) => ({ ...s, prosumers: false }));

}

// Stations — Active only.

try {

  const { data } = await api.get("/stations", {

    params: { status: "Active" }

  });

  if (!cancelled) setStations(Array.isArray(data) ? data : []);

} catch (e) {

  if (!cancelled) {

    setDropdownsError(

      (prev) =>

        prev ||

        e?.response?.data?.message ||

        "Could not load Stations list."
    )
  }
}

```

```

    );
  }
} finally {
  if (!cancelled) setDropdownsLoading((s) => ({ ...s, stations: false }));
}
})();

return () => {
  cancelled = true;
};
}, []);

// Whenever the selected station changes, load that station's slots.
useEffect(() => {
  const stationId = createForm.stationId;
  if (!stationId) {
    setSlots([]);
    return;
  }

  let cancelled = false;

  (async () => {
    setDropdownsLoading((s) => ({ ...s, slots: true }));

```

```

try {

  const { data } = await api.get(`/stations/${stationId}`);

  // StationDetailResponse shape — slots array inside.

  const list = Array.isArray(data?.slots) ? data.slots : [];

  if (!cancelled) {

    // Only show Available slots — Reserved/Inactive would fail the

    // server-side check anyway.

    setSlots(list.filter((s) => s.status === "Available"));

  }

} catch (e) {

  if (!cancelled) {

    setSlots([]);

    show(

      e?.response?.data?.message || "Could not load slots for station.",

      "error"

    );

  }

} finally {

  if (!cancelled) setDropdownsLoading((s) => ({ ...s, slots: false }));

}

})();

return () => {

```

```

    cancelled = true;

};

// eslint-disable-next-line react-hooks/exhaustive-deps
}, [createForm.stationId]);

// -----

// Create

// -----

// Resets the create modal to a clean slate.
function openCreate() {
    setCreateForm(emptyCreateForm());
    setSlots([]);
    setShowCreate(true);
}

// Submits a create to the API. Enforces nothing client-side —
// the server validates the 7-day window and slot availability.
async function submitCreate() {
    setCreateBusy(true);
    try {
        const payload = {
            prosumerNic: createForm.prosumerNic.trim(),

```



```

    stationId: createForm.stationId.trim(),

    slotId: createForm.slotId.trim(),

    scheduledAt: createForm.scheduledAt
      ? new Date(createForm.scheduledAt).toISOString()
      : null
  };

  await api.post("/reservations", payload);
  show("Reservation created.", "success");
  setShowCreate(false);
  setCreateForm(emptyCreateForm());
  setSlots([]);
  await load();
} catch (e) {

  const msg =

    e?.response?.data?.message ||

    e?.response?.data?.code ||

    e.message ||

    "Create failed.";

  show(msg, "error");

} finally {

  setCreateBusy(false);

}

```

```
}
```

```
// -----
```

```
// Cancel
```

```
// -----
```

```
async function confirmCancel() {  
  if (!cancelTarget) return;  
  setCancelBusy(true);  
  try {  
    await api.put(`/reservations/${cancelTarget.id}/cancel`, {  
      reason: cancelReason || null  
    });  
    show("Reservation cancelled.", "success");  
    setCancelTarget(null);  
    setCancelReason("");  
    await load();  
  } catch (e) {  
    const msg =  
      e?.response?.data?.message ||  
      e?.response?.data?.code ||  
      e.message ||  
      "Cancel failed.";  
  }  
}
```

```

    show(msg, "error");
  } finally {
    setCancelBusy(false);
  }
}

// -----
// Derived values
// -----

const counts = useMemo(
  () => ({
    total: reservations.length,
    confirmed: reservations.filter((r) => r.status === "Confirmed").length,
    completed: reservations.filter((r) => r.status === "Completed").length,
    cancelled: reservations.filter((r) => r.status === "Cancelled").length
  }),
  [reservations]
);

const canCreate =
  !createBusy &&
  createForm.prosumerNic &&

```

```
createForm.stationId &&
```

```
createForm.slotId &&
```

```
createForm.scheduledAt;
```

```
return (
```

```
<div className="mx-auto max-w-6xl px-6 py-8">
```

```
  { /* Page header */ }
```

```
<div className="mb-6 flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
```

```
<div>
```

```
<h1 className="text-xl font-semibold tracking-tight text-ink">
```

```
  Reservations
```

```
</h1>
```

```
<p className="mt-1 text-[13px] text-muted">
```

```
  All energy slot reservations across microgrid nodes. Operators may
```

```
  create bookings on behalf of Prosumers (phone-in or walk-in).
```

```
</p>
```

```
</div>
```

```
<div className="flex items-center gap-2">
```

```
<Button variant="primary" onClick={openCreate}>
```

```
  + New reservation
```

```
</Button>
```

```
<Button variant="secondary" onClick={load} disabled={loading}>
```

```
  {loading ? "Refreshing..." : "Refresh"}
```

```

    </Button>

  </div>

</div>

{/* Summary row */}

<div className="mb-6 grid grid-cols-2 gap-3 sm:grid-cols-4">

  <StatCard

    icon="calendar"

    label="Total"

    value={counts.total}

    hint="All reservations"

  />

  <StatCard

    icon="pulse"

    label="Confirmed"

    value={counts.confirmed}

    hint="Awaiting check-in"

  />

  <StatCard

    icon="check"

    label="Completed"

    value={counts.completed}

    hint="Transfer finished"

```

```
/>
```

```
<StatCard
```

```
  icon="bolt"
```

```
  label="Cancelled"
```

```
  value={counts.cancelled}
```

```
  hint="Slot released"
```

```
/>
```

```
</div>
```

```
{/* Filters */}
```

```
<div className="mb-4 grid grid-cols-1 gap-3 md:grid-cols-3">
```

```
  <Input
```

```
    label="Status"
```

```
    as="select"
```

```
    value={filterStatus}
```

```
    onChange={(e) => setFilterStatus(e.target.value)}
```

```
  >
```

```
    <option value="">All statuses</option>
```

```
    <option value="Confirmed">Confirmed</option>
```

```
    <option value="Completed">Completed</option>
```

```
    <option value="Cancelled">Cancelled</option>
```

```
  </Input>
```

```
  <Input
```

```

        label="Prosumer NIC"

        placeholder="e.g. 199012345678"

        value={filterNic}

        onChange={(e) => setFilterNic(e.target.value)}

    />

    <Input

        label="Station ID"

        placeholder="Mongo ObjectId"

        value={filterStationId}

        onChange={(e) => setFilterStationId(e.target.value)}

    />

</div>

{/* Error state */}

{error && (

    <p className="mb-4 rounded-md border border-error/30 bg-error-soft px-3 py-2 text-[13px] font-medium text-error">

        {error}

    </p>

)}

{/* Table */}

<Table>

```

```

<thead>

<tr>

  <Th>ID</Th>

  <Th>Prosumer NIC</Th>

  <Th>Scheduled</Th>

  <Th>Status</Th>

  <Th className="text-right">Actions</Th>

</tr>

</thead>

<tbody>

  {loading && (

    <tr>

      <Td colspan={5} className="text-center text-muted">

        Loading...

      </Td>

    </tr>

  )}

  {!loading && reservations.length === 0 && (

    <tr>

      <Td colspan={5} className="text-center text-muted">

        No reservations match the current filters.

      </Td>

    </tr>

  )}

```



```

  })

  {!loading &&

  reservations.map((r) => (

    <tr key={r.id}>

      <Td>

        <Link

          to={`/${reservations}/${r.id}`}

          className="font-mono text-xs text-primary hover:underline"

        >

          {r.id.slice(-8)}

        </Link>

      </Td>

      <Td>{r.prosumerNic}</Td>

      <Td className="tnum">{formatDateTime(r.scheduledAt)}</Td>

      <Td>

        <Badge tone={statusTone(r.status)}>{r.status}</Badge>

      </Td>

      <Td className="text-right">

        <div className="flex items-center gap-2 justify-end">

          <Button

            variant="ghost"

            size="sm"

            onClick={() => navigate(`/${reservations}/${r.id}`)}

```

```

    >

    View

  </Button>

  {r.status === "Confirmed" && (

    <Button

      variant="danger"

      size="sm"

      onClick={() => {

        setCancelTarget(r);

        setCancelReason("");

      }}

    >

    Cancel

  </Button>

  )}

</div>

</Td>

</tr>

  )}]

</tbody>

</Table>

{ /* Create modal — dropdowns for Prosumer / Station / Slot */ }

```

```

<Modal
  open={showCreate}
  onClose={() => !createBusy && setShowCreate(false)}
  title="New reservation"
>

<div className="space-y-4">

  <p className="text-xs text-muted">

    Create a reservation on behalf of a Prosumer. The server enforces

    the 7-day booking window and slot availability.

  </p>

  {dropdownsError && (
    <p className="rounded-md border border-error/30 bg-error-soft px-3 py-2 text-xs font-
medium text-error">

      {dropdownsError}

    </p>
  )}

  {/* Prosumer dropdown */}

  <Input

    label="Prosumer *"

    as="select"

    value={createForm.prosumerNic}

```

```

onChange={(e) => {

  // Changing prosumer does not reset station/slot.

  setCreateForm({ ...createForm, prosumerNic: e.target.value });

}}

disabled={dropdownsLoading.prosumers}
>

<option value="">

  {dropdownsLoading.prosumers

    ? "Loading prosumers..."

    : prosumers.length === 0

    ? "No active prosumers"

    : "Select a prosumer..."}

</option>

{prosumers.map((p) => (

  <option key={p.nic} value={p.nic}>

    {p.fullName} — {p.nic}

  </option>

))}

</Input>

{/* Station dropdown — clears slot when it changes */}

<Input

  label="Station *"

```

```

as="select"

value={createForm.stationId}

onChange={(e) => {

  setCreateForm({

    ...createForm,

    stationId: e.target.value,

    slotId: "" // reset slot; new list loads

  });

  setSlots([]);

}}

disabled={dropdownsLoading.stations}
>

<option value="">

  {dropdownsLoading.stations

    ? "Loading stations..."

    : stations.length === 0

    ? "No active stations"

    : "Select a station..."}

</option>

{stations.map((s) => (

  <option key={s.id} value={s.id}>

    {s.name}

  </option>

```

```

    )})
  </Input>

  { /* Slot dropdown — disabled until a station is chosen */ }

  <Input
    label="Slot *"
    as="select"
    value={createForm.slotId}
    onChange={(e) =>
      setCreateForm({ ...createForm, slotId: e.target.value })
    }
    disabled={
      !createForm.stationId ||
      dropdownsLoading.slots ||
      slots.length === 0
    }
  >
    <option value="">
      {!createForm.stationId
        ? "Pick a station first"
        : dropdownsLoading.slots
          ? "Loading slots..."
          : slots.length === 0
      }
    </option>
  </Input>

```

```

        ? "No available slots at this station"

        : "Select a slot..."
    }
</option>

{slots.map((s) => (
    <option key={s.id} value={s.id}>

        #{s.slotNumber} — {s.type} — {s.capacityKWh} kWh

    </option>

))}
</Input>

{ /* Scheduled at — free text (datetime-local) */ }

<Input

    label="Scheduled at *"

    type="datetime-local"

    value={createForm.scheduledAt}

    onChange={(e) =>

        setCreateForm({ ...createForm, scheduledAt: e.target.value })

    }

/>

<div className="flex justify-end gap-2 pt-2">

    <Button

        variant="secondary"

```

```

        onClick={() => setShowCreate(false)}

        disabled={createBusy}
    >

    Cancel

</Button>

<Button

    variant="primary"

    onClick={submitCreate}

    disabled={!canCreate}

    >

        {createBusy ? "Creating..." : "Create reservation"}

    </Button>

</div>

</div>

</Modal>

{/* Cancel confirmation modal */}

<Modal

    open={!cancelTarget}

    onClose={() => !cancelBusy && setCancelTarget(null)}

    title="Cancel reservation"

    >

        {cancelTarget && (

```



```

<div className="space-y-4">

  <p className="text-sm text-body">

    Are you sure you want to cancel reservation{" "}

    <span className="font-mono text-ink">

      {cancelTarget.id.slice(-8)}

    </span>{" "}

    for NIC{" "}

    <span className="font-semibold">{cancelTarget.prosumerNic}</span>?

  </p>

  <p className="text-xs text-muted">

    The server enforces the 12-hour notice rule — a late cancel will

    be rejected and the reason shown below.

  </p>

  <Input

    label="Reason (optional)"

    placeholder="e.g. Prosumer requested via phone"

    value={cancelReason}

    onChange={(e) => setCancelReason(e.target.value)}

  />

  <div className="flex justify-end gap-2 pt-2">

    <Button

      variant="secondary"

      onClick={() => setCancelTarget(null)}

```

```

        disabled={cancelBusy}

    >

    Keep reservation

</Button>

<Button

    variant="danger"

    onClick={confirmCancel}

    disabled={cancelBusy}

    >

        {cancelBusy ? "Cancelling..." : "Confirm cancel"}

    </Button>

</div>

</div>

)}

</Modal>

{ /* Toast */}

<Toast

    message={toast.message}

    tone={toast.tone}

    onDismiss={() => setToast({ message: "", tone: "error" })}

/>

</div>

```

```

    );
}

ScanQrActivity.kt
// =====

// File: ScanQrActivity.kt

// Purpose: Thin Activity host for ScanQrScreen — needed so the
// scanner can also be launched as a standalone Intent
// outside the Home bottom-nav shell.

// Author: Dinil

// =====

package com.smartmicrogrid.ui.operator

import android.content.Context
import android.content.Intent
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.smartmicrogrid.ui.theme.SmartMicrogridTheme

class ScanQrActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

```

```

enableEdgeToEdge()

setContent {

    SmartMicrogridTheme {

        ScanQrScreen()

    }

}

}

companion object {

    fun intentFor(context: Context): Intent =

        Intent(context, ScanQrActivity::class.java)

}

}

ScanQrScreen.kt
// =====
// File: ScanQrScreen.kt
// Purpose: Grid Operator QR scanner. CameraX preview + ML Kit
//      decode -> POST /api/reservations/verify-qr. Shows the
//      server's response (or rejection message) verbatim;
//      no business rule is computed on the client.
// Author: Dinil
// =====
package com.smartmicrogrid.ui.operator

import android.Manifest
import android.content.Context
import android.util.Size
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView

```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalLifecycleOwner
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.core.content.ContextCompat
import com.google.accompanist.permissions.ExperimentalPermissionsApi
import com.google.accompanist.permissions.isGranted
import com.google.accompanist.permissions.rememberPermissionState
import com.smartmicrogrid.data.local.AppDbHelper
import com.smartmicrogrid.data.local.ReservationCacheDao
import com.smartmicrogrid.data.local.VerifiedScan
import com.smartmicrogrid.data.remote.ApiClient
import com.smartmicrogrid.data.remote.dto.ApiErrorBody
import com.smartmicrogrid.data.remote.dto.VerifyQrRequest
import com.smartmicrogrid.data.remote.dto.VerifyQrResponse
import com.smartmicrogrid.util.QrScannerHelper
import com.google.gson.Gson
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.util.concurrent.Executors

@OptIn(ExperimentalPermissionsApi::class)
@Composable
fun ScanQrScreen() {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val scope = rememberCoroutineScope()

    // Camera permission gate.
    val cameraPermission = rememberPermissionState(Manifest.permission.CAMERA)

    // UI state.
    var result by remember { mutableStateOf<VerifyQrResponse?>(null) }
    var errorMessage by remember { mutableStateOf<String?>(null) }
    var busy by remember { mutableStateOf(false) }
    var scanResetKey by remember { mutableStateOf(0) } // bump to restart scanner

```

```

// Ask for permission on first composition.
LaunchedEffect(Unit) {
    //if (!cameraPermission.hasPermission) cameraPermission.launchPermissionRequest()
    if (cameraPermission.status.isGranted) cameraPermission.launchPermissionRequest()
}

Column(
    modifier = Modifier.fillMaxSize().padding(16.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp)
) {
    Text("Scan Prosumer QR", style = MaterialTheme.typography.headlineSmall)
    Text(
        "Point the camera at the prosumer's transaction QR. The server verifies " +
        "the token and completes the reservation.",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )

    // Camera preview + overlay.
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .weight(1f)
            .background(Color.Black)
    ) {
        // if (cameraPermission.hasPermission) {
        if (cameraPermission.status.isGranted) {
            key(scanResetKey) {
                CameraPreview(
                    onQrDecoded = { token ->
                        if (busy) return@CameraPreview
                        busy = true
                        errorMessage = null
                        scope.launch {
                            try {
                                val resp = callVerify(context, token)
                                if (resp.isSuccessful && resp.body() != null) {
                                    val body = resp.body()!!
                                    result = body
                                    cacheVerifiedScan(context, body)
                                } else {
                                    errorMessage = parseError(resp.errorBody()?.string())
                                    ?: "Verification failed (${resp.code()})"
                                }
                            }
                        }
                    }
                )
            }
        }
    }
}

```

```

        }
    } catch (e: Exception) {
        errorMessage = "Network error: ${e.message}"
    } finally {
        busy = false
    }
}
},
onError = { errorMessage = it }
)
}
} else {
    Text(
        "Camera permission required to scan QR codes.",
        color = Color.White,
        modifier = Modifier.align(Alignment.Center).padding(16.dp)
    )
}

if (busy) {
    CircularProgressIndicator(
        modifier = Modifier.align(Alignment.Center)
    )
}
}

// Result card or error.
result?.let { r ->
    Card(modifier = Modifier.fillMaxWidth()) {
        Column(Modifier.padding(12.dp), verticalArrangement =
Arrangement.spacedBy(4.dp)) {
            Text("✓ Verified", style = MaterialTheme.typography.titleMedium)
            Text("Reservation: ${r.reservationId}")
            Text("Prosumer: ${r.prosumerName}")
            Text("Station: ${r.stationName}")
            Text("Slot: ${r.slotNumber}")
            Text("Status: ${r.status}")
            Spacer(Modifier.height(8.dp))
            Button(onClick = {
                result = null
                errorMessage = null
                scanResetKey++ // restart the scanner
            }) { Text("Scan another") }
        }
    }
}

```

```

    }
}

if (errorMessage != null) {
    Card(modifier = Modifier.fillMaxWidth()) {
        Column(Modifier.padding(12.dp)) {
            Text("X ${errorMessage!!}", color = MaterialTheme.colorScheme.error)
            Spacer(Modifier.height(8.dp))
            Button(onClick = {
                errorMessage = null
                scanResetKey++
            }) { Text("Retry scan") }
        }
    }
}
}
}

@Composable
private fun CameraPreview(
    onQrDecoded: (String) -> Unit,
    onError: (String) -> Unit
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current

    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = { ctx ->
            val previewView = PreviewView(ctx).apply {
                scaleType = PreviewView.ScaleType.FILL_CENTER
            }

            val cameraProviderFuture = ProcessCameraProvider.getInstance(ctx)
            cameraProviderFuture.addListener({
                val cameraProvider = cameraProviderFuture.get()
                val preview = Preview.Builder().build().also {
                    it.setSurfaceProvider(previewView.surfaceProvider)
                }
                val analyzer = ImageAnalysis.Builder()
                    .setTargetResolution(Size(1280, 720))
                    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                    .build()
                    .also {

```



```

        it.setAnalyzer(
            Executors.newSingleThreadExecutor(),
            QrScannerHelper(onDecoded = onQrDecoded, onError = onError)
        )
    }
    try {
        cameraProvider.unbindAll()
        cameraProvider.bindToLifecycle(
            lifecycleOwner,
            CameraSelector.DEFAULT_BACK_CAMERA,
            preview,
            analyzer
        )
    } catch (e: Exception) {
        onError("Camera bind failed: ${e.message}")
    }
}, ContextCompat.getMainExecutor(ctx))

    previewView
}
)
}

// Calls the verify-qr endpoint. Runs the network call off the main thread.
private suspend fun callVerify(context: Context, token: String) = withContext(Dispatchers.IO) {
    ApiClient.service.verifyQr(VerifyQrRequest(token))
}

// Writes a successful verification into the operator's local scan history.
private suspend fun cacheVerifiedScan(context: Context, r: VerifyQrResponse) =
    withContext(Dispatchers.IO) {
        val db = AppDbHelper(context).writableDatabase
        ReservationCacheDao.upsert(db, VerifiedScan(
            reservationId = r.reservationId,
            prosumerName = r.prosumerName,
            stationName = r.stationName,
            slotNumber = r.slotNumber,
            status = r.status,
            verifiedAt = System.currentTimeMillis()
        ))
    }
}

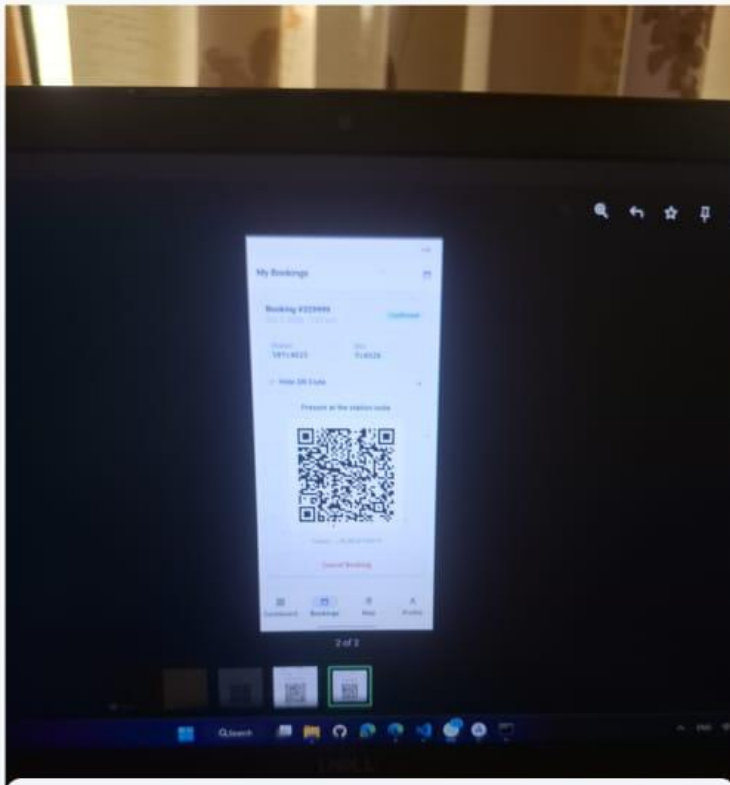
// Extracts a human-readable message from an API error body.
private fun parseError(body: String?): String? {

```

```
if (body.isNullOrBlank()) return null
return try {
    val err = Gson().fromJson(body, ApiErrorBody::class.java)
    err.message ?: err.code
} catch (_: Exception) { null }
}
```

1:13

25



✓ Verified

Reservation:

6abb6ad70a2fe15a0e329999

Prosumer: 200020002000

Station: Test Station Beta

Slot: 1

Status: Completed

Scan another



Dashboard



Scan QR



Map



Profile

11:13

25

Operator Dashboard

Today's verification activity, live from the API.

Confirmed Today

7

Completed Today

3

Pending by station

station new

Station 834c9ef8

1

Test Station Alpha

Station 58fc3fc6

1

Dankotuwa

Station 43a7c698

1



Dashboard



Scan QR



Map



Profile

Joule

OVERVIEW

Dashboard

NODE MANAGEMENT

Microgrid hubs

RESERVATIONS

Reservations

ADMINISTRATION

Users

Dinil

Backoffice

Sign out

Reservations

+ New reservation

Refresh

Total

8

All reservations

Confirmed

3

Awaiting check-in

Completed

3

Transfer finished

Cancelled

2

Slot released

Status

All statuses

Prosumer NIC

e.g. 199012345678

Station ID

Mongo ObjectId

ID	PROSUMER NIC	SCHEDULED	STATUS	ACTIONS
e8b40760	200212803369	Sep 29, 2026, 02:00 AM	Confirmed	View Cancel
58fc3fcd	991234567v	Sep 29, 2026, 09:00 AM	Confirmed	View Cancel
58fc4027	200020002000	Sep 30, 2026, 10:00 AM	Cancelled	View
da243ef2	200020002000	Sep 30, 2026, 10:03 AM	Completed	View
5fdcd980	200212803369	Sep 30, 2026, 12:00 PM	Confirmed	View Cancel

Reservations > Details

Reservations / e8b40760

Confirmed

Refresh

Edit

Cancel reservation

Reservation e8b40760

Full lifecycle detail. Edit and cancel are available while the reservation is Confirmed.

PROSUMER

NIC

200212803369

Profile not available (may be pending activation, deactivated, or lookup restricted).

BOOKING

Station ID

6ab8951c5b592345834c9ef8

Slot ID

6ab895bf5b592345834c9ef9

Scheduled at

Sep 29, 2026, 02:00 AM

LIFECYCLE

Created

Sep 28, 2026, 09:27 PM

Completed

—

Cancel reason

—

QR TOKEN

The token is issued at creation and verified by a Grid Operator's mobile scan. The web app never renders or scans it — this is a text view only.

NmF1YTh1NGYzMTAyYWVhZU4YjQulzYwFDE3OTA3NTMhMD8yVFh0F0h1NGxpZUFod3AzaDViUnhrRkl0yMmVIZE1Gb3J1neG1jMvVaT00

Edit reservation

Update the reservation details. The server enforces the 7-day window and the 12-hour modification notice.

Prosumer (not editable)

200212803369

Station *

station new

Slot *

#1 — Charging — 12000 kWh (current)

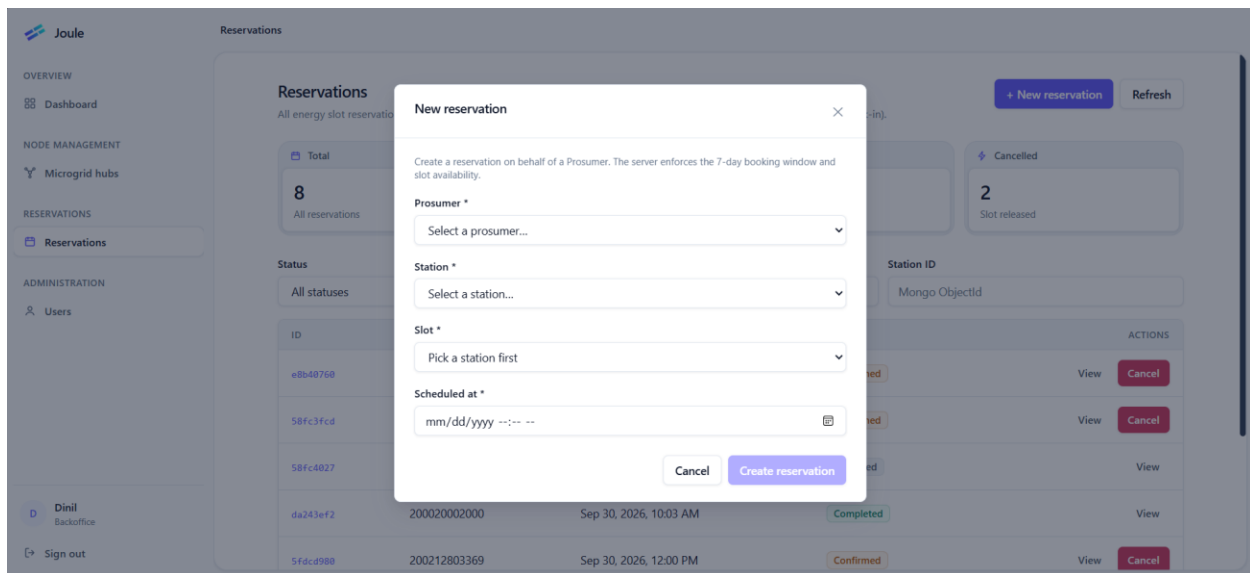
Scheduled at *

09/29/2026 02:00 AM

Discard changes

Save changes

345



6. Technical Challenges and Reflections

Enforcing the seven day booking window turned out to need more care than expected, because a user could in principle set their device clock incorrectly or simply try to submit a date outside the allowed range. The fix has two layers. On the mobile side, the date picker itself only allows dates between the current time and seven days ahead, so the app steers the user correctly by default. On the server side, the Reservations controller independently checks that the scheduled time falls between five minutes in the past and seven days ahead, and rejects anything outside that range with an HTTP 400 response. The server check is the one that actually matters, since the client side limit is only a convenience.

The camera based QR scanner caused repeated crashes early on, mostly around returning to the app from the background with the camera preview left in a bad state. The fix was to bind the CameraX process camera provider explicitly to the screen's lifecycle, re check camera permission every time the screen resumed, and unbind the camera cleanly when the screen was disposed rather than leaving it running.

Finding nearby hubs also needed a fallback plan, since not every prosumer grants location permission and not every device gets a GPS lock indoors. The app first tries to read the last known location from the device. If that comes back empty or the permission was denied, it falls back to a fixed default point in Colombo so the map still has something sensible to show. On the server side, the nearby stations query defaults to a generous five hundred kilometre radius when no better location is available, so every registered hub still shows up somewhere on the list.

A more subtle problem was the risk of two prosumers booking the same battery slot at the same moment. This is solved with an atomic conditional update rather than a read then write pattern. The update only succeeds if the slot's status is still Available at the moment the write runs. If the update matches zero

documents, the server returns a 409 Conflict immediately, and the client shows the prosumer that the slot has just been taken.

7. Use of AI Tools & Reflection

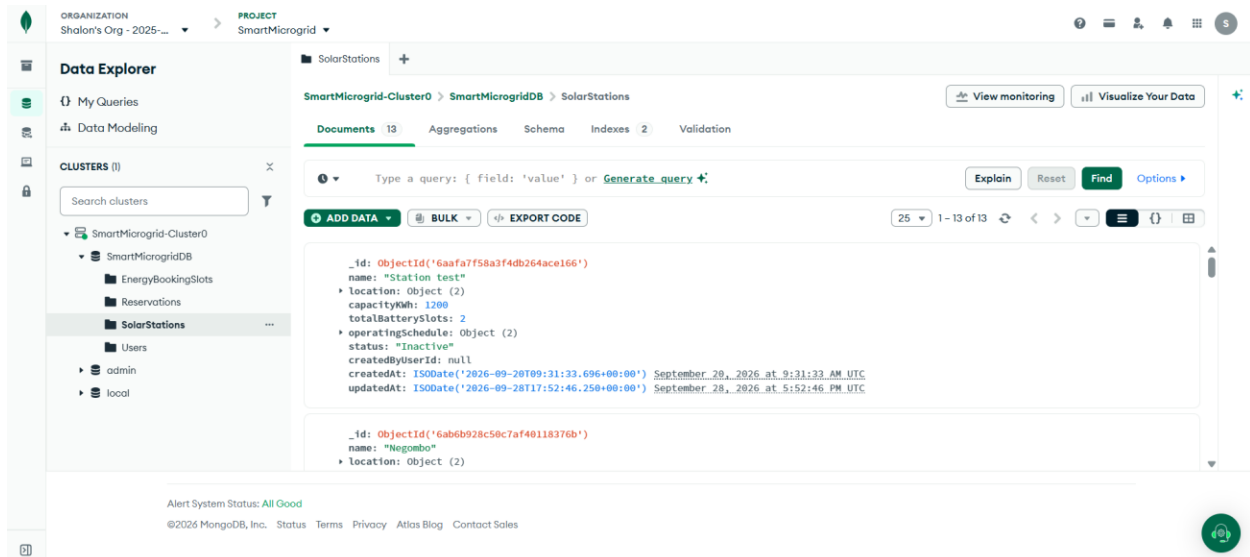
- **Planning, Technical Research & Infrastructure Guidance:** AI was utilized as a thought partner and technical reference during the initial architecture planning phase to review microgrid domain requirements and understand enterprise hosting patterns, understand what's required to accomplish. Additionally, external AI assistance was referenced for infrastructure configuration guidance specifically troubleshooting Windows IIS setup (Application Pool configuration with No Managed Code, ASP.NET Core Module registration, and environment port/socket binding diagnostics).
- **Multimedia & Presentation Assets:** AI generative media tools were used exclusively for creative marketing assets like generating video clips for the promotional trailer video and the web application hero background. The 5 minute technical demo video was recorded manually to show live, authentic system functionality.
- **Independent Implementation and Integrity:** All source code across the solution including the ASP.NET Core Web API, business logic rules, native Android activities, SQLite helper classes, React UI components, and MongoDB database queries was authored, tested, and implemented entirely by hand without AI assisted code generation. Every technical decision, data model, and API route has been developed independently under incremental Git version control.

8. Hosting and Deployment

8.1 MongoDB Cloud Database Setup

The server side NoSQL database is hosted on MongoDB Atlas in the cloud. The ASP.NET Core Web API connects directly to this cloud cluster to manage user records, station configurations, and reservation schedules.

The database contains all four required collections: Users, SolarStations, EnergyBookingSlots, and Reservations. Each document contains full business attributes, including geospatial coordinates (location) for proximity queries and operational capacity specs (capacityKwh and battery slots).



MongoDB Atlas Data Explorer Showing Cloud Collections and Station Documents

7.2 IIS and Kestrel Web API Deployment

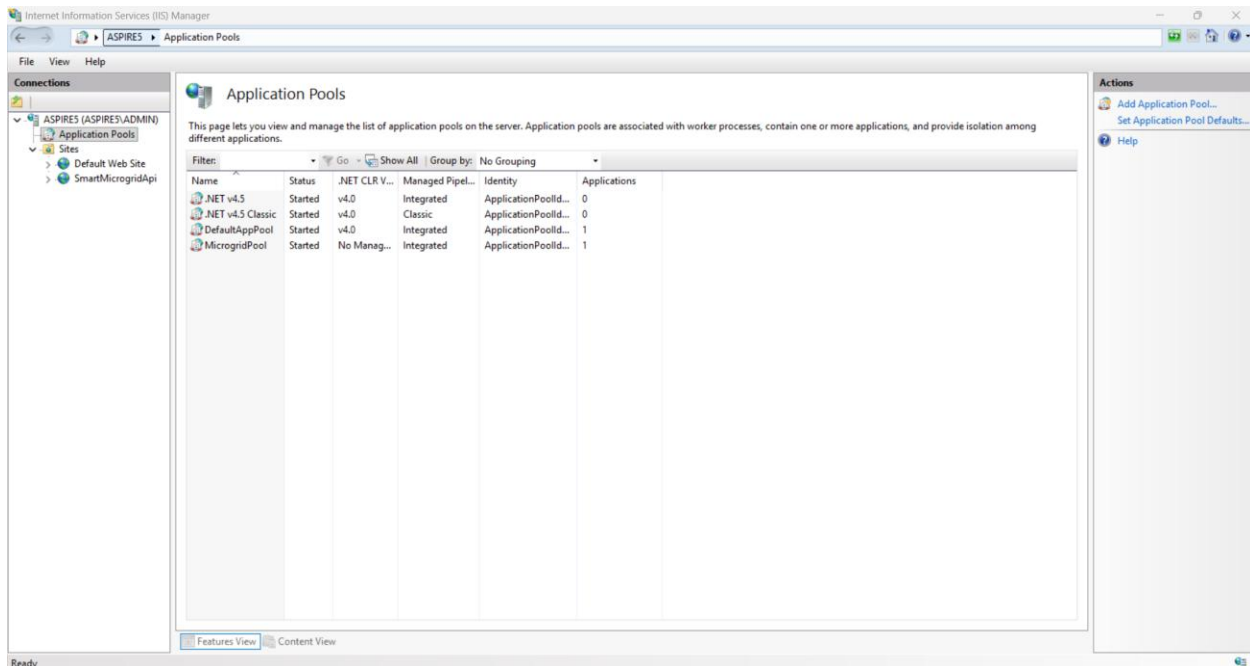
To meet the enterprise system architecture requirements, the central C# ASP.NET Core Web API was deployed directly on a local Windows Internet Information Services (IIS) server rather than running inside a temporary development terminal.

1. IIS Configuration and Hosting Bundle: Windows IIS was enabled with Application Development Features, including WebSocket Protocol and .NET Extensibility. The ASP.NET Core 9.0 Hosting Bundle was installed on the machine to register the native `AspNetCoreModuleV2` handler required for in-process hosting.
2. Publishing and Application Pool Setup:

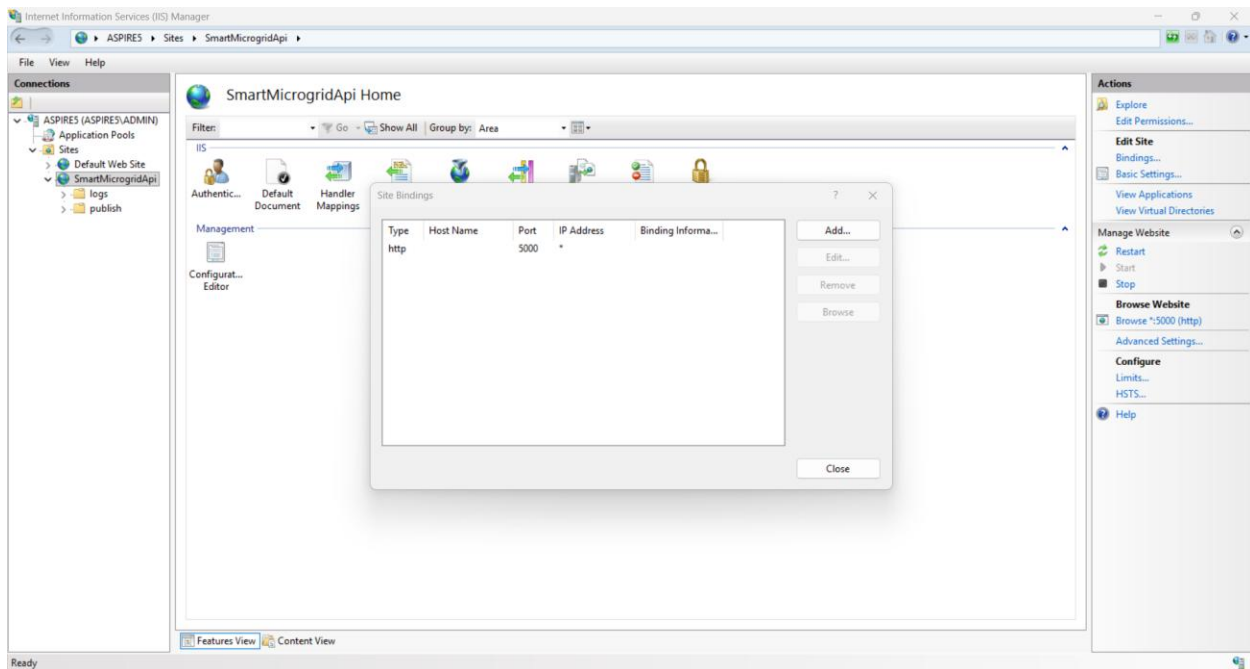
The API source code was built in Release mode and published directly into the standard IIS root folder:

```
cd F:\Smart-Solar-Grid\WebService\SmartMicrogrid.Api
dotnet publish -c Release -o C:\inetpub\wwwroot\smartmicrogrid-api
```

In IIS Manager (inetmgr), a dedicated Application Pool named `MicrogridPool` was configured with its .NET CLR Version set strictly to "No Managed Code". Because modern ASP.NET Core manages its own runtime, IIS acts as an in-process reverse proxy. A new website was added pointing to the published folder and bound to HTTP port 5000.



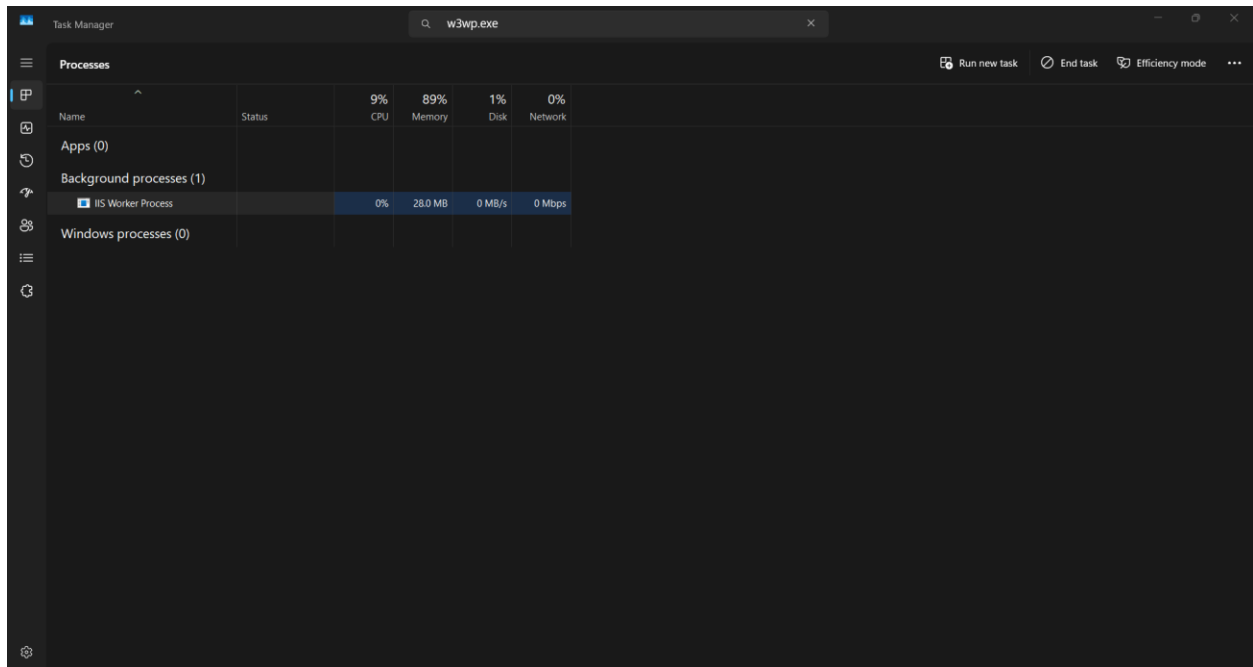
IIS Manager Site Configuration and Application Pool Setup



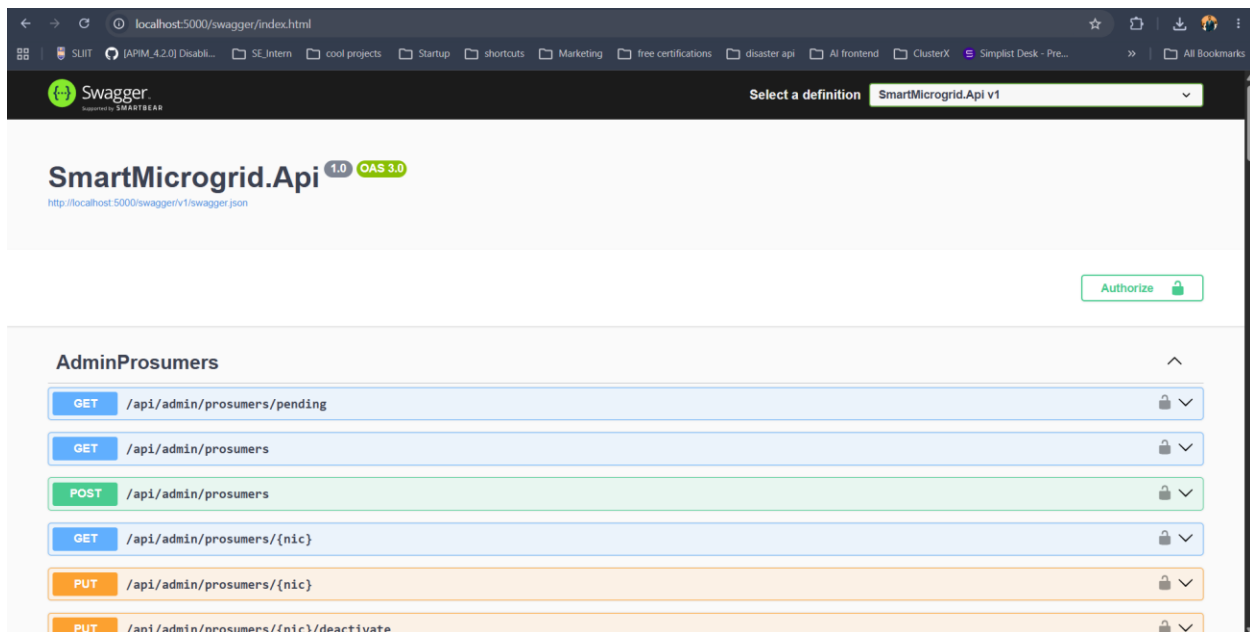
IIS Port Binding (HTTP Port 5000)

- In-Process web.config Setup: The web.config file generated in C:\inetpub\wwwroot\smartmicrogrid-api passes incoming web requests to the compiled application binary (SmartMicrogrid.Api.dll) using in-process hosting, with the ASPNETCORE_ENVIRONMENT variable set to Development:

- Live Deployment Verification: The API runs under the native Windows IIS worker process (w3wp.exe) and responds to incoming requests on port 5000. The live Swagger interface confirms that all API routes are accessible and communicating with MongoDB Atlas.



Active IIS Worker Process (w3wp.exe) Hosting the Web API



Live Web API Endpoints on Windows IIS (Swagger UI)

7.3 Web and Mobile Client Build

Build the web app with the two commands below, which install dependencies and produce a production bundle in the dist folder.

```
cd f:\Smart-Solar-Grid\WebApp
npm install
npm run build
```

Build and install the Android app with Gradle. The first command produces a debug APK under app build outputs apk debug, and the second installs it on a connected device or emulator.

```
cd f:\Smart-Solar-Grid\MobileApp
./gradlew assembleDebug
./gradlew installDebug
```

8. Git Repository and References

8.1 Git Repository

Repository URL - <https://github.com/3hal0n/smart-solar-microgrid-system>

8.2 References

Microsoft Corporation, 2026. Overview of ASP.NET Core Web API and Security Architecture. Microsoft Learn Documentation. Available at, <https://learn.microsoft.com/aspnet/core/>

MongoDB Inc, 2026. Geospatial Queries with 2dsphere Indexes and the C# Driver. MongoDB Documentation. Available at, <https://www.mongodb.com/docs/drivers/csharp/>

Google Developers, 2026. Building Modern Android UI with Jetpack Compose and CameraX. Android Developer Guides. Available at, <https://developer.android.com/jetpack/compose>

BCrypt.Net, 2026. Cryptographic Password Hashing and Salt Verification. Open Source Cryptography. Available at, <https://github.com/BcryptNet/bcrypt.net>

ZXing and CameraX Integration Team, 2026. High Performance Barcode and QR Code Processing on Android. Available at, <https://github.com/zxing/zxing>